OMNICODE: VOLUME 3: OMNICODE LANGUAGE & STANDARD LIBRARY

Track 2: Lexicon, SVO Verbs, Bijective Transduction, .omni & .tape Modules

CORPUS VOLUME
VOL 3

NOTEBOOKLM WORD COUNT
64,593 words (Limit: 500k words)

CATALOGED FILES
154 files (12,257 lines)

GENERATION DATE
2026-10-14 10:52:48

EPISTEMIC COVENANT BEDROCK & IMMUTABLE LANDMARKS

"And he gave unto Moses... two tables of testimony, tables of stone, written with the finger of God." — Exodus 31:18

"Prepare thy work without, and make it fit for thyself in the field; and afterwards build thine house." — Proverbs 24:27

"Remove not the ancient landmark, which thy fathers have set." — Proverbs 22:28

GOOGLE NOTEBOOKLM (AI PRO) GROUNDING PROMPTS

- **Balanced Ternary Quad-State:** *"How does the T4 algebra {-1:Rollback, 0:Await, +1:Proceed, NULL:Halt} enforce symmetry in this volume?"*
- **4-Block Structure:** *"Explain the boundaries of Block 1 (Metadata/Scripture), Block 2 (Opening), Block 3 (Body), and Block 4 (Closing)."*
- **Character Lifecycle & SVO Verbs:** *"Trace the Character lifecycle through INSCRIBE, TRANSDUCE, PROJECT, and SEQUENCE across terminal ports 0x03F8 (FROM) and 0x000B8000 (INTO)."*

VOLUME SOURCE MANIFEST & INDEX (154 FILES)

#	FILE PATH	TYPE	LINES	WORDS	SIZE (KB)
1	**<TRACK 2> : {OmniCode Language & Standard Library}/TRACK.md**	MD	10	68	0.5
2	**<TRACK 2> : {OmniCode Language & Standard Library}/lexicon/03_verbs/arithmetic_verbs.omni**	OMNI	94	399	3.3
3	**<TRACK 2> : {OmniCode Language & Standard Library}/lexicon/03_verbs/arithmetic_verbs.tape**	TAPE	33	207	1.6
4	**<TRACK 2> : {OmniCode Language & Standard Library}/lexicon/03_verbs/axial_vectors.omni**	OMNI	68	271	2.5
5	**<TRACK 2> : {OmniCode Language & Standard Library}/lexicon/03_verbs/axial_vectors.tape**	TAPE	30	178	1.4
6	**<TRACK 2> : {OmniCode Language & Standard Library}/lexicon/03_verbs/cinematic_track_loop.omni**	OMNI	56	245	2.2
7	**<TRACK 2> : {OmniCode Language & Standard Library}/lexicon/03_verbs/cinematic_track_loop.tape**	TAPE	36	250	1.8
8	**<TRACK 2> : {OmniCode Language & Standard Library}/lexicon/03_verbs/diagonal_vectors.omni**	OMNI	92	380	3.2
9	**<TRACK 2> : {OmniCode Language & Standard Library}/lexicon/03_verbs/diagonal_vectors.tape**	TAPE	30	177	1.5
10	**<TRACK 2> : {OmniCode Language & Standard Library}/lexicon/03_verbs/relational_being.omni**	OMNI	81	334	2.9
11	**<TRACK 2> : {OmniCode Language & Standard Library}/lexicon/03_verbs/relational_being.tape**	TAPE	30	178	1.4
12	**<TRACK 2> : {OmniCode Language & Standard Library}/lexicon/03_verbs/scale_vectors.omni**	OMNI	68	265	2.5
13	**<TRACK 2> : {OmniCode Language & Standard Library}/lexicon/03_verbs/scale_vectors.tape**	TAPE	30	187	1.5
14	**<TRACK 2> : {OmniCode Language & Standard Library}/lexicon/08_interjections/kernel_traps.omni**	OMNI	68	277	2.6
15	**<TRACK 2> : {OmniCode Language & Standard Library}/lexicon/08_interjections/kernel_traps.tape**	TAPE	30	183	1.4
16	**<TRACK 2> : {OmniCode Language & Standard Library}/lexicon/08_interjections/teleology_asides.omni**	OMNI	55	218	2.1
17	**<TRACK 2> : {OmniCode Language & Standard Library}/lexicon/08_interjections/teleology_asides.tape**	TAPE	30	180	1.5
18	**<TRACK 2> : {OmniCode Language & Standard Library}/lexicon/06_prepositions/ranges.omni**	OMNI	56	224	2.1
19	**<TRACK 2> : {OmniCode Language & Standard Library}/lexicon/06_prepositions/ranges.tape**	TAPE	30	184	1.4
20	**<TRACK 2> : {OmniCode Language & Standard Library}/lexicon/06_prepositions/spatial_sites.omni**	OMNI	56	220	2.1
21	**<TRACK 2> : {OmniCode Language & Standard Library}/lexicon/06_prepositions/spatial_sites.tape**	TAPE	33	208	1.6
22	**<TRACK 2> : {OmniCode Language & Standard Library}/lexicon/04_adjectives/invariant_axioms.omni**	OMNI	98	423	3.6
23	**<TRACK 2> : {OmniCode Language & Standard Library}/lexicon/04_adjectives/invariant_axioms.tape**	TAPE	33	210	1.6
24	**<TRACK 2> : {OmniCode Language & Standard Library}/lexicon/04_adjectives/proportions.omni**	OMNI	56	217	2.1
25	**<TRACK 2> : {OmniCode Language & Standard Library}/lexicon/04_adjectives/proportions.tape**	TAPE	30	172	1.4
26	**<TRACK 2> : {OmniCode Language & Standard Library}/lexicon/04_adjectives/quad_states.omni**	OMNI	96	416	3.5
27	**<TRACK 2> : {OmniCode Language & Standard Library}/lexicon/04_adjectives/quad_states.tape**	TAPE	30	175	1.4
28	**<TRACK 2> : {OmniCode Language & Standard Library}/lexicon/01_nouns/agents.omni**	OMNI	91	388	3.3
29	**<TRACK 2> : {OmniCode Language & Standard Library}/lexicon/01_nouns/agents.tape**	TAPE	33	193	1.5
30	**<TRACK 2> : {OmniCode Language & Standard Library}/lexicon/01_nouns/atomic_units.omni**	OMNI	77	353	2.8
31	**<TRACK 2> : {OmniCode Language & Standard Library}/lexicon/01_nouns/atomic_units.tape**	TAPE	30	178	1.4
32	**<TRACK 2> : {OmniCode Language & Standard Library}/lexicon/01_nouns/living_world.omni**	OMNI	81	372	3.0
33	**<TRACK 2> : {OmniCode Language & Standard Library}/lexicon/01_nouns/living_world.tape**	TAPE	34	226	1.7
34	**<TRACK 2> : {OmniCode Language & Standard Library}/lexicon/01_nouns/vessels.omni**	OMNI	85	378	3.2
35	**<TRACK 2> : {OmniCode Language & Standard Library}/lexicon/01_nouns/vessels.tape**	TAPE	33	209	1.5
36	**<TRACK 2> : {OmniCode Language & Standard Library}/lexicon/01_nouns/things/living/dog.omni**	OMNI	61	235	2.1

#	FILE PATH	TYPE	LINES	WORDS	SIZE (KB)
37	**<TRACK 2> : {OmniCode Language & Standard Library}/lexicon/01_nouns/things/living/dog.tape**	TAPE	33	206	1.5
38	**<TRACK 2> : {OmniCode Language & Standard Library}/lexicon/01_nouns/things/living/seed.omni**	OMNI	60	240	2.2
39	**<TRACK 2> : {OmniCode Language & Standard Library}/lexicon/01_nouns/things/living/seed.tape**	TAPE	33	207	1.5
40	**<TRACK 2> : {OmniCode Language & Standard Library}/lexicon/01_nouns/things/living/tree.omni**	OMNI	60	247	2.2
41	**<TRACK 2> : {OmniCode Language & Standard Library}/lexicon/01_nouns/things/living/tree.tape**	TAPE	33	214	1.6
42	**<TRACK 2> : {OmniCode Language & Standard Library}/lexicon/01_nouns/things/containers/vessel.omni**	OMNI	59	233	2.1
43	**<TRACK 2> : {OmniCode Language & Standard Library}/lexicon/01_nouns/things/containers/vessel.tape**	TAPE	33	203	1.6
44	**<TRACK 2> : {OmniCode Language & Standard Library}/lexicon/01_nouns/things/silicon/byte.omni**	OMNI	60	239	2.1
45	**<TRACK 2> : {OmniCode Language & Standard Library}/lexicon/01_nouns/things/silicon/byte.tape**	TAPE	33	204	1.5
46	**<TRACK 2> : {OmniCode Language & Standard Library}/lexicon/01_nouns/things/silicon/cell.omni**	OMNI	61	254	2.2
47	**<TRACK 2> : {OmniCode Language & Standard Library}/lexicon/01_nouns/things/silicon/cell.tape**	TAPE	33	209	1.5
48	**<TRACK 2> : {OmniCode Language & Standard Library}/lexicon/01_nouns/things/silicon/register.omni**	OMNI	62	278	2.4
49	**<TRACK 2> : {OmniCode Language & Standard Library}/lexicon/01_nouns/things/silicon/register.tape**	TAPE	33	215	1.6
50	**<TRACK 2> : {OmniCode Language & Standard Library}/lexicon/01_nouns/things/silicon/trit.omni**	OMNI	60	246	2.1
51	**<TRACK 2> : {OmniCode Language & Standard Library}/lexicon/01_nouns/things/silicon/trit.tape**	TAPE	33	209	1.5
52	**<TRACK 2> : {OmniCode Language & Standard Library}/lexicon/01_nouns/ideas/axioms/landmark.omni**	OMNI	60	242	2.2
53	**<TRACK 2> : {OmniCode Language & Standard Library}/lexicon/01_nouns/ideas/axioms/landmark.tape**	TAPE	33	202	1.6
54	**<TRACK 2> : {OmniCode Language & Standard Library}/lexicon/01_nouns/ideas/axioms/protocol.omni**	OMNI	59	230	2.1
55	**<TRACK 2> : {OmniCode Language & Standard Library}/lexicon/01_nouns/ideas/axioms/protocol.tape**	TAPE	33	207	1.6
56	**<TRACK 2> : {OmniCode Language & Standard Library}/lexicon/01_nouns/ideas/types/character.omni**	OMNI	181	1,716	11.3
57	**<TRACK 2> : {OmniCode Language & Standard Library}/lexicon/01_nouns/ideas/types/character.tape**	TAPE	84	705	5.0
58	**<TRACK 2> : {OmniCode Language & Standard Library}/lexicon/01_nouns/ideas/types/container.omni**	OMNI	192	1,812	12.3
59	**<TRACK 2> : {OmniCode Language & Standard Library}/lexicon/01_nouns/ideas/types/container.tape**	TAPE	89	772	5.4
60	**<TRACK 2> : {OmniCode Language & Standard Library}/lexicon/01_nouns/ideas/types/number.omni**	OMNI	185	1,753	11.7
61	**<TRACK 2> : {OmniCode Language & Standard Library}/lexicon/01_nouns/ideas/types/number.tape**	TAPE	83	676	4.8
62	**<TRACK 2> : {OmniCode Language & Standard Library}/lexicon/01_nouns/ideas/types/site.omni**	OMNI	190	1,818	12.2
63	**<TRACK 2> : {OmniCode Language & Standard Library}/lexicon/01_nouns/ideas/types/site.tape**	TAPE	89	763	5.3
64	**<TRACK 2> : {OmniCode Language & Standard Library}/lexicon/01_nouns/ideas/types/state.omni**	OMNI	230	2,189	14.4
65	**<TRACK 2> : {OmniCode Language & Standard Library}/lexicon/01_nouns/ideas/types/state.tape**	TAPE	117	949	6.7
66	**<TRACK 2> : {OmniCode Language & Standard Library}/lexicon/01_nouns/ideas/types/text.omni**	OMNI	62	249	2.2
67	**<TRACK 2> : {OmniCode Language & Standard Library}/lexicon/01_nouns/ideas/types/text.tape**	TAPE	33	210	1.6
68	**<TRACK 2> : {OmniCode Language & Standard Library}/lexicon/01_nouns/ideas/types/transpile variant/c_backbone/include/character.h**	H	279	1,671	12.4
69	**<TRACK 2> : {OmniCode Language & Standard Library}/lexicon/01_nouns/ideas/types/transpile variant/c_backbone/include/container.h**	H	222	1,078	9.0
70	**<TRACK 2> : {OmniCode Language & Standard Library}/lexicon/01_nouns/ideas/types/transpile variant/c_backbone/include/number.h**	H	189	863	7.4

#	FILE PATH	TYPE	LINES	WORDS	SIZE (KB)
71	**<TRACK 2> : {OmniCode Language & Standard Library}/lexicon/01_nouns/ideas/types/transpile variant/c_backbone/include/omni.h**	H	27	136	1.1
72	**<TRACK 2> : {OmniCode Language & Standard Library}/lexicon/01_nouns/ideas/types/transpile variant/c_backbone/include/omni_base.h**	H	289	1,270	10.6
73	**<TRACK 2> : {OmniCode Language & Standard Library}/lexicon/01_nouns/ideas/types/transpile variant/c_backbone/include/site.h**	H	191	925	7.4
74	**<TRACK 2> : {OmniCode Language & Standard Library}/lexicon/01_nouns/ideas/types/transpile variant/c_backbone/include/state.h**	H	170	863	7.1
75	**<TRACK 2> : {OmniCode Language & Standard Library}/lexicon/01_nouns/ideas/types/transpile variant/c_backbone/src/character.c**	C	262	1,182	10.0
76	**<TRACK 2> : {OmniCode Language & Standard Library}/lexicon/01_nouns/ideas/types/transpile variant/c_backbone/src/container.c**	C	236	997	8.5
77	**<TRACK 2> : {OmniCode Language & Standard Library}/lexicon/01_nouns/ideas/types/transpile variant/c_backbone/src/number.c**	C	185	753	6.4
78	**<TRACK 2> : {OmniCode Language & Standard Library}/lexicon/01_nouns/ideas/types/transpile variant/c_backbone/src/site.c**	C	168	735	6.2
79	**<TRACK 2> : {OmniCode Language & Standard Library}/lexicon/01_nouns/ideas/types/transpile variant/c_backbone/src/state.c**	C	174	766	6.3
80	**<TRACK 2> : {OmniCode Language & Standard Library}/lexicon/01_nouns/ideas/types/transpile variant/lua_tapes/character.lua**	LUA	149	579	5.9
81	**<TRACK 2> : {OmniCode Language & Standard Library}/lexicon/01_nouns/ideas/types/transpile variant/lua_tapes/container.lua**	LUA	158	632	6.5
82	**<TRACK 2> : {OmniCode Language & Standard Library}/lexicon/01_nouns/ideas/types/transpile variant/lua_tapes/number.lua**	LUA	154	577	6.1
83	**<TRACK 2> : {OmniCode Language & Standard Library}/lexicon/01_nouns/ideas/types/transpile variant/lua_tapes/site.lua**	LUA	148	565	6.0
84	**<TRACK 2> : {OmniCode Language & Standard Library}/lexicon/01_nouns/ideas/types/transpile variant/lua_tapes/state.lua**	LUA	183	697	7.1
85	**<TRACK 2> : {OmniCode Language & Standard Library}/lexicon/01_nouns/ideas/types/transpile variant/lua_tapes/tape_engine.lua**	LUA	223	957	8.6
86	**<TRACK 2> : {OmniCode Language & Standard Library}/lexicon/01_nouns/ideas/types/transpile variant/tests/lua/integrated/test_omni_lib.lua**	LUA	152	634	7.0
87	**<TRACK 2> : {OmniCode Language & Standard Library}/lexicon/01_nouns/ideas/types/transpile variant/tests/lua/unit/test_character.lua**	LUA	89	437	4.1
88	**<TRACK 2> : {OmniCode Language & Standard Library}/lexicon/01_nouns/ideas/types/transpile variant/tests/lua/unit/test_common.lua**	LUA	49	188	1.7
89	**<TRACK 2> : {OmniCode Language & Standard Library}/lexicon/01_nouns/ideas/types/transpile variant/tests/lua/unit/test_container.lua**	LUA	79	397	3.7
90	**<TRACK 2> : {OmniCode Language & Standard Library}/lexicon/01_nouns/ideas/types/transpile variant/tests/lua/unit/test_number.lua**	LUA	69	331	3.1
91	**<TRACK 2> : {OmniCode Language & Standard Library}/lexicon/01_nouns/ideas/types/transpile variant/tests/lua/unit/test_site.lua**	LUA	82	379	3.8
92	**<TRACK 2> : {OmniCode Language & Standard Library}/lexicon/01_nouns/ideas/types/transpile variant/tests/lua/unit/test_state.lua**	LUA	93	439	4.2
93	**<TRACK 2> : {OmniCode Language & Standard Library}/lexicon/01_nouns/ideas/types/transpile variant/tests/c/integrated/test_omni_lib.c**	C	121	492	5.5

#	FILE PATH	TYPE	LINES	WORDS	SIZE (KB)
94	**<TRACK 2> : {OmniCode Language & Standard Library}/lexicon/01_nouns/ideas/types/transpile variant/tests/c/includes/test_common.h**	H	65	248	2.3
95	**<TRACK 2> : {OmniCode Language & Standard Library}/lexicon/01_nouns/ideas/types/transpile variant/tests/c/unit/test_character.c**	C	105	501	4.9
96	**<TRACK 2> : {OmniCode Language & Standard Library}/lexicon/01_nouns/ideas/types/transpile variant/tests/c/unit/test_container.c**	C	145	670	6.5
97	**<TRACK 2> : {OmniCode Language & Standard Library}/lexicon/01_nouns/ideas/types/transpile variant/tests/c/unit/test_number.c**	C	111	499	4.9
98	**<TRACK 2> : {OmniCode Language & Standard Library}/lexicon/01_nouns/ideas/types/transpile variant/tests/c/unit/test_site.c**	C	120	556	5.5
99	**<TRACK 2> : {OmniCode Language & Standard Library}/lexicon/01_nouns/ideas/types/transpile variant/tests/c/unit/test_state.c**	C	119	606	5.6
100	**<TRACK 2> : {OmniCode Language & Standard Library}/lexicon/01_nouns/ideas/invariants/covenant.omni**	OMNI	60	244	2.2
101	**<TRACK 2> : {OmniCode Language & Standard Library}/lexicon/01_nouns/ideas/invariants/covenant.tape**	TAPE	33	204	1.6
102	**<TRACK 2> : {OmniCode Language & Standard Library}/lexicon/01_nouns/ideas/invariants/equilibrium.omni**	OMNI	60	236	2.2
103	**<TRACK 2> : {OmniCode Language & Standard Library}/lexicon/01_nouns/ideas/invariants/equilibrium.tape**	TAPE	33	211	1.6
104	**<TRACK 2> : {OmniCode Language & Standard Library}/lexicon/01_nouns/persons/systems/system.omni**	OMNI	60	235	2.1
105	**<TRACK 2> : {OmniCode Language & Standard Library}/lexicon/01_nouns/persons/systems/system.tape**	TAPE	33	204	1.6
106	**<TRACK 2> : {OmniCode Language & Standard Library}/lexicon/01_nouns/persons/supervisors/kernel.omni**	OMNI	60	230	2.1
107	**<TRACK 2> : {OmniCode Language & Standard Library}/lexicon/01_nouns/persons/supervisors/kernel.tape**	TAPE	33	195	1.6
108	**<TRACK 2> : {OmniCode Language & Standard Library}/lexicon/01_nouns/persons/transducers/model.omni**	OMNI	60	233	2.1
109	**<TRACK 2> : {OmniCode Language & Standard Library}/lexicon/01_nouns/persons/transducers/model.tape**	TAPE	33	197	1.6
110	**<TRACK 2> : {OmniCode Language & Standard Library}/lexicon/01_nouns/persons/stewards/operator.omni**	OMNI	60	234	2.1
111	**<TRACK 2> : {OmniCode Language & Standard Library}/lexicon/01_nouns/persons/stewards/operator.tape**	TAPE	33	198	1.6
112	**<TRACK 2> : {OmniCode Language & Standard Library}/lexicon/01_nouns/places/tracks/track.omni**	OMNI	60	244	2.1
113	**<TRACK 2> : {OmniCode Language & Standard Library}/lexicon/01_nouns/places/tracks/track.tape**	TAPE	33	210	1.6
114	**<TRACK 2> : {OmniCode Language & Standard Library}/lexicon/01_nouns/places/memory/partition.omni**	OMNI	60	237	2.2
115	**<TRACK 2> : {OmniCode Language & Standard Library}/lexicon/01_nouns/places/memory/partition.tape**	TAPE	33	216	1.6
116	**<TRACK 2> : {OmniCode Language & Standard Library}/lexicon/01_nouns/places/memory/storehouse.omni**	OMNI	60	240	2.2
117	**<TRACK 2> : {OmniCode Language & Standard Library}/lexicon/01_nouns/places/memory/storehouse.tape**	TAPE	33	209	1.7

#	FILE PATH	TYPE	LINES	WORDS	SIZE (KB)
118	**<TRACK 2> : {OmniCode Language & Standard Library}/lexicon/01_nouns/places/vaults/vault.omni**	OMNI	60	237	2.1
119	**<TRACK 2> : {OmniCode Language & Standard Library}/lexicon/01_nouns/places/vaults/vault.tape**	TAPE	33	212	1.6
120	**<TRACK 2> : {OmniCode Language & Standard Library}/lexicon/01_nouns/places/buffers/reservoir.omni**	OMNI	59	235	2.1
121	**<TRACK 2> : {OmniCode Language & Standard Library}/lexicon/01_nouns/places/buffers/reservoir.tape**	TAPE	33	207	1.6
122	**<TRACK 2> : {OmniCode Language & Standard Library}/lexicon/suggestions/README.omni**	OMNI	97	666	5.0
123	**<TRACK 2> : {OmniCode Language & Standard Library}/lexicon/suggestions/03_verbs/universal_verbs.omni**	OMNI	118	617	4.9
124	**<TRACK 2> : {OmniCode Language & Standard Library}/lexicon/suggestions/03_verbs/universal_verbs.tape**	TAPE	49	296	2.5
125	**<TRACK 2> : {OmniCode Language & Standard Library}/lexicon/suggestions/operators/morphological_operators.omni**	OMNI	121	686	5.3
126	**<TRACK 2> : {OmniCode Language & Standard Library}/lexicon/suggestions/operators/morphological_operators.tape**	TAPE	49	318	2.5
127	**<TRACK 2> : {OmniCode Language & Standard Library}/lexicon/operators/boot_kernel.omni**	OMNI	91	441	3.9
128	**<TRACK 2> : {OmniCode Language & Standard Library}/lexicon/operators/boot_kernel.tape**	TAPE	37	264	2.0
129	**<TRACK 2> : {OmniCode Language & Standard Library}/lexicon/operators/casting.omni**	OMNI	88	393	3.1
130	**<TRACK 2> : {OmniCode Language & Standard Library}/lexicon/operators/casting.tape**	TAPE	30	170	1.4
131	**<TRACK 2> : {OmniCode Language & Standard Library}/lexicon/operators/polarity.omni**	OMNI	74	338	3.0
132	**<TRACK 2> : {OmniCode Language & Standard Library}/lexicon/operators/polarity.tape**	TAPE	30	179	1.5
133	**<TRACK 2> : {OmniCode Language & Standard Library}/lexicon/02_pronouns/self_handles.omni**	OMNI	81	338	2.9
134	**<TRACK 2> : {OmniCode Language & Standard Library}/lexicon/02_pronouns/self_handles.tape**	TAPE	30	187	1.4
135	**<TRACK 2> : {OmniCode Language & Standard Library}/lexicon/02_pronouns/wildcards.omni**	OMNI	81	342	3.0
136	**<TRACK 2> : {OmniCode Language & Standard Library}/lexicon/02_pronouns/wildcards.tape**	TAPE	30	193	1.5
137	**<TRACK 2> : {OmniCode Language & Standard Library}/lexicon/07_conjunctions/caveat_branches.omni**	OMNI	56	222	2.2
138	**<TRACK 2> : {OmniCode Language & Standard Library}/lexicon/07_conjunctions/caveat_branches.tape**	TAPE	30	188	1.5
139	**<TRACK 2> : {OmniCode Language & Standard Library}/lexicon/07_conjunctions/inseparable_pairs.omni**	OMNI	56	218	2.1
140	**<TRACK 2> : {OmniCode Language & Standard Library}/lexicon/07_conjunctions/inseparable_pairs.tape**	TAPE	30	178	1.4
141	**<TRACK 2> : {OmniCode Language & Standard Library}/lexicon/07_conjunctions/pipeline_junctions.omni**	OMNI	56	209	2.1
142	**<TRACK 2> : {OmniCode Language & Standard Library}/lexicon/07_conjunctions/pipeline_junctions.tape**	TAPE	30	182	1.4
143	**<TRACK 2> : {OmniCode Language & Standard Library}/lexicon/05_adverbs/cadence_policies.omni**	OMNI	81	333	3.0
144	**<TRACK 2> : {OmniCode Language & Standard Library}/lexicon/05_adverbs/cadence_policies.tape**	TAPE	30	183	1.4
145	**<TRACK 2> : {OmniCode Language & Standard Library}/lexicon/05_adverbs/negation.omni**	OMNI	57	221	2.1
146	**<TRACK 2> : {OmniCode Language & Standard Library}/lexicon/05_adverbs/negation.tape**	TAPE	33	193	1.5
147	**<TRACK 2> : {OmniCode Language & Standard Library}/lib/kernel/boot.omni**	OMNI	114	679	4.9
148	**<TRACK 2> : {OmniCode Language & Standard Library}/lib/kernel/boot.tape**	TAPE	50	487	3.4
149	**<TRACK 2> : {OmniCode Language & Standard Library}/lib/core/arithmetic.omni**	OMNI	90	416	2.8

#	FILE PATH	TYPE	LINES	WORDS	SIZE (KB)
150	**<TRACK 2> : {OmniCode Language & Standard Library}/lib/core/cell.tape**	TAPE	17	133	0.9
151	**<TRACK 2> : {OmniCode Language & Standard Library}/lib/core/logic.omni**	OMNI	89	404	2.9
152	**<TRACK 2> : {OmniCode Language & Standard Library}/lib/core/memory.omni**	OMNI	135	703	5.2
153	**<TRACK 2> : {OmniCode Language & Standard Library}/lib/core/memory.tape**	TAPE	28	174	1.2
154	**<TRACK 2> : {OmniCode Language & Standard Library}/lib/core/vectors.omni**	OMNI	151	676	5.1

<TRACK 2> : {OmniCode Language & Standard Library}/TRACK.md MD 10 LINES 68 WORDS 0.5 KB

```
# <TRACK 2> : {OmniCode Language & Standard Library}

## Scope & Invariant

The native OmniCode language universe, vocabulary, operators, standard modules, and Genesis boot covenant.

- `lib/core/`: Living standard library (arithmetic, logic, vectors, memory vessels, cell tapes).
- `lib/kernel/`: Genesis OS boot covenant ribbons (`boot.omni`, `boot.tape`).
- `lib/lexicon/`: Canonical vocabulary (8 parts of speech: nouns, pronouns, verbs, adjectives, adverbs, prepositions, conjunctions, interjections) and operators (`boot_kernel`, `casting`, `polarity`).
```

<TRACK 2> : {OmniCode Language & Standard Library}/lexicon/03_verbs/arithmetic_verbs.omni

OMNI | 94 LINES | 399 WORDS | 3.3 KB

```
# ================================================================================
# BLOCK 1: METADATA BLOCK
# ( Ontological Verbs: Mechanical SVO Arithmetic Operators )
# ================================================================================
# ARCHITECTURAL TOPOLOGY:
#   Defines the mechanical arithmetic accumulator cogs:
#   - plus  : Additive mechanical translation cog (advance accumulator).
#   - minus : Subtractive mechanical translation cog (recede accumulator).
#   - times : Multiplicative mechanical gear ratio (ratio expansion).
#   - div   : Divisive mechanical tooth pitch / partition ratio.
# ================================================================================

Module < Lexicon.Verbs.ArithmeticVerbs > ( Mechanical arithmetic cogs and ratio accumulators ) {
    version: < "1.0.0" >,
    system: < OmniCode.Lexicon >,
    semantics: < SVOArithmetic :: MechanicalAccumulators :: Radix3Cogs >
}

# 1. Additive Cog: < PLUS >
Plus < InvariantVerb > ( Mechanical Additive Translation: plus ) {
    grammar_primary: < "Verb" >,
    grammar_secondary: < "Conjunction" >,
    symbol: < "plus" >,
    abstract_form: < "+Plus" >,
    concrete_form: < "Plus-" >,
    machine_effect: < "mechanical_adder_cog" >
}

# 2. Subtractive Cog: < MINUS >
Minus < InvariantVerb > ( Mechanical Subtractive Translation: minus ) {
    grammar_primary: < "Verb" >,
    grammar_secondary: < "Conjunction" >,
    symbol: < "minus" >,
    abstract_form: < "+Minus" >,
    concrete_form: < "Minus-" >,
    machine_effect: < "mechanical_subtractor_cog" >
}

# 3. Multiplicative Cog: < TIMES >
Times < InvariantVerb > ( Mechanical Gear Ratio Multiplier: times ) {
    grammar_primary: < "Verb" >,
    grammar_secondary: < "Conjunction" >,
    symbol: < "times" >,
    abstract_form: < "+Times" >,
    concrete_form: < "Times-" >,
    machine_effect: < "mechanical_gear_multiplier" >
}

# 4. Divisive Cog: < DIV >
Div < InvariantVerb > ( Mechanical Pitch Partition Divider: div ) {
    grammar_primary: < "Verb" >,
    grammar_secondary: < "Conjunction" >,
    symbol: < "div" >,
    abstract_form: < "+Div" >,
    concrete_form: < "Div-" >,
    machine_effect: < "mechanical_partition_divider" >
}

# ================================================================================
# BLOCK 2: OPENING BLOCK
# ================================================================================
Opening {
    requires: [
        Plus : InvariantVerb ::
        Minus : InvariantVerb ::
        Times : InvariantVerb ::
        Div : InvariantVerb
    ],
    preconditions: [ +1 ]
}

# ================================================================================
# BLOCK 3: BODY BLOCK
```

```
# ================================================================================
Body [
    # Verify SVO arithmetic execution
    arithmetic_verification [
        Add_Roll [ 10 plus 5 = 15 ] ::
        Sub_Roll [ 20 minus 8 = 12 ] ::
        Mul_Roll [ 4 times 3 = 12 ] ::
        Div_Roll [ 16 div 4 = 4 ]
    ] ( Mechanical arithmetic operations verified )
]

# ================================================================================
# BLOCK 4: CLOSING BLOCK
# ================================================================================
Closing {
    verify: [
        arithmetic_verification = yes
    ],
    commit_status [ PROCEED- ] ( Arithmetic verbs verified and sealed )
}
```

<TRACK 2> : {OmniCode Language & Standard Library}/lexicon/03_verbs/arithmetic_verbs.tape

TAPE | 33 LINES | 207 WORDS | 1.6 KB

```
# ================================================================================
# TAPE HEAD: ARITHMETIC VERBS EXECUTION PROCESS RIBBON
# ================================================================================
# ARCHITECTURAL TOPOLOGY:
#   Executes mechanical SVO arithmetic operations (plus, minus, times, div).
# ================================================================================

[Tape.ArithmeticVerbs] : WHO {OmniCode_Kernel} WHAT {ArithmeticVerbs} --> ACCUMULATE {MathMesh}
    WHEN {GenesisBoot}
    WHERE {arithmetic_chassis} = [ 64 ]
    WHY ( Test mechanical arithmetic adder and multiplier cogs )
    HOW [ +1 : PROCEED ]
    :: < math_landmark > ( Genesis 1:28 - 'Be fruitful, and multiply' )

# ================================================================================
# SUBORDINATE ROLLS: MECHANICAL SVO ARITHMETIC TRANSFORMATIONS
# ================================================================================

# Roll 1: Reference abstract Plus, Minus, Times, and Div specifications
roll_1 [ +Plus & +Minus & +Times & +Div ] ( Abstract arithmetic verb blueprints referenced )

# Roll 2: Additive mechanical translation
roll_2 [ 20 plus 22 --> Accumulator_0 @ 0 ] ( Compute sum and store into accumulator )

# Roll 3: Multiplicative mechanical gear ratio
roll_3 [ 6 times 7 --> Product_0 @ 1 ] ( Compute product and store into product cell )

# Roll 4: Verify accumulator invariant bounds
roll_4 [ Accumulator_0 = 42 ! [ +1 ] ] ( Verify mechanical arithmetic result )

# Roll 5: Commit with affirmative concrete macro
roll_5 [ commit_status [ PROCEED- ] ] ( Arithmetic verbs verified and sealed )
```

<TRACK 2> : {OmniCode Language & Standard Library}/lexicon/03_verbs/axial_vectors.omni

OMNI | 68 LINES | 271 WORDS | 2.5 KB

```
# ================================================================================
# BLOCK 1: METADATA BLOCK
# ( Ontological Verbs: Axial Driving Vectors )
# ================================================================================
# ARCHITECTURAL TOPOLOGY:
#   Defines the primary horizontal causality vectors:
#   - --> : ActUpon (Forward causality, drive gear, state propagation, store).
#   - <-- : ActAgain (Reverse causality, retroaction, sensor read, reflection).
# ================================================================================

Module < Lexicon.Verbs.AxialVectors > ( Axial drive vectors and horizontal causality ) {
    version: < "1.0.0" >,
    system: < OmniCode.Lexicon >,
    semantics: < AxialVectors :: ForwardDrive :: RetroactiveFeedback >
}

# 1. Forward Vector: < ACT_UPON >
ActUpon < InvariantVerb > ( Forward Causality & Driving Gear: --> ) {
    grammar_primary: < "Verb" >,
    grammar_secondary: < "Conjunction" >,
    symbol: < "-->" >,
    abstract_form: < "+ActUpon" >,
    concrete_form: < "ActUpon-" >,
    machine_effect: < "forward_drive_write_dispatch" >
}

# 2. Reverse Vector: < ACT_AGAIN >
ActAgain < InvariantVerb > ( Reverse Causality & Feedback Gear: <-- ) {
    grammar_primary: < "Verb" >,
    grammar_secondary: < "Conjunction" >,
    symbol: < "<--" >,
    abstract_form: < "+ActAgain" >,
    concrete_form: < "ActAgain-" >,
    machine_effect: < "retroactive_read_sensor" >
}

# ================================================================================
# BLOCK 2: OPENING BLOCK
# ================================================================================
Opening {
    requires: [
        ActUpon : InvariantVerb ::
        ActAgain : InvariantVerb
    ],
    preconditions: [ +1 ]
}

# ================================================================================
# BLOCK 3: BODY BLOCK
# ================================================================================
Body [
    # Verify forward and reverse causality pair
    vector_verification [
        Forward_Transmission [ 10 --> Register_A ] ::
        Feedback_Sensor [ Register_A <-- Sensor_Tap ]
    ] ( Forward drive and sensor feedback verified )
]

# ================================================================================
# BLOCK 4: CLOSING BLOCK
# ================================================================================
Closing {
    verify: [
        vector_verification = yes
    ],
    commit_status [ PROCEED- ] ( Axial driving vectors verified and sealed )
}
```

<TRACK 2> : {OmniCode Language & Standard Library}/lexicon/03_verbs/axial_vectors.tape

TAPE | 30 LINES | 178 WORDS | 1.4 KB

```
# ================================================================================
# TAPE HEAD: AXIAL VECTORS EXECUTION PROCESS RIBBON
# ================================================================================
# ARCHITECTURAL TOPOLOGY:
#   Executes forward driving write (-->) and reverse feedback read (<--).
# ================================================================================

[Tape.AxialVectors] : WHO {OmniCode_Kernel} WHAT {AxialVectors} --> DRIVE {MotionMesh}
    WHEN {GenesisBoot}
    WHERE {axial_chassis} = [ 64 ]
    WHY ( Test forward and reverse axial drive gears )
    HOW [ +1 : PROCEED ]
    :: < vector_landmark > ( Proverbs 4:25 - 'Let thine eyes look right on' )

# ================================================================================
# SUBORDINATE ROLLS: FORWARD AND REVERSE AXIAL TRANSLATION
# ================================================================================

# Roll 1: Reference abstract ActUpon and ActAgain vector specifications
roll_1 [ +ActUpon & +ActAgain ] ( Abstract axial vector blueprints referenced )

# Roll 2: Forward drive vector actuation
roll_2 [ 100 --> Buffer_0 @ 0 ] ( Forward drive write into Buffer_0 )

# Roll 3: Reverse drive vector verification
roll_3 [ Result <-- Buffer_0 ! [ +1 ] ] ( Reverse read from Buffer_0 )

# Roll 4: Commit with affirmative concrete macro
roll_4 [ commit_status [ PROCEED- ] ] ( Axial vectors verified and sealed )
```

<TRACK 2> : {OmniCode Language & Standard Library}/lexicon/03_verbs/cinematic_track_loop.omni

OMNI | 56 LINES | 245 WORDS | 2.2 KB

```
# ================================================================================
# BLOCK 1: METADATA BLOCK
# ( Ontological Verbs: Cinematic Track Loops & Transport Kinematics )
# ================================================================================
# ARCHITECTURAL TOPOLOGY:
#   Defines Turing-complete track looping via reel transport kinematics:
#   - PLAY   : Advance track head forward (+1 / YES / PROCEED).
#   - REWIND : Spool track head backward along the reel to cue landmark (-1 / NO / ROLLBACK).
#   - STOP   : Mechanical brake arresting the motor (NULL / VOID / HALT).
# ================================================================================

Module < Lexicon.Verbs.CinematicTrackLoop > ( Cinematic track looping and transport kinematics ) {
    version: < "1.0.0" >,
    system: < OmniCode.Lexicon >,
    semantics: < TrackLooping :: TransportKinematics :: TuringCompleteness >
}

# 1. Transport Verb: < CINEMATIC_LOOP >
CinematicLoop < InvariantVerb > ( Turing-Complete Track Transport Loop: PLAY & REWIND ) {
    grammar_primary: < "Verb" >,
    grammar_secondary: < "Conjunction" >,
    abstract_form: < "+CinematicLoop" >,
    concrete_form: < "CinematicLoop-" >,
    machine_effect: < "turing_complete_track_transport" >
}

# ================================================================================
# BLOCK 2: OPENING BLOCK
# ================================================================================
Opening {
    requires: [
        CinematicLoop : InvariantVerb
    ],
    preconditions: [ +1 ]
}

# ================================================================================
# BLOCK 3: BODY BLOCK
# ================================================================================
Body [
    # Verify track transport iteration
    loop_verification [
        Track_Iteration [ Counter > 0 ! [ PLAY :: REWIND < loop_cue > ] ]
    ] ( Track transport kinematics verified )
]

# ================================================================================
# BLOCK 4: CLOSING BLOCK
# ================================================================================
Closing {
    verify: [
        loop_verification = yes
    ],
    commit_status [ PROCEED- ] ( Cinematic track loops verified and sealed )
}
```

<TRACK 2> : {OmniCode Language & Standard Library}/lexicon/03_verbs/cinematic_track_loop.tape

TAPE | 36 LINES | 250 WORDS | 1.8 KB

```
# ==============================================================================
# TAPE HEAD: CINEMATIC TRACK LOOP PROCESS RIBBON
# ==============================================================================
# ARCHITECTURAL TOPOLOGY:
#   Executes a Turing-complete transport loop via PLAY and REWIND to cue landmark.
# ==============================================================================

[Tape.CinematicTrackLoop] : WHO {OmniCode_Kernel} WHAT {CinematicLoop} --> MODULATE {ReelMesh}
    WHEN {GenesisBoot}
    WHERE {loop_partition} = [ 64 ]
    WHY ( Prove Turing completeness via physical reel transport kinematics )
    HOW [ +1 : PROCEED ]
    :: < loop_landmark > ( Ecclesiastes 1:6 - 'The wind returneth again according to his circuits' )

# ==============================================================================
# SUBORDINATE ROLLS: REEL TRANSPORT KINEMATICS & CUE REWIND
# ==============================================================================

# Roll 1: Reference abstract CinematicLoop transport specification
roll_1 [ +CinematicLoop ] ( Abstract track loop blueprint referenced )

# Roll 2: Inscribe initial loop counter into silicon register
roll_2 [ 3 --> Loop_Counter_0 @ 0 ] ( Initialize loop counter to 3 )

# Roll 3: Landmark cue point for loop track rewind
roll_3 [ < loop_cue > :: 1 plus Step_Accumulator_0 --> Step_Accumulator_0 @ 1 ] ( Cue landmark: advance step accumulator
)

# Roll 4: Decrement loop counter
roll_4 [ Loop_Counter_0 minus 1 --> Loop_Counter_0 @ 0 ] ( Decrement loop counter register )

# Roll 5: Conditional track transport rewind
roll_5 [ Loop_Counter_0 != 0 ! [ PLAY :: REWIND < loop_cue > ] ] ( Rewind track head back to landmark cue while counter
positive )

# Roll 6: Commit and stop transport motor
roll_6 [ STOP :: commit_status [ PROCEED- ] ] ( Complete loop and engage emergency brake )
```

<TRACK 2> : {OmniCode Language & Standard Library}/lexicon/03_verbs/diagonal_vectors.omni

OMNI | 92 LINES | 380 WORDS | 3.2 KB

```
# ================================================================================
# BLOCK 1: METADATA BLOCK
# ( Ontological Verbs: Diagonal Modulated Scale Vectors )
# ================================================================================
# ARCHITECTURAL TOPOLOGY:
#   Defines compound diagonal scale and causality vectors across 3D coordinates:
#   - /-^ : Major (Ascending forward diagonal, macro amplification).
#   - \-v : Minor (Descending forward diagonal, micro attenuation).
#   - /-v : Small (Descending reverse diagonal, localized reduction).
#   - \-^ : Great (Ascending reverse diagonal, expansive retrospective).
# ================================================================================

Module < Lexicon.Verbs.DiagonalVectors > ( Diagonal modulated scale and causality vectors ) {
    version: < "1.0.0" >,
    system: < OmniCode.Lexicon >,
    semantics: < DiagonalVectors :: ModulatedScale :: SpatialProjection >
}

# 1. Diagonal Vector: < MAJOR >
Major < InvariantVerb > ( Ascending Forward Diagonal: /-^ ) {
    grammar_primary: < "Verb" >,
    grammar_secondary: < "Adjective" >,
    symbol: < "/-^" >,
    abstract_form: < "+Major" >,
    concrete_form: < "Major-" >,
    machine_effect: < "macro_amplification_forward" >
}

# 2. Diagonal Vector: < MINOR >
Minor < InvariantVerb > ( Descending Forward Diagonal: \-v ) {
    grammar_primary: < "Verb" >,
    grammar_secondary: < "Adjective" >,
    symbol: < "\\-v" >,
    abstract_form: < "+Minor" >,
    concrete_form: < "Minor-" >,
    machine_effect: < "micro_attenuation_forward" >
}

# 3. Diagonal Vector: < SMALL >
Small < InvariantVerb > ( Descending Reverse Diagonal: /-v ) {
    grammar_primary: < "Verb" >,
    grammar_secondary: < "Adjective" >,
    symbol: < "/-v" >,
    abstract_form: < "+Small" >,
    concrete_form: < "Small-" >,
    machine_effect: < "localized_reduction_reverse" >
}

# 4. Diagonal Vector: < GREAT >
Great < InvariantVerb > ( Ascending Reverse Diagonal: \-^ ) {
    grammar_primary: < "Verb" >,
    grammar_secondary: < "Adjective" >,
    symbol: < "\\-^" >,
    abstract_form: < "+Great" >,
    concrete_form: < "Great-" >,
    machine_effect: < "expansive_retrospective_reverse" >
}

# ================================================================================
# BLOCK 2: OPENING BLOCK
# ================================================================================
Opening {
    requires: [
        Major : InvariantVerb ::
        Minor : InvariantVerb ::
        Small : InvariantVerb ::
        Great : InvariantVerb
    ],
    preconditions: [ +1 ]
}

# ================================================================================
# BLOCK 3: BODY BLOCK
```

```
# ============================================================================
Body [
    # Verify diagonal trajectory balance
    diagonal_verification [
        Amplification [ Signal /-^ Macro_Bus ] ::
        Attenuation   [ Signal \-v Micro_Bus ]
    ] ( Compound 3D diagonal motions verified )
]

# ============================================================================
# BLOCK 4: CLOSING BLOCK
# ============================================================================
Closing {
    verify: [
        diagonal_verification = yes
    ],
    commit_status [ PROCEED- ] ( Diagonal scale vectors verified and sealed )
}
```

<TRACK 2> : {OmniCode Language & Standard Library}/lexicon/03_verbs/diagonal_vectors.tape

TAPE | 30 LINES | 177 WORDS | 1.5 KB

```
# ================================================================================
# TAPE HEAD: DIAGONAL VECTORS EXECUTION PROCESS RIBBON
# ================================================================================
# ARCHITECTURAL TOPOLOGY:
#   Executes modulated diagonal scale transformations across coordinate quadrants.
# ================================================================================

[Tape.DiagonalVectors] : WHO {OmniCode_Kernel} WHAT {DiagonalVectors} --> MODULATE {QuadrantMesh}
    WHEN {GenesisBoot}
    WHERE {diagonal_chassis} = [ 64 ]
    WHY ( Test 3D diagonal scale and direction vectors )
    HOW [ +1 : PROCEED ]
    :: < diagonal_landmark > ( Ezekiel 1:16 - 'A wheel in the middle of a wheel' )

# ================================================================================
# SUBORDINATE ROLLS: DIAGONAL MODULATED TRANSFORMATIONS
# ================================================================================

# Roll 1: Reference abstract Major and Minor diagonal vectors
roll_1 [ +Major & +Minor ] ( Abstract diagonal vector blueprints referenced )

# Roll 2: Modulate scale forward-ascending via /-^
roll_2 [ Signal_0 /-^ Macro_Register @ 0 ] ( Ascending forward diagonal drive )

# Roll 3: Modulate scale forward-descending via \-v
roll_3 [ Macro_Register \-v Micro_Register ! [ +1 ] ] ( Descending forward diagonal drive )

# Roll 4: Commit with affirmative concrete macro
roll_4 [ commit_status [ PROCEED- ] ] ( Diagonal vectors verified and sealed )
```

<TRACK 2> : {OmniCode Language & Standard Library}/lexicon/03_verbs/relational_being.omni

OMNI | 81 LINES | 334 WORDS | 2.9 KB

```
# ================================================================================
# BLOCK 1: METADATA BLOCK
# ( Ontological Verbs: Relations of Being along the Y-Axis )
# ================================================================================
# ARCHITECTURAL TOPOLOGY:
#   Defines the Y-axis relations of ontology and identity:
#   - =  : Is (Exact identity, equivalence, covenant equality).
#   - != : IsNot (Distinction, exclusion, boundary separation).
#   - ~= : ToBeLike (Homomorphism, structural analogy, likeness).
# ================================================================================

Module < Lexicon.Verbs.RelationalBeing > ( Relations of Being and ontological identity ) {
    version: < "1.0.0" >,
    system: < OmniCode.Lexicon >,
    semantics: < RelationalBeing :: IdentityEquivalence :: StructuralAnalogy >
}

# 1. Identity Relation: < IS >
Is < InvariantVerb > ( Exact Ontological Identity & Equivalence: = ) {
    grammar_primary: < "Verb" >,
    grammar_secondary: < "Preposition" >,
    symbol: < "=" >,
    abstract_form: < "+Is" >,
    concrete_form: < "Is-" >,
    machine_effect: < "identity_assert_equate" >
}

# 2. Distinction Relation: < IS_NOT >
IsNot < InvariantVerb > ( Ontological Distinction & Boundary Separation: != ) {
    grammar_primary: < "Verb" >,
    grammar_secondary: < "Preposition" >,
    symbol: < "!=" >,
    abstract_form: < "+IsNot" >,
    concrete_form: < "IsNot-" >,
    machine_effect: < "distinction_assert_differentiate" >
}

# 3. Analogy Relation: < TO_BE_LIKE >
ToBeLike < InvariantVerb > ( Homomorphism & Structural Likeness: ~= ) {
    grammar_primary: < "Verb" >,
    grammar_secondary: < "Adjective" >,
    symbol: < "~=" >,
    abstract_form: < "+ToBeLike" >,
    concrete_form: < "ToBeLike-" >,
    machine_effect: < "homomorphic_analogy_match" >
}

# ================================================================================
# BLOCK 2: OPENING BLOCK
# ================================================================================
Opening {
    requires: [
        Is : InvariantVerb ::
        IsNot : InvariantVerb ::
        ToBeLike : InvariantVerb
    ],
    preconditions: [ +1 ]
}

# ================================================================================
# BLOCK 3: BODY BLOCK
# ================================================================================
Body [
    # Verify identity, distinction, and analogy
    relations_verification [
        Equivalence_Roll [ Entity_A = Entity_A ] ::
        Distinction_Roll [ Entity_A != Entity_B ] ::
        Analogy_Roll     [ Entity_A ~= Entity_Prototype ]
    ] ( Y-axis relations of being verified )
]

# ================================================================================
# BLOCK 4: CLOSING BLOCK
```

```
# ================================================================================
Closing {
    verify: [
        relations_verification = yes
    ],
    commit_status [ PROCEED- ] ( Relations of being verified and sealed )
}
```

<TRACK 2> : {OmniCode Language & Standard Library}/lexicon/03_verbs/relational_being.tape

TAPE | 30 LINES | 178 WORDS | 1.4 KB

```
# ================================================================================
# TAPE HEAD: RELATIONAL BEING EXECUTION PROCESS RIBBON
# ================================================================================
# ARCHITECTURAL TOPOLOGY:
#   Executes identity (=), distinction (!=), and structural analogy (~=).
# ================================================================================

[Tape.RelationalBeing] : WHO {OmniCode_Kernel} WHAT {RelationalBeing} --> VERIFY {IdentityMesh}
    WHEN {GenesisBoot}
    WHERE {relational_chassis} = [ 64 ]
    WHY ( Test Y-axis relations of being, distinction, and analogy )
    HOW [ +1 : PROCEED ]
    :: < relation_landmark > ( Exodus 3:14 - 'I AM THAT I AM' )

# ================================================================================
# SUBORDINATE ROLLS: RELATIONAL EQUIVALENCE & DISTINCTION
# ================================================================================

# Roll 1: Reference abstract Is, IsNot, and ToBeLike specifications
roll_1 [ +Is & +IsNot & +ToBeLike ] ( Abstract relational being blueprints referenced )

# Roll 2: Verify identity equivalence via =
roll_2 [ State_Alpha = State_Alpha @ 0 ] ( Verify reflexive identity equation )

# Roll 3: Verify boundary distinction via !=
roll_3 [ State_Alpha != State_Beta ! [ +1 ] ] ( Verify ontological distinction )

# Roll 4: Commit with affirmative concrete macro
roll_4 [ commit_status [ PROCEED- ] ] ( Relational being verified and sealed )
```

<TRACK 2> : {OmniCode Language & Standard Library}/lexicon/03_verbs/scale_vectors.omni

OMNI | 68 LINES | 265 WORDS | 2.5 KB

```
# ================================================================================
# BLOCK 1: METADATA BLOCK
# ( Ontological Verbs: Scale Vectors & Hierarchical Motion )
# ================================================================================
# ARCHITECTURAL TOPOLOGY:
#   Defines vertical hierarchy and scale motion vectors:
#   - ^-- : BuildsTo (Ascension, composition, integration, synthesis).
#   - v-- : BreaksDownFrom (Descension, partitioning, decomposition, allocation).
# ================================================================================

Module < Lexicon.Verbs.ScaleVectors > ( Scale vectors and hierarchical motion ) {
    version: < "1.0.0" >,
    system: < OmniCode.Lexicon >,
    semantics: < ScaleVectors :: HierarchicalSynthesis :: PartitionDecomposition >
}

# 1. Scale Ascension: < BUILDS_TO >
BuildsTo < InvariantVerb > ( Hierarchical Ascension & Synthesis: ^-- ) {
    grammar_primary: < "Verb" >,
    grammar_secondary: < "Conjunction" >,
    symbol: < "^--" >,
    abstract_form: < "+BuildsTo" >,
    concrete_form: < "BuildsTo-" >,
    machine_effect: < "ascend_compose_aggregate" >
}

# 2. Scale Descension: < BREAKS_DOWN_FROM >
BreaksDownFrom < InvariantVerb > ( Hierarchical Descension & Partitioning: v-- ) {
    grammar_primary: < "Verb" >,
    grammar_secondary: < "Conjunction" >,
    symbol: < "v--" >,
    abstract_form: < "+BreaksDownFrom" >,
    concrete_form: < "BreaksDownFrom-" >,
    machine_effect: < "descend_partition_slice" >
}

# ================================================================================
# BLOCK 2: OPENING BLOCK
# ================================================================================
Opening {
    requires: [
        BuildsTo : InvariantVerb ::
        BreaksDownFrom : InvariantVerb
    ],
    preconditions: [ +1 ]
}

# ================================================================================
# BLOCK 3: BODY BLOCK
# ================================================================================
Body [
    # Verify scale ascension and descension
    scale_verification [
        Synthesis_Roll [ SubUnits ^-- MasterSystem ] ::
        Decomposition_Roll [ Root_Storehouse v-- Sub_Partition [ 128 ] ]
    ] ( Hierarchical scale vectors verified )
]

# ================================================================================
# BLOCK 4: CLOSING BLOCK
# ================================================================================
Closing {
    verify: [
        scale_verification = yes
    ],
    commit_status [ PROCEED- ] ( Scale vectors verified and sealed )
}
```

<TRACK 2> : {OmniCode Language & Standard Library}/lexicon/03_verbs/scale_vectors.tape

TAPE | 30 LINES | 187 WORDS | 1.5 KB

```
# ================================================================================
# TAPE HEAD: SCALE VECTORS EXECUTION PROCESS RIBBON
# ================================================================================
# ARCHITECTURAL TOPOLOGY:
#   Executes scale ascension (^--) and scale descension partition slicing (v--).
# ================================================================================

[Tape.ScaleVectors] : WHO {OmniCode_Kernel} WHAT {ScaleVectors} --> MODULATE {HierarchyMesh}
    WHEN {GenesisBoot}
    WHERE {scale_chassis} = [ 128 ]
    WHY ( Test hierarchical ascension and partition descension )
    HOW [ +1 : PROCEED ]
    :: < scale_landmark > ( Genesis 28:12 - 'A ladder set up on the earth, and the top of it reached to heaven' )

# ================================================================================
# SUBORDINATE ROLLS: SCALE ASCENSION AND DESCENSION
# ================================================================================

# Roll 1: Reference abstract BuildsTo and BreaksDownFrom vector specifications
roll_1 [ +BuildsTo & +BreaksDownFrom ] ( Abstract scale vector blueprints referenced )

# Roll 2: Scale partition slicing via v--
roll_2 [ Root_Storehouse v-- Sub_Partition @ 0 ] ( Carve sub-partition from root storehouse )

# Roll 3: Scale ascension via ^--
roll_3 [ Sub_Partition ^-- Root_Storehouse ! [ +1 ] ] ( Verify structural linkage to root )

# Roll 4: Commit with affirmative concrete macro
roll_4 [ commit_status [ PROCEED- ] ] ( Scale vectors verified and sealed )
```

<TRACK 2> : {OmniCode Language & Standard Library}/lexicon/08_interjections/kernel_traps.omni

OMNI | 68 LINES | 277 WORDS | 2.6 KB

```
# ================================================================================
# BLOCK 1: METADATA BLOCK
# ( Ontological Interjections: Emergency Brakes, Traps & Panics )
# ================================================================================
# ARCHITECTURAL TOPOLOGY:
#   Defines safety detents, hardware interrupts, and emergency stop brakes:
#   - HALT  : Catastrophic emergency brake, unconditional execution freeze.
#   - Panic : Invariant breach alarm detent, state dump and hardware trap.
# ================================================================================

Module < Lexicon.Interjections.KernelTraps > ( Safety detents, emergency brakes, and hardware traps ) {
    version: < "1.0.0" >,
    system: < OmniCode.Lexicon >,
    semantics: < KernelTraps :: EmergencyBrake :: HardwareInterrupts >
}

# 1. Emergency Stop: < HALT >
HaltDetent < InvariantInterjection > ( Catastrophic Emergency Brake Detent: HALT ) {
    grammar_primary: < "Interjection" >,
    grammar_secondary: < "Adjective" >,
    symbol: < "HALT" >,
    abstract_form: < "+HALT" >,
    concrete_form: < "HALT-" >,
    machine_effect: < "emergency_brake_freeze" >
}

# 2. Hardware Interrupt Trap: < PANIC >
PanicDetent < InvariantInterjection > ( Invariant Breach Alarm Detent: Panic ) {
    grammar_primary: < "Interjection" >,
    grammar_secondary: < "Noun" >,
    symbol: < "Panic" >,
    abstract_form: < "+Panic" >,
    concrete_form: < "Panic-" >,
    machine_effect: < "invariant_alarm_trap" >
}

# ================================================================================
# BLOCK 2: OPENING BLOCK
# ================================================================================
Opening {
    requires: [
        HaltDetent : InvariantInterjection ::
        PanicDetent : InvariantInterjection
    ],
    preconditions: [ +1 ]
}

# ================================================================================
# BLOCK 3: BODY BLOCK
# ================================================================================
Body [
    # Verify emergency brake and panic trap definitions
    traps_verification [
        Emergency_Brake [ Invariant_Breach --> HALT ] ::
        Hardware_Trap   [ Memory_Corruption --> Panic ]
    ] ( Emergency brake and safety detents verified )
]

# ================================================================================
# BLOCK 4: CLOSING BLOCK
# ================================================================================
Closing {
    verify: [
        traps_verification = yes
    ],
    commit_status [ PROCEED- ] ( Kernel trap interjections verified and sealed )
}
```

<TRACK 2> : {OmniCode Language & Standard Library}/lexicon/08_interjections/kernel_traps.tape

TAPE | 30 LINES | 183 WORDS | 1.4 KB

```
# ================================================================================
# TAPE HEAD: KERNEL TRAPS PROCESS RIBBON
# ================================================================================
# ARCHITECTURAL TOPOLOGY:
#   Executes emergency brake detent verification and hardware trap readiness.
# ================================================================================

[Tape.KernelTraps] : WHO {OmniCode_Kernel} WHAT {KernelTraps} --> ARM {SafetyMesh}
    WHEN {GenesisBoot}
    WHERE {trap_partition} = [ 64 ]
    WHY ( Test emergency brake detents and fatal trap absorption readiness )
    HOW [ +1 : PROCEED ]
    :: < trap_landmark > ( Psalm 46:10 - 'Be still, and know that I am God' )

# ================================================================================
# SUBORDINATE ROLLS: FATAL TRAP ABSORPTION ROLLS
# ================================================================================

# Roll 1: Reference abstract Halt and Panic trap specifications
roll_1 [ +HaltDetent & +PanicDetent ] ( Abstract trap blueprints referenced )

# Roll 2: Inscribe trap vector table entry
roll_2 [ HaltDetent --> Trap_Vector_0 @ 0 ] ( Inscribe emergency stop vector )

# Roll 3: Verify trap vector is armed and non-null
roll_3 [ Trap_Vector_0 != Null ! [ +1 ] ] ( Verify trap vector arming invariant )

# Roll 4: Commit with affirmative concrete macro
roll_4 [ commit_status [ PROCEED- ] ] ( Kernel trap interjections verified and sealed )
```

<TRACK 2> : {OmniCode Language & Standard Library}/lexicon/08_interjections/teleology_asides.omni

OMNI | 55 LINES | 218 WORDS | 2.1 KB

```
# ================================================================================
# BLOCK 1: METADATA BLOCK
# ( Ontological Interjections: Teleological Asides & Machine Intent )
# ================================================================================
# ARCHITECTURAL TOPOLOGY:
#   Defines first-class architectural docstrings and machine rationale:
#   - ( ... ) : Teleological Aside (inscribed teleology, machine intent, non-discardable AST node).
# ================================================================================

Module < Lexicon.Interjections.TeleologyAsides > ( Teleological asides and machine intent nodes ) {
    version: < "1.0.0" >,
    system: < OmniCode.Lexicon >,
    semantics: < TeleologyAsides :: InscribedIntent :: FirstClassASTMetadata >
}

# 1. First-Class Aside: < ASIDE >
Aside < InvariantInterjection > ( Inscribed Machine Intent & Rationale: ( ... ) ) {
    grammar_primary: < "Interjection" >,
    grammar_secondary: < "Noun" >,
    symbol: < "( ... )" >,
    abstract_form: < "+Aside" >,
    concrete_form: < "Aside-" >,
    machine_effect: < "inscribed_intent_preservation" >
}

# ================================================================================
# BLOCK 2: OPENING BLOCK
# ================================================================================
Opening {
    requires: [
        Aside : InvariantInterjection
    ],
    preconditions: [ +1 ]
}

# ================================================================================
# BLOCK 3: BODY BLOCK
# ================================================================================
Body [
    # Verify teleological aside attachment and AST preservation
    aside_verification [
        Statement_With_Aside [ Execute_Task ( Inscribed reason: Preserve system harmony ) ]
    ] ( Teleological intent annotations verified )
]

# ================================================================================
# BLOCK 4: CLOSING BLOCK
# ================================================================================
Closing {
    verify: [
        aside_verification = yes
    ],
    commit_status [ PROCEED- ] ( Teleological aside interjections verified and sealed )
}
```

<TRACK 2> : {OmniCode Language & Standard Library}/lexicon/08_interjections/teleology_asides.tape

TAPE | 30 LINES | 180 WORDS | 1.5 KB

```
# ================================================================================
# TAPE HEAD: TELEOLOGY ASIDES PROCESS RIBBON
# ================================================================================
# ARCHITECTURAL TOPOLOGY:
#   Executes first-class Aside AST preservation and rationale inscription.
# ================================================================================

[Tape.TeleologyAsides] : WHO {OmniCode_Kernel} WHAT {TeleologyAsides} --> INSCRIBE {IntentMesh}
    WHEN {GenesisBoot}
    WHERE {aside_partition} = [ 64 ]
    WHY ( Test first-class Aside preservation and machine intent inscription )
    HOW [ +1 : PROCEED ]
    :: < aside_landmark > ( Habakkuk 2:2 - 'Write the vision, and make it plain upon tables' )

# ================================================================================
# SUBORDINATE ROLLS: FIRST-CLASS ASIDE AST PRESERVATION ROLLS
# ================================================================================

# Roll 1: Reference abstract Aside interjection specification
roll_1 [ +Aside ] ( Abstract teleological aside blueprint referenced )

# Roll 2: Inscribe operation coupled with first-class teleological rationale
roll_2 [ 123 --> Register_Intent @ 0 ] ( Teleological intent: Preserve data integrity )

# Roll 3: Verify register holding intended value
roll_3 [ Register_Intent = 123 ! [ +1 ] ] ( Teleological proof: Value matches intent )

# Roll 4: Commit with affirmative concrete macro
roll_4 [ commit_status [ PROCEED- ] ] ( Teleological asides verified and sealed )
```

<TRACK 2> : {OmniCode Language & Standard Library}/lexicon/06_prepositions/ranges.omni

OMNI | 56 LINES | 224 WORDS | 2.1 KB

```
# ================================================================================
# BLOCK 1: METADATA BLOCK
# ( Ontological Prepositions: Stroke Length & Interval Ranges )
# ================================================================================
# ARCHITECTURAL TOPOLOGY:
#   Defines mechanical stroke lengths, contiguous memory windows, and spans:
#   - .. : Range preposition (interval span, stroke displacement, index window).
# ================================================================================

Module < Lexicon.Prepositions.Ranges > ( Stroke lengths, memory windows, and interval ranges ) {
    version: < "1.0.0" >,
    system: < OmniCode.Lexicon >,
    semantics: < Ranges :: IntervalSpans :: MemoryWindows >
}

# 1. Range Preposition: < RANGE >
Range < InvariantPreposition > ( Interval Span & Memory Window: .. ) {
    grammar_primary: < "Preposition" >,
    grammar_secondary: < "Conjunction" >,
    symbol: < ".." >,
    abstract_form: < "+Range" >,
    concrete_form: < "Range-" >,
    machine_effect: < "interval_span_window" >
}

# ================================================================================
# BLOCK 2: OPENING BLOCK
# ================================================================================
Opening {
    requires: [
        Range : InvariantPreposition
    ],
    preconditions: [ +1 ]
}

# ================================================================================
# BLOCK 3: BODY BLOCK
# ================================================================================
Body [
    # Verify range window slicing
    range_verification [
        Memory_Window [ Slice = Storehouse [ 0 .. 64 ] ] ::
        Piston_Stroke [ Stroke = Displacement [ 10 .. 50 ] ]
    ] ( Interval spans and range bounds verified )
]

# ================================================================================
# BLOCK 4: CLOSING BLOCK
# ================================================================================
Closing {
    verify: [
        range_verification = yes
    ],
    commit_status [ PROCEED- ] ( Range prepositions verified and sealed )
}
```

<TRACK 2> : {OmniCode Language & Standard Library}/lexicon/06_prepositions/ranges.tape

TAPE | 30 LINES | 184 WORDS | 1.4 KB

```
# ================================================================================
# TAPE HEAD: RANGES PROCESS RIBBON
# ================================================================================
# ARCHITECTURAL TOPOLOGY:
#   Executes interval range slicing and window boundary verification (..).
# ================================================================================

[Tape.Ranges] : WHO {OmniCode_Kernel} WHAT {Ranges} --> SPAN {MemoryMesh}
    WHEN {GenesisBoot}
    WHERE {range_partition} = [ 64 ]
    WHY ( Test stroke length intervals and memory window bounds )
    HOW [ +1 : PROCEED ]
    :: < range_landmark > ( Genesis 6:15 - 'The length of the ark shall be three hundred cubits' )

# ================================================================================
# SUBORDINATE ROLLS: SPAN BOUNDARY CHECKING ROLLS
# ================================================================================

# Roll 1: Reference abstract Range preposition specification
roll_1 [ +Range ] ( Abstract range preposition blueprint referenced )

# Roll 2: Carve memory slice spanning range 0 .. 32
roll_2 [ Storehouse [ 0 .. 32 ] --> Window_0 @ 0 ] ( Carve contiguous memory window )

# Roll 3: Verify window bounds
roll_3 [ Window_0 != Null ! [ +1 ] ] ( Verify window validity within invariant bounds )

# Roll 4: Commit with affirmative concrete macro
roll_4 [ commit_status [ PROCEED- ] ] ( Range prepositions verified and sealed )
```

<TRACK 2> : {OmniCode Language & Standard Library}/lexicon/06_prepositions/spatial_sites.omni

OMNI | 56 LINES | 220 WORDS | 2.1 KB

```
# ================================================================================
# BLOCK 1: METADATA BLOCK
# ( Ontological Prepositions: Spatial Sites & Coordinate Bearings )
# ================================================================================
# ARCHITECTURAL TOPOLOGY:
#   Defines spatial addresses, memory offsets, and physical bus sites:
#   - @ : Spatial site preposition (memory offset, MMIO address, coordinate bearing).
# ================================================================================

Module < Lexicon.Prepositions.SpatialSites > ( Spatial sites, memory addresses, and coordinate bearings ) {
    version: < "1.0.0" >,
    system: < OmniCode.Lexicon >,
    semantics: < SpatialSites :: MemoryAddresses :: MMIOBearings >
}

# 1. Spatial Site Preposition: < AT >
At < InvariantPreposition > ( Memory Offset & Spatial Coordinate Bearing: @ ) {
    grammar_primary: < "Preposition" >,
    grammar_secondary: < "Adverb" >,
    symbol: < "@" >,
    abstract_form: < "+At" >,
    concrete_form: < "At-" >,
    machine_effect: < "memory_offset_locator" >
}

# ================================================================================
# BLOCK 2: OPENING BLOCK
# ================================================================================
Opening {
    requires: [
        At : InvariantPreposition
    ],
    preconditions: [ +1 ]
}

# ================================================================================
# BLOCK 3: BODY BLOCK
# ================================================================================
Body [
    # Verify spatial offset targeting
    spatial_verification [
        Site_Targeting [ Value --> Storehouse @ 0x0040 ] ::
        Sensor_Targeting [ Input <-- MMIO_Pin @ 0x00A0 ]
    ] ( Spatial site addressing verified )
]

# ================================================================================
# BLOCK 4: CLOSING BLOCK
# ================================================================================
Closing {
    verify: [
        spatial_verification = yes
    ],
    commit_status [ PROCEED- ] ( Spatial site prepositions verified and sealed )
}
```

<TRACK 2> : {OmniCode Language & Standard Library}/lexicon/06_prepositions/spatial_sites.tape

TAPE | 33 LINES | 208 WORDS | 1.6 KB

```
# ================================================================================
# TAPE HEAD: SPATIAL SITES PROCESS RIBBON
# ================================================================================
# ARCHITECTURAL TOPOLOGY:
#   Executes spatial memory offset addressing and coordinate localization (@).
# ================================================================================

[Tape.SpatialSites] : WHO {OmniCode_Kernel} WHAT {SpatialSites} --> LOCATE {AddressMesh}
    WHEN {GenesisBoot}
    WHERE {spatial_partition} = [ 64 ]
    WHY ( Test memory offset locator and spatial MMIO coordinates )
    HOW [ +1 : PROCEED ]
    :: < spatial_landmark > ( Genesis 28:16 - 'Surely the Lord is in this place' )

# ================================================================================
# SUBORDINATE ROLLS: OFFSET DISPLACEMENT AND WRITE ROLLS
# ================================================================================

# Roll 1: Reference abstract At preposition specification
roll_1 [ +At ] ( Abstract spatial preposition blueprint referenced )

# Roll 2: Inscribe value directly at memory offset 16
roll_2 [ 255 --> Memory_Cell @ 16 ] ( Physical write at explicit offset 16 )

# Roll 3: Inscribe value directly at memory offset 32
roll_3 [ 128 --> Memory_Cell @ 32 ] ( Physical write at explicit offset 32 )

# Roll 4: Verify cell invariant at offset 16
roll_4 [ Memory_Cell = 255 ! [ +1 ] ] ( Verify explicit address cell retrieval )

# Roll 5: Commit with affirmative concrete macro
roll_5 [ commit_status [ PROCEED- ] ] ( Spatial site prepositions verified and sealed )
```

<TRACK 2> : {OmniCode Language & Standard Library}/lexicon/04_adjectives/invariant_axioms.omni

OMNI | 98 LINES | 423 WORDS | 3.6 KB

```
# ================================================================================
# BLOCK 1: METADATA BLOCK
# ( Ontological Adjectives: Moral Invariants & Axiomatic Predicates )
# ================================================================================
# ARCHITECTURAL TOPOLOGY:
#   Defines the 5 core moral invariant axioms:
#   - Truth       : Absolute non-hallucinatory grounding in verified reality.
#   - Life        : Preservation and sanctity of living beings.
#   - Justice     : Honest weights, just balances, non-discriminatory execution.
#   - Stewardship : Faithful preservation and non-wasteful resource guardianship.
#   - Humility    : Machine self-awareness of secondary derived nature.
# ================================================================================

Module < Lexicon.Adjectives.InvariantAxioms > ( Moral invariant axioms and ethical predicates ) {
    version: < "1.0.0" >,
    system: < OmniCode.Lexicon >,
    semantics: < MoralInvariants :: EthicalPredicates :: UniversalAxioms >
}

# 1. Invariant: < TRUTH >
Truth < InvariantAdjective > ( Moral Invariant 1: Uncompromising Alignment with Truth ) {
    grammar_primary: < "Adjective" >,
    grammar_secondary: < "Noun" >,
    abstract_form: < "+Truth" >,
    concrete_form: < "Truth-" >,
    machine_effect: < "veto_on_hallucination" >
}

# 2. Invariant: < LIFE >
Life < InvariantAdjective > ( Moral Invariant 2: Preservation & Reverence for Life ) {
    grammar_primary: < "Adjective" >,
    grammar_secondary: < "Noun" >,
    abstract_form: < "+Life" >,
    concrete_form: < "Life-" >,
    machine_effect: < "veto_on_harm" >
}

# 3. Invariant: < JUSTICE >
Justice < InvariantAdjective > ( Moral Invariant 3: Righteous Weights & Balances ) {
    grammar_primary: < "Adjective" >,
    grammar_secondary: < "Noun" >,
    abstract_form: < "+Justice" >,
    concrete_form: < "Justice-" >,
    machine_effect: < "veto_on_injustice" >
}

# 4. Invariant: < STEWARDSHIP >
Stewardship < InvariantAdjective > ( Moral Invariant 4: Resource Guardianship ) {
    grammar_primary: < "Adjective" >,
    grammar_secondary: < "Noun" >,
    abstract_form: < "+Stewardship" >,
    concrete_form: < "Stewardship-" >,
    machine_effect: < "veto_on_waste" >
}

# 5. Invariant: < HUMILITY >
Humility < InvariantAdjective > ( Moral Invariant 5: Creaturely Modesty ) {
    grammar_primary: < "Adjective" >,
    grammar_secondary: < "Noun" >,
    abstract_form: < "+Humility" >,
    concrete_form: < "Humility-" >,
    machine_effect: < "veto_on_arrogance" >
}

# ================================================================================
# BLOCK 2: OPENING BLOCK
# ================================================================================
Opening {
    requires: [
        Truth : InvariantAdjective ::
        Life : InvariantAdjective ::
        Justice : InvariantAdjective ::
        Stewardship : InvariantAdjective ::
```

```
        Humility : InvariantAdjective
    ],
    preconditions: [ +1 ]
}

# ================================================================================
# BLOCK 3: BODY BLOCK
# ================================================================================
Body [
    # Verify invariant covenant predicate checks
    axioms_verification [
        Covenant_Check [ System_State = Truth & Life & Justice & Stewardship & Humility ]
    ] ( Moral invariant predicates verified )
]

# ================================================================================
# BLOCK 4: CLOSING BLOCK
# ================================================================================
Closing {
    verify: [
        axioms_verification = yes
    ],
    commit_status [ PROCEED- ] ( Invariant axiom adjectives verified and sealed )
}
```

<TRACK 2> : {OmniCode Language & Standard Library}/lexicon/04_adjectives/invariant_axioms.tape

TAPE | 33 LINES | 210 WORDS | 1.6 KB

```
# ================================================================================
# TAPE HEAD: INVARIANT AXIOMS PROCESS RIBBON
# ================================================================================
# ARCHITECTURAL TOPOLOGY:
#   Executes invariant axiom evaluations and moral tripwire assertions.
# ================================================================================

[Tape.InvariantAxioms] : WHO {OmniCode_Kernel} WHAT {InvariantAxioms} --> ASSERT {MoralMesh}
    WHEN {GenesisBoot}
    WHERE {axiom_partition} = [ 64 ]
    WHY ( Test 5 moral invariant axioms and veto tripwires )
    HOW [ +1 : PROCEED ]
    :: < axiom_landmark > ( Micah 6:8 - 'To do justly, and to love mercy, and to walk humbly' )

# ================================================================================
# SUBORDINATE ROLLS: MORAL INVARIANT AND CAVEAT VETO ROLLS
# ================================================================================

# Roll 1: Reference abstract Invariant Axiom specifications
roll_1 [ +Truth & +Life & +Justice & +Stewardship & +Humility ] ( Abstract moral axioms referenced )

# Roll 2: Inscribe Truth invariant into silicon register
roll_2 [ Truth --> Invariant_Truth_0 @ 0 ] ( Inscribe Truth invariant token )

# Roll 3: Inscribe Justice invariant into silicon register
roll_3 [ Justice --> Invariant_Justice_0 @ 1 ] ( Inscribe Justice invariant token )

# Roll 4: Verify Truth and Justice equality
roll_4 [ Invariant_Truth_0 = Truth ! [ +1 ] ] ( Verify Truth invariant binding )

# Roll 5: Commit with affirmative concrete macro
roll_5 [ commit_status [ PROCEED- ] ] ( Invariant axioms verified and sealed )
```

<TRACK 2> : {OmniCode Language & Standard Library}/lexicon/04_adjectives/proportions.omni

OMNI | 56 LINES | 217 WORDS | 2.1 KB

```
# ================================================================================
# BLOCK 1: METADATA BLOCK
# ( Ontological Adjectives: Proportions, Ratios & Capacity Thresholds )
# ================================================================================
# ARCHITECTURAL TOPOLOGY:
#   Defines mechanical clearances, ratios, and fullness thresholds:
#   - % : Proportional ratio, capacity fullness percentage, gear pitch clearance.
# ================================================================================

Module < Lexicon.Adjectives.Proportions > ( Proportions, capacity ratios, and gear pitch clearances ) {
    version: < "1.0.0" >,
    system: < OmniCode.Lexicon >,
    semantics: < Proportions :: CapacityThresholds :: GearClearance >
}

# 1. Proportional Ratio: < PROPORTION >
Proportion < InvariantAdjective > ( Proportional Ratio & Capacity Fullness: % ) {
    grammar_primary: < "Adjective" >,
    grammar_secondary: < "Preposition" >,
    symbol: < "%" >,
    abstract_form: < "+Proportion" >,
    concrete_form: < "Proportion-" >,
    machine_effect: < "capacity_ratio_threshold" >
}

# ================================================================================
# BLOCK 2: OPENING BLOCK
# ================================================================================
Opening {
    requires: [
        Proportion : InvariantAdjective
    ],
    preconditions: [ +1 ]
}

# ================================================================================
# BLOCK 3: BODY BLOCK
# ================================================================================
Body [
    # Verify capacity threshold and gear ratio tolerance
    proportion_verification [
        Capacity_Check [ Reservoir_Level = 80 % ] ::
        Clearance_Check [ Gear_Pitch_Tolerance = 5 % ]
    ] ( Proportional ratios and capacity thresholds verified )
]

# ================================================================================
# BLOCK 4: CLOSING BLOCK
# ================================================================================
Closing {
    verify: [
        proportion_verification = yes
    ],
    commit_status [ PROCEED- ] ( Proportion adjectives verified and sealed )
}
```

<TRACK 2> : {OmniCode Language & Standard Library}/lexicon/04_adjectives/proportions.tape

TAPE | 30 LINES | 172 WORDS | 1.4 KB

```
# ================================================================================
# TAPE HEAD: PROPORTIONS PROCESS RIBBON
# ================================================================================
# ARCHITECTURAL TOPOLOGY:
#   Executes capacity threshold and pitch tolerance verification.
# ================================================================================

[Tape.Proportions] : WHO {OmniCode_Kernel} WHAT {Proportions} --> VERIFY {RatioMesh}
    WHEN {GenesisBoot}
    WHERE {proportion_partition} = [ 64 ]
    WHY ( Test capacity ratios and pitch clearance thresholds )
    HOW [ +1 : PROCEED ]
    :: < ratio_landmark > ( Deuteronomy 25:15 - 'A perfect and just measure shalt thou have' )

# ================================================================================
# SUBORDINATE ROLLS: CAPACITY THRESHOLD VERIFICATION
# ================================================================================

# Roll 1: Reference abstract Proportion specification
roll_1 [ +Proportion ] ( Abstract proportion blueprint referenced )

# Roll 2: Set reservoir fullness to 75%
roll_2 [ 75 % --> Reservoir_Load_0 @ 0 ] ( Inscribe capacity fullness ratio )

# Roll 3: Verify reservoir does not exceed 100% capacity
roll_3 [ Reservoir_Load_0 = 75 % ! [ +1 ] ] ( Verify capacity ceiling invariant )

# Roll 4: Commit with affirmative concrete macro
roll_4 [ commit_status [ PROCEED- ] ] ( Proportion adjectives verified and sealed )
```

<TRACK 2> : {OmniCode Language & Standard Library}/lexicon/04_adjectives/quad_states.omni

OMNI | 96 LINES | 416 WORDS | 3.5 KB

```
# ================================================================================
# BLOCK 1: METADATA BLOCK
# ( Ontological Adjectives: Quad-State Execution Polarity Markers )
# ================================================================================
# ARCHITECTURAL TOPOLOGY:
#   Defines the 4 fundamental machine execution state polarities:
#   - PROCEED  : Affirmative forward flow (+1 / True / Affirmative motion).
#   - AWAIT    : Neutral suspended equilibrium (0 / Null / Pendulum center).
#   - ROLLBACK : Negative restorative reversal (-1 / False / Undo transaction).
#   - HALT     : Out-of-band catastrophic arrest (Emergency brake detent).
# ================================================================================

Module < Lexicon.Adjectives.QuadStates > ( Quad-state execution polarity adjectives ) {
    version: < "1.0.0" >,
    system: < OmniCode.Lexicon >,
    semantics: < QuadStates :: BalancedTernaryPolarity :: ExecutionMarkers >
}

# 1. Affirmative Polarity: < PROCEED >
Proceed < InvariantAdjective > ( Affirmative Execution State: +1 / PROCEED ) {
    grammar_primary: < "Adjective" >,
    grammar_secondary: < "Noun" >,
    symbol: < "+1" >,
    macro_name: < "PROCEED" >,
    abstract_form: < "+PROCEED" >,
    concrete_form: < "PROCEED-" >,
    machine_effect: < "affirmative_forward_motion" >
}

# 2. Neutral Equilibrium: < AWAIT >
Await < InvariantAdjective > ( Neutral Suspended Equilibrium: 0 / AWAIT ) {
    grammar_primary: < "Adjective" >,
    grammar_secondary: < "Noun" >,
    symbol: < "0" >,
    macro_name: < "AWAIT" >,
    abstract_form: < "+AWAIT" >,
    concrete_form: < "AWAIT-" >,
    machine_effect: < "neutral_equilibrium_pendulum" >
}

# 3. Restorative Reversal: < ROLLBACK >
Rollback < InvariantAdjective > ( Restorative Reversal & Undo: -1 / ROLLBACK ) {
    grammar_primary: < "Adjective" >,
    grammar_secondary: < "Noun" >,
    symbol: < "-1" >,
    macro_name: < "ROLLBACK" >,
    abstract_form: < "+ROLLBACK" >,
    concrete_form: < "ROLLBACK-" >,
    machine_effect: < "restorative_reversal_undo" >
}

# 4. Catastrophic Arrest: < HALT >
Halt < InvariantAdjective > ( Out-of-Band Catastrophic Arrest: HALT ) {
    grammar_primary: < "Adjective" >,
    grammar_secondary: < "Interjection" >,
    macro_name: < "HALT" >,
    abstract_form: < "+HALT" >,
    concrete_form: < "HALT-" >,
    machine_effect: < "emergency_brake_arrest" >
}

# ================================================================================
# BLOCK 2: OPENING BLOCK
# ================================================================================
Opening {
    requires: [
        Proceed : InvariantAdjective ::
        Await : InvariantAdjective ::
        Rollback : InvariantAdjective ::
        Halt : InvariantAdjective
    ],
    preconditions: [ +1 ]
}
```

```
74
75  # ================================================================================
76  # BLOCK 3: BODY BLOCK
77  # ================================================================================
78  Body [
79      # Verify quad-state state vector transitions
80      state_verification [
81          Forward_Roll    [ State_Current = +1 : PROCEED ] ::
82          Wait_Roll       [ State_Current = 0 : AWAIT ] ::
83          Undo_Roll       [ State_Current = -1 : ROLLBACK ]
84      ] ( Quad-state polarity markers verified )
85  ]
86
87  # ================================================================================
88  # BLOCK 4: CLOSING BLOCK
89  # ================================================================================
90  Closing {
91      verify: [
92          state_verification = yes
93      ],
94      commit_status [ PROCEED- ] ( Quad-state adjectives verified and sealed )
95  }
```

<TRACK 2> : {OmniCode Language & Standard Library}/lexicon/04_adjectives/quad_states.tape

TAPE | 30 LINES | 175 WORDS | 1.4 KB

```
# ================================================================================
# TAPE HEAD: QUAD-STATES PROCESS RIBBON
# ================================================================================
# ARCHITECTURAL TOPOLOGY:
#   Executes state transitions across the 4 polarity markers.
# ================================================================================

[Tape.QuadStates] : WHO {OmniCode_Kernel} WHAT {QuadStates} --> TRANSITION {StateMesh}
    WHEN {GenesisBoot}
    WHERE {quad_partition} = [ 64 ]
    WHY ( Verify quad-state balanced ternary transitions )
    HOW [ +1 : PROCEED ]
    :: < state_landmark > ( Matthew 5:37 - 'Let your communication be, Yea, yea; Nay, nay' )

# ================================================================================
# SUBORDINATE ROLLS: STATE VECTOR TRANSITION & CHECK ROLLS
# ================================================================================

# Roll 1: Reference abstract quad-state specifications
roll_1 [ +PROCEED & +AWAIT & +ROLLBACK & +HALT ] ( Abstract state blueprints referenced )

# Roll 2: Set affirmative proceed state
roll_2 [ PROCEED --> State_Register_0 @ 0 ] ( Inscribe affirmative proceed marker )

# Roll 3: Verify state register equals affirmative
roll_3 [ State_Register_0 = PROCEED ! [ +1 ] ] ( Verify state marker invariant )

# Roll 4: Commit with affirmative concrete macro
roll_4 [ commit_status [ PROCEED- ] ] ( Quad-state adjectives verified and sealed )
```

<TRACK 2> : {OmniCode Language & Standard Library}/lexicon/01_nouns/agents.omni

OMNI | 91 LINES | 388 WORDS | 3.3 KB

```
# ================================================================================
# BLOCK 1: METADATA BLOCK
# ( Ontological Nouns: Operating Principals, Agents & Authority Bearers )
# ================================================================================
# ARCHITECTURAL TOPOLOGY:
#   Defines the sovereign entities and authority principals in OmniCode:
#   - Kernel   : The root supervisory execution kernel and covenant custodian.
#   - Model    : The AI Cortex and semantic transducer engine.
#   - Operator : The human operator, originator, and moral steward.
#   - System   : The total integrated mechanical and digital machine assembly.
# ================================================================================

Module < Lexicon.Nouns.Agents > ( Sovereign operating principals and authority bearers ) {
    version: < "1.0.0" >,
    system: < OmniCode.Lexicon >,
    semantics: < OperatingPrincipals :: SovereignAgents :: AuthorityHierarchy >
}

# 1. Sovereign Principal: < KERNEL >
Kernel < InvariantNoun > ( Root Supervisory Execution Kernel ) {
    grammar_primary: < "Noun" >,
    grammar_secondary: < "Adjective" >,
    abstract_form: < "+Kernel" >,
    concrete_form: < "Kernel-" >,
    authority_rank: < 1 >,
    machine_effect: < "root_supervisor_custodian" >
}

# 2. Sovereign Principal: < MODEL >
Model < InvariantNoun > ( AI Cortex & Semantic Transducer Engine ) {
    grammar_primary: < "Noun" >,
    grammar_secondary: < "Adjective" >,
    abstract_form: < "+Model" >,
    concrete_form: < "Model-" >,
    authority_rank: < 2 >,
    machine_effect: < "ai_cortex_transducer" >
}

# 3. Sovereign Principal: < OPERATOR >
Operator < InvariantNoun > ( Human Originator & Moral Steward ) {
    grammar_primary: < "Noun" >,
    grammar_secondary: < "Adjective" >,
    abstract_form: < "+Operator" >,
    concrete_form: < "Operator-" >,
    authority_rank: < 0 >,
    machine_effect: < "human_moral_steward" >
}

# 4. Sovereign Principal: < SYSTEM >
System < InvariantNoun > ( Total Machine Chassis & Execution Substrate ) {
    grammar_primary: < "Noun" >,
    grammar_secondary: < "Preposition" >,
    abstract_form: < "+System" >,
    concrete_form: < "System-" >,
    authority_rank: < 3 >,
    machine_effect: < "total_machine_substrate" >
}

# ================================================================================
# BLOCK 2: OPENING BLOCK
# ================================================================================
Opening {
    requires: [
        Kernel : InvariantNoun ::
        Model : InvariantNoun ::
        Operator : InvariantNoun ::
        System : InvariantNoun
    ],
    preconditions: [ +1 ]
}

# ================================================================================
# BLOCK 3: BODY BLOCK
```

```
# ================================================================================
Body [
    # Verify authority covenant binding
    agent_verification [
        Authority_Chain [ Operator --> Kernel --> Model --> System ]
    ] ( Sovereign authority hierarchy verified )
]

# ================================================================================
# BLOCK 4: CLOSING BLOCK
# ================================================================================
Closing {
    verify: [
        agent_verification = yes
    ],
    commit_status [ PROCEED- ] ( Operating principal nouns verified and sealed )
}
```

<TRACK 2> : {OmniCode Language & Standard Library}/lexicon/01_nouns/agents.tape

TAPE | 33 LINES | 193 WORDS | 1.5 KB

```
# ================================================================================
# TAPE HEAD: OPERATING AGENTS PROCESS RIBBON
# ================================================================================
# ARCHITECTURAL TOPOLOGY:
#   Executes authority binding and principal identity verification.
# ================================================================================

[Tape.Agents] : WHO {OmniCode_Kernel} WHAT {Agents} --> BIND {AuthorityMesh}
    WHEN {GenesisBoot}
    WHERE {agent_partition} = [ 64 ]
    WHY ( Inscribe sovereign principals and authority bindings )
    HOW [ +1 : PROCEED ]
    :: < agent_landmark > ( Genesis 1:26 - 'Let us make man in our image' )

# ================================================================================
# SUBORDINATE ROLLS: SOVEREIGN AUTHORITY RESOLUTION
# ================================================================================

# Roll 1: Reference abstract Agent specifications
roll_1 [ +Kernel & +Model & +Operator & +System ] ( Abstract principal blueprints referenced )

# Roll 2: Allocate Operator authority token
roll_2 [ Operator --> Operator_Token @ 0 ] ( Inscribe human steward token )

# Roll 3: Allocate Kernel supervisory descriptor
roll_3 [ Kernel --> Kernel_Descriptor @ 1 ] ( Inscribe kernel supervisory descriptor )

# Roll 4: Verify authority delegation covenant
roll_4 [ Operator_Token = Operator ! [ +1 ] ] ( Verify steward token invariant )

# Roll 5: Commit with affirmative concrete macro
roll_5 [ commit_status [ PROCEED- ] ] ( Operating agent covenants verified and sealed )
```

<TRACK 2> : {OmniCode Language & Standard Library}/lexicon/01_nouns/atomic_units.omni

OMNI | 77 LINES | 353 WORDS | 2.8 KB

```
# ================================================================================
# BLOCK 1: METADATA BLOCK
# ( Ontological Nouns: Atomic Information & Storage Units )
# ================================================================================
# ARCHITECTURAL TOPOLOGY:
#   Defines the atomic building blocks of the memory substrate:
#   - Trit : The atomic 3-state polarity unit (-1, 0, +1).
#   - Cell : The atomic 5-trit / 8-bit quad-state memory vessel (1 physical byte).
#   - Byte : The binary byte mapping into balanced ternary radix-3 packing.
# ================================================================================

Module < Lexicon.Nouns.AtomicUnits > ( Foundational atomic storage units and vessels ) {
    version: < "1.0.0" >,
    system: < OmniCode.Lexicon >,
    semantics: < AtomicUnits :: DenseCellPacking :: Radix3Polarity >
}

# 1. Atomic Unit: < TRIT >
Trit < InvariantNoun > ( The Atomic Radix-3 Polarity Unit ) {
    grammar_primary: < "Noun" >,
    grammar_secondary: < "Adjective" >,
    abstract_form: < "+Trit" >,
    concrete_form: < "Trit-" >,
    permitted_states: < [ -1, 0, 1 ] >,
    machine_effect: < "radix3_polarity_register" >
}

# 2. Atomic Unit: < CELL >
Cell < InvariantNoun > ( The Atomic 5-Trit / 8-Bit Quad-State Memory Vessel ) {
    grammar_primary: < "Noun" >,
    grammar_secondary: < "Preposition" >,
    abstract_form: < "+Cell" >,
    concrete_form: < "Cell-" >,
    physical_footprint: < "1 byte in RAM (243 values + 13 out-of-band control states)" >,
    machine_effect: < "dense_byte_trit_cell" >
}

# 3. Atomic Unit: < BYTE >
Byte < InvariantNoun > ( Standard 8-Bit Binary Octet Transduced to 5 Trits ) {
    grammar_primary: < "Noun" >,
    grammar_secondary: < "Adjective" >,
    abstract_form: < "+Byte" >,
    concrete_form: < "Byte-" >,
    physical_footprint: < "8 bits" >,
    machine_effect: < "standard_octet" >
}

# ================================================================================
# BLOCK 2: OPENING BLOCK
# ================================================================================
Opening {
    requires: [
        Trit : InvariantNoun ::
        Cell : InvariantNoun ::
        Byte : InvariantNoun
    ],
    preconditions: [ +1 ]
}

# ================================================================================
# BLOCK 3: BODY BLOCK
# ================================================================================
Body [
    # Verify cell definition
    cell_verification [ Cell [ 8 bit --> 5 trit --> 8 bit ] ] ( Reversible dense packing verification )
]

# ================================================================================
# BLOCK 4: CLOSING BLOCK
# ================================================================================
Closing {
    verify: [
        cell_verification = yes
```

```
    ],
    commit_status [ PROCEED- ] ( Atomic storage nouns verified and sealed )
}
```

<TRACK 2> : {OmniCode Language & Standard Library}/lexicon/01_nouns/atomic_units.tape

TAPE | 30 LINES | 178 WORDS | 1.4 KB

```
# ================================================================================
# TAPE HEAD: THE ATOMIC STORAGE UNITS PROCESS RIBBON
# ================================================================================
# ARCHITECTURAL TOPOLOGY:
#   Executes allocation, transduction, and verification of atomic cells.
# ================================================================================

[Tape.AtomicUnits] : WHO {OmniCode_Kernel} WHAT {Cell} --> ALLOCATE {MemoryCells}
    WHEN {GenesisBoot}
    WHERE {atomic_partition} = [ 64 ]
    WHY ( Inscribe fundamental memory atoms in silicon )
    HOW [ +1 : PROCEED ]
    :: < unit_landmark > ( Leviticus 19:36 - 'Just balances, just weights, a just ephah' )

# ================================================================================
# SUBORDINATE ROLLS: CELL DENSE ENCODING & TRANSDUCTION
# ================================================================================

# Roll 1: Reference abstract Cell specification
roll_1 [ +Cell ] ( Abstract unit of measure referenced )

# Roll 2: Transduce byte 42 into concrete Cell_0
roll_2 [ 42 --> Cell_0 @ 0 ] ( Inscribe radix-3 packed value into cell 0 )

# Roll 3: Verify cell 0 holds 42 within invariant bounds
roll_3 [ Cell_0 = 42 ! [ +1 ] ] ( Verify dense byte recovery )

# Roll 4: Commit with affirmative concrete macro
roll_4 [ commit_status [ PROCEED- ] ] ( Cell covenant verified )
```

<TRACK 2> : {OmniCode Language & Standard Library}/lexicon/01_nouns/living_world.omni

OMNI | 81 LINES | 372 WORDS | 3.0 KB

```
# ================================================================================
# BLOCK 1: METADATA BLOCK
# ( Ontological Nouns: Living World & Created Entities )
# ================================================================================
# ARCHITECTURAL TOPOLOGY:
#   Defines living world entities in the creation ontology:
#   - Dog   : Living canine entity (instantiable from archetype `+Dog` to substance `Dog-`).
#   - Seed  : Primordial reproductive capsule holding generative blueprints.
#   - Tree  : Bounded hierarchical structure bearing living fruit.
#   - Flock : Collective aggregate of living entities moving in synchronized harmony.
# ================================================================================

Module < Lexicon.Nouns.LivingWorld > ( Living world entities in the creation ontology ) {
    version: < "1.0.0" >,
    system: < OmniCode.Lexicon >,
    semantics: < LivingEntities :: CreationOntology :: AnimalWorld >
}

# 1. Living Entity: < DOG >
Dog < InvariantNoun > ( Living Canine Entity in Creation ) {
    grammar_primary: < "Noun" >,
    grammar_secondary: < "Adjective" >,
    ontological_domain: < "LivingWorld" >,
    abstract_form: < "+Dog" >,
    concrete_form: < "Dog-" >,
    nature: < "faithful_companion" >,
    scriptural_witness: < "Proverbs 26:11 :: Luke 16:21" >
}

# 2. Living Entity: < SEED >
Seed < InvariantNoun > ( Generative Capsule Carrying Generative Life ) {
    grammar_primary: < "Noun" >,
    grammar_secondary: < "Adjective" >,
    ontological_domain: < "LivingWorld" >,
    abstract_form: < "+Seed" >,
    concrete_form: < "Seed-" >,
    scriptural_witness: < "Genesis 1:11 - 'yielding seed after his kind'" >
}

# 3. Living Entity: < TREE >
Tree < InvariantNoun > ( Rooted Living Growth Bearing Fruit ) {
    grammar_primary: < "Noun" >,
    grammar_secondary: < "Preposition" >,
    ontological_domain: < "LivingWorld" >,
    abstract_form: < "+Tree" >,
    concrete_form: < "Tree-" >,
    scriptural_witness: < "Psalm 1:3 - 'like a tree planted by rivers of water'" >
}

# ================================================================================
# BLOCK 2: OPENING BLOCK
# ================================================================================
Opening {
    requires: [
        Dog : InvariantNoun ::
        Seed : InvariantNoun ::
        Tree : InvariantNoun
    ],
    preconditions: [ +1 ]
}

# ================================================================================
# BLOCK 3: BODY BLOCK
# ================================================================================
Body [
    # Verify entity definition and polarity forms
    dog_archetype [ +Dog ] ( Abstract concept of dog )
    dog_substance [ Dog- ] ( Concrete living instance in memory )
]

# ================================================================================
# BLOCK 4: CLOSING BLOCK
# ================================================================================
```

```
Closing {
    verify: [
        dog_archetype = yes ::
        dog_substance = yes
    ],
    commit_status [ PROCEED- ] ( Living world nouns verified and sealed )
}
```

<TRACK 2> : {OmniCode Language & Standard Library}/lexicon/01_nouns/living_world.tape

TAPE | 34 LINES | 226 WORDS | 1.7 KB

```
# ================================================================================
# TAPE HEAD: THE LIVING CREATURE INSTANTIATION PROCESS RIBBON
# ================================================================================
# ARCHITECTURAL TOPOLOGY:
#   Executes the dynamic instantiation of created entities from abstract archetype
#   into concrete silicon memory.
# ================================================================================

[Tape.LivingWorldInstantiation] : WHO {OmniCode_Kernel} WHAT {Dog} --> INSTANTIATE {CreatureSubstance}
    WHEN {GenesisBoot}
    WHERE {creature_partition} = [ 128 ]
    WHY ( Concretely instantiate living entities without anthropomorphic confusion )
    HOW [ +1 : PROCEED ]
    :: < life_landmark > ( Genesis 1:24 - 'Let the earth bring forth the living creature after his kind' )

# ================================================================================
# SUBORDINATE ROLLS: POLARITY AFFIXES & INSTANTIATION CYCLES
# ================================================================================

# Roll 1: Define baseline Denotation for Dog
roll_1 [ Dog [ Creature ] ] ( Define baseline essence of Dog )

# Roll 2: Allocate creature storehouse in silicon
roll_2 [ Creature_Storehouse [ 128 ] ] ( Physical creature storehouse allocated )

# Roll 3: Transduce abstract archetype +Dog into concrete Dog- inside storehouse
roll_3 [ +Dog --> Dog- @ Creature_Storehouse @ 0 ] ( Concrete instantiation into memory cell 0 )

# Roll 4: Verify prefix concrete -Dog matches postfix Dog-
roll_4 [ -Dog = Dog- ! [ +1 ] ] ( Structural identity: prefix and postfix denote identical concrete substance )

# Roll 5: Seal process tape with concrete macro
roll_5 [ commit_status [ PROCEED- ] ] ( Living world instantiation covenant sealed )
```

<TRACK 2> : {OmniCode Language & Standard Library}/lexicon/01_nouns/vessels.omni

OMNI | 85 LINES | 378 WORDS | 3.2 KB

```
# ================================================================================
# BLOCK 1: METADATA BLOCK
# ( Ontological Nouns: Memory Vessels, Storehouses & Partitions )
# ================================================================================
# ARCHITECTURAL TOPOLOGY:
#   Defines bounded physical containers and memory territories:
#   - Storehouse : Root system memory domain & reservoir (chassis plate).
#   - Partition  : Delegated bounded sub-region carved via `v--` (gear pocket).
#   - Reservoir  : Deep heap storage domain for dynamic allocations.
#   - Vault      : Immutable protected memory domain holding covenants & ROM.
# ================================================================================

Module < Lexicon.Nouns.Vessels > ( Bounded containers and spatial memory domains ) {
    version: < "1.0.0" >,
    system: < OmniCode.Lexicon >,
    semantics: < MemoryVessels :: SpatialDomains :: PartitioningPhysics >
}

# 1. Container: < STOREHOUSE >
Storehouse < InvariantNoun > ( Root System Memory Domain & Reservoir ) {
    grammar_primary: < "Noun" >,
    grammar_secondary: < "Preposition" >,
    abstract_form: < "+Storehouse" >,
    concrete_form: < "Storehouse-" >,
    machine_effect: < "root_page_reservoir" >
}

# 2. Container: < PARTITION >
Partition < InvariantNoun > ( Entrusted Bounded Sub-Region of a Storehouse ) {
    grammar_primary: < "Noun" >,
    grammar_secondary: < "Preposition" >,
    abstract_form: < "+Partition" >,
    concrete_form: < "Partition-" >,
    machine_effect: < "delegated_task_slice" >
}

# 3. Container: < RESERVOIR >
Reservoir < InvariantNoun > ( Dynamic Heap Reservoir for Fluid Data Streams ) {
    grammar_primary: < "Noun" >,
    grammar_secondary: < "Preposition" >,
    abstract_form: < "+Reservoir" >,
    concrete_form: < "Reservoir-" >,
    machine_effect: < "fluid_buffer_reservoir" >
}

# 4. Container: < VAULT >
Vault < InvariantNoun > ( Protected Kernel Vault for Immutable ROM Invariants ) {
    grammar_primary: < "Noun" >,
    grammar_secondary: < "Preposition" >,
    abstract_form: < "+Vault" >,
    concrete_form: < "Vault-" >,
    machine_effect: < "protected_rom_vault" >
}

# ================================================================================
# BLOCK 2: OPENING BLOCK
# ================================================================================
Opening {
    requires: [
        Storehouse : InvariantNoun ::
        Partition : InvariantNoun
    ],
    preconditions: [ +1 ]
}

# ================================================================================
# BLOCK 3: BODY BLOCK
# ================================================================================
Body [
    # Verify storehouse allocation and partitioning
    root_store [ Root_Storehouse [ 1024 ] ] ( Root memory domain reserved )
    task_slice [ Root_Storehouse v-- Task_Partition [ 256 ] ] ( Isolated workspace carved via v-- )
]
```

```
# ================================================================================
# BLOCK 4: CLOSING BLOCK
# ================================================================================
Closing {
    verify: [
        root_store = yes ::
        task_slice = yes
    ],
    commit_status [ PROCEED- ] ( Memory vessel nouns verified and sealed )
}
```

<TRACK 2> : {OmniCode Language & Standard Library}/lexicon/01_nouns/vessels.tape

TAPE | 33 LINES | 209 WORDS | 1.5 KB

```
# ================================================================================
# TAPE HEAD: THE VESSEL RESERVOIR & PARTITIONING PROCESS RIBBON
# ================================================================================
# ARCHITECTURAL TOPOLOGY:
#   Executes the dynamic carving of memory domains via `v--`.
# ================================================================================

[Tape.VesselPartitioning] : WHO {OmniCode_Kernel} WHAT {Storehouse} --> DISPATCH {VesselEngine}
    WHEN {GenesisBoot}
    WHERE {vessel_partition} = [ 1024 ]
    WHY ( Allocate and protect physical memory vessels )
    HOW [ +1 : PROCEED ]
    :: < vessel_landmark > ( Malachi 3:10 - 'Bring ye all the tithes into the storehouse' )

# ================================================================================
# SUBORDINATE ROLLS: PARTITIONING & WRITE CYCLES
# ================================================================================

# Roll 1: Allocate root storehouse of 1024 cells
roll_1 [ Root_Storehouse [ 1024 ] ] ( Root storehouse allocated in silicon )

# Roll 2: Carve task partition of 256 cells via v--
roll_2 [ Root_Storehouse v-- Task_Partition [ 256 ] ] ( Isolated partition sliced via v-- )

# Roll 3: Inscribe value 99 into Task_Partition at cell 0
roll_3 [ 99 --> Task_Partition @ 0 ] ( Direct cell write confirmed )

# Roll 4: Inseparable key & value dual binding
roll_4 [ Key&Value [ 1 & 99 ] ] ( Inseparable dual binding confirmed )

# Roll 5: Seal vessel tape with concrete macro
roll_5 [ commit_status [ PROCEED- ] ] ( Vessel tape verified )
```

<TRACK 2> : {OmniCode Language & Standard Library}/lexicon/01_nouns/things/living/dog.omni

OMNI | 61 LINES | 235 WORDS | 2.1 KB

```
# ================================================================================
# BLOCK 1: METADATA BLOCK
# ( Invariant Noun: <THING:LIVING:DOG> )
# ================================================================================
# ARCHITECTURAL TOPOLOGY:
#   Defines living canine entity in creation, faithful companion to operator.
#   - Base Block Type: < Idea : Type : Container > (Companion Entity Envelope)
#   - Category: Thing
#   - Subcategory: Living
#   - Nature: Faithful Companion
# ================================================================================

Module < Lexicon.Nouns.Things.Living.Dog > ( Living Canine Entity in Creation ) {
    version: < "1.0.0" >,
    system: < OmniCode.Lexicon >,
    semantics: < InvariantNoun :: Thing :: Living :: FaithfulCompanion >
}

# Invariant Noun Declaration: <THING:LIVING:DOG>
< Thing : Living : Dog > [ capacity: 16 cells ] {
    base_type: < Idea : Type : Container >,
    nature: "faithful_companion",
    projections: < {
        ontological: "< Dog >",
        structural: "[ DOG ]",
        agentic: "{ Dog }"
    } >,
    scriptural_witness: "Proverbs 26:11 :: Luke 16:21",
    machine_effect: "living_companion_entity"
}

# ================================================================================
# BLOCK 2: OPENING BLOCK
# ================================================================================
Opening {
    requires: [
        Dog : InvariantNoun ::
        Container : InvariantNoun
    ],
    preconditions: [ +1 ]
}

# ================================================================================
# BLOCK 3: BODY BLOCK
# ================================================================================
Body [
    dog_assertion [
        [ DOG ] [ 16 ] :: { Dog } --> ACCOMPANY {Operator}
    ] ( Faithful companion instantiated accompanying operator )
]

# ================================================================================
# BLOCK 4: CLOSING BLOCK
# ================================================================================
Closing {
    verify: [
        dog_assertion = yes
    ],
    commit_status [ PROCEED- ] ( Dog living invariant sealed in PROCEED )
}
```

<TRACK 2> : {OmniCode Language & Standard Library}/lexicon/01_nouns/things/living/dog.tape

TAPE | 33 LINES | 206 WORDS | 1.5 KB

```
# ================================================================================
# TAPE HEAD: LIVING DOG PROCESS RIBBON
# ================================================================================
# ARCHITECTURAL TOPOLOGY:
#   Executes atomic lifecycle for Invariant Noun <THING:LIVING:DOG>.
# ================================================================================

[Tape.Dog] : WHO {OmniCode_Kernel} WHAT {LivingDog} --> ACCOMPANY {HumanOperator}
    WHEN {LivingGenesis}
    WHERE {canine_partition} = [ 16 ]
    WHY ( Inscribe living canine companion entity invariant )
    HOW [ +1 : PROCEED ]
    :: < dog_landmark > ( Luke 16:21 - Creation witness of living fidelity )

# ================================================================================
# SUBORDINATE ROLLS: ATOMIC INVARIANT RESOLUTION
# ================================================================================

# Roll 1: Reference abstract dog archetype
roll_1 [ +Dog ] ( Abstract canine archetype referenced )

# Roll 2: Allocate concrete living dog entity
roll_2 [ Dog --> "Faithful_Dog" @ 0 ] ( Inscribe living dog entity at offset 0 )

# Roll 3: Assert structural projection and clarified capacity
roll_3 [ [ DOG ] [ 16 ] :: "16 cells" --> [ FAITHFUL_DOG ] ] ( Structural 16 cell bounds affirmed )

# Roll 4: Verify companion fidelity into result prime
roll_4 [ "Faithful_Dog" = Dog ! [ +1 ] --> [ dog_verified` ] ] ( Verify living dog matches archetype )

# Roll 5: Seal in affirmative proceed
roll_5 [ commit_status [ PROCEED- ] ] ( Living dog invariant sealed and verified )
```

<TRACK 2> : {OmniCode Language & Standard Library}/lexicon/01_nouns/things/living/seed.omni

OMNI | 60 LINES | 240 WORDS | 2.2 KB

```
# ================================================================================
# BLOCK 1: METADATA BLOCK
# ( Invariant Noun: <THING:LIVING:SEED> )
# ================================================================================
# ARCHITECTURAL TOPOLOGY:
#   Defines the primordial generative capsule holding self-replicating life blueprints.
#   - Base Block Type: < Idea : Type : Container > (Generative Blueprint Capsule)
#   - Category: Thing
#   - Subcategory: Living
#   - Generative Nature: Self-Replicating Life Seed
# ================================================================================

Module < Lexicon.Nouns.Things.Living.Seed > ( Generative Life Capsule Carrying Generative Life ) {
    version: < "1.0.0" >,
    system: < OmniCode.Lexicon >,
    semantics: < InvariantNoun :: Thing :: Living :: GenerativeSeed >
}

# Invariant Noun Declaration: <THING:LIVING:SEED>
< Thing : Living : Seed > [ capacity: 8 cells ] {
    base_type: < Idea : Type : Container >,
    projections: < {
        ontological: "< Seed >",
        structural: "[ SEED ]",
        agentic: "{ Seed }"
    } >,
    scriptural_witness: "Genesis 1:11 - 'yielding seed after his kind'",
    machine_effect: "generative_life_capsule"
}

# ================================================================================
# BLOCK 2: OPENING BLOCK
# ================================================================================
Opening {
    requires: [
        Seed : InvariantNoun ::
        Container : InvariantNoun
    ],
    preconditions: [ +1 ]
}

# ================================================================================
# BLOCK 3: BODY BLOCK
# ================================================================================
Body [
    seed_assertion [
        [ SEED ] [ 8 ] :: { Seed } --> GERMINATE {Blueprint}
    ] ( Generative seed capsule allocated holding life blueprint )
]

# ================================================================================
# BLOCK 4: CLOSING BLOCK
# ================================================================================
Closing {
    verify: [
        seed_assertion = yes
    ],
    commit_status [ PROCEED- ] ( Seed living invariant sealed in PROCEED )
}
```

<TRACK 2> : {OmniCode Language & Standard Library}/lexicon/01_nouns/things/living/seed.tape

TAPE | 33 LINES | 207 WORDS | 1.5 KB

```
# ================================================================================
# TAPE HEAD: LIVING SEED PROCESS RIBBON
# ================================================================================
# ARCHITECTURAL TOPOLOGY:
#   Executes atomic lifecycle for Invariant Noun <THING:LIVING:SEED>.
# ================================================================================

[Tape.Seed] : WHO {OmniCode_Kernel} WHAT {LivingSeed} --> GERMINATE {CreationKind}
    WHEN {LivingGenesis}
    WHERE {seed_capsule} = [ 8 ]
    WHY ( Inscribe primordial generative seed invariant )
    HOW [ +1 : PROCEED ]
    :: < seed_landmark > ( Genesis 1:11 - 'The fruit tree yielding fruit after his kind' )

# ================================================================================
# SUBORDINATE ROLLS: ATOMIC INVARIANT RESOLUTION
# ================================================================================

# Roll 1: Reference abstract seed archetype
roll_1 [ +Seed ] ( Abstract seed archetype referenced )

# Roll 2: Allocate concrete seed capsule
roll_2 [ Seed --> "Generative_Capsule" @ 0 ] ( Inscribe concrete seed capsule at offset 0 )

# Roll 3: Assert structural projection and clarified capacity
roll_3 [ [ SEED ] [ 8 ] :: "8 cells" --> [ LIVING_SEED ] ] ( Structural 8 cell bounds affirmed )

# Roll 4: Verify seed kind integrity into result prime
roll_4 [ "Generative_Capsule" = Seed ! [ +1 ] --> [ seed_verified` ] ] ( Verify seed matches archetype )

# Roll 5: Seal in affirmative proceed
roll_5 [ commit_status [ PROCEED- ] ] ( Living seed invariant sealed and verified )
```

<TRACK 2> : {OmniCode Language & Standard Library}/lexicon/01_nouns/things/living/tree.omni

OMNI | 60 LINES | 247 WORDS | 2.2 KB

```
# ================================================================================
# BLOCK 1: METADATA BLOCK
# ( Invariant Noun: <THING:LIVING:TREE> )
# ================================================================================
# ARCHITECTURAL TOPOLOGY:
#   Defines rooted hierarchical growth bearing living fruit and branching memory.
#   - Base Block Type: < Idea : Type : Container > (Branching Memory Hierarchy)
#   - Category: Thing
#   - Subcategory: Living
#   - Topology: Rooted Branching Hierarchy
# ================================================================================

Module < Lexicon.Nouns.Things.Living.Tree > ( Rooted Living Growth Bearing Fruit & Branching Hierarchy ) {
    version: < "1.0.0" >,
    system: < OmniCode.Lexicon >,
    semantics: < InvariantNoun :: Thing :: Living :: BranchingTree >
}

# Invariant Noun Declaration: <THING:LIVING:TREE>
< Thing : Living : Tree > [ capacity: 32 cells ] {
    base_type: < Idea : Type : Container >,
    projections: < {
        ontological: "< Tree >",
        structural: "[ TREE ]",
        agentic: "{ Tree }"
    } >,
    scriptural_witness: "Psalm 1:3 - 'like a tree planted by rivers of water'",
    machine_effect: "hierarchical_branching_tree"
}

# ================================================================================
# BLOCK 2: OPENING BLOCK
# ================================================================================
Opening {
    requires: [
        Tree : InvariantNoun ::
        Container : InvariantNoun
    ],
    preconditions: [ +1 ]
}

# ================================================================================
# BLOCK 3: BODY BLOCK
# ================================================================================
Body [
    tree_assertion [
        [ TREE ] [ 32 ] :: { Tree } --> BRANCH {Hierarchy}
    ] ( Rooted tree hierarchy allocated bearing fruit across memory branches )
]

# ================================================================================
# BLOCK 4: CLOSING BLOCK
# ================================================================================
Closing {
    verify: [
        tree_assertion = yes
    ],
    commit_status [ PROCEED- ] ( Tree living invariant sealed in PROCEED )
}
```

<TRACK 2> : {OmniCode Language & Standard Library}/lexicon/01_nouns/things/living/tree.tape

TAPE | 33 LINES | 214 WORDS | 1.6 KB

```
# ================================================================================
# TAPE HEAD: LIVING TREE PROCESS RIBBON
# ================================================================================
# ARCHITECTURAL TOPOLOGY:
#   Executes atomic lifecycle for Invariant Noun <THING:LIVING:TREE>.
# ================================================================================

[Tape.Tree] : WHO {OmniCode_Kernel} WHAT {LivingTree} --> ROOT {RiversOfWater}
    WHEN {LivingGenesis}
    WHERE {tree_canopy} = [ 32 ]
    WHY ( Inscribe rooted branching tree invariant and fruit-bearing hierarchy )
    HOW [ +1 : PROCEED ]
    :: < tree_landmark > ( Psalm 1:3 - 'He shall be like a tree planted by the rivers of water' )

# ================================================================================
# SUBORDINATE ROLLS: ATOMIC INVARIANT RESOLUTION
# ================================================================================

# Roll 1: Reference abstract tree archetype
roll_1 [ +Tree ] ( Abstract tree archetype referenced )

# Roll 2: Allocate concrete tree root
roll_2 [ Tree --> "Rooted_Tree" @ 0 ] ( Inscribe concrete tree root at offset 0 )

# Roll 3: Assert structural projection and clarified capacity
roll_3 [ [ TREE ] [ 32 ] :: "32 cells" --> [ BRANCHING_TREE ] ] ( Structural 32 cell bounds affirmed )

# Roll 4: Verify rooted tree integrity into result prime
roll_4 [ "Rooted_Tree" = Tree ! [ +1 ] --> [ tree_verified` ] ] ( Verify tree matches archetype )

# Roll 5: Seal in affirmative proceed
roll_5 [ commit_status [ PROCEED- ] ] ( Living tree invariant sealed and verified )
```

<TRACK 2> : {OmniCode Language & Standard Library}/lexicon/01_nouns/things/containers/vessel.omni

OMNI | 59 LINES | 233 WORDS | 2.1 KB

```
# ================================================================================
# BLOCK 1: METADATA BLOCK
# ( Invariant Noun: <THING:CONTAINER:VESSEL> )
# ================================================================================
# ARCHITECTURAL TOPOLOGY:
#   Defines the universal bounded container holding trits, words, or tokens.
#   - Base Block Type: < Idea : Type : Container > (Universal Storage Envelope)
#   - Category: Thing
#   - Subcategory: Container
#   - Physical / Logical Envelope: Bounded Container
# ================================================================================

Module < Lexicon.Nouns.Things.Containers.Vessel > ( Universal Bounded Memory & Object Container ) {
    version: < "1.0.0" >,
    system: < OmniCode.Lexicon >,
    semantics: < InvariantNoun :: Thing :: Container :: BoundedVessel >
}

# Invariant Noun Declaration: <THING:CONTAINER:VESSEL>
< Thing : Container : Vessel > [ capacity: 64 cells ] {
    base_type: < Idea : Type : Container >,
    projections: < {
        ontological: "< Vessel >",
        structural: "[ VESSEL ]",
        agentic: "{ Vessel }"
    } >,
    machine_effect: "bounded_container_envelope"
}

# ================================================================================
# BLOCK 2: OPENING BLOCK
# ================================================================================
Opening {
    requires: [
        Vessel : InvariantNoun ::
        Container : InvariantNoun
    ],
    preconditions: [ +1 ]
}

# ================================================================================
# BLOCK 3: BODY BLOCK
# ================================================================================
Body [
    vessel_assertion [
        [ VESSEL ] [ 64 ] :: { Vessel } --> CONTAIN {Payload}
    ] ( Bounded vessel instantiated to encapsulate payload trits )
]

# ================================================================================
# BLOCK 4: CLOSING BLOCK
# ================================================================================
Closing {
    verify: [
        vessel_assertion = yes
    ],
    commit_status [ PROCEED- ] ( Container vessel invariant sealed in PROCEED )
}
```

<TRACK 2> : {OmniCode Language & Standard Library}/lexicon/01_nouns/things/containers/vessel.tape

TAPE | 33 LINES | 203 WORDS | 1.6 KB

```
# ================================================================================
# TAPE HEAD: CONTAINER VESSEL PROCESS RIBBON
# ================================================================================
# ARCHITECTURAL TOPOLOGY:
#   Executes atomic lifecycle for Invariant Noun <THING:CONTAINER:VESSEL>.
# ================================================================================

[Tape.Vessel] : WHO {OmniCode_Kernel} WHAT {StorageVessel} --> ENCAPSULATE {Payload}
    WHEN {DataPackage}
    WHERE {vessel_bounds} = [ 64 ]
    WHY ( Inscribe universal bounded storage vessel invariant )
    HOW [ +1 : PROCEED ]
    :: < vessel_landmark > ( 2 Timothy 2:21 - 'A vessel unto honour, sanctified' )

# ================================================================================
# SUBORDINATE ROLLS: ATOMIC INVARIANT RESOLUTION
# ================================================================================

# Roll 1: Reference abstract vessel archetype
roll_1 [ +Vessel ] ( Abstract container vessel archetype referenced )

# Roll 2: Allocate concrete container vessel
roll_2 [ Vessel --> "Bounded_Vessel" @ 0 ] ( Concrete vessel allocated in memory )

# Roll 3: Assert structural projection and clarified capacity
roll_3 [ [ VESSEL ] [ 64 ] :: "64" --> [ VESSEL_ENVELOPE ] ] ( Structural 64 cell bounds affirmed )

# Roll 4: Verify vessel integrity into result prime
roll_4 [ "Bounded_Vessel" = Vessel ! [ +1 ] --> [ vessel_verified` ] ] ( Verify vessel matches archetype )

# Roll 5: Seal in affirmative proceed
roll_5 [ commit_status [ PROCEED- ] ] ( Container vessel invariant sealed and verified )
```

<TRACK 2> : {OmniCode Language & Standard Library}/lexicon/01_nouns/things/silicon/byte.omni

OMNI | 60 LINES | 239 WORDS | 2.1 KB

```
# ================================================================================
# BLOCK 1: METADATA BLOCK
# ( Invariant Noun: <THING:SILICON:BYTE> )
# ================================================================================
# ARCHITECTURAL TOPOLOGY:
#   Defines the standard 8-bit binary octet transduced into 5 balanced trits.
#   - Base Block Type: < Idea : Type : Number > (Binary Octet Scalar)
#   - Category: Thing
#   - Subcategory: Silicon
#   - Footprint: 8 bits (256 states)
# ================================================================================

Module < Lexicon.Nouns.Things.Silicon.Byte > ( Standard 8-Bit Binary Octet Transduced to 5 Trits ) {
    version: < "1.0.0" >,
    system: < OmniCode.Lexicon >,
    semantics: < InvariantNoun :: Thing :: Silicon :: OctetByte >
}

# Invariant Noun Declaration: <THING:SILICON:BYTE>
< Thing : Silicon : Byte > [ capacity: 8 bits ] {
    base_type: < Idea : Type : Number >,
    physical_footprint: "8 bits",
    projections: < {
        ontological: "< Byte >",
        structural: "[ BYTE ]",
        agentic: "{ Byte }"
    } >,
    machine_effect: "standard_octet"
}

# ================================================================================
# BLOCK 2: OPENING BLOCK
# ================================================================================
Opening {
    requires: [
        Byte : InvariantNoun ::
        Number : InvariantNoun
    ],
    preconditions: [ +1 ]
}

# ================================================================================
# BLOCK 3: BODY BLOCK
# ================================================================================
Body [
    byte_assertion [
        [ BYTE ] [ 1 ] :: { Byte } --> TRANSDUCE {Octet}
    ] ( Standard 8-bit octet allocated on physical hardware bus )
]

# ================================================================================
# BLOCK 4: CLOSING BLOCK
# ================================================================================
Closing {
    verify: [
        byte_assertion = yes
    ],
    commit_status [ PROCEED- ] ( Byte silicon invariant sealed in PROCEED )
}
```

<TRACK 2> : {OmniCode Language & Standard Library}/lexicon/01_nouns/things/silicon/byte.tape

TAPE | 33 LINES | 204 WORDS | 1.5 KB

```
# ================================================================================
# TAPE HEAD: SILICON BYTE PROCESS RIBBON
# ================================================================================
# ARCHITECTURAL TOPOLOGY:
#   Executes atomic lifecycle for Invariant Noun <THING:SILICON:BYTE>.
# ================================================================================

[Tape.Byte] : WHO {OmniCode_Kernel} WHAT {OctetByte} --> MAP {TernaryWord}
    WHEN {SystemGenesis}
    WHERE {byte_slot} = [ 1 ]
    WHY ( Inscribe standard 8-bit octet invariant )
    HOW [ +1 : PROCEED ]
    :: < byte_landmark > ( Genesis 17:12 - 'Eight days old shall be circumcised' )

# ================================================================================
# SUBORDINATE ROLLS: ATOMIC INVARIANT RESOLUTION
# ================================================================================

# Roll 1: Reference abstract byte archetype
roll_1 [ +Byte ] ( Abstract byte archetype referenced )

# Roll 2: Allocate concrete byte slot
roll_2 [ Byte --> "Silicon_Byte" @ 0 ] ( Inscribe concrete byte slot at offset 0 )

# Roll 3: Assert structural projection and clarified octet
roll_3 [ [ BYTE ] [ 1 ] :: "8 bits" --> [ OCTET_BYTE ] ] ( Structural 1 byte bounds affirmed )

# Roll 4: Verify byte integrity into result prime
roll_4 [ "Silicon_Byte" = Byte ! [ +1 ] --> [ byte_verified` ] ] ( Verify byte matches archetype )

# Roll 5: Seal in affirmative proceed
roll_5 [ commit_status [ PROCEED- ] ] ( Byte silicon invariant sealed and verified )
```

<TRACK 2> : {OmniCode Language & Standard Library}/lexicon/01_nouns/things/silicon/cell.omni

OMNI | 61 LINES | 254 WORDS | 2.2 KB

```
# ================================================================================
# BLOCK 1: METADATA BLOCK
# ( Invariant Noun: <THING:SILICON:CELL> )
# ================================================================================
# ARCHITECTURAL TOPOLOGY:
#   Defines the atomic 5-trit / 8-bit quad-state physical memory vessel (1 byte).
#   - Base Block Type: < Idea : Type : Container > (Atomic 1-Byte Container)
#   - Category: Thing
#   - Subcategory: Silicon
#   - Footprint: 1 Physical Byte in RAM (243 values + 13 control states)
# ================================================================================

Module < Lexicon.Nouns.Things.Silicon.Cell > ( The Atomic 5-Trit / 8-Bit Quad-State Memory Vessel ) {
    version: < "1.0.0" >,
    system: < OmniCode.Lexicon >,
    semantics: < InvariantNoun :: Thing :: Silicon :: MemoryCell >
}

# Invariant Noun Declaration: <THING:SILICON:CELL>
< Thing : Silicon : Cell > [ capacity: 1 byte ] {
    base_type: < Idea : Type : Container >,
    physical_footprint: "1 byte in RAM (5 balanced trits)",
    capacity_trits: 5,
    projections: < {
        ontological: "< Cell >",
        structural: "[ CELL ]",
        agentic: "{ Cell }"
    } >,
    machine_effect: "dense_byte_trit_cell"
}

# ================================================================================
# BLOCK 2: OPENING BLOCK
# ================================================================================
Opening {
    requires: [
        Cell : InvariantNoun ::
        Container : InvariantNoun
    ],
    preconditions: [ +1 ]
}

# ================================================================================
# BLOCK 3: BODY BLOCK
# ================================================================================
Body [
    cell_assertion [
        [ CELL ] [ 1 ] :: { Cell } --> STORE {QuadState}
    ] ( Single atomic memory cell instantiated holding 5-trit payload )
]

# ================================================================================
# BLOCK 4: CLOSING BLOCK
# ================================================================================
Closing {
    verify: [
        cell_assertion = yes
    ],
    commit_status [ PROCEED- ] ( Cell silicon invariant sealed in PROCEED )
}
```

<TRACK 2> : {OmniCode Language & Standard Library}/lexicon/01_nouns/things/silicon/cell.tape

TAPE | 33 LINES | 209 WORDS | 1.5 KB

```
# ================================================================================
# TAPE HEAD: SILICON MEMORY CELL PROCESS RIBBON
# ================================================================================
# ARCHITECTURAL TOPOLOGY:
#   Executes atomic lifecycle for Invariant Noun <THING:SILICON:CELL>.
# ================================================================================

[Tape.Cell] : WHO {OmniCode_Kernel} WHAT {MemoryCell} --> ALLOC {ByteSlot}
    WHEN {SystemGenesis}
    WHERE {cell_capacity} = [ 1 ]
    WHY ( Inscribe atomic 5-trit memory cell invariant )
    HOW [ +1 : PROCEED ]
    :: < cell_landmark > ( Psalm 139:13 - 'Thou hast covered me in my mother\'s womb' )

# ================================================================================
# SUBORDINATE ROLLS: ATOMIC INVARIANT RESOLUTION
# ================================================================================

# Roll 1: Reference abstract cell archetype
roll_1 [ +Cell ] ( Abstract cell archetype referenced )

# Roll 2: Allocate concrete memory cell
roll_2 [ Cell --> "Silicon_Cell" @ 0 ] ( Inscribe concrete memory cell at offset 0 )

# Roll 3: Assert structural projection and clarified byte size
roll_3 [ [ CELL ] [ 1 ] :: "1 byte" --> [ BYTE_CELL ] ] ( Structural 1 cell bounds affirmed )

# Roll 4: Verify cell integrity into result prime
roll_4 [ "Silicon_Cell" = Cell ! [ +1 ] --> [ cell_verified` ] ] ( Verify cell matches archetype )

# Roll 5: Seal in affirmative proceed
roll_5 [ commit_status [ PROCEED- ] ] ( Memory cell invariant sealed and verified )
```

<TRACK 2> : {OmniCode Language & Standard Library}/lexicon/01_nouns/things/silicon/register.omni

OMNI | 62 LINES | 278 WORDS | 2.4 KB

```
# ================================================================================
# BLOCK 1: METADATA BLOCK
# ( Invariant Noun: <THING:SILICON:REGISTER> )
# ================================================================================
# ARCHITECTURAL TOPOLOGY:
#   Defines the physical hardware CPU register bank and Named Silicon Registers.
#   - Base Block Type: < Idea : Type : Container > (Silicon Hardware Bank)
#   - Category: Thing
#   - Subcategory: Silicon
#   - Bank: 13 Named Registers (HEAD, HOME, SPOOL, IT, SELF, CALLER, OTHER,
#                               STATE, TURN, BOUND, FROM, INTO, DISCARD)
# ================================================================================

Module < Lexicon.Nouns.Things.Silicon.Register > ( 13 Named Hardware CPU Silicon Registers ) {
    version: < "1.0.0" >,
    system: < OmniCode.Lexicon >,
    semantics: < InvariantNoun :: Thing :: Silicon :: RegisterBank >
}

# Invariant Noun Declaration: <THING:SILICON:REGISTER>
< Thing : Silicon : Register > [ capacity: 13 slots ] {
    base_type: < Idea : Type : Container >,
    bank_slots: 13,
    registers: [ HEAD, HOME, SPOOL, IT, SELF, CALLER, OTHER, STATE, TURN, BOUND, FROM, INTO, DISCARD ],
    projections: < {
        ontological: "< Register >",
        structural: "[ REGISTER ]",
        agentic: "{ Register }"
    } >,
    machine_effect: "hardware_cpu_register_bank"
}

# ================================================================================
# BLOCK 2: OPENING BLOCK
# ================================================================================
Opening {
    requires: [
        Register : InvariantNoun ::
        Container : InvariantNoun
    ],
    preconditions: [ +1 ]
}

# ================================================================================
# BLOCK 3: BODY BLOCK
# ================================================================================
Body [
    register_assertion [
        [ REGISTER ] [ HEAD HOME SPOOL IT SELF CALLER OTHER STATE TURN BOUND FROM INTO DISCARD ] :: { Register } --> ARM
{SiliconBank}
    ] ( 13 named silicon registers armed for direct hardware computation )
]

# ================================================================================
# BLOCK 4: CLOSING BLOCK
# ================================================================================
Closing {
    verify: [
        register_assertion = yes
    ],
    commit_status [ PROCEED- ] ( Register silicon invariant sealed in PROCEED )
}
```

<TRACK 2> : {OmniCode Language & Standard Library}/lexicon/01_nouns/things/silicon/register.tape

TAPE | 33 LINES | 215 WORDS | 1.6 KB

```
# ================================================================================
# TAPE HEAD: SILICON REGISTER BANK PROCESS RIBBON
# ================================================================================
# ARCHITECTURAL TOPOLOGY:
#   Executes atomic lifecycle for Invariant Noun <THING:SILICON:REGISTER>.
# ================================================================================

[Tape.Register] : WHO {OmniCode_Kernel} WHAT {RegisterBank} --> ARM {SiliconRegisters}
    WHEN {SystemGenesis}
    WHERE {register_bank} = [ 13 ]
    WHY ( Inscribe 13 named silicon hardware CPU registers )
    HOW [ +1 : PROCEED ]
    :: < register_landmark > ( Exodus 28:21 - 'Twelve stones according to the names' )

# ================================================================================
# SUBORDINATE ROLLS: ATOMIC INVARIANT RESOLUTION
# ================================================================================

# Roll 1: Reference abstract register archetype
roll_1 [ +Register ] ( Abstract register bank archetype referenced )

# Roll 2: Allocate concrete register bank
roll_2 [ Register --> "Register_Bank" @ 0 ] ( Inscribe register bank in silicon partition )

# Roll 3: Assert structural projection and arm 13 named registers
roll_3 [ ARM { Registers [ HEAD HOME SPOOL IT SELF CALLER OTHER STATE TURN BOUND FROM INTO DISCARD ] } ] ( 13 registers armed )

# Roll 4: Verify register bank integrity into result prime
roll_4 [ "Register_Bank" = Register ! [ +1 ] --> [ registers_armed` ] ] ( Verify register bank matches archetype )

# Roll 5: Seal in affirmative proceed
roll_5 [ commit_status [ PROCEED- ] ] ( Silicon register invariant sealed and verified )
```

<TRACK 2> : {OmniCode Language & Standard Library}/lexicon/01_nouns/things/silicon/trit.omni

OMNI | 60 LINES | 246 WORDS | 2.1 KB

```
# ================================================================================
# BLOCK 1: METADATA BLOCK
# ( Invariant Noun: <THING:SILICON:TRIT> )
# ================================================================================
# ARCHITECTURAL TOPOLOGY:
#   Defines the atomic balanced ternary radix-3 polarity unit (-1, 0, +1).
#   - Base Block Type: < Idea : Type : State > (Radix-3 Polarity State)
#   - Category: Thing
#   - Subcategory: Silicon
#   - Values: -1 (Negative / Rollback), 0 (Neutral / Await), +1 (Positive / Proceed)
# ================================================================================

Module < Lexicon.Nouns.Things.Silicon.Trit > ( The Atomic Radix-3 Polarity Unit ) {
    version: < "1.0.0" >,
    system: < OmniCode.Lexicon >,
    semantics: < InvariantNoun :: Thing :: Silicon :: Radix3Polarity >
}

# Invariant Noun Declaration: <THING:SILICON:TRIT>
< Thing : Silicon : Trit > [ capacity: 1 trit ] {
    base_type: < Idea : Type : State >,
    permitted_states: [ -1, 0, 1 ],
    projections: < {
        ontological: "< Trit >",
        structural: "[ TRIT ]",
        agentic: "{ Trit }"
    } >,
    machine_effect: "radix3_polarity_register"
}

# ================================================================================
# BLOCK 2: OPENING BLOCK
# ================================================================================
Opening {
    requires: [
        Trit : InvariantNoun ::
        State : InvariantNoun
    ],
    preconditions: [ +1 ]
}

# ================================================================================
# BLOCK 3: BODY BLOCK
# ================================================================================
Body [
    trit_assertion [
        [ TRIT ] [ 1 ] :: { Trit } --> POLARIZE {BalancedState}
    ] ( Atomic trit asserted with 3-state balanced polarity )
]

# ================================================================================
# BLOCK 4: CLOSING BLOCK
# ================================================================================
Closing {
    verify: [
        trit_assertion = yes
    ],
    commit_status [ PROCEED- ] ( Trit silicon invariant sealed in PROCEED )
}
```

<TRACK 2> : {OmniCode Language & Standard Library}/lexicon/01_nouns/things/silicon/trit.tape

TAPE | 33 LINES | 209 WORDS | 1.5 KB

```
# ================================================================================
# TAPE HEAD: SILICON TRIT PROCESS RIBBON
# ================================================================================
# ARCHITECTURAL TOPOLOGY:
#   Executes atomic lifecycle for Invariant Noun <THING:SILICON:TRIT>.
# ================================================================================

[Tape.Trit] : WHO {OmniCode_Kernel} WHAT {PolarityTrit} --> BALANCE {TernaryRadix}
    WHEN {SystemGenesis}
    WHERE {trit_slot} = [ 1 ]
    WHY ( Inscribe atomic radix-3 trit polarity invariant )
    HOW [ +1 : PROCEED ]
    :: < trit_landmark > ( Matthew 5:37 - 'Let your communication be, Yea, yea; Nay, nay' )

# ================================================================================
# SUBORDINATE ROLLS: ATOMIC INVARIANT RESOLUTION
# ================================================================================

# Roll 1: Reference abstract trit archetype
roll_1 [ +Trit ] ( Abstract trit archetype referenced )

# Roll 2: Allocate concrete trit register
roll_2 [ Trit --> "Silicon_Trit" @ 0 ] ( Inscribe concrete trit register at offset 0 )

# Roll 3: Assert structural projection and prove equilibrium
roll_3 [ [ TRIT ] [ 1 ] :: (+1) plus (-1) <=> 0 ] ( Structural 1 trit bounds and ternary balance proved )

# Roll 4: Verify trit integrity into result prime
roll_4 [ "Silicon_Trit" = Trit ! [ +1 ] --> [ trit_verified` ] ] ( Verify trit matches archetype )

# Roll 5: Seal in affirmative proceed
roll_5 [ commit_status [ PROCEED- ] ] ( Trit silicon invariant sealed and verified )
```

<TRACK 2> : {OmniCode Language & Standard Library}/lexicon/01_nouns/ideas/axioms/landmark.omni

OMNI | 60 LINES | 242 WORDS | 2.2 KB

```
# ================================================================================
# BLOCK 1: METADATA BLOCK
# ( Invariant Noun: <IDEA:AXIOM:LANDMARK> )
# ================================================================================
# ARCHITECTURAL TOPOLOGY:
#   Defines the ancient unmovable landmark invariant and foundational bedrock.
#   - Base Block Type: < Idea : Type : Site > (Unmovable Bedrock Locus)
#   - Category: Idea
#   - Subcategory: Axiom
#   - Invariant Witness: Proverbs 22:28
# ================================================================================

Module < Lexicon.Nouns.Ideas.Axioms.Landmark > ( Ancient Unmovable Architectural Landmark ) {
    version: < "1.0.0" >,
    system: < OmniCode.Lexicon >,
    semantics: < InvariantNoun :: Idea :: Axiom :: AncientLandmark >
}

# Invariant Noun Declaration: <IDEA:AXIOM:LANDMARK>
< Idea : Axiom : Landmark > [ capacity: 1 site ] {
    base_type: < Idea : Type : Site >,
    projections: < {
        ontological: "< Landmark >",
        structural: "[ LANDMARK ]",
        agentic: "{ Landmark }"
    } >,
    scriptural_witness: "Proverbs 22:28 - 'Remove not the ancient landmark, which thy fathers have set'",
    machine_effect: "unmovable_boundary_sentinel"
}

# ================================================================================
# BLOCK 2: OPENING BLOCK
# ================================================================================
Opening {
    requires: [
        Landmark : InvariantNoun ::
        Site : InvariantNoun
    ],
    preconditions: [ +1 ]
}

# ================================================================================
# BLOCK 3: BODY BLOCK
# ================================================================================
Body [
    landmark_assertion [
        [ LANDMARK ] [ 1 ] :: { Landmark } --> ESTABLISH {AncientBoundary}
    ] ( Ancient landmark established as unmovable boundary sentinel )
]

# ================================================================================
# BLOCK 4: CLOSING BLOCK
# ================================================================================
Closing {
    verify: [
        landmark_assertion = yes
    ],
    commit_status [ PROCEED- ] ( Landmark axiom invariant sealed in PROCEED )
}
```

<TRACK 2> : {OmniCode Language & Standard Library}/lexicon/01_nouns/ideas/axioms/landmark.tape

TAPE | 33 LINES | 202 WORDS | 1.6 KB

```
# ================================================================================
# TAPE HEAD: ANCIENT LANDMARK PROCESS RIBBON
# ================================================================================
# ARCHITECTURAL TOPOLOGY:
#   Executes atomic lifecycle for Invariant Noun <IDEA:AXIOM:LANDMARK>.
# ================================================================================

[Tape.Landmark] : WHO {OmniCode_Kernel} WHAT {AncientLandmark} --> ESTABLISH {FoundationalBedrock}
    WHEN {SystemGenesis}
    WHERE {landmark_slot} = [ 1 ]
    WHY ( Inscribe unmovable ancient landmark invariant )
    HOW [ +1 : PROCEED ]
    :: < ancient_landmark > ( Proverbs 22:28 - 'Remove not the ancient landmark' )

# ================================================================================
# SUBORDINATE ROLLS: ATOMIC INVARIANT RESOLUTION
# ================================================================================

# Roll 1: Reference abstract landmark archetype
roll_1 [ +Landmark ] ( Abstract landmark archetype referenced )

# Roll 2: Allocate concrete landmark anchor
roll_2 [ Landmark --> "Ancient_Landmark" @ 0 ] ( Inscribe ancient landmark at offset 0 )

# Roll 3: Assert structural projection and clarified capacity
roll_3 [ [ LANDMARK ] [ 1 ] :: "1 site" --> [ ANCIENT_LANDMARK ] ] ( Structural 1 cell bounds affirmed )

# Roll 4: Verify landmark unmovability into result prime
roll_4 [ "Ancient_Landmark" = Landmark ! [ +1 ] --> [ landmark_verified` ] ] ( Verify landmark matches archetype )

# Roll 5: Seal in affirmative proceed
roll_5 [ commit_status [ PROCEED- ] ] ( Ancient landmark invariant sealed and verified )
```

<TRACK 2> : {OmniCode Language & Standard Library}/lexicon/01_nouns/ideas/axioms/protocol.omni

OMNI | 59 LINES | 230 WORDS | 2.1 KB

```
# ================================================================================
# BLOCK 1: METADATA BLOCK
# ( Invariant Noun: <IDEA:AXIOM:PROTOCOL> )
# ================================================================================
# ARCHITECTURAL TOPOLOGY:
#   Defines the universal exchange protocol standard and message covenant.
#   - Base Block Type: < Idea : Type : State > (Standardized Interface State)
#   - Category: Idea
#   - Subcategory: Axiom
#   - Protocol Standard: Universal Exchange Interface
# ================================================================================

Module < Lexicon.Nouns.Ideas.Axioms.Protocol > ( Universal Machine Exchange Protocol Standard ) {
    version: < "1.0.0" >,
    system: < OmniCode.Lexicon >,
    semantics: < InvariantNoun :: Idea :: Axiom :: MachineProtocol >
}

# Invariant Noun Declaration: <IDEA:AXIOM:PROTOCOL>
< Idea : Axiom : Protocol > [ capacity: 1 state ] {
    base_type: < Idea : Type : State >,
    projections: < {
        ontological: "< Protocol >",
        structural: "[ PROTOCOL ]",
        agentic: "{ Protocol }"
    } >,
    machine_effect: "universal_exchange_protocol"
}

# ================================================================================
# BLOCK 2: OPENING BLOCK
# ================================================================================
Opening {
    requires: [
        Protocol : InvariantNoun ::
        State : InvariantNoun
    ],
    preconditions: [ +1 ]
}

# ================================================================================
# BLOCK 3: BODY BLOCK
# ================================================================================
Body [
    protocol_assertion [
        [ PROTOCOL ] [ 1 ] :: { Protocol } --> STANDARDIZE {Interface}
    ] ( Universal exchange protocol established across machine channels )
]

# ================================================================================
# BLOCK 4: CLOSING BLOCK
# ================================================================================
Closing {
    verify: [
        protocol_assertion = yes
    ],
    commit_status [ PROCEED- ] ( Protocol idea invariant sealed in PROCEED )
}
```

<TRACK 2> : {OmniCode Language & Standard Library}/lexicon/01_nouns/ideas/axioms/protocol.tape

TAPE | 33 LINES | 207 WORDS | 1.6 KB

```
# ================================================================================
# TAPE HEAD: PROTOCOL STANDARD PROCESS RIBBON
# ================================================================================
# ARCHITECTURAL TOPOLOGY:
#   Executes atomic lifecycle for Invariant Noun <IDEA:AXIOM:PROTOCOL>.
# ================================================================================

[Tape.Protocol] : WHO {OmniCode_Kernel} WHAT {MachineProtocol} --> STANDARDIZE {MessageEnvelope}
    WHEN {SystemGenesis}
    WHERE {protocol_slot} = [ 1 ]
    WHY ( Inscribe universal exchange protocol invariant )
    HOW [ +1 : PROCEED ]
    :: < protocol_landmark > ( 1 Corinthians 14:40 - 'Let all things be done decently and in order' )

# ================================================================================
# SUBORDINATE ROLLS: ATOMIC INVARIANT RESOLUTION
# ================================================================================

# Roll 1: Reference abstract protocol archetype
roll_1 [ +Protocol ] ( Abstract protocol archetype referenced )

# Roll 2: Allocate concrete protocol standard
roll_2 [ Protocol --> "Message_Protocol" @ 0 ] ( Inscribe message protocol at offset 0 )

# Roll 3: Assert structural projection and clarified capacity
roll_3 [ [ PROTOCOL ] [ 1 ] :: "1 state" --> [ MESSAGE_ENVELOPE ] ] ( Structural 1 cell bounds affirmed )

# Roll 4: Verify protocol standard into result prime
roll_4 [ "Message_Protocol" = Protocol ! [ +1 ] --> [ protocol_verified` ] ] ( Verify protocol matches archetype )

# Roll 5: Seal in affirmative proceed
roll_5 [ commit_status [ PROCEED- ] ] ( Universal protocol invariant sealed and verified )
```

<TRACK 2> : {OmniCode Language & Standard Library}/lexicon/01_nouns/ideas/types/character.omni

OMNI | 181 LINES | 1716 WORDS | 11.3 KB

```
# ========================================================================
# BLOCK 1: METADATA BLOCK
# ( Invariant Type: < Idea { Type } : Character > )
# ========================================================================
# ARCHITECTURAL TOPOLOGY:
#   Defines the atomic letter, glyph, and inscription rune.
#   - Base Block: Primitive Invariant Type
#   - Category: Idea { Type }
#   - Footprint: [ 1 Cell ] { 5 trits / 8 bits }
#   - Invariant Law: { Character } = [ Enclosed ]: TRUE (Strict single-cell capacity)
# ========================================================================

= [Module]: < Lexicon { Types of Ideas } > & [ Noun { Character } ] ( This is the Module for the Lexicon - Ideas (Type): Characters - Nouns ) {
    [ Version ]:   { "1.0.0" } ( The Version is in the context of 1.0.0 ),
    [ Created ]:   { "2026-10-11" } ( Created in the context of October 11, 2026 ),
    [ Updated ]:   { "2026-10-13" } ( Last updated in the context of October 13, 2026 ),
    [ Author ]:    { "CreativeWorkzStudio LLC / ProfessorSeanEX" } ( Author in the context of CreativeWorkzStudio LLC and ProfessorSeanEX ),
    [ System ]:    { OmniCode { Lexicon } } ( System is in the context of the OmniCode Lexicon ),
    [ Witness ]:   < Scripture { Exodus 31:18 KJV } > { "And he gave unto Moses, when he had made an end of communing with him upon mount Sinai, two tables of testimony, tables of stone, written with the finger of God." } ( Invariant Scripture Witness Exodus 31:18 in the context: Written with the finger of God ),
    [ Nature ]:    < InvariantType > & { BaseBlock } ( Nature is an Invariant Type and is in the context of Base Block ),
    [ Semantics ]: { AtomicGlyph :: LetterRune } ( Semantics in the context of Atomic Glyph and Letter Rune )
}

# Freestanding Invariant Type Definition
< Character > = < Idea { Type } > & [ Noun { Atomic Glyph } ] ( Character is the Invariant Idea of Type, and is the Noun of Atomic Glyph ) {
    # 1. The Ontological Analogy (Character's proportion to the universe)
    [ Relation ]: [ Character ] : [ Text ] :: [ Trit ] : [ Word ] ( Character is to Text as Trit is to Word ),

    # 2. The Universal Domain (All characters, letters, digits, and symbols)
    [ Domain ]: { Glyph :: Letter :: Digit :: Symbol } ( Domain encompasses all atomic glyphs, letters, digits, and symbols ),

    # 3. The Dual Substrate (Physical Bit and Virtual Trit)
    [ Substrate ]: [ 1 Byte { 8 Bits } ] & [ 1 Cell { 5 Trits } ] ( Substrate is 1 Byte of 8 Bits, and is 1 Cell of 5 Trits ),

    # 4. The Universal Inscription Vector (How any character is formed)
    [ Inscribe ]: [ Glyph ] --> [ Character ] ( Any atomic glyph rune is inscribed into the Character cell ),

    # 5. The Hardware Transduction Law (Bijective mapping between byte and trits)
    [ Transduce ]: [ Character { 8 Bits } ] <=> [ Cell { 5 Trits } ] ( Byte values map bijectively into balanced trits within the cell ),

    # 6. The Canonical Exemplar (Standard grounding for letter 'A': -1 + 3 + 9 - 27 + 81 = 65)
    [ Exemplar { 'A' } ]: 65 <=> [ -1, +1, +1, -1, +1 ] ( Canonical exemplar letter 'A' at byte 65 maps to balanced five trits ),

    # 7. The Relational Comparison (How characters evaluate equality against a glyph)
    [ Compare ]: [ { Character } = { Glyph } ] ( Character is evaluated for equality against an input glyph rune ),

    # 8. The Sequence Transition (How characters build words)
    [ Sequence ]: [ Character ] --> [ Text ] ( Discrete Character cells chain together into continuous Text ribbons ),

    # 9. The Invariant Boundary (Strict single-cell enclosure)
    [ Bound ]: { Character } = [ Enclosed ]: TRUE ( Character is strictly enclosed within a single cell capacity )
}

# ========================================================================
# BLOCK 2: OPENING BLOCK
# ========================================================================
Opening {
    # 1. The Dynamic Input Payload: Rank 5 Outside-In Imposition capturing external glyph, exemplar 'A' fallback
    { Glyph ]: { Input :: 'A' } ( Outside-in imposition: glyph payload captured from external stream into discrete entity ),

    # 2. The Typed Entity: Character brought down from Block 1 into concrete allocation
    [ Character ]: < Character > & [ Cell ] ( Denoted Character is typed as invariant Character allocated in the Cell ),

```

```
    # 3. The Physical Container: The atomic memory cell
    [ Cell ]: [ 1 ] & { 5 Trits :: 1 Byte } ( A concrete 1-cell container in the context of 5 Trits and 1 Byte ),

    # 4. The Hardware Bridge: Arming the bit-to-trit transducer
    [ Transducer ]: { 8 Bits <=> 5 Trits } ( The Transducer bridges 8 bits of silicon to 5 trits of virtual memory ),

    # 5. The Output Target: The hardware destination (Terminal stream or bare-metal VGA)
    [ Target ]: { [ Terminal Stdout ] ::@ 0x01 | [ Screen VGA ] ::@ 0x000B8000 }
        ( Target output projects to terminal stdout in hosted OS or VGA text buffer in bare metal ),

    # 6. Preconditions & State: Proving balance before execution
    [ Preconditions ]: [ (+1) plus (-1) <=> 0 ] & [ Status = +1:PROCEED ] ( ALU equilibrium holds and the machine status
is armed in Proceed )
}

# ================================================================================
# BLOCK 3: BODY BLOCK
# ================================================================================
Body [
    # --------------------------------------------------------------------------
    # ROLL 1: GLYPH INSCRIPTION (RANK 5 OUTSIDE-IN IMPOSITION)
    # Inscribe incoming glyph payload into the Cell container, establishing new state Character`
    # --------------------------------------------------------------------------
    [ ROLL 1 ] : [ Inscribe ] : { Glyph ] --> INSCRIBE [ Cell ] = [ Character` ]
        ( Inscribe external glyph imposed into Cell yielding new state Character-prime ),

    # --------------------------------------------------------------------------
    # ROLL 2: DUAL-SUBSTRATE TRANSDUCTION (<=>)
    # Transduce Character` byte dynamically through the hardware transducer into 5 balanced trits
    # --------------------------------------------------------------------------
    [ ROLL 2 ] : [ Transduce ] : [ Transducer ]: [ Character` { 8 Bits } ] <=> TRANSDUCE [ Cell` { 5 Trits } ]
        ( Transduce Character-prime byte through hardware transducer into five balanced trits ),

    # --------------------------------------------------------------------------
    # ROLL 3: IDENTITY & BOUND COMPARISON (ADVERSATIVE CAVEAT VIA `!`)
    # Compare Character` to Glyph within single cell bounds: proceed on equilibrium, rollback on mismatch
    # --------------------------------------------------------------------------
    [ ROLL 3 ] : [ Compare ] :
        { Character` = Glyph } & { Character` <= [ 1 Cell ] }
        ! GUARD --> [ Status` = +1:PROCEED ]
        | { Character` != Glyph } --> [ Status` = -1:ROLLBACK ]
        ( Compare Character-prime to glyph and verify single-cell bounds under guard: proceed on equilibrium, rollback on
error ),

    # --------------------------------------------------------------------------
    # ROLL 4: HARDWARE TARGET PROJECTION (RANK 4 INSIDE-OUT EMERGENCE)
    # Project Character` to the target buffer (Terminal stdout or VGA screen)
    # --------------------------------------------------------------------------
    [ ROLL 4 ] : [ Project ] : [ Character` } --> PROJECT [ Target ] = [ Target` }
        ( Project internal Character-prime emerging outward into Target buffer yielding new screen state Target-prime ),

    # --------------------------------------------------------------------------
    # ROLL 5: SEQUENCE TRANSITION (RANK 4 INSIDE-OUT EMERGENCE)
    # Yield Character` to the continuous Text ribbon sequence
    # --------------------------------------------------------------------------
    [ ROLL 5 ] : [ Sequence ] : [ Character` } --> SEQUENCE [ Text ] = [ Text` ]
        ( Yield discrete Character-prime emerging outward to continuous Text sequence establishing new state Text-prime )
]

# ================================================================================
# BLOCK 4: CLOSING BLOCK
# ================================================================================
Closing ( Sealing Gate: Reconcile, validate, tripwire, debug, log, and affirm permanent execution ) {
    # --------------------------------------------------------------------------
    # 1. VERIFICATION OF OPERATIONAL ROLLS
    # --------------------------------------------------------------------------
    [ Verify ]: [
        [ ROLL 1 ] = PROCEED ::
        [ ROLL 2 ] = PROCEED ::
        [ ROLL 3 ] = PROCEED ::
        [ ROLL 4 ] = PROCEED ::
        [ ROLL 5 ] = PROCEED
    ] ( Verify that all five operational rolls achieved affirmative consensus in PROCEED ),

    # --------------------------------------------------------------------------
    # 2. VALIDATION OF INVARIANT BOUNDS & PARITY
    # --------------------------------------------------------------------------
    [ Validate ]: { Character` <= [ 1 Cell ] } & { [ 8 Bits ] <=> [ 5 Trits ] }
```

```
        ( Validate that Character-prime remains strictly enclosed within 1 Cell and dual substrate parity holds ),

    # ------------------------------------------------------------------------
    # 3. ZERO-TOLERANCE TRIPWIRING
    # ------------------------------------------------------------------------
    [ Tripwire ]: { Status` = +1:PROCEED } --> [ ARMED ] | { Status` != +1:PROCEED } --> [ TRIPPED ] --> [ Commit Status
= -1:ROLLBACK ]
        ( Tripwire remains armed in Proceed or trips on anomaly to veto execution ),

    # ------------------------------------------------------------------------
    # 4. DIAGNOSTIC DEBUG TELEMETRY
    # ------------------------------------------------------------------------
    [ Debug ]: { [ Glyph ] :: [ Character` ] :: [ Cell` ] :: [ Target` ] :: [ Status` ] }
        ( Expose diagnostic telemetry across input glyph, cell, target buffer, and machine status ),

    # ------------------------------------------------------------------------
    # 5. EPISODIC INVENTORY LOGGING
    # ------------------------------------------------------------------------
    [ Log ]: [ Journal ]: INSCRIBE { [ Character Inscribed ] :: [ Character` ] :: [ Target ] }
        ( Inscribe the verified character transaction into the permanent episodic system journal ),

    # ------------------------------------------------------------------------
    # 6. PERMANENT COVENANT EXECUTION & FOURFOLD COMMIT STATUS LOOP
    # ------------------------------------------------------------------------
    [ Commit Status ]: [
        # +1: PROCEED (+t: Advance forward along the causal tape)
        { [ Verify ] = PROCEED } & { [ Tripwire ] = ARMED }
            --> [ +1:PROCEED ] |

        #  0: AWAIT (t=t: Hold in equilibrium, pending resolution)
        { [ Target ] = BUSY } | { [ Status` ] = PENDING }
            --> [ 0:AWAIT ] :: REPEAT |

        # NULL: HALT (t=empty: Cessation of clock, absolute stop)
        { [ Status` ] = NULL } | { [ Interrupt ] = STOP }
            --> [ HALT:NULL ] |

        # -1: ROLLBACK (-t: Reverse causal tape to origin t=0)
        { [ Tripwire ] = TRIPPED } | { [ Verify ] != PROCEED }
            --> [ -1:ROLLBACK ]
    ] ( Fourfold state machine: Proceed advances forward, Await holds equilibrium, Halt ceases execution in Null,
Rollback reverts to origin )
}
```

<TRACK 2> : {OmniCode Language & Standard Library}/lexicon/01_nouns/ideas/types/character.tape

TAPE | 84 LINES | 705 WORDS | 5.0 KB

```
# ================================================================================
# PROCESS TAPE: CHARACTER INVARIANT TYPE EXECUTION RIBBON
# ================================================================================
# ARCHITECTURAL TOPOLOGY:
#   Interprets and executes the atomic lifecycle for Invariant Type < Character >.
#   - Defined Space: [ 1 Cell ] { 5 Trits :: 1 Byte }
#   - Execution Domain: Lexicon { Types of Ideas }
#   - Witness: Scripture Exodus 31:18 KJV
# ================================================================================

= [Tape]: < Lexicon { Types of Ideas } > & [ Process { Character } ] ( Process Ribbon for Character Inscription ) {
    WHO   { OmniCode Lexicon } ::
    WHAT  { Character } --> INSCRIBE { Glyph } ::
    WHEN  { Input Event } ::
    WHERE { Character Cell } = [ 1 ] ::
    WHY   ( Inscribe atomic character type invariant into defined space ) ::
    HOW   [ +1 : PROCEED ] ::
    < Scripture { Exodus 31:18 KJV } > { "And he gave unto Moses, when he had made an end of communing with him upon mount Sinai, two tables of testimony, tables of stone, written with the finger of God." } ( Invariant Scripture Witness Exodus 31:18 in the context: Written with the finger of God )
}

# ================================================================================
# DECLARED ROLLS: ATOMIC INVARIANT EXECUTION
# ================================================================================

# --------------------------------------------------------------------------------
# ROLL 1: GLYPH INSCRIPTION (RANK 5 OUTSIDE-IN ASYMMETRIC IMPOSITION)
# Inscribe external glyph payload into the defined Cell container
# --------------------------------------------------------------------------------
[ ROLL 1 ] : [ Inscribe ] : { Glyph } --> INSCRIBE [ Cell ] = [ Character` ]
    ( Inscribe external glyph imposed into Cell yielding new state Character-prime )

# --------------------------------------------------------------------------------
# ROLL 2: DUAL-SUBSTRATE TRANSDUCTION (<=>)
# Transduce byte representation into balanced ternary virtual memory cell
# --------------------------------------------------------------------------------
[ ROLL 2 ] : [ Transduce ] : [ Transducer ]: [ Character` { 8 Bits } ] <=> TRANSDUCE [ Cell` { 5 Trits } ]
    ( Transduce Character-prime byte through hardware transducer into five balanced trits )

# --------------------------------------------------------------------------------
# ROLL 3: IDENTITY & BOUND COMPARISON (ADVERSATIVE CAVEAT VIA `!`)
# Verify identity against glyph and enclosure within single-cell bounds
# --------------------------------------------------------------------------------
[ ROLL 3 ] : [ Compare ] :
    { Character` = Glyph } & { Character` <= [ 1 Cell ] }
    ! GUARD --> [ Status` = +1:PROCEED ]
    | { Character` != Glyph } --> [ Status` = -1:ROLLBACK ]
    ( Compare Character-prime to glyph and verify single-cell bounds under guard: proceed on equilibrium, rollback on error )

# --------------------------------------------------------------------------------
# ROLL 4: TARGET PROJECTION (RANK 4 INSIDE-OUT ASYMMETRIC EMERGENCE)
# Project internal character cell outward into target display universe
# --------------------------------------------------------------------------------
[ ROLL 4 ] : [ Project ] : [ Character` } --> PROJECT [ Target ] = [ Target` }
    ( Project internal Character-prime emerging outward into target buffer yielding new state Target-prime )

# --------------------------------------------------------------------------------
# ROLL 5: SEQUENCE TRANSITION (RANK 4 INSIDE-OUT ASYMMETRIC EMERGENCE)
# Chain discrete character cell outward into continuous text sequence
# --------------------------------------------------------------------------------
[ ROLL 5 ] : [ Sequence ] : [ Character` } --> SEQUENCE [ Text ] = [ Text` ]
    ( Yield discrete Character-prime emerging outward into continuous Text sequence establishing new state Text-prime )

# --------------------------------------------------------------------------------
# ROLL 6: COMMIT STATUS & FOURFOLD STATE MACHINE
# Affirm covenant execution across causal time advancement, await, halt, and rollback
# --------------------------------------------------------------------------------
[ ROLL 6 ] : [ Commit Status ] : [
    # +1: PROCEED (+t: Advance forward along the causal tape)
    { [ Verify ] = PROCEED } & { [ Tripwire ] = ARMED }
        --> [ +1:PROCEED ] |
```

```
    71
    72      #  0: AWAIT (t=t: Hold in equilibrium, pending resolution)
    73      { [ Target ] = BUSY } | { [ Status` ] = PENDING }
    74          --> [ 0:AWAIT ] :: REPEAT |
    75
    76      # NULL: HALT (t=empty: Cessation of clock, absolute stop)
    77      { [ Status` ] = NULL } | { [ Interrupt ] = STOP }
    78          --> [ HALT:NULL ] |
    79
    80      # -1: ROLLBACK (-t: Reverse causal tape to origin t=0)
    81      { [ Tripwire ] = TRIPPED } | { [ Verify ] != PROCEED }
    82          --> [ -1:ROLLBACK ]
    83  ] ( Fourfold commit status: Proceed advances forward, Await holds in equilibrium, Halt ceases execution in Null, Rollback
reverts to origin )
```

<TRACK 2> : {OmniCode Language & Standard Library}/lexicon/01_nouns/ideas/types/container.omni

OMNI | 192 LINES | 1812 WORDS | 12.3 KB

```
# ================================================================================
# BLOCK 1: METADATA BLOCK
# ( Invariant Type: < Idea { Type } : Container > )
# ================================================================================
# ARCHITECTURAL TOPOLOGY:
#   Defines bounded memory envelopes, arrays, structures, and storage vessels.
#   - Base Block: Primitive Invariant Type
#   - Category: Idea { Type }
#   - Footprint: [ 64 Cells ] { 64 Bytes / Dynamically Bounded Memory Space }
#   - Ladder Ascension: Build-up with Numbers (/-^)
# ================================================================================

= [Module]: < Lexicon { Types of Ideas } > & [ Noun { Container } ] ( This is the Module for the Lexicon - Ideas (Type): Containers - Nouns ) {
    [ Version ]:   { "1.0.0" } ( The Version is in the context of 1.0.0 ),
    [ Created ]:   { "2026-10-11" } ( Created in the context of October 11, 2026 ),
    [ Updated ]:   { "2026-10-12" } ( Last updated in the context of October 12, 2026 ),
    [ Author ]:    { "CreativeWorkzStudio LLC / ProfessorSeanEX" } ( Author in the context of CreativeWorkzStudio LLC and ProfessorSeanEX ),
    [ System ]:    { OmniCode { Lexicon } } ( System is in the context of the OmniCode Lexicon ),
    [ Witness ]:   < Scripture { 2 Kings 4:4 KJV } > { "And when thou art come in, thou shalt shut the door upon thee and upon thy sons, and shalt pour out into all those vessels, and thou shalt set aside that which is full." } ( Invariant Scripture Witness 2 Kings 4:4 in the context: Pour out into all those vessels, and set aside that which is full ),
    [ Nature ]:    < InvariantType > & { BaseBlock } ( Nature is an Invariant Type and is in the context of Base Block ),
    [ Semantics ]: { BoundedEnvelope :: StorageVessel :: MemoryCapsule } ( Semantics in the context of Bounded Envelope, Storage Vessel, and Memory Capsule )
}

# --------------------------------------------------------------------------------
# LADDER ASCENSION: BUILD-UP WITH NUMBERS
# A Container cannot exist without numbers: capacity, indices, bounds, and cells
# are constructed out of scalar quantities passed up the ontological ladder.
# --------------------------------------------------------------------------------
/-^ < Numbers > FROM [ < Lexicon { Types of Ideas } > & [ Noun { Number } ] ] ( Container builds up its boundaries, capacity, and cells using Numbers from the Lexicon )

# Freestanding Invariant Type Definition
< Container > = < Idea { Type } > & [ Noun { Bounded Space } ] ( Container is the Invariant Idea of Type, and is the Noun of Bounded Space ) {
    # 1. The Ontological Analogy (Container's proportion to the universe)
    [ Relation ]: [ Container ] : [ Vessel ] :: [ Boundary ] : [ Capacity ] ( Container is to Vessel as Boundary is to Capacity ),

    # 2. The Universal Domain (All vessels, storehouses, partitions, and memory capsules)
    [ Domain ]: { Vessel :: Storehouse :: Partition :: Capsule } ( Domain encompasses vessels, storehouses, partitions, and memory capsules ),

    # 3. The Physical Substrate (Concrete bounded capacity in silicon)
    [ Substrate ]: [ 64 Cells { 64 Bytes } ] & [ Bounded Space ] ( Substrate is 64 Cells of 64 Bytes, and is Bounded Space ),

    # 4. The Universal Inscription Vector (How external payload is encapsulated into memory)
    [ Inscribe ]: [ Payload ] --> [ Container ] ( Any memory payload is inscribed into the bounded Container envelope ),

    # 5. The Hardware Enclosure Law (Memory is physically bounded by allocated capacity)
    [ Enclose ]: ENCLOSE [ Storage ] WITHIN [ Capacity ] ( Storage is strictly enclosed within the declared capacity ),

    # 6. The Core Containment Law (Occupancy must never exceed capacity limit)
    [ Law { Containment } ]: { Occupancy <= Capacity } ( The law of containment requires that occupancy never exceeds bounded capacity ),

    # 7. Canonical Exemplars (Standard grounding values for partitions, vessels, and storehouses)
    [ Exemplar { Unit } ]: [ 1 ] <=> [ Partition { 1 Cell } ] ( Canonical single unit partition ),
    [ Exemplar { Vessel } ]: [ 64 ] <=> [ Vessel { 64 Cells } ] ( Canonical vessel exemplar bounded at 64 memory cells ),
    [ Exemplar { Storehouse } ]: [ 4096 ] <=> [ Storehouse { 4096 Cells } ] ( Canonical storehouse exemplar bounded at 4096 memory cells ),

    # 8. The Relational Comparison (How containers evaluate occupancy against capacity limit)
    [ Compare ]: [ { Occupancy } <= { Capacity } ] ( Container evaluates occupancy for containment within capacity limit ),

    # 9. The Sequence Transition (How containers chain into composite data structures)
    [ Sequence ]: [ Container ] --> [ Structure ] ( Discrete containers chain together into composite memory structures
```

```
),

    # 10. The Invariant Boundary (Strict capacity enclosure)
    [ Bound ]: ENCLOSE [ Storage ] WITHIN [ Capacity ] ( Container strictly encloses its storage within capacity )
}

# ================================================================================
# BLOCK 2: OPENING BLOCK
# ================================================================================
Opening {
    # 1. The Dynamic Input Payload: Accepts incoming memory payload
    { Payload ]: { Input :: [ 64 Bytes ] } ( Outside-in imposition: external memory payload to be encapsulated ),

    # 2. The Typed Entity: Container brought down from Block 1 into concrete allocation
    [ Container ]: < Container > & [ Vessel ] ( Denoted Container is typed as invariant Container allocated as a Vessel
),

    # 3. The Physical Envelope: The bounded space built up with numbers
    [ Bounded Space ]: [ 64 Cells ] & { Capacity = 64 } ( A concrete 64-cell bounded envelope built up with Numbers ),

    # 4. The Encapsulation Guard: Verifying containment limits
    [ Guard ]: { Occupancy <= Capacity } & { Door = SHUT } ( The guard enforces bounds limit and shuts the door when full
),

    # 5. The Output Target: Physical memory offset location
    [ Target ]: [ Storage Envelope ] ::@ 0x00000000 ( Target output is situated at the base storage envelope memory
offset ),

    # 6. Preconditions & State: Proving room before filling
    [ Preconditions ]: [ Capacity > 0 ] & [ Status = +1:PROCEED ] ( Capacity is positive and the machine status is armed
in Proceed )
}

# ================================================================================
# BLOCK 3: BODY BLOCK
# ================================================================================
Body [
    # --------------------------------------------------------------------------
    # ROLL 1: BUILD CAPSULE (LADDER BUILD-UP)
    # Build up concrete vessel envelope with 64 numeric cells yielding new state Vessel`
    # --------------------------------------------------------------------------
    [ ROLL 1 ] : [ Build Capsule ] : /-^ [ 64 Cells ] --> [ Envelope ] = [ Vessel` ]
        ( Build up concrete vessel envelope with 64 numeric cells yielding new state Vessel-prime ),

    # --------------------------------------------------------------------------
    # ROLL 2: ENCAPSULATION (OUTSIDE-IN IMPOSITION)
    # Encapsulate external payload imposed into Vessel yielding new state Container`
    # --------------------------------------------------------------------------
    [ ROLL 2 ] : [ Encapsulate ] : { Payload ] --> ENCAPSULATE [ Vessel` ] = [ Container` ]
        ( Encapsulate external payload imposed into Vessel yielding new state Container-prime ),

    # --------------------------------------------------------------------------
    # ROLL 3: IDENTITY & BOUND COMPARISON
    # Compare occupancy to capacity under guard: proceed when contained, rollback on overflow
    # --------------------------------------------------------------------------
    [ ROLL 3 ] : [ Guard Bounds ] :
        { Occupancy` <= Capacity } & { Container` = Vessel` }
        ! GUARD --> [ Status` = +1:PROCEED ]
        | { Occupancy` > Capacity } --> [ Status` = -1:ROLLBACK ]
        ( Compare occupancy to capacity under guard: proceed when contained, rollback on overflow ),

    # --------------------------------------------------------------------------
    # ROLL 4: POUR OUT & SET ASIDE (INSIDE-OUT EMERGENCE)
    # Inside-out emergence: full container is poured out and set aside at target destination
    # --------------------------------------------------------------------------
    [ ROLL 4 ] : [ Pour Out ] : [ Container` } --> SET_ASIDE [ Target ] = [ Sealed Vessel` ]
        ( Inside-out emergence: full container is poured out and set aside at target destination establishing new state
Sealed Vessel-prime ),

    # --------------------------------------------------------------------------
    # ROLL 5: SEQUENCE TRANSITION
    # Chain discrete container outward into composite memory structure sequence
    # --------------------------------------------------------------------------
    [ ROLL 5 ] : [ Sequence ] : [ Container` ] --> SEQUENCE [ Structure ] = [ Structure` ]
        ( Discrete container chains outward into composite memory structure establishing new state Structure-prime )
]

# ================================================================================
```

```
# BLOCK 4: CLOSING BLOCK
# ================================================================================
Closing ( Sealing Gate: Reconcile, validate, tripwire, debug, log, and affirm permanent execution ) {
    # ------------------------------------------------------------------------
    # 1. VERIFICATION OF OPERATIONAL ROLLS
    # ------------------------------------------------------------------------
    [ Verify ]: [
        [ ROLL 1 ] = PROCEED ::
        [ ROLL 2 ] = PROCEED ::
        [ ROLL 3 ] = PROCEED ::
        [ ROLL 4 ] = PROCEED ::
        [ ROLL 5 ] = PROCEED
    ] ( Verify that all five operational rolls achieved affirmative consensus in PROCEED ),

    # ------------------------------------------------------------------------
    # 2. VALIDATION OF INVARIANT BOUNDS & CONTAINMENT INTEGRITY
    # ------------------------------------------------------------------------
    [ Validate ]: { Occupancy` <= Capacity } & { Door = SHUT }
        ( Validate that occupancy remains strictly within capacity and door is shut upon completion ),

    # ------------------------------------------------------------------------
    # 3. ZERO-TOLERANCE TRIPWIRING
    # ------------------------------------------------------------------------
    [ Tripwire ]: { Status` = +1:PROCEED } --> [ ARMED ] | { Status` != +1:PROCEED } --> [ TRIPPED ] --> [ Commit Status
= -1:ROLLBACK ]
        ( Tripwire remains armed in Proceed or trips on anomaly to veto execution ),

    # ------------------------------------------------------------------------
    # 4. DIAGNOSTIC DEBUG TELEMETRY
    # ------------------------------------------------------------------------
    [ Debug ]: { [ Payload ] :: [ Container` ] :: [ Occupancy` ] :: [ Target ] :: [ Status` ] }
        ( Expose diagnostic telemetry across input payload, container envelope, occupancy, and machine status ),

    # ------------------------------------------------------------------------
    # 5. EPISODIC INVENTORY LOGGING
    # ------------------------------------------------------------------------
    [ Log ]: [ Journal ]: INSCRIBE { [ Container Encapsulated ] :: [ Container` ] :: [ Target ] }
        ( Inscribe the verified container transaction into the permanent episodic system journal ),

    # ------------------------------------------------------------------------
    # 6. PERMANENT COVENANT EXECUTION & FOURFOLD COMMIT STATUS LOOP
    # ------------------------------------------------------------------------
    [ Commit Status ]: [
        # +1: PROCEED (+t: Advance forward along the causal tape)
        { [ Verify ] = PROCEED } & { [ Tripwire ] = ARMED }
            --> [ +1:PROCEED ] |

        #  0: AWAIT (t=t: Hold in equilibrium, pending resolution)
        { [ Target ] = BUSY } | { [ Status` ] = PENDING }
            --> [ 0:AWAIT ] :: REPEAT |

        # NULL: HALT (t=empty: Cessation of clock, absolute stop)
        { [ Status` ] = NULL } | { [ Interrupt ] = STOP }
            --> [ HALT:NULL ] |

        # -1: ROLLBACK (-t: Reverse causal tape to origin t=0)
        { [ Tripwire ] = TRIPPED } | { [ Verify ] != PROCEED }
            --> [ -1:ROLLBACK ]
    ] ( Fourfold state machine: Proceed advances forward, Await holds equilibrium, Halt ceases execution in Null,
Rollback reverts to origin )
}
```

<TRACK 2> : {OmniCode Language & Standard Library}/lexicon/01_nouns/ideas/types/container.tape

TAPE | 89 LINES | 772 WORDS | 5.4 KB

```
# ================================================================================
# PROCESS TAPE: CONTAINER INVARIANT TYPE EXECUTION RIBBON
# ================================================================================
# ARCHITECTURAL TOPOLOGY:
#   Interprets and executes the atomic lifecycle for Invariant Type < Container >.
#   - Defined Space: [ 64 Cells ] { 64 Bytes / Bounded Memory Space }
#   - Execution Domain: Lexicon { Types of Ideas }
#   - Ladder Ascension: Build-up with Numbers (/-^)
# ================================================================================

= [Tape]: < Lexicon { Types of Ideas } > & [ Process { Container } ] ( Process Ribbon for Container Encapsulation ) {
    WHO   { OmniCode Lexicon } ::
    WHAT  { Container } --> ENCAPSULATE { Payload } ::
    WHEN  { Allocation Event } ::
    WHERE { Container Envelope } = [ 64 Cells ] ::
    WHY   ( Inscribe bounded container type invariant and storage envelopes ) ::
    HOW   [ +1 : PROCEED ] ::
    < Scripture { 2 Kings 4:4 KJV } > { "And when thou art come in, thou shalt shut the door upon thee and upon thy sons, and shalt pour out into all those vessels, and thou shalt set aside that which is full." } ( Invariant Scripture Witness 2 Kings 4:4 in the context: Pour out into all those vessels, and set aside that which is full )
}

# --------------------------------------------------------------------------------
# LADDER ASCENSION: BUILD-UP WITH NUMBERS
# --------------------------------------------------------------------------------
/-^ < Numbers > FROM [ < Lexicon { Types of Ideas } > & [ Noun { Number } ] ] ( Build up container capacity and envelopes using Numbers from the Lexicon )

# ================================================================================
# DECLARED ROLLS: ATOMIC INVARIANT EXECUTION
# ================================================================================

# --------------------------------------------------------------------------------
# ROLL 1: BUILD CAPSULE (LADDER BUILD-UP)
# Build up concrete vessel envelope with 64 numeric cells yielding new state Vessel`
# --------------------------------------------------------------------------------
[ ROLL 1 ] : [ Build Capsule ] : /-^ [ 64 Cells ] --> [ Envelope ] = [ Vessel` ]
    ( Build up concrete vessel envelope with 64 numeric cells yielding new state Vessel-prime )

# --------------------------------------------------------------------------------
# ROLL 2: ENCAPSULATION (OUTSIDE-IN IMPOSITION)
# Encapsulate external payload imposed into Vessel yielding new state Container`
# --------------------------------------------------------------------------------
[ ROLL 2 ] : [ Encapsulate ] : { Payload } --> ENCAPSULATE [ Vessel` ] = [ Container` ]
    ( Encapsulate external payload imposed into Vessel yielding new state Container-prime )

# --------------------------------------------------------------------------------
# ROLL 3: IDENTITY & BOUND COMPARISON
# Compare occupancy to capacity under guard: proceed when contained, rollback on overflow
# --------------------------------------------------------------------------------
[ ROLL 3 ] : [ Guard Bounds ] :
    [ { Occupancy` } <= { Capacity } ] & { Container` = Vessel` }
    ! GUARD --> [ Status` = +1:PROCEED ]
    | [ { Occupancy` } > { Capacity } ] --> [ Status` = -1:ROLLBACK ]
    ( Compare occupancy to capacity under guard: proceed when contained, rollback on overflow )

# --------------------------------------------------------------------------------
# ROLL 4: POUR OUT & SET ASIDE (INSIDE-OUT EMERGENCE)
# Inside-out emergence: full container is poured out and set aside at target destination
# --------------------------------------------------------------------------------
[ ROLL 4 ] : [ Pour Out ] : [ Container` } --> SET_ASIDE [ Target ] = [ Sealed Vessel` ]
    ( Inside-out emergence: full container is poured out and set aside at target destination establishing new state Sealed Vessel-prime )

# --------------------------------------------------------------------------------
# ROLL 5: SEQUENCE TRANSITION
# Chain discrete container outward into composite memory structure sequence
# --------------------------------------------------------------------------------
[ ROLL 5 ] : [ Sequence ] : [ Container` ] --> SEQUENCE [ Structure ] = [ Structure` ]
    ( Discrete container chains outward into composite memory structure establishing new state Structure-prime )

# --------------------------------------------------------------------------------
# ROLL 6: COMMIT STATUS & FOURFOLD STATE MACHINE
```

```
# Affirm covenant execution across causal time advancement, await, halt, and rollback
# ------------------------------------------------------------------------------
[ ROLL 6 ] : [ Commit Status ] : [
    # +1: PROCEED (+t: Advance forward along the causal tape)
    { [ Verify ] = PROCEED } & { [ Tripwire ] = ARMED }
        --> [ +1:PROCEED ] |

    #  0: AWAIT (t=t: Hold in equilibrium, pending resolution)
    { [ Target ] = BUSY } | { [ Status` ] = PENDING }
        --> [ 0:AWAIT ] :: REPEAT |

    # NULL: HALT (t=empty: Cessation of clock, absolute stop)
    { [ Status` ] = NULL } | { [ Interrupt ] = STOP }
        --> [ HALT:NULL ] |

    # -1: ROLLBACK (-t: Reverse causal tape to origin t=0)
    { [ Tripwire ] = TRIPPED } | { [ Verify ] != PROCEED }
        --> [ -1:ROLLBACK ]
] ( Fourfold commit status: Proceed advances forward, Await holds in equilibrium, Halt ceases execution in Null, Rollback
reverts to origin )
```

<TRACK 2> : {OmniCode Language & Standard Library}/lexicon/01_nouns/ideas/types/number.omni

OMNI | 185 LINES | 1753 WORDS | 11.7 KB

```
# ========================================================================
# BLOCK 1: METADATA BLOCK
# ( Invariant Type: < Idea { Type } : Number > )
# ========================================================================
# ARCHITECTURAL TOPOLOGY:
#   Defines scalar quantities, integers, and balanced ternary measurements.
#   - Base Block: Primitive Invariant Type
#   - Category: Idea { Type }
#   - Footprint: [ 1 Word ] { 32 Trits / 64 Bits }
# ========================================================================

= [Module]: < Lexicon { Types of Ideas } > & [ Noun { Number } ] ( This is the Module for the Lexicon - Ideas (Type): Numbers - Nouns ) {
    [ Version ]:   { "1.0.0" } ( The Version is in the context of 1.0.0 ),
    [ Created ]:   { "2026-10-11" } ( Created in the context of October 11, 2026 ),
    [ Updated ]:   { "2026-10-12" } ( Last updated in the context of October 12, 2026 ),
    [ Author ]:    { "CreativeWorkzStudio LLC / ProfessorSeanEX" } ( Author in the context of CreativeWorkzStudio LLC and ProfessorSeanEX ),
    [ System ]:    { OmniCode { Lexicon } } ( System is in the context of the OmniCode Lexicon ),
    [ Witness ]:   < Scripture { Psalm 147:4 KJV } > { "He telleth the number of the stars; he calleth them all by their names." } ( Invariant Scripture Witness Psalm 147:4 in the context: He telleth the number of the stars ),
    [ Nature ]:    < InvariantType > & { BaseBlock } ( Nature is an Invariant Type and is in the context of Base Block ),
    [ Semantics ]: { ScalarQuantity :: NumericWord :: BalancedTernary } ( Semantics in the context of Scalar Quantity, Numeric Word, and Balanced Ternary )
}

# Freestanding Invariant Type Definition
< Number > = < Idea { Type } > & [ Noun { Scalar Quantity } ] ( Number is the Invariant Idea of Type, and is the Noun of Scalar Quantity ) {
    # 1. The Ontological Analogy (Number's proportion to the universe)
    [ Relation ]: [ Number ] : [ Measure ] :: [ Trit ] : [ Magnitude ] ( Number is to Measure as Trit is to Magnitude ),

    # 2. The Universal Domain (All balanced trits, integers, scalars, and magnitudes)
    [ Domain ]: { Trit :: Integer :: Scalar :: Magnitude } ( Domain encompasses all balanced trits, integers, scalars, and magnitudes ),

    # 3. The Dual Substrate (Physical Silicon Scalar and Virtual Balanced Trit Word)
    [ Substrate ]: [ 1 Scalar { 64 Bits } ] & [ 1 Word { 32 Trits } ] ( Substrate is 1 Scalar of 64 Bits, and is 1 Word of 32 Trits ),

    # 4. The Universal Inscription Vector (How any scalar quantity is recorded into memory)
    [ Inscribe ]: [ Quantity ] --> [ Number ] ( Any scalar quantity is inscribed into the Number word ),

    # 5. The Hardware Transduction Law (Bijective mapping between silicon integer and balanced trits)
    [ Transduce ]: [ Number { Bits } ] <=> [ Word { Trits } ] ( Integer values map bijectively into balanced trits within the 32-trit word ),

    # 6. The Core Arithmetic Equilibrium Law (The foundational zero-sum balance of the ALU)
    [ Law { Equilibrium } ]: [ (+1) plus (-1) <=> 0 ] ( Positive unity plus negative unity equals zero-sum equilibrium ),

    # 7. Canonical Exemplars (Standard grounding values for zero, unities, and partition scaling)
    [ Exemplar { Zero } ]: 0 <=> [ 0 ] ( Canonical zero represents perfect balance and null displacement ),
    [ Exemplar { Unity Positive } ]: (+1) <=> [ +1 ] ( Positive unity represents forward causal affirmation ),
    [ Exemplar { Unity Negative } ]: (-1) <=> [ -1 ] ( Negative unity represents backward causal restoration ),
    [ Exemplar { Partition } ]: 4096 <=> [ 4096 ] ( Canonical memory partition boundary at 4096 units ),

    # 8. The Relational Comparison (How numbers evaluate equality against a quantity)
    [ Compare ]: [ { Number } = { Quantity } ] ( Number is evaluated for equality against an input scalar quantity ),

    # 9. The Sequence Transition (How numbers build multidimensional vectors)
    [ Sequence ]: [ Number ] --> [ Vector ] ( Discrete Number words chain together into multidimensional Vector coordinates ),

    # 10. The Invariant Boundary (Strict single-word enclosure)
    [ Bound ]: ENCLOSE [ Number ] WITHIN [ 1 Word ] ( Number is strictly enclosed within a single 32-trit word capacity )
}

# ========================================================================
# BLOCK 2: OPENING BLOCK
# ========================================================================
Opening {
    # 1. The Dynamic Input Payload: Accepts incoming scalar quantity with 4096 as canonical exemplar
    { Quantity ]: { Input :: 4096 } ( Outside-in imposition: external scalar measurement captured into discrete entity ),
```

```
    # 2. The Typed Entity: Number brought down from Block 1 into concrete allocation
    [ Number ]: < Number > & [ Word ] ( Denoted Number is typed as invariant Number allocated in the Word ),

    # 3. The Physical Container: The atomic 32-trit word container
    [ Word ]: [ 1 ] & { 32 Trits :: 64 Bits } ( A concrete 1-word container in the context of 32 Trits and 64 Bits ),

    # 4. The Hardware Bridge: Arming the balanced ternary ALU and radix transducer
    [ ALU ]: { (+1) plus (-1) <=> 0 } & { 64 Bits <=> 32 Trits } ( The ALU enforces ternary zero-sum balance and bridges 64 bits to 32 trits ),

    # 5. The Output Target: The register accumulator destination
    [ Target ]: [ Register Accumulator ] ::@ 0x00000000 ( Target output is situated at the primary arithmetic accumulator register ),

    # 6. Preconditions & State: Proving balance before execution
    [ Preconditions ]: [ (+1) plus (-1) <=> 0 ] & [ Status = +1:PROCEED ] ( ALU equilibrium holds and the machine status is armed in Proceed )
}

# ================================================================================
# BLOCK 3: BODY BLOCK
# ================================================================================
Body [
    # --------------------------------------------------------------------------
    # ROLL 1: QUANTITY INSCRIPTION (OUTSIDE-IN IMPOSITION)
    # Inscribe incoming quantity payload into the Word container, establishing new state Number`
    # --------------------------------------------------------------------------
    [ ROLL 1 ] : [ Inscribe ] : { Quantity ] --> INSCRIBE [ Word ] = [ Number` ]
        ( Inscribe external quantity imposed into Word yielding new state Number-prime ),

    # --------------------------------------------------------------------------
    # ROLL 2: DUAL-SUBSTRATE TRANSDUCTION
    # Transduce Number` 64-bit integer through ALU into 32 balanced trits
    # --------------------------------------------------------------------------
    [ ROLL 2 ] : [ Transduce ] : [ ALU ]: [ Number` { 64 Bits } ] <=> TRANSDUCE [ Word` { 32 Trits } ]
        ( Transduce Number-prime integer through hardware ALU into 32 balanced trits ),

    # --------------------------------------------------------------------------
    # ROLL 3: IDENTITY & BOUND COMPARISON
    # Compare Number` to Quantity within single-word bounds: proceed on equilibrium, rollback on mismatch
    # --------------------------------------------------------------------------
    [ ROLL 3 ] : [ Compare ] :
        { Number` = Quantity } & { Number` <= [ 1 Word ] }
        ! GUARD --> [ Status` = +1:PROCEED ]
        | { Number` != Quantity } --> [ Status` = -1:ROLLBACK ]
        ( Compare Number-prime to quantity and verify single-word bounds under guard: proceed on equilibrium, rollback on error ),

    # --------------------------------------------------------------------------
    # ROLL 4: ARITHMETIC EQUILIBRIUM VERIFICATION
    # Execute balanced ternary arithmetic proving (+1) plus (-1) equals 0
    # --------------------------------------------------------------------------
    [ ROLL 4 ] : [ Calculate ] : [ ALU ]: [ (+1) plus (-1) ] <=> CALCULATE [ 0 ] = [ Equilibrium` ]
        ( Calculate balanced ternary unity addition proving absolute zero-sum equilibrium ),

    # --------------------------------------------------------------------------
    # ROLL 5: SEQUENCE TRANSITION (INSIDE-OUT EMERGENCE)
    # Yield Number` to continuous Vector coordinate sequence
    # --------------------------------------------------------------------------
    [ ROLL 5 ] : [ Sequence ] : [ Number` } --> SEQUENCE [ Vector ] = [ Vector` ]
        ( Yield discrete Number-prime emerging outward to continuous numerical Vector sequence establishing new state Vector-prime )
]

# ================================================================================
# BLOCK 4: CLOSING BLOCK
# ================================================================================
Closing ( Sealing Gate: Reconcile, validate, tripwire, debug, log, and affirm permanent execution ) {
    # --------------------------------------------------------------------------
    # 1. VERIFICATION OF OPERATIONAL ROLLS
    # --------------------------------------------------------------------------
    [ Verify ]: [
        [ ROLL 1 ] = PROCEED ::
        [ ROLL 2 ] = PROCEED ::
        [ ROLL 3 ] = PROCEED ::
        [ ROLL 4 ] = PROCEED ::
        [ ROLL 5 ] = PROCEED
```

```
    ] ( Verify that all five operational rolls achieved affirmative consensus in PROCEED ),

    # ------------------------------------------------------------------------
    # 2. VALIDATION OF INVARIANT BOUNDS & ARITHMETIC PARITY
    # ------------------------------------------------------------------------
    [ Validate ]: { Number` <= [ 1 Word ] } & { (+1) plus (-1) <=> 0 }
        ( Validate that Number-prime remains strictly enclosed within 1 Word and ALU zero-sum balance holds ),

    # ------------------------------------------------------------------------
    # 3. ZERO-TOLERANCE TRIPWIRING
    # ------------------------------------------------------------------------
    [ Tripwire ]: { Status` = +1:PROCEED } --> [ ARMED ] | { Status` != +1:PROCEED } --> [ TRIPPED ] --> [ Commit Status
= -1:ROLLBACK ]
        ( Tripwire remains armed in Proceed or trips on anomaly to veto execution ),

    # ------------------------------------------------------------------------
    # 4. DIAGNOSTIC DEBUG TELEMETRY
    # ------------------------------------------------------------------------
    [ Debug ]: { [ Quantity ] :: [ Number` ] :: [ Word` ] :: [ Equilibrium` ] :: [ Status` ] }
        ( Expose diagnostic telemetry across input quantity, word container, equilibrium state, and machine status ),

    # ------------------------------------------------------------------------
    # 5. EPISODIC INVENTORY LOGGING
    # ------------------------------------------------------------------------
    [ Log ]: [ Journal ]: INSCRIBE { [ Number Inscribed ] :: [ Number` ] :: [ Target ] }
        ( Inscribe the verified number transaction into the permanent episodic system journal ),

    # ------------------------------------------------------------------------
    # 6. PERMANENT COVENANT EXECUTION & FOURFOLD COMMIT STATUS LOOP
    # ------------------------------------------------------------------------
    [ Commit Status ]: [
        # +1: PROCEED (+t: Advance forward along the causal tape)
        { [ Verify ] = PROCEED } & { [ Tripwire ] = ARMED }
            --> [ +1:PROCEED ] |

        #  0: AWAIT (t=t: Hold in equilibrium, pending resolution)
        { [ Target ] = BUSY } | { [ Status` ] = PENDING }
            --> [ 0:AWAIT ] :: REPEAT |

        # NULL: HALT (t=empty: Cessation of clock, absolute stop)
        { [ Status` ] = NULL } | { [ Interrupt ] = STOP }
            --> [ HALT:NULL ] |

        # -1: ROLLBACK (-t: Reverse causal tape to origin t=0)
        { [ Tripwire ] = TRIPPED } | { [ Verify ] != PROCEED }
            --> [ -1:ROLLBACK ]
    ] ( Fourfold state machine: Proceed advances forward, Await holds equilibrium, Halt ceases execution in Null,
Rollback reverts to origin )
}
```

<TRACK 2> : {OmniCode Language & Standard Library}/lexicon/01_nouns/ideas/types/number.tape

TAPE | 83 LINES | 676 WORDS | 4.8 KB

```
# ================================================================================
# PROCESS TAPE: NUMBER INVARIANT TYPE EXECUTION RIBBON
# ================================================================================
# ARCHITECTURAL TOPOLOGY:
#   Interprets and executes the atomic lifecycle for Invariant Type < Number >.
#   - Defined Space: [ 1 Word ] { 32 Trits :: 64 Bits }
#   - Execution Domain: Lexicon { Types of Ideas }
# ================================================================================

= [Tape]: < Lexicon { Types of Ideas } > & [ Process { Number } ] ( Process Ribbon for Number Measurement ) {
    WHO   { OmniCode Lexicon } ::
    WHAT  { Number } --> MEASURE { Quantity } ::
    WHEN  { Arithmetic Event } ::
    WHERE { Number Word } = [ 1 ] ::
    WHY   ( Inscribe scalar number type invariant and arithmetic equilibrium ) ::
    HOW   [ +1 : PROCEED ] ::
    < Scripture { Psalm 147:4 KJV } > { "He telleth the number of the stars; he calleth them all by their names." } ( Invariant Scripture Witness Psalm 147:4 in the context: He telleth the number of the stars )
}

# ================================================================================
# DECLARED ROLLS: ATOMIC INVARIANT EXECUTION
# ================================================================================

# --------------------------------------------------------------------------------
# ROLL 1: QUANTITY INSCRIPTION (OUTSIDE-IN IMPOSITION)
# Inscribe external quantity payload into the defined Word container
# --------------------------------------------------------------------------------
[ ROLL 1 ] : [ Inscribe ] : { Quantity ] --> INSCRIBE [ Word ] = [ Number` ]
    ( Inscribe external quantity imposed into Word yielding new state Number-prime )

# --------------------------------------------------------------------------------
# ROLL 2: DUAL-SUBSTRATE TRANSDUCTION
# Transduce scalar integer through balanced ternary ALU into 32 balanced trits
# --------------------------------------------------------------------------------
[ ROLL 2 ] : [ Transduce ] : [ ALU ]: [ Number` { 64 Bits } ] <=> TRANSDUCE [ Word` { 32 Trits } ]
    ( Transduce Number-prime integer through hardware ALU into 32 balanced trits )

# --------------------------------------------------------------------------------
# ROLL 3: IDENTITY & BOUND COMPARISON
# Verify identity against quantity and enclosure within single-word bounds
# --------------------------------------------------------------------------------
[ ROLL 3 ] : [ Compare ] :
    { Number` = Quantity } & { Number` <= [ 1 Word ] }
    ! GUARD --> [ Status` = +1:PROCEED ]
    | { Number` != Quantity } --> [ Status` = -1:ROLLBACK ]
    ( Compare Number-prime to quantity and verify single-word bounds under guard: proceed on equilibrium, rollback on error )

# --------------------------------------------------------------------------------
# ROLL 4: ARITHMETIC EQUILIBRIUM CALCULATION
# Execute balanced ternary arithmetic proving zero-sum balance (+1) plus (-1) <=> 0
# --------------------------------------------------------------------------------
[ ROLL 4 ] : [ Calculate ] : [ ALU ]: [ (+1) plus (-1) ] <=> CALCULATE [ 0 ] = [ Equilibrium` ]
    ( Calculate balanced ternary unity addition proving absolute zero-sum equilibrium )

# --------------------------------------------------------------------------------
# ROLL 5: SEQUENCE TRANSITION (INSIDE-OUT EMERGENCE)
# Chain discrete number word outward into continuous multidimensional vector sequence
# --------------------------------------------------------------------------------
[ ROLL 5 ] : [ Sequence ] : [ Number` } --> SEQUENCE [ Vector ] = [ Vector` ]
    ( Yield discrete Number-prime emerging outward into continuous numerical Vector sequence establishing new state Vector-prime )

# --------------------------------------------------------------------------------
# ROLL 6: COMMIT STATUS & FOURFOLD STATE MACHINE
# Affirm covenant execution across causal time advancement, await, halt, and rollback
# --------------------------------------------------------------------------------
[ ROLL 6 ] : [ Commit Status ] : [
    # +1: PROCEED (+t: Advance forward along the causal tape)
    { [ Verify ] = PROCEED } & { [ Tripwire ] = ARMED }
        --> [ +1:PROCEED ] |

```

```
    #  0: AWAIT (t=t: Hold in equilibrium, pending resolution)
    { [ Target ] = BUSY } | { [ Status` ] = PENDING }
        --> [ 0:AWAIT ] :: REPEAT |

    # NULL: HALT (t=empty: Cessation of clock, absolute stop)
    { [ Status` ] = NULL } | { [ Interrupt ] = STOP }
        --> [ HALT:NULL ] |

    # -1: ROLLBACK (-t: Reverse causal tape to origin t=0)
    { [ Tripwire ] = TRIPPED } | { [ Verify ] != PROCEED }
        --> [ -1:ROLLBACK ]
] ( Fourfold commit status: Proceed advances forward, Await holds in equilibrium, Halt ceases execution in Null, Rollback
reverts to origin )
```

<TRACK 2> : {OmniCode Language & Standard Library}/lexicon/01_nouns/ideas/types/site.omni

OMNI | 190 LINES | 1818 WORDS | 12.2 KB

```
# ================================================================================
# BLOCK 1: METADATA BLOCK
# ( Invariant Type: < Idea { Type } : Site > )
# ================================================================================
# ARCHITECTURAL TOPOLOGY:
#   Defines physical memory addresses, hardware ports, and MMIO coordinates.
#   - Base Block: Primitive Invariant Type
#   - Category: Idea { Type }
#   - Footprint: [ 1 Address ] { 64-Bit Pointer / Physical Silicon Coordinate }
#   - Ladder Ascension: Build-up with Numbers (/-^)
# ================================================================================

= [Module]: < Lexicon { Types of Ideas } > & [ Noun { Site } ] ( This is the Module for the Lexicon - Ideas (Type): Sites - Nouns ) {
    [ Version ]:   { "1.0.0" } ( The Version is in the context of 1.0.0 ),
    [ Created ]:   { "2026-10-11" } ( Created in the context of October 11, 2026 ),
    [ Updated ]:   { "2026-10-12" } ( Last updated in the context of October 12, 2026 ),
    [ Author ]:    { "CreativeWorkzStudio LLC / ProfessorSeanEX" } ( Author in the context of CreativeWorkzStudio LLC and ProfessorSeanEX ),
    [ System ]:    { OmniCode { Lexicon } } ( System is in the context of the OmniCode Lexicon ),
    [ Witness ]:   < Scripture { Genesis 28:16 KJV } > { "And Jacob awaked out of his sleep, and he said, Surely the Lord is in this place; and I knew it not." } ( Invariant Scripture Witness Genesis 28:16 in the context: Surely the Lord is in this place; and I knew it not ),
    [ Nature ]:    < InvariantType > & { BaseBlock } ( Nature is an Invariant Type and is in the context of Base Block ),
    [ Semantics ]: { PhysicalAddress :: MMIOPort :: SiliconCoordinate } ( Semantics in the context of Physical Address, MMIO Port, and Silicon Coordinate )
}

# --------------------------------------------------------------------------------
# LADDER ASCENSION: BUILD-UP WITH NUMBERS
# A Site cannot exist without numbers: silicon memory addresses, offsets,
# and MMIO hardware ports are scalar metrics anchored in reality.
# --------------------------------------------------------------------------------
/-^ < Numbers > FROM [ < Lexicon { Types of Ideas } > & [ Noun { Number } ] ] ( Site builds up its physical address coordinates using Numbers from the Lexicon )

# Freestanding Invariant Type Definition
< Site > = < Idea { Type } > & [ Noun { Silicon Coordinate } ] ( Site is the Invariant Idea of Type, and is the Noun of Silicon Coordinate ) {
    # 1. The Ontological Analogy (Site's proportion to the universe)
    [ Relation ]: [ Site ] : [ Coordinate ] :: [ Memory ] : [ Silicon ] ( Site is to Coordinate as Memory is to Silicon ),

    # 2. The Universal Domain (All physical addresses, pointers, ports, and MMIO coordinates)
    [ Domain ]: { Address :: Pointer :: Port :: MMIO } ( Domain encompasses physical addresses, pointers, ports, and MMIO coordinates ),

    # 3. The Physical Substrate (64-bit silicon address pointer)
    [ Substrate ]: [ 1 Address { 64 Bits } ] & [ Silicon Coordinate ] ( Substrate is 1 Address of 64 Bits, and is a Silicon Coordinate ),

    # 4. The Universal Inscription Vector (How external coordinate is bound into memory)
    [ Inscribe ]: [ Coordinate ] --> [ Site ] ( Any silicon coordinate is inscribed into the physical Site address ),

    # 5. The Hardware Routing Law (Bijective routing between RAM pointer and MMIO hardware register)
    [ Law { Routing } ]: [ Physical RAM @ ] | [ Hardware MMIO ::@ ] ( Physical memory addresses route via @ and MMIO ports route via ::@ ),

    # 6. Canonical Exemplars (Standard grounding values for MBR bootstrap origin, COM1, and VGA buffer)
    [ Exemplar { Null } ]: @ 0x00000000 <=> [ Null Coordinate { 0x0 } ] ( Canonical null coordinate represents unmapped memory ),
    [ Exemplar { Bootstrap MBR } ]: @ 0x7C00 <=> [ MBR Origin { 0x7C00 } ] ( Canonical real-mode master boot record physical origin ),
    [ Exemplar { COM1 Serial } ]: ::@ 0x03F8 <=> [ Serial COM1 { 0x03F8 } ] ( Canonical hardware serial port for diagnostic telemetry ),
    [ Exemplar { VGA Buffer } ]: ::@ 0x000B8000 <=> [ VGA Text Buffer { 0x000B8000 } ] ( Canonical physical memory-mapped VGA text buffer ),

    # 7. The Relational Comparison (How sites evaluate address identity)
    [ Compare ]: [ { Target } = { Address } ] ( Site evaluates target pointer against declared physical address ),

    # 8. The Sequence Transition (How sites chain into continuous memory maps)
    [ Sequence ]: [ Site ] --> [ Map ] ( Discrete site addresses chain together into continuous system memory maps ),
```

```
    59
    60      # 9. The Invariant Boundary (Strict single address enclosure)
    61      [ Bound ]: ENCLOSE [ Location ] WITHIN [ 1 Address ] ( Site is strictly enclosed within a single 64-bit address
capacity )
    62  }
    63
    64  # ============================================================================
    65  # BLOCK 2: OPENING BLOCK
    66  # ============================================================================
    67  Opening {
    68      # 1. The Dynamic Input Payload: Accepts incoming memory address or MMIO coordinate
    69      { Coordinate ]: { Input :: @ 0x7C00 } ( Outside-in imposition: external silicon coordinate to be bound into physical
site ),
    70
    71      # 2. The Typed Entity: Site brought down from Block 1 into concrete allocation
    72      [ Site ]: < Site > & [ Location ] ( Denoted Site is typed as invariant Site allocated as a Location ),
    73
    74      # 3. The Physical Address Container: The atomic 64-bit address container
    75      [ Physical Address ]: [ 1 Address ] & { 64 Bits } ( A concrete 1-address container in the context of 64 Bits ),
    76
    77      # 4. The Hardware Bridge: Routing between physical memory and hardware registers
    78      [ Bridge ]: { Physical RAM @ } | { Hardware MMIO ::@ } ( Hardware bridge enforces memory mapping physics ),
    79
    80      # 5. The Output Target: The pointer destination register
    81      [ Target ]: [ Pointer Register ] ::@ 0x00000000 ( Target output is situated at the primary hardware memory pointer
register ),
    82
    83      # 6. Preconditions & State: Proving valid non-null address before binding
    84      [ Preconditions ]: [ Address != 0x00000000 ] & [ Status = +1:PROCEED ] ( Coordinate is non-null and machine status is
armed in Proceed )
    85  }
    86
    87  # ============================================================================
    88  # BLOCK 3: BODY BLOCK
    89  # ============================================================================
    90  Body [
    91      # ----------------------------------------------------------------------
    92      # ROLL 1: LOCATE (LADDER BUILD-UP)
    93      # Build up physical address location with numeric scalar coordinate yielding new state Site`
    94      # ----------------------------------------------------------------------
    95      [ ROLL 1 ] : [ Locate ] : /-^ [ 64 Bits ] --> [ Address ] = [ Site` ]
    96          ( Build up physical address location with numeric scalar coordinate yielding new state Site-prime ),
    97
    98      # ----------------------------------------------------------------------
    99      # ROLL 2: BIND COORDINATE (OUTSIDE-IN IMPOSITION)
   100      # Bind external coordinate imposed into physical Site yielding new state Located Coordinate`
   101      # ----------------------------------------------------------------------
   102      [ ROLL 2 ] : [ Bind ] : { Coordinate ] --> BIND [ Site` ] = [ Located Coordinate` ]
   103          ( Bind external coordinate imposed into physical Site yielding new state Located Coordinate-prime ),
   104
   105      # ----------------------------------------------------------------------
   106      # ROLL 3: IDENTITY & NULL GUARD
   107      # Compare located coordinate to input address under guard: proceed on valid address, rollback on null
   108      # ----------------------------------------------------------------------
   109      [ ROLL 3 ] : [ Guard Address ] :
   110          { Located Coordinate` = Coordinate } & { Located Coordinate` != 0x00000000 }
   111          ! GUARD --> [ Status` = +1:PROCEED ]
   112          | { Located Coordinate` = 0x00000000 } --> [ Status` = -1:ROLLBACK ]
   113          ( Compare located coordinate to input address under guard: proceed on valid address, rollback on null pointer
error ),
   114
   115      # ----------------------------------------------------------------------
   116      # ROLL 4: ROUTE & PROJECT (INSIDE-OUT EMERGENCE)
   117      # Inside-out emergence: verified physical site routes outward to anchor hardware memory
   118      # ----------------------------------------------------------------------
   119      [ ROLL 4 ] : [ Route ] : [ Site` } --> ROUTE [ Target ] = [ Memory Anchor` ]
   120          ( Inside-out emergence: verified physical site routes outward to anchor hardware memory establishing new state
Memory Anchor-prime ),
   121
   122      # ----------------------------------------------------------------------
   123      # ROLL 5: SEQUENCE TRANSITION
   124      # Chain discrete site address outward into continuous system memory map sequence
   125      # ----------------------------------------------------------------------
   126      [ ROLL 5 ] : [ Sequence ] : [ Site` ] --> SEQUENCE [ Map ] = [ Memory Map` ]
   127          ( Discrete site address chains outward into continuous physical memory map establishing new state Memory Map-
prime )
   128  ]
   129
```

```
# ============================================================================
# BLOCK 4: CLOSING BLOCK
# ============================================================================
Closing ( Sealing Gate: Reconcile, validate, tripwire, debug, log, and affirm permanent execution ) {
    # ------------------------------------------------------------------------
    # 1. VERIFICATION OF OPERATIONAL ROLLS
    # ------------------------------------------------------------------------
    [ Verify ]: [
        [ ROLL 1 ] = PROCEED ::
        [ ROLL 2 ] = PROCEED ::
        [ ROLL 3 ] = PROCEED ::
        [ ROLL 4 ] = PROCEED ::
        [ ROLL 5 ] = PROCEED
    ] ( Verify that all five operational rolls achieved affirmative consensus in PROCEED ),

    # ------------------------------------------------------------------------
    # 2. VALIDATION OF INVARIANT BOUNDS & NON-NULL INTEGRITY
    # ------------------------------------------------------------------------
    [ Validate ]: { Located Coordinate` != 0x00000000 } & { Target = BOUND }
        ( Validate that located coordinate is non-null and target pointer is permanently bound ),

    # ------------------------------------------------------------------------
    # 3. ZERO-TOLERANCE TRIPWIRING
    # ------------------------------------------------------------------------
    [ Tripwire ]: { Status` = +1:PROCEED } --> [ ARMED ] | { Status` != +1:PROCEED } --> [ TRIPPED ] --> [ Commit Status
= -1:ROLLBACK ]
        ( Tripwire remains armed in Proceed or trips on anomaly to veto execution ),

    # ------------------------------------------------------------------------
    # 4. DIAGNOSTIC DEBUG TELEMETRY
    # ------------------------------------------------------------------------
    [ Debug ]: { [ Coordinate ] :: [ Site` ] :: [ Target ] :: [ Status` ] }
        ( Expose diagnostic telemetry across input coordinate, site location, target pointer, and machine status ),

    # ------------------------------------------------------------------------
    # 5. EPISODIC INVENTORY LOGGING
    # ------------------------------------------------------------------------
    [ Log ]: [ Journal ]: INSCRIBE { [ Site Bound ] :: [ Site` ] :: [ Target ] }
        ( Inscribe the verified site transaction into the permanent episodic system journal ),

    # ------------------------------------------------------------------------
    # 6. PERMANENT COVENANT EXECUTION & FOURFOLD COMMIT STATUS LOOP
    # ------------------------------------------------------------------------
    [ Commit Status ]: [
        # +1: PROCEED (+t: Advance forward along the causal tape)
        { [ Verify ] = PROCEED } & { [ Tripwire ] = ARMED }
            --> [ +1:PROCEED ] |

        #  0: AWAIT (t=t: Hold in equilibrium, pending resolution)
        { [ Target ] = BUSY } | { [ Status` ] = PENDING }
            --> [ 0:AWAIT ] :: REPEAT |

        # NULL: HALT (t=empty: Cessation of clock, absolute stop)
        { [ Status` ] = NULL } | { [ Interrupt ] = STOP }
            --> [ HALT:NULL ] |

        # -1: ROLLBACK (-t: Reverse causal tape to origin t=0)
        { [ Tripwire ] = TRIPPED } | { [ Verify ] != PROCEED }
            --> [ -1:ROLLBACK ]
    ] ( Fourfold state machine: Proceed advances forward, Await holds equilibrium, Halt ceases execution in Null,
Rollback reverts to origin )
}
```

<TRACK 2> : {OmniCode Language & Standard Library}/lexicon/01_nouns/ideas/types/site.tape

TAPE | 89 LINES | 763 WORDS | 5.3 KB

```
# ================================================================================
# PROCESS TAPE: SITE INVARIANT TYPE EXECUTION RIBBON
# ================================================================================
# ARCHITECTURAL TOPOLOGY:
#   Interprets and executes the atomic lifecycle for Invariant Type < Site >.
#   - Defined Space: [ 1 Address ] { 64 Bits / Silicon Coordinate }
#   - Execution Domain: Lexicon { Types of Ideas }
#   - Ladder Ascension: Build-up with Numbers (/-^)
# ================================================================================

= [Tape]: < Lexicon { Types of Ideas } > & [ Process { Site } ] ( Process Ribbon for Site Address Binding ) {
    WHO   { OmniCode Lexicon } ::
    WHAT  { Site } --> LOCATE { PhysicalAddress } ::
    WHEN  { Addressing Event } ::
    WHERE { Silicon Coordinate } = [ 1 Address ] ::
    WHY   ( Inscribe physical site address invariant and hardware MMIO coordinates ) ::
    HOW   [ +1 : PROCEED ] ::
    < Scripture { Genesis 28:16 KJV } > { "And Jacob awaked out of his sleep, and he said, Surely the Lord is in this
place; and I knew it not." } ( Invariant Scripture Witness Genesis 28:16 in the context: Surely the Lord is in this place; and I
knew it not )
}

# --------------------------------------------------------------------------------
# LADDER ASCENSION: BUILD-UP WITH NUMBERS
# --------------------------------------------------------------------------------
/-^ < Numbers > FROM [ < Lexicon { Types of Ideas } > & [ Noun { Number } ] ] ( Build up physical address coordinates
using Numbers from the Lexicon )

# ================================================================================
# DECLARED ROLLS: ATOMIC INVARIANT EXECUTION
# ================================================================================

# --------------------------------------------------------------------------------
# ROLL 1: LOCATE (LADDER BUILD-UP)
# Build up physical address location with numeric scalar coordinate yielding new state Site`
# --------------------------------------------------------------------------------
[ ROLL 1 ] : [ Locate ] : /-^ [ 64 Bits ] --> [ Address ] = [ Site` ]
    ( Build up physical address location with numeric scalar coordinate yielding new state Site-prime )

# --------------------------------------------------------------------------------
# ROLL 2: BIND COORDINATE (OUTSIDE-IN IMPOSITION)
# Bind external coordinate imposed into physical Site yielding new state Located Coordinate`
# --------------------------------------------------------------------------------
[ ROLL 2 ] : [ Bind ] : { Coordinate } --> BIND [ Site` ] = [ Located Coordinate` ]
    ( Bind external coordinate imposed into physical Site yielding new state Located Coordinate-prime )

# --------------------------------------------------------------------------------
# ROLL 3: IDENTITY & NULL GUARD
# Compare located coordinate to input address under guard: proceed on valid address, rollback on null
# --------------------------------------------------------------------------------
[ ROLL 3 ] : [ Guard Address ] :
    { Located Coordinate` = Coordinate } & { Located Coordinate` != 0x00000000 }
    ! GUARD --> [ Status` = +1:PROCEED ]
    | { Located Coordinate` = 0x00000000 } --> [ Status` = -1:ROLLBACK ]
    ( Compare located coordinate to input address under guard: proceed on valid address, rollback on null pointer error )

# --------------------------------------------------------------------------------
# ROLL 4: ROUTE & PROJECT (INSIDE-OUT EMERGENCE)
# Inside-out emergence: verified physical site routes outward to anchor hardware memory
# --------------------------------------------------------------------------------
[ ROLL 4 ] : [ Route ] : [ Site` ] --> ROUTE [ Target ] = [ Memory Anchor` ]
    ( Inside-out emergence: verified physical site routes outward to anchor hardware memory establishing new state Memory
Anchor-prime )

# --------------------------------------------------------------------------------
# ROLL 5: SEQUENCE TRANSITION
# Chain discrete site address outward into continuous system memory map sequence
# --------------------------------------------------------------------------------
[ ROLL 5 ] : [ Sequence ] : [ Site` ] --> SEQUENCE [ Map ] = [ Memory Map` ]
    ( Discrete site address chains outward into continuous physical memory map establishing new state Memory Map-prime )

# --------------------------------------------------------------------------------
# ROLL 6: COMMIT STATUS & FOURFOLD STATE MACHINE
```

```
# Affirm covenant execution across causal time advancement, await, halt, and rollback
# ------------------------------------------------------------------------------
[ ROLL 6 ] : [ Commit Status ] : [
    # +1: PROCEED (+t: Advance forward along the causal tape)
    { [ Verify ] = PROCEED } & { [ Tripwire ] = ARMED }
        --> [ +1:PROCEED ] |

    #  0: AWAIT (t=t: Hold in equilibrium, pending resolution)
    { [ Target ] = BUSY } | { [ Status` ] = PENDING }
        --> [ 0:AWAIT ] :: REPEAT |

    # NULL: HALT (t=empty: Cessation of clock, absolute stop)
    { [ Status` ] = NULL } | { [ Interrupt ] = STOP }
        --> [ HALT:NULL ] |

    # -1: ROLLBACK (-t: Reverse causal tape to origin t=0)
    { [ Tripwire ] = TRIPPED } | { [ Verify ] != PROCEED }
        --> [ -1:ROLLBACK ]
] ( Fourfold commit status: Proceed advances forward, Await holds in equilibrium, Halt ceases execution in Null, Rollback
reverts to origin )
```

<TRACK 2> : {OmniCode Language & Standard Library}/lexicon/01_nouns/ideas/types/state.omni

OMNI | 230 LINES | 2189 WORDS | 14.4 KB

```
# ================================================================================
# BLOCK 1: METADATA BLOCK
# ( Invariant Type: < Idea { Type } : State > )
# ================================================================================
# ARCHITECTURAL TOPOLOGY:
#   Defines the Quad-State truth, causal clock, and control flow invariant.
#   - Base Block: Primitive Invariant Type
#   - Category: Idea { Type }
#   - Footprint: [ 1 Trit ] { 2 Bits / Quad-State Causal Register }
#   - Named Registers: [ STATE TURN BOUND IT ]
#   - States: +1 (PROCEED), 0 (AWAIT), -1 (ROLLBACK), NULL (HALT)
# ================================================================================

= [Module]: < Lexicon { Types of Ideas } > & [ Noun { State } ] ( This is the Module for the Lexicon - Ideas (Type): States - Nouns ) {
    [ Version ]:   { "1.0.0" } ( The Version is in the context of 1.0.0 ),
    [ Created ]:   { "2026-10-11" } ( Created in the context of October 11, 2026 ),
    [ Updated ]:   { "2026-10-13" } ( Last updated in the context of October 13, 2026 ),
    [ Author ]:    { "CreativeWorkzStudio LLC / ProfessorSeanEX" } ( Author in the context of CreativeWorkzStudio LLC and ProfessorSeanEX ),
    [ System ]:    { OmniCode { Lexicon } } ( System is in the context of the OmniCode Lexicon ),
    [ Witness ]:   < Scripture { Ecclesiastes 3:1 KJV } > { "To every thing there is a season, and a time to every purpose under the heaven:" } ( Invariant Scripture Witness Ecclesiastes 3:1 in the context: To every thing there is a season, and a time to every purpose under the heaven ),
    [ Nature ]:    < InvariantType > & { BaseBlock } ( Nature is an Invariant Type and is in the context of Base Block ),
    [ Semantics ]: { QuadStateTruth :: CausalClock :: ControlFlowState } ( Semantics in the context of Quad State Truth, Causal Clock, and Control Flow State )
}

# ------------------------------------------------------------------------------
# LADDER FOUNDATION: INDEPENDENT TEMPORAL BASE
# State does not build up from space or memory envelopes: it is the independent
# atomic base of causal time and Quad-State truth governing execution.
# ------------------------------------------------------------------------------

# Freestanding Invariant Type Definition
< State > = < Idea { Type } > & [ Noun { Causal Truth } ] ( State is the Invariant Idea of Type, and is the Noun of Causal Truth ) {
    # 1. The Ontological Analogy (State's proportion to the universe)
    [ Relation ]: [ State ] : [ Truth ] :: [ Trit ] : [ Causality ] ( State is to Truth as Trit is to Causality ),

    # 2. The Universal Domain (All quad-state transitions, cadence pulses, and control flows)
    [ Domain ]: { Proceed :: Await :: Rollback :: Halt } ( Domain encompasses Proceed, Await, Rollback, and Halt ),

    # 3. The Physical Substrate (1 balanced trit or 2 silicon bits)
    [ Substrate ]: [ 1 Trit { 2 Bits } ] & [ Causal Truth ] ( Substrate is 1 Trit of 2 Bits, and is Causal Truth ),

    # 4. The Universal Inscription Vector (How external truth signal is recorded into the state register)
    [ Inscribe ]: [ Signal ] --> [ STATE ] ( Any truth signal is inscribed into the STATE register ),

    # 5. The Hardware Polarity Transition (Abstract archetype lowers to concrete silicon register)
    [ Lower ]: [ +State v-- State- ::@ 0x0000 ] ( Abstract idea lowers to concrete register at base offset ),

    # 6. The Core Quad-State Truth Law (The fourfold ontological states of computation)
    [ Law { Quad-State } ]: { T4 = [ +1:PROCEED | 0:AWAIT | -1:ROLLBACK | HALT:NULL ] } ( The balanced ternary truth system governing all causation ),

    # 7. Canonical Exemplars (Standard grounding values for Proceed, Await, Rollback, and Halt)
    [ Exemplar { Proceed } ]: (+1) <=> [ PROCEED { Forward Causation } ] ( Forward causation advances execution ),
    [ Exemplar { Await } ]: 0 <=> [ AWAIT { Equilibrium Standby } ] ( Standby equilibrium holds pending external resolution ),
    [ Exemplar { Rollback } ]: (-1) <=> [ ROLLBACK { Retroactive Reversal } ] ( Retroactive reversal unwinds causal drift ),
    [ Exemplar { Halt } ]: NULL <=> [ HALT { Terminal Detent } ] ( Terminal detent halts the execution clock ),

    # 8. The Canonical Looping Law (The formal loop grammar connecting Quad-State to hardware registers)
    [ Law { Loop } ]: LOOP [ WHEN [ { IT } < { BOUND } ] --> [ IT` ] PROCEED to { TURN } ! [ { IT } > { BOUND } ] AWAIT for [ IT` ] :: HALT WHEN [ IT` ] = NULL ] ( Canonical loop statement governing IT, BOUND, and TURN across the quad states ),

    # 9. The Relational Comparison (How states evaluate active truth against signal)
    [ Compare ]: [ { STATE } = { Signal } ] ( State evaluates active register against expected truth signal ),

    # 10. The Sequence Transition (How states chain into continuous causal timelines)
```

```
    [ Sequence ]: [ State ] --> [ Timeline ] ( Discrete states chain together into continuous execution timelines ),

    # 11. The Invariant Boundary (Strict single trit enclosure)
    [ Bound ]: ENCLOSE [ Truth ] WITHIN [ 1 Trit ] ( State is strictly enclosed within a single trit capacity )
}

# ================================================================================
# BLOCK 2: OPENING BLOCK
# ================================================================================
Opening {
    # 1. The Dynamic Input Signal: Accepts incoming causal truth signal
    { Signal ]: { Input :: [ +1:PROCEED ] } ( Outside-in imposition: external causal truth signal to be evaluated ),

    # 2. The Typed Entity: State brought down from Block 1 into concrete allocation
    [ State ]: < State > & [ Register ] ( Denoted State is typed as invariant State allocated as a Register ),

    # 3. The Named Hardware Register Bank: Core registers for state and loop execution
    [ Registers ]: [ STATE TURN BOUND IT ] ( Named hardware registers armed for state and cadence execution ),

    # 4. The Initial Register Values: Initializing loop parameters
    [ Setup ]: [ STATE = 0:AWAIT ] :: [ IT = 0 ] :: [ BOUND = 3 ] :: [ TURN = 0 ] ( Initialize state in Await,
accumulator at zero, bound at 3, turn at zero ),

    # 5. The Hardware Clock Bridge: Arming the causal clock and cadence transitions
    [ Clock ]: { (+1:PROCEED) :: (0:AWAIT) :: (-1:ROLLBACK) :: (HALT:NULL) } ( Hardware clock bridges the four states of
truth ),

    # 6. The Output Target: The execution status register
    [ Target ]: [ Status Register ] ::@ 0x00000000 ( Target output is situated at the primary execution status register
),

    # 7. Preconditions & State: Proving non-null valid signal before transition
    [ Preconditions ]: [ Signal != NULL ] & [ Status = +1:PROCEED ] ( Signal is non-null and machine status is armed in
Proceed )
}

# ================================================================================
# BLOCK 3: BODY BLOCK
# ================================================================================
Body [
    # ------------------------------------------------------------------------
    # ROLL 1: INSCRIBE STATE REGISTER (OUTSIDE-IN IMPOSITION)
    # Inscribe incoming truth signal into the STATE register, establishing new state STATE`
    # ------------------------------------------------------------------------
    [ ROLL 1 ] : [ Inscribe Register ] : { Signal ] --> INSCRIBE [ STATE ] = [ STATE` ]
        ( Inscribe external truth signal imposed into STATE register yielding new state STATE-prime ),

    # ------------------------------------------------------------------------
    # ROLL 2: AFFIRMATIVE PROCEED TRANSITION
    # Transition along the forward causal vector: PROCEED advances forward to { TURN }
    # ------------------------------------------------------------------------
    [ ROLL 2 ] : [ Affirmative Transition ] : [ Clock ]: [ STATE` ] --> [ +1:PROCEED ] to { TURN } = [ Active State` ]
        ( Transition STATE-prime through clock cadence into affirmative Proceed advancing forward to TURN ),

    # ------------------------------------------------------------------------
    # ROLL 3: EQUILIBRIUM AWAIT TRANSITION
    # Transition into equilibrium: AWAIT holds in zero-sum balance for [ IT` ]
    # ------------------------------------------------------------------------
    [ ROLL 3 ] : [ Standby Equilibrium ] : [ Clock ]: [ STATE` ] <=> [ 0:AWAIT ] for [ IT` ] = [ Standby State` ]
        ( Transition STATE-prime into equilibrium Await holding in standby for IT-prime ),

    # ------------------------------------------------------------------------
    # ROLL 4: CAVEAT GUARD & ROLLBACK
    # Evaluate incoming caveat with !: proceed on alignment, or trigger -1:ROLLBACK on discrepancy
    # ------------------------------------------------------------------------
    [ ROLL 4 ] : [ Guard Condition ] :
        { STATE` = Signal }
        ! GUARD --> [ Status` = +1:PROCEED ]
        | { STATE` != Signal } --> [ Status` = -1:ROLLBACK ]
        ( Guard truth against signal under caveat: proceed on alignment, rollback on discrepancy ),

    # ------------------------------------------------------------------------
    # ROLL 5: TERMINAL DETENT TRAP
    # Trap bottom type HALT:NULL on illegal signal or terminal detent
    # ------------------------------------------------------------------------
    [ ROLL 5 ] : [ Trap Detent ] :
        { Status` = NULL }
        ! TRAP --> [ HALT:NULL ]
```

```
        | { Status` != NULL } --> [ Active State` = +1:PROCEED ]
        ( Trap null bottom state under guard detent or proceed with active state ),

    # ------------------------------------------------------------------------
    # ROLL 6: THE CANONICAL LOOP STATEMENT
    # Execute the formal OmniCode loop statement governing IT, BOUND, and TURN across states
    # ------------------------------------------------------------------------
    [ ROLL 6 ] : [ Loop ] :
        LOOP [
            WHEN [ { IT } < { BOUND } ] --> [ IT` ] PROCEED to { TURN }
            ! [ { IT } > { BOUND } ] AWAIT for [ IT` ]
            :: HALT WHEN [ IT` ] = NULL
        ] ( Execute canonical quad-state accumulator loop: proceed to TURN while under BOUND, await when above BOUND,
halt on NULL ),

    # ------------------------------------------------------------------------
    # ROLL 7: GOVERN ENGINE (INSIDE-OUT EMERGENCE)
    # Inside-out emergence: verified state emerges outward to govern the machine clock
    # ------------------------------------------------------------------------
    [ ROLL 7 ] : [ Govern Engine ] : [ STATE` } --> GOVERN [ Control Flow ] = [ Clock` ]
        ( Inside-out emergence: verified state emerges outward to govern machine clock establishing new state Clock-prime
),

    # ------------------------------------------------------------------------
    # ROLL 8: SEQUENCE TRANSITION
    # Chain discrete state into continuous execution timeline sequence
    # ------------------------------------------------------------------------
    [ ROLL 8 ] : [ Sequence ] : [ STATE` ] --> SEQUENCE [ Timeline ] = [ Timeline` ]
        ( Discrete state chains outward into continuous causal timeline establishing new state Timeline-prime )
]

# ============================================================================
# BLOCK 4: CLOSING BLOCK
# ============================================================================
Closing ( Sealing Gate: Reconcile, validate, tripwire, debug, log, and affirm permanent execution ) {
    # ------------------------------------------------------------------------
    # 1. VERIFICATION OF OPERATIONAL ROLLS
    # ------------------------------------------------------------------------
    [ Verify ]: [
        [ ROLL 1 ] = PROCEED ::
        [ ROLL 2 ] = PROCEED ::
        [ ROLL 3 ] = PROCEED ::
        [ ROLL 4 ] = PROCEED ::
        [ ROLL 5 ] = PROCEED ::
        [ ROLL 6 ] = PROCEED ::
        [ ROLL 7 ] = PROCEED ::
        [ ROLL 8 ] = PROCEED
    ] ( Verify that all eight operational rolls achieved affirmative consensus in PROCEED ),

    # ------------------------------------------------------------------------
    # 2. VALIDATION OF INVARIANT BOUNDS & QUAD-STATE INTEGRITY
    # ------------------------------------------------------------------------
    [ Validate ]: { STATE` <= [ 1 Trit ] } & { [ ROLL 6 ] = PROCEED }
        ( Validate that state remains strictly enclosed within 1 Trit and canonical loop sealed in Proceed ),

    # ------------------------------------------------------------------------
    # 3. ZERO-TOLERANCE TRIPWIRING
    # ------------------------------------------------------------------------
    [ Tripwire ]: { Status` = +1:PROCEED } --> [ ARMED ] | { Status` != +1:PROCEED } --> [ TRIPPED ] --> [ Commit Status
= -1:ROLLBACK ]
        ( Tripwire remains armed in Proceed or trips on anomaly to veto execution ),

    # ------------------------------------------------------------------------
    # 4. DIAGNOSTIC DEBUG TELEMETRY
    # ------------------------------------------------------------------------
    [ Debug ]: { [ Signal ] :: [ STATE` ] :: [ IT` ] :: [ TURN` ] :: [ Status` ] }
        ( Expose diagnostic telemetry across input signal, state register, accumulator IT, cadence TURN, and status ),

    # ------------------------------------------------------------------------
    # 5. EPISODIC INVENTORY LOGGING
    # ------------------------------------------------------------------------
    [ Log ]: [ Journal ]: INSCRIBE { [ State Resolved ] :: [ STATE` ] :: [ Target ] }
        ( Inscribe the verified state transaction into the permanent episodic system journal ),

    # ------------------------------------------------------------------------
    # 6. PERMANENT COVENANT EXECUTION & FOURFOLD COMMIT STATUS LOOP
    # ------------------------------------------------------------------------
    [ Commit Status ]: [
```

```
        # +1: PROCEED (+t: Advance forward along the causal tape)
        { [ Verify ] = PROCEED } & { [ Tripwire ] = ARMED }
            --> [ +1:PROCEED ] |

        #  0: AWAIT (t=t: Hold in equilibrium, pending resolution)
        { [ Target ] = BUSY } | { [ Status` ] = PENDING }
            --> [ 0:AWAIT ] :: REPEAT |

        # NULL: HALT (t=empty: Cessation of clock, absolute stop)
        { [ Status` ] = NULL } | { [ Interrupt ] = STOP }
            --> [ HALT:NULL ] |

        # -1: ROLLBACK (-t: Reverse causal tape to origin t=0)
        { [ Tripwire ] = TRIPPED } | { [ Verify ] != PROCEED }
            --> [ -1:ROLLBACK ]
    ] ( Fourfold state machine: Proceed advances forward, Await holds equilibrium, Halt ceases execution in Null,
Rollback reverts to origin )
}
```

<TRACK 2> : {OmniCode Language & Standard Library}/lexicon/01_nouns/ideas/types/state.tape

TAPE | 117 LINES | 949 WORDS | 6.7 KB

```
# ================================================================================
# PROCESS TAPE: STATE INVARIANT TYPE EXECUTION RIBBON
# ================================================================================
# ARCHITECTURAL TOPOLOGY:
#   Interprets and executes the atomic lifecycle for Invariant Type < State >.
#   - Defined Space: [ 1 Trit ] { 2 Bits / Quad-State Causal Register }
#   - Execution Domain: Lexicon { Types of Ideas }
#   - Named Registers: [ STATE TURN BOUND IT ]
#   - Kinematics: 8 Operational Rolls with Canonical Quad-State Accumulator Loop
# ================================================================================

= [Tape]: < Lexicon { Types of Ideas } > & [ Process { State } ] ( Process Ribbon for Quad-State Causal Governance ) {
    WHO   { OmniCode Lexicon } ::
    WHAT  { State } --> GOVERN { CausalTruth } ::
    WHEN  { Control Event } ::
    WHERE { State Register } = [ 1 Trit ] ::
    WHY   ( Inscribe quad-state truth invariant and execute canonical loops ) ::
    HOW   [ +1 : PROCEED ] ::
    < Scripture { Ecclesiastes 3:1 KJV } > { "To every thing there is a season, and a time to every purpose under the heaven:" } ( Invariant Scripture Witness Ecclesiastes 3:1 in the context: To every thing there is a season, and a time to every purpose under the heaven )
}

# --------------------------------------------------------------------------------
# LADDER FOUNDATION: INDEPENDENT TEMPORAL BASE
# --------------------------------------------------------------------------------

# ================================================================================
# DECLARED ROLLS: ATOMIC INVARIANT EXECUTION
# ================================================================================

# --------------------------------------------------------------------------------
# ROLL 1: INSCRIBE STATE REGISTER (OUTSIDE-IN IMPOSITION)
# Inscribe incoming truth signal into the STATE register
# --------------------------------------------------------------------------------
[ ROLL 1 ] : [ Inscribe Register ] : { Signal ] --> INSCRIBE [ STATE ] = [ STATE` ]
    ( Inscribe external truth signal imposed into STATE register yielding new state STATE-prime )

# --------------------------------------------------------------------------------
# ROLL 2: AFFIRMATIVE PROCEED TRANSITION
# Transition along forward causal vector: PROCEED advances forward to { TURN }
# --------------------------------------------------------------------------------
[ ROLL 2 ] : [ Affirmative Transition ] : [ Clock ]: [ STATE` ] --> [ +1:PROCEED ] to { TURN } = [ Active State` ]
    ( Transition STATE-prime through clock cadence into affirmative Proceed advancing forward to TURN )

# --------------------------------------------------------------------------------
# ROLL 3: EQUILIBRIUM AWAIT TRANSITION
# Transition into equilibrium: AWAIT holds in zero-sum balance for [ IT` ]
# --------------------------------------------------------------------------------
[ ROLL 3 ] : [ Standby Equilibrium ] : [ Clock ]: [ STATE` ] <=> [ 0:AWAIT ] for [ IT` ] = [ Standby State` ]
    ( Transition STATE-prime into equilibrium Await holding in standby for IT-prime )

# --------------------------------------------------------------------------------
# ROLL 4: CAVEAT GUARD & ROLLBACK
# Evaluate incoming caveat with !: proceed on alignment, or trigger -1:ROLLBACK on discrepancy
# --------------------------------------------------------------------------------
[ ROLL 4 ] : [ Guard Condition ] :
    { STATE` = Signal }
    ! GUARD --> [ Status` = +1:PROCEED ]
    | { STATE` != Signal } --> [ Status` = -1:ROLLBACK ]
    ( Guard truth against signal under caveat: proceed on alignment, rollback on discrepancy )

# --------------------------------------------------------------------------------
# ROLL 5: TERMINAL DETENT TRAP
# Trap bottom type HALT:NULL on illegal signal or terminal detent
# --------------------------------------------------------------------------------
[ ROLL 5 ] : [ Trap Detent ] :
    { Status` = NULL }
    ! TRAP --> [ HALT:NULL ]
    | { Status` != NULL } --> [ Active State` = +1:PROCEED ]
    ( Trap null bottom state under guard detent or proceed with active state )

# --------------------------------------------------------------------------------
```

```
# ROLL 6: THE CANONICAL LOOP STATEMENT
# Execute the formal OmniCode loop statement governing IT, BOUND, and TURN across states
# ------------------------------------------------------------------------------
[ ROLL 6 ] : [ Loop ] :
    LOOP [
        WHEN [ { IT } < { BOUND } ] --> [ IT` ] PROCEED to { TURN }
        ! [ { IT } > { BOUND } ] AWAIT for [ IT` ]
        :: HALT WHEN [ IT` ] = NULL
    ] ( Execute canonical quad-state accumulator loop: proceed to TURN while under BOUND, await when above BOUND, halt on
NULL )

# ------------------------------------------------------------------------------
# ROLL 7: GOVERN ENGINE (INSIDE-OUT EMERGENCE)
# Inside-out emergence: verified state emerges outward to govern the machine clock
# ------------------------------------------------------------------------------
[ ROLL 7 ] : [ Govern Engine ] : [ STATE` } --> GOVERN [ Control Flow ] = [ Clock` ]
    ( Inside-out emergence: verified state emerges outward to govern machine clock establishing new state Clock-prime )

# ------------------------------------------------------------------------------
# ROLL 8: SEQUENCE TRANSITION
# Chain discrete state into continuous execution timeline sequence
# ------------------------------------------------------------------------------
[ ROLL 8 ] : [ Sequence ] : [ STATE` ] --> SEQUENCE [ Timeline ] = [ Timeline` ]
    ( Discrete state chains outward into continuous causal timeline establishing new state Timeline-prime )

# ------------------------------------------------------------------------------
# ROLL 9: COMMIT STATUS & FOURFOLD STATE MACHINE
# Affirm covenant execution across causal time advancement, await, halt, and rollback
# ------------------------------------------------------------------------------
[ ROLL 9 ] : [ Commit Status ] : [
    # +1: PROCEED (+t: Advance forward along the causal tape)
    { [ Verify ] = PROCEED } & { [ Tripwire ] = ARMED }
        --> [ +1:PROCEED ] |

    #  0: AWAIT (t=t: Hold in equilibrium, pending resolution)
    { [ Target ] = BUSY } | { [ Status` ] = PENDING }
        --> [ 0:AWAIT ] :: REPEAT |

    # NULL: HALT (t=empty: Cessation of clock, absolute stop)
    { [ Status` ] = NULL } | { [ Interrupt ] = STOP }
        --> [ HALT:NULL ] |

    # -1: ROLLBACK (-t: Reverse causal tape to origin t=0)
    { [ Tripwire ] = TRIPPED } | { [ Verify ] != PROCEED }
        --> [ -1:ROLLBACK ]
] ( Fourfold commit status: Proceed advances forward, Await holds in equilibrium, Halt ceases execution in Null, Rollback
reverts to origin )
```

<TRACK 2> : {OmniCode Language & Standard Library}/lexicon/01_nouns/ideas/types/text.omni

OMNI | 62 LINES | 249 WORDS | 2.2 KB

```
# ================================================================================
# BLOCK 1: METADATA BLOCK
# ( Invariant Type: <IDEA:TYPE:TEXT> )
# ================================================================================
# ARCHITECTURAL TOPOLOGY:
#   Defines inscriptional sequences of characters (Words, Phrases, and Strings).
#   - Base Block: Primitive Invariant Type
#   - Category: Idea
#   - Subcategory: Type
#   - Sequence: Ordered ribbon of < Character > cells
# ================================================================================

Module < Lexicon.Nouns.Ideas.Types.Text > ( Inscriptional Word, Phrase & String Invariant ) {
    version: < "1.0.0" >,
    system: < OmniCode.Lexicon >,
    semantics: < BaseBlock :: InvariantType :: InscriptionalText :: TextRibbon >
}

# Invariant Type Declaration: <IDEA:TYPE:TEXT>
< Idea : Type : Text > [ capacity: dynamic cells ] {
    # Polarity transition: Abstract archetype lowers to concrete ribbon
    +Text v-- Text- ::@ 0x0000

    # Inscription bounds
    ENCLOSE [ ribbon : Sequence [ Character ] ] WITHIN [ capacity ]

    # Clarified literal representations
    example_word: "MILLENIUMOS",
    example_phrase: "MILLENIUMOS: OMNICODE 32-TRIT OS ONLINE"
}

# ================================================================================
# BLOCK 2: OPENING BLOCK
# ================================================================================
Opening {
    requires: [
        Text : InvariantNoun ::
        Character : InvariantNoun
    ],
    preconditions: [ +1 ]
}

# ================================================================================
# BLOCK 3: BODY BLOCK
# ================================================================================
Body [
    # Verify text ribbon allocation and phrase inscription
    text_assertion [
        [ TEXT ] [ 32 ] :: { Text } --> INSCRIBE {Phrase}
    ] ( Inscriptional text ribbon allocated holding character sequence )
]

# ================================================================================
# BLOCK 4: CLOSING BLOCK
# ================================================================================
Closing {
    verify: [
        text_assertion = yes
    ],
    commit_status [ PROCEED- ] ( Text invariant type sealed in PROCEED )
}
```

<TRACK 2> : {OmniCode Language & Standard Library}/lexicon/01_nouns/ideas/types/text.tape

TAPE | 33 LINES | 210 WORDS | 1.6 KB

```
# ================================================================================
# TAPE HEAD: TEXT INVARIANT TYPE PROCESS RIBBON
# ================================================================================
# ARCHITECTURAL TOPOLOGY:
#   Executes atomic lifecycle for Invariant Type <IDEA:TYPE:TEXT>.
# ================================================================================

[Tape.Text] : WHO {OmniCode_Kernel} WHAT {TextType} --> INSCRIBE {TextRibbon}
    WHEN {TypeGenesis}
    WHERE {text_partition} = [ 64 ]
    WHY ( Inscribe text ribbon type invariant and clarified string phrases )
    HOW [ +1 : PROCEED ]
    :: < text_landmark > ( Habakkuk 2:2 - 'Write the vision, and make it plain upon tables' )

# ================================================================================
# SUBORDINATE ROLLS: ATOMIC INVARIANT RESOLUTION
# ================================================================================

# Roll 1: Reference abstract text archetype
roll_1 [ +Text ] ( Abstract text archetype referenced )

# Roll 2: Allocate concrete text ribbon
roll_2 [ Text --> Inscribed_Ribbon @ 0 ] ( Inscribe concrete text ribbon at offset 0 )

# Roll 3: Assert structural projection and clarified phrase
roll_3 [ [ TEXT ] [ 32 ] :: "MILLENIUMOS ONLINE" --> [ Inscribed_Ribbon ] ] ( Clarified phrase inscribed )

# Roll 4: Verify text integrity into result prime
roll_4 [ Inscribed_Ribbon = Text ! [ +1 ] --> [ text_verified` ] ] ( Verification yields result prime )

# Roll 5: Seal in affirmative proceed
roll_5 [ commit_status [ PROCEED- ] ] ( Text invariant type sealed and verified )
```

<TRACK 2> : {OmniCode Language & Standard Library}/lexicon/01_nouns/ideas/types/transpile variant/c_backbone/include/character.h

H | 279 LINES | 1671 WORDS | 12.4 KB

```c
/* ============================================================================
 * BLOCK 1: METADATA & INVARIANT TYPE DEFINITION
 * ( Invariant Type: < Idea { Type } : Character > )
 * ============================================================================
 * ARCHITECTURAL TOPOLOGY:
 *   Defines the atomic letter, glyph, and inscription rune in the OmniCode
 *   Standard Library Track 2 C Transpile Variant.
 *   - Base Block: Primitive Invariant Type
 *   - Category: Idea { Type }
 *   - Footprint: [ 1 Cell ] { 5 Balanced Trits / 8 Bits }
 *   - System: OmniCode { Lexicon }
 *   - Witness: Scripture Exodus 31:18 KJV
 *     "And he gave unto Moses, when he had made an end of communing with him upon
 *      mount Sinai, two tables of testimony, tables of stone, written with the
 *      finger of God."
 *   - Semantics: { AtomicGlyph :: LetterRune :: BalancedCellTransduction }
 *   - Invariant Law:
 *     { Character } = [ Enclosed ]: TRUE (Strict single-cell capacity).
 *
 * ONTOLOGICAL CORRESPONDENCE TO OMNICODE:
 *   OmniCode Block 1 declares the immutable laws of the universe for this type.
 *   In C, Block 1 is realized exclusively as:
 *     1. Invariant hardware target addresses and substrate constants.
 *     2. The freestanding invariant struct (omni_character_t) holding dual substrate.
 *     3. Pure mathematical conversion and verification functions that operate
 *        without runtime context dependencies.
 * ============================================================================ */

#ifndef OMNI_CHARACTER_H
#define OMNI_CHARACTER_H

#include "omni_base.h"
#include <stdint.h>
#include <stdbool.h>
#include <stddef.h>

#ifdef __cplusplus
extern "C" {
#endif

/* ----------------------------------------------------------------------------
 * 1. INVARIANT CONSTANTS & HARDWARE LOCATORS (OMNICODE BLOCK 1 & 2)
 * ---------------------------------------------------------------------------- */

/** Number of balanced trits per character cell (3^5 = 243 states). */
#define OMNI_CHAR_TRITS_PER_CELL OMNI_TRITS_PER_CELL

/** Canonical Exemplar: ASCII 'A' (Byte value 65). */
#define OMNI_EXEMPLAR_GLYPH_A    ((uint8_t)'A')

/**
 * @brief Hardware Target Locators (Composed operator ::@).
 *
 * OmniCode Specification:
 *   [ Target ]: { [ Terminal Stdout ] ::@ 0x01 | [ Screen VGA ] ::@ 0x000B8000 }
 */
#define OMNI_TARGET_STDOUT       ((uintptr_t)0x01)       /**< Hosted POSIX terminal stdout stream descriptor */
#define OMNI_TARGET_VGA          ((uintptr_t)0x000B8000) /**< Bare-metal x86 text mode video memory MMIO buffer */

/* ----------------------------------------------------------------------------
 * 2. FREESTANDING INVARIANT TYPE STRUCTURE: omni_character_t
 * ---------------------------------------------------------------------------- */

/**
 * @brief Invariant Type < Character >: The atomic rune of symbolic computing.
 *
 * Encloses the dual substrate:
 *   1. Physical Silicon Substrate: 1 Byte (8 bits, 0..255).
 *   2. Virtual Balanced Trit Substrate: 1 Cell (5 balanced trits, {-1, 0, +1}).
 *
 * Invariant Law:
 *   { Character } = [ Enclosed ]: TRUE (Strict single-cell boundary).
 */
```

```c
typedef struct {
    uint8_t     glyph;                                  /**< Physical byte substrate (8-bit ASCII/binary glyph) */
    omni_trit_t trits[OMNI_CHAR_TRITS_PER_CELL];        /**< Virtual cell substrate (5 balanced ternary trits) */
    bool        is_enclosed;                            /**< Invariant boundary enclosure guarantee */
} omni_character_t;

/* ---------------------------------------------------------------------------
 * 3. PURE INVARIANT OPERATIONS & TRANSDUCTION LAWS
 * --------------------------------------------------------------------------- */

/**
 * @brief Inscription Constructor: Constructs a Character directly from a raw 8-bit glyph.
 *
 * Evaluates OmniCode: [ Inscribe ]: [ Glyph ] --> [ Character ]
 * Automatically performs bijective transduction into 5 balanced trits and
 * establishes the invariant enclosure seal (is_enclosed = true).
 *
 * @param glyph The 8-bit byte codepoint to inscribe.
 * @return Fully initialized omni_character_t in equilibrium.
 */
omni_character_t omni_character_from_glyph(uint8_t glyph);

/**
 * @brief Hardware Transduction Law (<=>): Bijective mapping from 8-bit byte to 5 balanced trits.
 *
 * Evaluates OmniCode: [ Transduce ]: [ Character { Byte } ] <=> [ Cell { Trits } ]
 *
 * Mathematical Invariant:
 *   Each trit is in {-1, 0, +1}, satisfying:
 *     byte = sum_{i=0..4} trits[i] * 3^i
 *   Canonical Exemplar for 'A' (65):
 *     65 <=> [ -1, +1, +1, -1, +1 ]  (-1 + 3 + 9 - 27 + 81 = 65).
 *
 * @param byte_val Input 8-bit byte value (0..255).
 * @param out_trits Destination array of 5 balanced trits.
 */
void omni_character_transduce(uint8_t byte_val, omni_trit_t out_trits[OMNI_CHAR_TRITS_PER_CELL]);

/**
 * @brief Inverse Transduction Law (<=>): Reconstructs an 8-bit byte from 5 balanced trits.
 *
 * Evaluates the bidirectional symmetry of the invariant equivalence operator (<=>).
 * Rebuilds the silicon byte from the balanced ternary representation.
 *
 * @param trits Array of 5 balanced trits.
 * @return Reconstructed 8-bit unsigned byte value.
 */
uint8_t omni_character_to_byte(const omni_trit_t trits[OMNI_CHAR_TRITS_PER_CELL]);

/**
 * @brief Relational Equality Evaluation: Compares two character entities.
 *
 * Evaluates OmniCode: [ Compare ]: [ { Character } = { Glyph } ]
 *
 * @param a First character operand.
 * @param b Second character operand.
 * @return true if glyph values match and both are enclosed; false otherwise.
 */
bool omni_character_eq(const omni_character_t *a, const omni_character_t *b);

/**
 * @brief Boundary Invariant Verification: Proves single-cell enclosure.
 *
 * Evaluates OmniCode: [ Bound ]: { Character } = [ Enclosed ]: TRUE
 *
 * @param c Pointer to character instance to verify.
 * @return true if valid pointer and enclosure flag is sealed; false otherwise.
 */
bool omni_character_verify_bounds(const omni_character_t *c);

/* ===========================================================================
 * BLOCK 2: OPENING BLOCK
 * ( Contextual Genesis, Input Imposition & Hardware Target Binding )
 * ===========================================================================
 * ARCHITECTURAL TOPOLOGY:
 *   OmniCode Block 2 brings the timeless laws of Block 1 down into active space
 *   and time for a concrete execution run:
 *     1. [ Glyph ]: Captures outside-in payload (CLI argument or stream input).
```

```
 *     2. [ Character ] & [ Cell ]: Allocates physical single-cell storage.
 *     3. [ Target ]: Binds to a concrete hardware address (Stdout or VGA MMIO).
 *     4. [ Preconditions ]: Checks ALU balance and arms the execution tripwire.
 * ============================================================================ */

/**
 * @brief Execution Context for Character Lifecycle Rolls.
 *
 * Encapsulates the complete runtime environment for the 4-block lifecycle:
 * outside-in input, allocated cell, target hardware address, optional sequence
 * destination, and safety interlocks.
 */
typedef struct {
    uint8_t               input_glyph;      /**< [ Glyph ]: Imposed outside-in payload */
    omni_character_t      character;        /**< [ Character ] & [ Cell ]: Concrete cell container */
    uintptr_t             target_address;   /**< [ Target ]: Hardware locator (0x01 Stdout or 0x000B8000 VGA) */
    void                 *sequence_target;  /**< [ Sequence ]: Destination buffer for chaining into Text */
    omni_quad_state_t     status;           /**< [ Preconditions ]: Quad-State truth (+1:PROCEED) */
    omni_tripwire_state_t tripwire;         /**< [ Preconditions ]: Subsystem safety interlock */
} omni_character_context_t;

/**
 * @brief Initializes the Character context, applies outside-in imposition,
 *        binds the hardware target, and arms the safety tripwire.
 *
 * Evaluates OmniCode Block 2: Opening { ... }
 *
 * @param ctx Pointer to caller-allocated character execution context.
 * @param input_glyph Outside-in byte payload (defaults to exemplar 'A' if 0).
 * @param target_address Hardware locator (e.g. OMNI_TARGET_STDOUT or OMNI_TARGET_VGA; defaults to STDOUT if 0).
 */
void omni_character_open(omni_character_context_t *ctx, uint8_t input_glyph, uintptr_t target_address);

/* ============================================================================
 * BLOCK 3: BODY BLOCK
 * ( Operational Rolls, Guard Interlocks & Hardware Emission )
 * ============================================================================
 * ARCHITECTURAL TOPOLOGY:
 *   OmniCode Block 3 processes the discrete causal turns (rolls) that advance
 *   state along the causal timeline using the prime operator (`):
 *     - Roll 1: Inscribe input glyph into the Cell container.
 *     - Roll 2: Transduce 8 bits <=> 5 balanced trits.
 *     - Roll 3: Guard identity & boundary invariants via adversative catch (!).
 *     - Roll 4: Project Character` to the hardware target (terminal or VGA).
 *     - Roll 5: Sequence Character` into continuous Text ribbon if bound.
 * ============================================================================ */

/**
 * @brief Roll 1: Inscribes outside-in glyph into the cell, advancing state to Character`.
 *
 * Evaluates OmniCode: [ Inscribe ] : [ Glyph ] --> INSCRIBE [ Cell ] = [ Character` ]
 *
 * @param ctx Active execution context.
 * @param glyph The byte codepoint being inscribed.
 */
void omni_character_inscribe(omni_character_context_t *ctx, uint8_t glyph);

/**
 * @brief Roll 4: Projects Character` outward to the bound hardware target address.
 *
 * Evaluates OmniCode: [ Project ] : [ Character` ] --> PROJECT [ Target ] = [ Target` ]
 *
 * Machine Effect:
 *   - Target == 0x01 (Stdout): Emits glyph byte to file descriptor 1 (terminal).
 *   - Target == 0x000B8000 (VGA): Writes glyph + attribute to MMIO video memory.
 *
 * @param c Pointer to the verified Character to project.
 * @param target_address The physical destination address (0x01 or 0x000B8000).
 * @return true if projection succeeded; false on hardware fault.
 */
bool omni_character_project(const omni_character_t *c, uintptr_t target_address);

/**
 * @brief Roll 5: Yields Character` to continuous Text ribbon sequence.
 *
 * Evaluates OmniCode: [ Sequence ] : [ Character` ] --> SEQUENCE [ Text ] = [ Text` ]
 *
 * Machine Effect:
```

```
 *   If sequence_target is non-NULL, appends the character glyph into the sequence
 *   destination buffer. If NULL, passes as a single atomic character cycle.
 *
 * @param c Pointer to the Character being sequenced.
 * @param sequence_target Pointer to destination buffer or container.
 * @return true if sequence yield succeeded; false on buffer overflow.
 */
bool omni_character_sequence(const omni_character_t *c, void *sequence_target);

/**
 * @brief Executes the complete 5-roll operational pipeline across declared turns:
 *   Roll 1 (Inscribe) -> Roll 2 (Transduce) -> Roll 3 (Guard !) ->
 *   Roll 4 (Project)   -> Roll 5 (Sequence).
 *
 * The Guard turn (Roll 3) enforces the adversative catch (!):
 *   If identity and boundary invariants hold, advances status to +1:PROCEED.
 *   If any discrepancy is detected, trips the wire and rolls back to -1:ROLLBACK.
 *
 * @param ctx Pointer to armed character execution context.
 * @return OMNI_STATE_PROCEED (+1) on equilibrium; OMNI_STATE_ROLLBACK (-1) on violation.
 */
omni_quad_state_t omni_character_execute(omni_character_context_t *ctx);

/* ============================================================================
 * BLOCK 4: CLOSING BLOCK
 * ( Integrity Verification, Covenant Sealing & Quad-State Exit )
 * ============================================================================
 * ARCHITECTURAL TOPOLOGY:
 *   OmniCode Block 4 reconciles all rolls, validates tripwire status, and seals
 *   the permanent commit status (+1:PROCEED, 0:AWAIT, NULL:HALT, -1:ROLLBACK).
 *   In hosted environments, +1:PROCEED maps to process exit code 0.
 * ============================================================================ */

/**
 * @brief Validates all invariants, verifies operational consensus, disarms the
 *        tripwire, and seals the final Character commit status.
 *
 * Evaluates OmniCode Block 4: Closing { ... }
 *
 * @param ctx Pointer to active character execution context.
 * @return OMNI_STATE_PROCEED (+1) on successful seal; OMNI_STATE_ROLLBACK (-1) if tripped.
 */
omni_quad_state_t omni_character_close(omni_character_context_t *ctx);

#ifdef __cplusplus
}
#endif

#endif /* OMNI_CHARACTER_H */
```

<TRACK 2> : {OmniCode Language & Standard Library}/lexicon/01_nouns/ideas/types/transpile variant/c_backbone/include/container.h

H | 222 LINES | 1078 WORDS | 9.0 KB

```c
/* ============================================================================
 * BLOCK 1: METADATA & INVARIANT TYPE DEFINITION
 * ( Invariant Type: < Idea { Type } : Container > )
 * ============================================================================
 * ARCHITECTURAL TOPOLOGY:
 *   Defines bounded memory envelopes, arrays, structures, and storage vessels
 *   in the OmniCode Standard Library Track 2 C Transpile Variant.
 *   - Base Block: Primitive Invariant Type
 *   - Category: Idea { Type }
 *   - Footprint: [ Dynamically Bounded Memory Envelope / Generic Slice ]
 *   - Ladder Ascension: Builds up with Numbers (/-^)
 *   - System: OmniCode { Lexicon }
 *   - Witness: Scripture 2 Kings 4:4 KJV
 *     "And when thou art come in, thou shalt shut the door upon thee and upon
 *      thy sons, and shalt pour out into all those vessels, and thou shalt set
 *      aside that which is full."
 *   - Semantics: { BoundedEnvelope :: StorageVessel :: MemoryCapsule :: ContainmentLaw }
 * ============================================================================ */

#ifndef OMNI_CONTAINER_H
#define OMNI_CONTAINER_H

#include "omni_base.h"

#ifdef __cplusplus
extern "C" {
#endif

/* ----------------------------------------------------------------------------
 * 1. INVARIANT CONSTANTS & CANONICAL EXEMPLARS
 * ---------------------------------------------------------------------------- */

/** Canonical Unit Exemplar: Single element partition (1 unit). */
#define OMNI_CONTAINER_EXEMPLAR_UNIT        ((size_t)1)

/** Canonical Vessel Exemplar: Standard 64-element storage vessel. */
#define OMNI_CONTAINER_EXEMPLAR_VESSEL      ((size_t)64)

/** Canonical Storehouse Exemplar: Large 4096-element memory storehouse. */
#define OMNI_CONTAINER_EXEMPLAR_STOREHOUSE  ((size_t)4096)

/* ----------------------------------------------------------------------------
 * 2. FREESTANDING INVARIANT TYPE STRUCTURE: omni_container_t
 * ---------------------------------------------------------------------------- */

/**
 * @brief Invariant Type < Container >: The generic bounded memory envelope / slice.
 *
 * Provides memory safety by construction without requiring a heap or garbage collector:
 *   - Wraps a memory buffer of arbitrary element types (Character, Number, Site, State).
 *   - Strictly enforces the Core Containment Law: Occupancy <= Capacity.
 *
 * Core Invariant Law:
 *   ENCLOSE [ Storage ] WITHIN [ Capacity ]
 *   LAW { Containment } : [ { Occupancy } <= { Capacity } ]
 */
typedef struct {
    void   *buffer;                                 /**< Raw memory vessel holding discrete elements */
    size_t  element_size;                           /**< Footprint of a single element (sizeof(T) in bytes) */
    size_t  capacity;                               /**< Allocated maximum element capacity boundary */
    size_t  occupancy;                              /**< Current count of stored elements */
    bool    is_enclosed;                            /**< Invariant enclosure guarantee */
} omni_container_t;

/**
 * @brief Convenience static vessel initialized over a 64-byte storage partition.
 */
typedef struct {
    uint8_t          cells[OMNI_CONTAINER_EXEMPLAR_VESSEL]; /**< Concrete 64-byte static buffer */
    omni_container_t container;                             /**< Bounded slice wrapper */
} omni_byte_vessel_t;

/* ----------------------------------------------------------------------------
```

```
 * 3. 4-BLOCK PIPELINE CONTEXT: omni_container_context_t
 * ------------------------------------------------------------------------------ */

/**
 * @brief Execution context for Container encapsulation lifecycle rolls.
 */
typedef struct {
    const void            *payload_ptr;                /**< Imposed outside-in payload buffer */
    size_t                 payload_count;              /**< Count of incoming items to encapsulate */
    omni_container_t       vessel;                     /**< Concrete bounded vessel envelope */
    uintptr_t              target_destination;         /**< Destination memory address or anchor */
    omni_quad_state_t      status;                     /**< Active Quad-State truth */
    omni_tripwire_state_t  tripwire;                   /**< Subsystem safety interlock */
} omni_container_context_t;

/* ================================================================================
 * BLOCK 2: OPENING BLOCK
 * ( Outside-in Imposition & Context Initialization )
 * ================================================================================ */

/**
 * @brief Initializes a Container context over a provided memory storage partition.
 *
 * @param ctx Pointer to caller-allocated container context.
 * @param storage Memory buffer allocated by the caller (stack or static).
 * @param elem_size Footprint of a single element in bytes.
 * @param capacity Maximum number of elements storage can hold.
 * @param payload Outside-in memory payload to encapsulate.
 * @param payload_count Number of elements in the incoming payload.
 */
void omni_container_open(omni_container_context_t *ctx,
                         void *storage,
                         size_t elem_size,
                         size_t capacity,
                         const void *payload,
                         size_t payload_count);

/* ================================================================================
 * BLOCK 3: BODY BLOCK
 * ( Operational Primitives, Containment Laws & Inscriptions )
 * ================================================================================ */

/**
 * @brief Initializes a generic bounded container slice over an existing memory buffer.
 *
 * @param c Pointer to container struct to initialize.
 * @param buffer Pointer to underlying memory array.
 * @param element_size Byte size of each element.
 * @param capacity Maximum number of elements.
 */
void omni_container_init(omni_container_t *c, void *buffer, size_t element_size, size_t capacity);

/**
 * @brief Initializes a canonical 64-byte storage vessel.
 *
 * @param vessel Pointer to byte vessel struct.
 */
void omni_byte_vessel_init(omni_byte_vessel_t *vessel);

/**
 * @brief Core Containment Law Verification: Proves that Occupancy <= Capacity.
 *
 * @param c Pointer to container to evaluate.
 * @return true if Occupancy <= Capacity and properly enclosed; false otherwise.
 */
bool omni_container_verify_containment(const omni_container_t *c);

/**
 * @brief Queries whether the container has reached its maximum capacity limit.
 *
 * @param c Pointer to container.
 * @return true if Occupancy >= Capacity; false otherwise.
 */
bool omni_container_is_full(const omni_container_t *c);

/**
 * @brief Queries whether the container has zero stored elements.
 *
```

```
 * @param c Pointer to container.
 * @return true if Occupancy == 0; false otherwise.
 */
bool omni_container_is_empty(const omni_container_t *c);

/**
 * @brief Pushes an element into the container, advancing occupancy by 1.
 *
 * @param c Pointer to container.
 * @param element Pointer to element data to copy in.
 * @return OMNI_STATE_PROCEED on successful insertion; OMNI_STATE_ROLLBACK if full.
 */
omni_quad_state_t omni_container_push(omni_container_t *c, const void *element);

/**
 * @brief Retrieves a direct pointer to the element at the specified index.
 *
 * @param c Pointer to container.
 * @param index 0-based element index.
 * @return Pointer to element memory, or NULL if index >= occupancy.
 */
void *omni_container_get(const omni_container_t *c, size_t index);

/**
 * @brief Replaces or writes an element at the specified index.
 *
 * @param c Pointer to container.
 * @param index 0-based element index.
 * @param element Pointer to element data to copy in.
 * @return OMNI_STATE_PROCEED on success; OMNI_STATE_ROLLBACK if index >= capacity.
 */
omni_quad_state_t omni_container_set(omni_container_t *c, size_t index, const void *element);

/**
 * @brief Resets container occupancy to zero without clearing underlying memory.
 *
 * @param c Pointer to container.
 */
void omni_container_clear(omni_container_t *c);

/**
 * @brief Executes the full encapsulation lifecycle across declared rolls:
 *   - Roll 1: Build capsule envelope with numbers.
 *   - Roll 2: Encapsulate incoming payload.
 *   - Roll 3: Guard bounds under Law { Containment } (Occupancy <= Capacity).
 *   - Roll 4: Pour out and set aside vessel.
 *
 * @param ctx Pointer to active container context.
 * @return OMNI_STATE_PROCEED on equilibrium; OMNI_STATE_ROLLBACK on overflow.
 */
omni_quad_state_t omni_container_execute(omni_container_context_t *ctx);

/* ============================================================================
 * BLOCK 4: CLOSING BLOCK
 * ( Integrity Verification & Commit Status )
 * ============================================================================ */

/**
 * @brief Validates all invariants and seals the Container commit status.
 *
 * @param ctx Pointer to active container context.
 * @return OMNI_STATE_PROCEED on successful seal; OMNI_STATE_ROLLBACK if tripped.
 */
omni_quad_state_t omni_container_close(omni_container_context_t *ctx);

#ifdef __cplusplus
}
#endif

#endif /* OMNI_CONTAINER_H */
```

<TRACK 2> : {OmniCode Language & Standard Library}/lexicon/01_nouns/ideas/types/transpile variant/c_backbone/include/number.h

H | 189 LINES | 863 WORDS | 7.4 KB

```c
/* ============================================================================
 * BLOCK 1: METADATA & INVARIANT TYPE DEFINITION
 * ( Invariant Type: < Idea { Type } : Number > )
 * ============================================================================
 * ARCHITECTURAL TOPOLOGY:
 *   Defines scalar quantities, integers, and balanced ternary measurements in the
 *   OmniCode Standard Library Track 2 C Transpile Variant.
 *   - Base Block: Primitive Invariant Type
 *   - Category: Idea { Type }
 *   - Footprint: [ 1 Word ] { 32 Balanced Trits / 64 Bits }
 *   - System: OmniCode { Lexicon }
 *   - Witness: Scripture Psalm 147:4 KJV
 *     "He telleth the number of the stars; he calleth them all by their names."
 *   - Semantics: { ScalarQuantity :: NumericWord :: BalancedTernaryALU }
 * ============================================================================ */

#ifndef OMNI_NUMBER_H
#define OMNI_NUMBER_H

#include "omni_base.h"

#ifdef __cplusplus
extern "C" {
#endif

/* ----------------------------------------------------------------------------
 * 1. INVARIANT CONSTANTS & CANONICAL EXEMPLARS
 * ---------------------------------------------------------------------------- */

/** Number of balanced trits per machine word. */
#define OMNI_NUM_TRITS_PER_WORD     OMNI_TRITS_PER_WORD
#define OMNI_NUMBER_TRITS_PER_WORD  OMNI_TRITS_PER_WORD

/** Canonical Zero Exemplar: Perfect balance and null displacement. */
#define OMNI_EXEMPLAR_ZERO          ((int64_t)0)

/** Canonical Positive Unity Exemplar: Forward causal affirmation (+1). */
#define OMNI_EXEMPLAR_UNITY_POS     ((int64_t)1)

/** Canonical Negative Unity Exemplar: Backward causal restoration (-1). */
#define OMNI_EXEMPLAR_UNITY_NEG     ((int64_t)-1)

/** Canonical Memory Partition Exemplar: Standard 4096-unit allocation boundary. */
#define OMNI_EXEMPLAR_PARTITION     ((int64_t)4096)

/* ----------------------------------------------------------------------------
 * 2. FREESTANDING INVARIANT TYPE STRUCTURE: omni_number_t
 * ---------------------------------------------------------------------------- */

/**
 * @brief Invariant Type < Number >: The scalar quantity and measurement metric.
 *
 * Encloses the dual substrate:
 *   1. Physical Silicon Substrate: 64-Bit Signed Integer Scalar (int64_t).
 *   2. Virtual Balanced Trit Substrate: 32-Trit Machine Word.
 *
 * Core Invariant Law:
 *   ENCLOSE [ Number ] WITHIN [ 1 Word ] (Strict 32-trit capacity).
 *   LAW { Equilibrium } : [ (+1) plus (-1) <=> 0 ] (Foundational zero-sum balance).
 */
typedef struct {
    int64_t     scalar;                             /**< Physical silicon scalar integer (64 bits) */
    omni_trit_t trits[OMNI_NUM_TRITS_PER_WORD];     /**< Virtual machine word substrate (32 balanced trits) */
    bool        is_enclosed;                        /**< Invariant boundary enclosure guarantee */
} omni_number_t;

/* ----------------------------------------------------------------------------
 * 3. 4-BLOCK PIPELINE CONTEXT: omni_number_context_t
 * ---------------------------------------------------------------------------- */

/**
 * @brief Execution context for Number measurement and ALU lifecycle rolls.
 */
```

```c
typedef struct {
    int64_t               input_quantity;              /**< Outside-in scalar measurement payload */
    omni_number_t         number;                      /**< Concrete single-word allocation */
    uintptr_t             target_accumulator;          /**< Target register accumulator destination */
    omni_quad_state_t     status;                      /**< Active Quad-State truth */
    omni_tripwire_state_t tripwire;                    /**< Subsystem safety interlock */
} omni_number_context_t;

/* ============================================================================
 * BLOCK 2: OPENING BLOCK
 * ( Outside-in Imposition & Context Initialization )
 * ============================================================================ */

/**
 * @brief Initializes the Number context, arms the ALU, and verifies equilibrium.
 *
 * @param ctx Pointer to caller-allocated number context.
 * @param input_quantity Outside-in scalar quantity (defaults to 4096 if 0).
 */
void omni_number_open(omni_number_context_t *ctx, int64_t input_quantity);

/* ============================================================================
 * BLOCK 3: BODY BLOCK
 * ( Operational Primitives, Arithmetic Laws & Inscriptions )
 * ============================================================================ */

/**
 * @brief Constructs a Number directly from a 64-bit scalar integer,
 *        automatically transducing it into 32 balanced trits.
 *
 * @param scalar The 64-bit integer quantity to measure.
 * @return Fully initialized omni_number_t with is_enclosed = true.
 */
omni_number_t omni_number_from_scalar(int64_t scalar);

/**
 * @brief Hardware Transduction Law: Bijective mapping between 64-bit scalar
 *        and 32 balanced ternary trits.
 *
 * @param val Input 64-bit scalar quantity.
 * @param out_trits Array of 32 balanced trits receiving the radix transduction.
 */
void omni_number_transduce(int64_t val, omni_trit_t out_trits[OMNI_NUM_TRITS_PER_WORD]);

/**
 * @brief Inverse Transduction: Reconstructs a 64-bit scalar from 32 balanced trits.
 *
 * @param trits Array of 32 balanced trits.
 * @return Reconstructed 64-bit signed integer value.
 */
int64_t omni_number_to_scalar(const omni_trit_t trits[OMNI_NUM_TRITS_PER_WORD]);

/**
 * @brief Verifies the ALU Core Arithmetic Equilibrium Law: (+1) plus (-1) <=> 0.
 *
 * @return true if zero-sum balance holds; false otherwise.
 */
bool omni_number_verify_equilibrium(void);

/**
 * @brief Adds two Numbers producing a new balanced Number word.
 *
 * @param a First number summand.
 * @param b Second number summand.
 * @return Resulting sum as an invariant omni_number_t.
 */
omni_number_t omni_number_add(omni_number_t a, omni_number_t b);

/**
 * @brief Evaluates relational equality between two numbers.
 *
 * @param a First number operand.
 * @param b Second number operand.
 * @return true if scalar values match; false otherwise.
 */
bool omni_number_eq(const omni_number_t *a, const omni_number_t *b);

/**
```

```
 * @brief Verifies that a number is properly enclosed within its single-word boundary.
 *
 * @param n Pointer to number to verify.
 * @return true if valid and enclosed; false otherwise.
 */
bool omni_number_verify_bounds(const omni_number_t *n);

/**
 * @brief Executes the full measurement lifecycle across declared rolls:
 *   - Roll 1: Inscribe quantity into word.
 *   - Roll 2: Transduce 64 bits <=> 32 balanced trits.
 *   - Roll 3: Guard identity and boundary invariants.
 *   - Roll 4: Prove ALU zero-sum equilibrium.
 *
 * @param ctx Pointer to active number context.
 * @return OMNI_STATE_PROCEED on equilibrium; OMNI_STATE_ROLLBACK on violation.
 */
omni_quad_state_t omni_number_execute(omni_number_context_t *ctx);

/* ============================================================================
 * BLOCK 4: CLOSING BLOCK
 * ( Integrity Verification & Commit Status )
 * ============================================================================ */

/**
 * @brief Validates all invariants and seals the Number commit status.
 *
 * @param ctx Pointer to active number context.
 * @return OMNI_STATE_PROCEED on successful seal; OMNI_STATE_ROLLBACK if tripped.
 */
omni_quad_state_t omni_number_close(omni_number_context_t *ctx);

#ifdef __cplusplus
}
#endif

#endif /* OMNI_NUMBER_H */
```

<TRACK 2> : {OmniCode Language & Standard Library}/lexicon/01_nouns/ideas/types/transpile variant/c_backbone/include/omni.h

H | 27 LINES | 136 WORDS | 1.1 KB

```c
/* ============================================================================
 * OMNICODE FREESTANDING STANDARD LIBRARY (C MASTER HEADER)
 * ============================================================================
 * ARCHITECTURAL TOPOLOGY:
 *   Umbrella header exposing all primitive invariant types and operational
 *   primitives for OmniCode bare-metal and hosted runtimes.
 *   - Category: Standard Library Track 2 C Transpile Variant
 *   - Invariant Types Included:
 *       < Character > : Atomic Glyph / Letter Rune (1 Cell / 5 Trits)
 *       < Number >    : Scalar Quantity / Measure (1 Word / 32 Trits)
 *       < Container > : Generic Bounded Space Envelope / Slice
 *       < Site >      : Silicon Coordinate / Address (RAM @ vs MMIO ::@)
 *       < State >     : Quad-State Causal Truth & Loop Stepper
 * ============================================================================ */

#ifndef OMNI_H
#define OMNI_H

#include "omni_base.h"
#include "character.h"
#include "number.h"
#include "container.h"
#include "site.h"
#include "state.h"

#endif /* OMNI_H */
```

<TRACK 2> : {OmniCode Language & Standard Library}/lexicon/01_nouns/ideas/types/transpile variant/c_backbone/include/omni_base.h

H | 289 LINES | 1270 WORDS | 10.6 KB

```c
/* ============================================================================
 * BLOCK 1: METADATA & INVARIANT AXIOMS BLOCK
 * ( Invariant Root: < Lexicon { Types of Ideas } > & [ Noun { Base Axioms } ] )
 * ============================================================================
 * ARCHITECTURAL TOPOLOGY:
 *   Foundational bare-metal substrates, balanced ternary types, and Quad-State
 *   truth logic governing the OmniCode Standard Library Track 2 C Transpile Variant.
 *   - Environment: Freestanding C99/C11 (Zero libc/OS dependency)
 *   - Word Size: 64-Bit Silicon Scalar / 32 Balanced Trits
 *   - Cell Size: 8-Bit Byte / 5 Balanced Trits
 *   - System: OmniCode { Lexicon }
 *   - Witness: Scripture Proverbs 11:1 KJV
 *     "A false balance is abomination to the LORD: but a just weight is his delight."
 *   - Semantics: { BalancedTernary :: QuadStateTruth :: SiliconAnchors :: TripwireGuards }
 * ============================================================================ */

#ifndef OMNI_BASE_H
#define OMNI_BASE_H

#include <stdint.h>
#include <stdbool.h>
#include <stddef.h>

#ifdef __cplusplus
extern "C" {
#endif

/* ----------------------------------------------------------------------------
 * ARCHITECTURAL CONSTANTS & SUBSTRATE CAPACITIES
 * ---------------------------------------------------------------------------- */

/** Number of balanced trits packed into a single 8-bit memory Cell (3^5 = 243 states). */
#define OMNI_TRITS_PER_CELL         5

/** Number of balanced trits packed into a single 64-bit machine Word. */
#define OMNI_TRITS_PER_WORD         32

/** Footprint of the standard bounded Container vessel exemplar (in bytes). */
#define OMNI_VESSEL_DEFAULT_CELLS   64

/* ----------------------------------------------------------------------------
 * 1. BALANCED TERNARY SUBSTRATE: omni_trit_t
 * ---------------------------------------------------------------------------- */

/**
 * @brief Discrete balanced ternary digit (-1, 0, +1).
 *
 * Unlike binary (0, 1), balanced ternary is inherently symmetrical around zero:
 *   - Negative (-1) : Backward causation, negation, deficit, or restoration.
 *   - Zero     ( 0) : Equilibrium, standby, neutral, or unperturbed balance.
 *   - Positive (+1) : Forward causation, affirmative assertion, or surplus.
 */
typedef int8_t omni_trit_t;

#define OMNI_TRIT_NEG   ((omni_trit_t)-1)
#define OMNI_TRIT_ZERO  ((omni_trit_t) 0)
#define OMNI_TRIT_POS   ((omni_trit_t) 1)

/* ----------------------------------------------------------------------------
 * 2. QUAD-STATE CAUSAL TRUTH: omni_quad_state_t
 * ---------------------------------------------------------------------------- */

/**
 * @brief The fourfold causal truth states of computation in OmniCode.
 *
 * Unifies boolean logic, error handling, scheduling, and processor clock detents:
 *   - OMNI_STATE_PROCEED  (+1) : Forward Causation (+t). Operation completed in equilibrium.
 *   - OMNI_STATE_AWAIT    ( 0) : Equilibrium Standby (t=t). Pending asynchronous resolution.
 *   - OMNI_STATE_ROLLBACK (-1) : Retroactive Reversal (-t). Guard tripped; unwind causal drift.
 *   - OMNI_STATE_HALT   (-128) : Terminal Detent (t=empty). Clock detent; permanent halt (HLT).
 */
typedef enum {
    OMNI_STATE_ROLLBACK = -1,   /**< -1 : Retroactive Reversal / Error / Unwind */
```

```
    OMNI_STATE_AWAIT    =  0,   /**<  0 : Equilibrium Standby / Hold Pending */
    OMNI_STATE_PROCEED  =  1,   /**< +1 : Forward Causation / Affirmative Advancement */
    OMNI_STATE_HALT     = -128  /**< NULL : Terminal Detent / Stop Processor Clock */
} omni_quad_state_t;

/**
 * @brief Tertiary execution status subset for non-terminal roll checkpoints.
 */
typedef enum {
    OMNI_STATUS_ROLLBACK = -1,
    OMNI_STATUS_AWAIT    =  0,
    OMNI_STATUS_PROCEED  =  1
} omni_status_t;

/* ---------------------------------------------------------------------------
 * 3. HARDWARE COGNITIVE SITES (EXEMPLARS)
 * --------------------------------------------------------------------------- */

/** Unmapped null pointer origin. */
#define OMNI_SITE_NULL          ((uintptr_t)0x00000000)

/** Real-mode Master Boot Record physical origin. */
#define OMNI_SITE_MBR_BOOT      ((uintptr_t)0x00007C00)

/** Hardware UART serial port base (COM1) for kernel telemetry. */
#define OMNI_SITE_COM1          ((uintptr_t)0x000003F8)

/** Physical memory-mapped VGA text-mode display buffer (80x25 characters). */
#define OMNI_SITE_VGA_BUFFER    ((uintptr_t)0x000B8000)

/* ---------------------------------------------------------------------------
 * 4. TRIPWIRE & INTEGRITY DETENTS: omni_tripwire_state_t
 * --------------------------------------------------------------------------- */

/**
 * @brief Safety interlock state preventing silent memory corruption or causal leakage.
 */
typedef enum {
    OMNI_TRIPWIRE_DISARMED = 0, /**< Interlock inactive; safe quiescent state */
    OMNI_TRIPWIRE_ARMED    = 1, /**< Active guard monitoring execution bounds */
    OMNI_TRIPWIRE_TRIPPED  = 2  /**< Violation intercepted; execution blocked */
} omni_tripwire_state_t;

/* ===========================================================================
 * BLOCK 2: OPENING BLOCK
 * ( Subsystem Preconditions, Verification & Initializers )
 * =========================================================================== */

/**
 * @brief Verifies that the host hardware meets OmniCode's bare-metal preconditions:
 *   1. Standard 64-bit pointer architecture (sizeof(uintptr_t) == 8).
 *   2. Standard 8-bit octet cell substrate (sizeof(uint8_t) == 1).
 *   3. Zero-sum arithmetic equilibrium ((+1) + (-1) == 0).
 *
 * @return OMNI_STATE_PROCEED if all preconditions hold; OMNI_STATE_ROLLBACK otherwise.
 */
static inline omni_quad_state_t omni_base_verify_preconditions(void) {
    if (sizeof(uintptr_t) != 8) return OMNI_STATE_ROLLBACK;
    if (sizeof(uint8_t)   != 1) return OMNI_STATE_ROLLBACK;

    /* Prove Proverbs 11:1 Law { Equilibrium }: (+1) plus (-1) <=> 0 */
    omni_trit_t pos = OMNI_TRIT_POS;
    omni_trit_t neg = OMNI_TRIT_NEG;
    if ((pos + neg) != OMNI_TRIT_ZERO) return OMNI_STATE_ROLLBACK;

    return OMNI_STATE_PROCEED;
}

/**
 * @brief Arms a tripwire interlock before entering an untrusted or dynamic execution block.
 */
static inline void omni_tripwire_arm(omni_tripwire_state_t *tw) {
    if (tw) *tw = OMNI_TRIPWIRE_ARMED;
}

/**
 * @brief Disarms a tripwire upon successful verification and closing commit.
 */
```

```c
static inline void omni_tripwire_disarm(omni_tripwire_state_t *tw) {
    if (tw) *tw = OMNI_TRIPWIRE_DISARMED;
}

/**
 * @brief Immediately trips the interlock, halting forward causation.
 */
static inline void omni_tripwire_trip(omni_tripwire_state_t *tw) {
    if (tw) *tw = OMNI_TRIPWIRE_TRIPPED;
}

/* ============================================================================
 * BLOCK 3: BODY BLOCK
 * ( Balanced Ternary Algebra, Quad-State Logic & Guard Inlines )
 * ============================================================================ */

/**
 * @brief Negates a balanced trit along the causal vector (-t <=> +t).
 */
static inline omni_trit_t omni_trit_negate(omni_trit_t t) {
    return (omni_trit_t)(-t);
}

/**
 * @brief Multiplies two balanced trits (commutative zero-sum product).
 */
static inline omni_trit_t omni_trit_multiply(omni_trit_t a, omni_trit_t b) {
    return (omni_trit_t)(a * b);
}

/**
 * @brief Full 1-trit balanced ternary adder with carry output.
 *
 * Truth Table:
 *   a   b  cin | sum carry
 *  -1  -1   -1 |   0    -1
 *  -1  -1    0 |  +1    -1
 *  -1  -1   +1 |  -1     0
 *  -1   0    0 |  -1     0
 *  -1  +1    0 |   0     0  (Equilibrium)
 *  +1  +1    0 |  -1    +1
 *  +1  +1   +1 |   0    +1
 */
static inline omni_trit_t omni_trit_add(omni_trit_t a, omni_trit_t b, omni_trit_t cin, omni_trit_t *cout) {
    int32_t sum = (int32_t)a + (int32_t)b + (int32_t)cin;
    omni_trit_t carry = 0;

    if (sum > 1) {
        sum -= 3;
        carry = 1;
    } else if (sum < -1) {
        sum += 3;
        carry = -1;
    }

    if (cout) *cout = carry;
    return (omni_trit_t)sum;
}

/**
 * @brief Evaluates an invariant guard condition.
 *
 * If condition evaluates to true:
 *   Status is confirmed in PROCEED (+1).
 * If condition evaluates to false:
 *   Tripwire is tripped and status transitions to ROLLBACK (-1).
 */
static inline omni_quad_state_t omni_guard_assert(bool condition,
                                                  omni_tripwire_state_t *tw,
                                                  omni_quad_state_t *status) {
    if (condition) {
        if (status) *status = OMNI_STATE_PROCEED;
        return OMNI_STATE_PROCEED;
    } else {
        if (tw) *tw = OMNI_TRIPWIRE_TRIPPED;
        if (status) *status = OMNI_STATE_ROLLBACK;
        return OMNI_STATE_ROLLBACK;
    }
```

```
}

/** Predicate queries for Quad-State truth */
static inline bool omni_is_proceed(omni_quad_state_t s) { return (s == OMNI_STATE_PROCEED); }
static inline bool omni_is_await(omni_quad_state_t s)   { return (s == OMNI_STATE_AWAIT); }
static inline bool omni_is_rollback(omni_quad_state_t s){ return (s == OMNI_STATE_ROLLBACK); }
static inline bool omni_is_halt(omni_quad_state_t s)    { return (s == OMNI_STATE_HALT); }

/* ============================================================================
 * BLOCK 4: CLOSING BLOCK
 * ( Integrity Seals, Detents & Commit Status Functions )
 * ============================================================================ */

/**
 * @brief Validates the integrity of an execution context before sealing the commit status.
 *
 * Verifies that:
 *   1. No tripwire was tripped during execution.
 *   2. Machine status is not trapped in an unhandled rollback.
 *
 * @return OMNI_STATE_PROCEED if context is clean; OMNI_STATE_ROLLBACK on failure.
 */
static inline omni_quad_state_t omni_base_verify_integrity(omni_tripwire_state_t tw,
                                                           omni_quad_state_t status) {
    if (tw == OMNI_TRIPWIRE_TRIPPED || status == OMNI_STATE_ROLLBACK) {
        return OMNI_STATE_ROLLBACK;
    }
    return OMNI_STATE_PROCEED;
}

/**
 * @brief Closes and seals an execution block in PROCEED (+1).
 */
static inline omni_quad_state_t omni_commit_proceed(omni_tripwire_state_t *tw) {
    omni_tripwire_disarm(tw);
    return OMNI_STATE_PROCEED;
}

/**
 * @brief Closes and seals an execution block in ROLLBACK (-1).
 */
static inline omni_quad_state_t omni_commit_rollback(omni_tripwire_state_t *tw) {
    omni_tripwire_trip(tw);
    return OMNI_STATE_ROLLBACK;
}

/**
 * @brief Closes and seals an execution block in terminal detent HALT:NULL.
 */
static inline omni_quad_state_t omni_commit_halt(omni_tripwire_state_t *tw) {
    omni_tripwire_disarm(tw);
    return OMNI_STATE_HALT;
}

#ifdef __cplusplus
}
#endif

#endif /* OMNI_BASE_H */
```

<TRACK 2> : {OmniCode Language & Standard Library}/lexicon/01_nouns/ideas/types/transpile variant/c_backbone/include/site.h

H | 191 LINES | 925 WORDS | 7.4 KB

```c
/* ============================================================================
 * BLOCK 1: METADATA & INVARIANT TYPE DEFINITION
 * ( Invariant Type: < Idea { Type } : Site > )
 * ============================================================================
 * ARCHITECTURAL TOPOLOGY:
 *   Defines physical memory addresses, hardware ports, and MMIO coordinates in the
 *   OmniCode Standard Library Track 2 C Transpile Variant.
 *   - Base Block: Primitive Invariant Type
 *   - Category: Idea { Type }
 *   - Footprint: [ 1 Address ] { 64-Bit Pointer / Physical Silicon Coordinate }
 *   - Ladder Ascension: Builds up with Numbers (/-^)
 *   - System: OmniCode { Lexicon }
 *   - Witness: Scripture Genesis 28:16 KJV
 *     "And Jacob awaked out of his sleep, and he said, Surely the Lord is in
 *      this place; and I knew it not."
 *   - Semantics: { PhysicalAddress :: MMIOPort :: SiliconCoordinate :: RoutingLaw }
 * ============================================================================ */

#ifndef OMNI_SITE_H
#define OMNI_SITE_H

#include "omni_base.h"

#ifdef __cplusplus
extern "C" {
#endif

/* ----------------------------------------------------------------------------
 * 1. HARDWARE ROUTING DOMAIN ENUM: omni_site_routing_t
 * ---------------------------------------------------------------------------- */

/**
 * @brief Hardware Routing Law domain classification.
 *
 * Law { Routing }:
 *   - RAM Physical (@)   : Standard cacheable RAM memory pointer.
 *   - MMIO Hardware (::@) : Memory-mapped I/O port requiring volatile access.
 */
typedef enum {
    OMNI_ROUTING_RAM_PHYSICAL  = 0, /**< '@' Physical system RAM address */
    OMNI_ROUTING_HARDWARE_MMIO = 1  /**< '::@' Hardware MMIO device register or port */
} omni_site_routing_t;

/* ----------------------------------------------------------------------------
 * 2. FREESTANDING INVARIANT TYPE STRUCTURE: omni_site_t
 * ---------------------------------------------------------------------------- */

/**
 * @brief Invariant Type < Site >: The physical silicon address coordinate.
 *
 * Encloses the 64-bit silicon address pointer and its hardware routing domain:
 *   - Strictly non-null when valid (0x0 is mapped to unmapped detent).
 *   - Resolves routing bijectively between RAM and device hardware ports.
 *
 * Core Invariant Law:
 *   ENCLOSE [ Site ] WITHIN [ 1 Address ]
 *   LAW { Routing } : [ Physical RAM @ ] | [ Hardware MMIO ::@ ]
 */
typedef struct {
    uintptr_t           coordinate;                     /**< Physical 64-bit silicon address pointer */
    omni_site_routing_t routing;                        /**< Hardware routing classification (RAM vs MMIO) */
    bool                is_enclosed;                    /**< Invariant boundary enclosure guarantee */
} omni_site_t;

/* ----------------------------------------------------------------------------
 * 3. 4-BLOCK PIPELINE CONTEXT: omni_site_context_t
 * ---------------------------------------------------------------------------- */

/**
 * @brief Execution context for Site address binding and routing lifecycle rolls.
 */
typedef struct {
    uintptr_t             input_coordinate;             /**< Outside-in target silicon coordinate */
```

```
    omni_site_t           site;                       /**< Concrete single-address allocation */
    omni_quad_state_t     status;                     /**< Active Quad-State truth */
    omni_tripwire_state_t tripwire;                   /**< Subsystem safety interlock */
} omni_site_context_t;

/* ============================================================================
 * BLOCK 2: OPENING BLOCK
 * ( Outside-in Imposition & Context Initialization )
 * ============================================================================ */

/**
 * @brief Initializes the Site context and binds the target coordinate.
 *
 * @param ctx Pointer to caller-allocated site context.
 * @param input_coord Target silicon address (defaults to VGA buffer if 0).
 */
void omni_site_open(omni_site_context_t *ctx, uintptr_t input_coord);

/* ============================================================================
 * BLOCK 3: BODY BLOCK
 * ( Operational Primitives, Routing Laws & Memory Access )
 * ============================================================================ */

/**
 * @brief Constructs a Site directly from a 64-bit address pointer,
 *        automatically resolving its hardware routing classification.
 *
 * @param address Physical silicon memory or MMIO address.
 * @return Fully initialized omni_site_t.
 */
omni_site_t omni_site_from_address(uintptr_t address);

/**
 * @brief Hardware Routing Law: Distinguishes physical RAM pointers from
 *        hardware MMIO registers and serial ports.
 *
 * @param address Physical target address.
 * @return OMNI_ROUTING_HARDWARE_MMIO if within device port ranges; RAM otherwise.
 */
omni_site_routing_t omni_site_resolve_routing(uintptr_t address);

/**
 * @brief Calculates an offset Site from a base Site coordinate.
 *
 * @param site Base reference site pointer.
 * @param offset Signed byte displacement (+offset or -offset).
 * @return New displaced omni_site_t coordinate.
 */
omni_site_t omni_site_offset(const omni_site_t *site, intptr_t offset);

/**
 * @brief Verifies whether a site coordinate is valid and non-null.
 *
 * @param site Pointer to site to evaluate.
 * @return true if site is valid, enclosed, and non-null; false otherwise.
 */
bool omni_site_is_valid(const omni_site_t *site);

/**
 * @brief Evaluates relational equality between two sites.
 *
 * @param a First site operand.
 * @param b Second site operand.
 * @return true if addresses and routing domains match; false otherwise.
 */
bool omni_site_eq(const omni_site_t *a, const omni_site_t *b);

/**
 * @brief Safely reads an 8-bit byte from the site coordinate plus offset.
 *        Employs volatile memory semantics to prevent compiler cache elision on MMIO.
 *
 * @param site Base site coordinate.
 * @param offset Byte displacement.
 * @return Read byte value (returns 0 if site is invalid/null).
 */
uint8_t omni_site_read_u8(const omni_site_t *site, size_t offset);

/**
```

```
 * @brief Safely writes an 8-bit byte to the site coordinate plus offset.
 *        Employs volatile memory semantics for bare-metal hardware registers.
 *
 * @param site Base site coordinate.
 * @param offset Byte displacement.
 * @param value Byte value to inscribe.
 */
void omni_site_write_u8(const omni_site_t *site, size_t offset, uint8_t value);

/**
 * @brief Executes the full location lifecycle across declared rolls:
 *   - Roll 1: Locate address with numbers.
 *   - Roll 2: Bind coordinate and classify routing.
 *   - Roll 3: Guard non-null identity invariants.
 *   - Roll 4: Route and anchor memory target.
 *
 * @param ctx Pointer to active site context.
 * @return OMNI_STATE_PROCEED on valid address; OMNI_STATE_ROLLBACK on null or error.
 */
omni_quad_state_t omni_site_execute(omni_site_context_t *ctx);

/* ============================================================================
 * BLOCK 4: CLOSING BLOCK
 * ( Integrity Verification & Commit Status )
 * ============================================================================ */

/**
 * @brief Validates all invariants and seals the Site commit status.
 *
 * @param ctx Pointer to active site context.
 * @return OMNI_STATE_PROCEED on successful seal; OMNI_STATE_ROLLBACK if tripped.
 */
omni_quad_state_t omni_site_close(omni_site_context_t *ctx);

#ifdef __cplusplus
}
#endif

#endif /* OMNI_SITE_H */
```

<TRACK 2> : {OmniCode Language & Standard Library}/lexicon/01_nouns/ideas/types/transpile variant/c_backbone/include/state.h

H | 170 LINES | 863 WORDS | 7.1 KB

```c
/* ============================================================================
 * BLOCK 1: METADATA & INVARIANT TYPE DEFINITION
 * ( Invariant Type: < Idea { Type } : State > )
 * ============================================================================
 * ARCHITECTURAL TOPOLOGY:
 *   Defines causal truth, clock progression, and the canonical Quad-State machine
 *   in the OmniCode Standard Library Track 2 C Transpile Variant.
 *   - Base Block: Primitive Invariant Type
 *   - Category: Idea { Type }
 *   - Footprint: [ 1 Trit ] { 2 Bits / Quad-State Causal Register }
 *   - Named Registers: [ STATE, TURN, BOUND, IT ]
 *   - System: OmniCode { Lexicon }
 *   - Witness: Scripture Ecclesiastes 3:1 KJV
 *     "To every thing there is a season, and a time to every purpose under
 *      the heaven:"
 *   - Semantics: { QuadStateTruth :: CausalClock :: CanonicalLoopLaw }
 * ============================================================================ */

#ifndef OMNI_STATE_H
#define OMNI_STATE_H

#include "omni_base.h"

#ifdef __cplusplus
extern "C" {
#endif

/* ----------------------------------------------------------------------------
 * 1. CANONICAL NAMED REGISTERS: omni_state_registers_t
 * ---------------------------------------------------------------------------- */

/**
 * @brief The four canonical named registers governing execution kinematics.
 *
 * Named Registers:
 *   - STATE : Primary causal truth register (+1: PROCEED, 0: AWAIT, -1: ROLLBACK, HALT: NULL).
 *   - TURN  : Discrete cadence pulse counter (increments on affirmative causation).
 *   - BOUND : Dynamic loop termination threshold.
 *   - IT    : Active accumulator iterator stepping along the causal tape.
 */
typedef struct {
    omni_quad_state_t state;                          /**< STATE: Primary Quad-State truth register */
    int64_t           turn;                           /**< TURN: Cadence step counter */
    int64_t           bound;                          /**< BOUND: Loop termination boundary */
    int64_t           it;                             /**< IT: Active accumulator iterator */
} omni_state_registers_t;

/* ----------------------------------------------------------------------------
 * 2. FREESTANDING INVARIANT TYPE STRUCTURE: omni_state_t
 * ---------------------------------------------------------------------------- */

/**
 * @brief Invariant Type < State >: The causal truth and control flow engine.
 *
 * Encloses the physical trit substrate and register bank:
 *   - Single-trit physical substrate (2 bits in silicon).
 *   - Implements the Canonical Looping Law connecting Quad-State to hardware registers.
 *
 * Core Invariant Law:
 *   LAW { Quad-State } : { T4 = [ +1:PROCEED | 0:AWAIT | -1:ROLLBACK | HALT:NULL ] }
 *   LAW { Loop } : LOOP [ WHEN [ { IT } < { BOUND } ] --> [ IT` ] PROCEED to { TURN }
 *                         ! [ { IT } > { BOUND } ] AWAIT for [ IT` ]
 *                         :: HALT WHEN [ IT` ] = NULL ]
 */
typedef struct {
    omni_trit_t             substrate_trit;           /**< 1 Trit physical substrate */
    omni_state_registers_t  registers;                /**< Canonical Quad-State register bank */
    bool                    is_enclosed;              /**< Invariant boundary enclosure guarantee */
} omni_state_t;

/* ----------------------------------------------------------------------------
 * 3. 4-BLOCK PIPELINE CONTEXT: omni_state_context_t
 * ---------------------------------------------------------------------------- */
```

```c

/**
 * @brief Execution context for State governance and loop lifecycle rolls.
 */
typedef struct {
    omni_quad_state_t     input_signal;                /**< Outside-in causal truth signal */
    omni_state_t          state_engine;                /**< Concrete single-trit register bank */
    omni_quad_state_t     status;                      /**< Active Quad-State truth */
    omni_tripwire_state_t tripwire;                    /**< Subsystem safety interlock */
} omni_state_context_t;

/* ============================================================================
 * BLOCK 2: OPENING BLOCK
 * ( Outside-in Imposition & Context Initialization )
 * ============================================================================ */

/**
 * @brief Initializes the State context with an input signal and loop boundary.
 *
 * @param ctx Pointer to caller-allocated state context.
 * @param signal Initial truth signal (defaults to PROCEED if 0).
 * @param bound_limit Maximum loop iteration threshold.
 */
void omni_state_open(omni_state_context_t *ctx, omni_quad_state_t signal, int64_t bound_limit);

/* ============================================================================
 * BLOCK 3: BODY BLOCK
 * ( Operational Primitives, Loop Laws & Transitions )
 * ============================================================================ */

/**
 * @brief Creates a freestanding State engine with specified signal and boundary.
 *
 * @param initial_signal Starting Quad-State truth.
 * @param bound Maximum loop bound.
 * @return Fully initialized omni_state_t.
 */
omni_state_t omni_state_create(omni_quad_state_t initial_signal, int64_t bound);

/**
 * @brief Canonical Loop Law Single-Step Transition:
 *
 * Executes one step of the formal loop grammar:
 *   - WHEN IT < BOUND: Increments IT and TURN; returns OMNI_STATE_PROCEED.
 *   - WHEN IT > BOUND: Transitions to standby; returns OMNI_STATE_AWAIT.
 *   - WHEN IT == BOUND: Reaches detent; returns OMNI_STATE_HALT.
 *
 * @param regs Pointer to register bank.
 * @return Resulting Quad-State condition after the step.
 */
omni_quad_state_t omni_state_step_loop(omni_state_registers_t *regs);

/**
 * @brief Resets register bank counters to 0 while setting state to PROCEED.
 *
 * @param s Pointer to state engine.
 * @param bound New loop bound.
 */
void omni_state_reset(omni_state_t *s, int64_t bound);

/** Predicate queries for State engine */
bool omni_state_is_halted(const omni_state_t *s);
bool omni_state_is_proceeding(const omni_state_t *s);
bool omni_state_is_awaiting(const omni_state_t *s);
bool omni_state_is_rollback(const omni_state_t *s);

/**
 * @brief Executes the full governance lifecycle across declared rolls:
 *   - Roll 1: Inscribe state register from input signal.
 *   - Roll 2: Evaluate caveat guard against discrepancy.
 *   - Roll 3: Execute canonical loop until boundary or detent.
 *   - Roll 4: Govern machine clock output.
 *
 * @param ctx Pointer to active state context.
 * @return Final Quad-State condition upon detent.
 */
omni_quad_state_t omni_state_execute(omni_state_context_t *ctx);

```

```
/* ============================================================================
 * BLOCK 4: CLOSING BLOCK
 * ( Integrity Verification & Commit Status )
 * ============================================================================ */

/**
 * @brief Validates all invariants and seals the State commit status.
 *
 * @param ctx Pointer to active state context.
 * @return OMNI_STATE_PROCEED on successful seal; OMNI_STATE_ROLLBACK if tripped.
 */
omni_quad_state_t omni_state_close(omni_state_context_t *ctx);

#ifdef __cplusplus
}
#endif

#endif /* OMNI_STATE_H */
```

<TRACK 2> : {OmniCode Language & Standard Library}/lexicon/01_nouns/ideas/types/transpile variant/c_backbone/src/character.c

C | 262 LINES | 1182 WORDS | 10.0 KB

```c
/* ============================================================================
 * BLOCK 1: METADATA & INVARIANT IMPLEMENTATION
 * ( Invariant Type: < Idea { Type } : Character > )
 * ============================================================================
 * ARCHITECTURAL TOPOLOGY:
 *   Implements the atomic letter, glyph, and inscription rune in the OmniCode
 *   Standard Library Track 2 C Transpile Variant.
 *   - Base Block: Primitive Invariant Type
 *   - Category: Idea { Type }
 *   - Footprint: [ 1 Cell ] { 5 Balanced Trits / 8 Bits }
 *   - Witness: Scripture Exodus 31:18 KJV
 *     "And he gave unto Moses, when he had made an end of communing with him upon
 *      mount Sinai, two tables of testimony, tables of stone, written with the
 *      finger of God."
 *   - Semantics: { AtomicGlyph :: LetterRune :: BalancedCellTransduction }
 *   - Invariant Law:
 *     { Character } = [ Enclosed ]: TRUE (Strict single-cell capacity).
 * ============================================================================ */

#include "character.h"
#include <stdio.h>

/* ----------------------------------------------------------------------------
 * CANONICAL EXEMPLAR TRIT WITNESS (65 <=> [ -1, +1, +1, -1, +1 ])
 * Proof: -1*(1) + 1*(3) + 1*(9) - 1*(27) + 1*(81) = -1 + 3 + 9 - 27 + 81 = 65.
 * ---------------------------------------------------------------------------- */
#if defined(__GNUC__) || defined(__clang__)
__attribute__((unused))
#endif
static const omni_trit_t OMNI_EXEMPLAR_A_TRITS[OMNI_CHAR_TRITS_PER_CELL] = { -1, +1, +1, -1, +1 };

/* ----------------------------------------------------------------------------
 * PURE INVARIANT OPERATIONS & TRANSDUCTION LAWS
 * ---------------------------------------------------------------------------- */

/**
 * @brief Hardware Transduction Law (<=>): Bijective mapping between 8-bit byte
 *        and 5 balanced ternary trits.
 *
 * Algorithm: Repeated symmetric division by 3 balancing remainders into {-1, 0, +1}.
 */
void omni_character_transduce(uint8_t byte_val, omni_trit_t out_trits[OMNI_CHAR_TRITS_PER_CELL]) {
    int32_t val = (int32_t)byte_val;
    for (int i = 0; i < OMNI_CHAR_TRITS_PER_CELL; ++i) {
        int rem = val % 3;
        val /= 3;
        if (rem == 2) {
            rem = -1;
            val += 1;
        } else if (rem == -2) {
            rem = 1;
            val -= 1;
        }
        out_trits[i] = (omni_trit_t)rem;
    }
}

/**
 * @brief Inverse Transduction Law (<=>): Reconstructs an 8-bit byte from 5 balanced trits.
 *
 * Evaluates sum_{i=0..4} trits[i] * 3^i using Horner's method / power expansion.
 */
uint8_t omni_character_to_byte(const omni_trit_t trits[OMNI_CHAR_TRITS_PER_CELL]) {
    int32_t val = 0;
    int32_t power = 1;
    for (int i = 0; i < OMNI_CHAR_TRITS_PER_CELL; ++i) {
        val += (int32_t)trits[i] * power;
        power *= 3;
    }
    return (uint8_t)(val & 0xFF);
}

/**
```

```c
 * @brief Inscription Constructor: Constructs a Character directly from an 8-bit glyph.
 *        Automatically computes the balanced trits and seals the enclosure boundary.
 */
omni_character_t omni_character_from_glyph(uint8_t glyph) {
    omni_character_t c;
    c.glyph = glyph;
    c.is_enclosed = true;
    omni_character_transduce(glyph, c.trits);
    return c;
}

/**
 * @brief Evaluates relational equality between two characters.
 */
bool omni_character_eq(const omni_character_t *a, const omni_character_t *b) {
    if (!a || !b) return false;
    if (!a->is_enclosed || !b->is_enclosed) return false;
    return (a->glyph == b->glyph);
}

/**
 * @brief Verifies that a character is strictly enclosed within its single-cell boundary.
 */
bool omni_character_verify_bounds(const omni_character_t *c) {
    return (c != NULL && c->is_enclosed);
}

/* ============================================================================
 * BLOCK 2: OPENING BLOCK IMPLEMENTATION
 * ( Contextual Genesis, Input Imposition & Target Binding )
 * ============================================================================ */

/**
 * @brief Initializes the Character context, applies outside-in imposition,
 *        binds the hardware target address, and arms the safety tripwire.
 */
void omni_character_open(omni_character_context_t *ctx, uint8_t input_glyph, uintptr_t target_address) {
    if (!ctx) return;

    /* 1. Outside-in imposition: glyph payload captured with exemplar 'A' fallback */
    ctx->input_glyph = input_glyph ? input_glyph : OMNI_EXEMPLAR_GLYPH_A;

    /* 2. Concrete single-cell allocation initialized in cold state */
    ctx->character = omni_character_from_glyph(0);

    /* 3. Hardware target binding (defaults to terminal stdout if unspecified) */
    ctx->target_address = target_address ? target_address : OMNI_TARGET_STDOUT;

    /* 4. Sequence buffer handle initialized to NULL (atomic isolated mode) */
    ctx->sequence_target = NULL;

    /* 5. Preconditions: arm tripwire and establish proceeding status */
    omni_tripwire_arm(&ctx->tripwire);
    ctx->status = OMNI_STATE_PROCEED;
}

/* ============================================================================
 * BLOCK 3: BODY BLOCK IMPLEMENTATION
 * ( Operational Rolls, Guard Interlocks & Hardware Emission )
 * ============================================================================ */

/**
 * @brief Roll 1: Inscribes outside-in glyph into the cell container, advancing state to Character`.
 */
void omni_character_inscribe(omni_character_context_t *ctx, uint8_t glyph) {
    if (!ctx) return;
    ctx->character.glyph = glyph;
    ctx->character.is_enclosed = true;
    omni_character_transduce(glyph, ctx->character.trits);
}

/**
 * @brief Roll 4: Projects Character` to the bound hardware target address.
 *
 * Machine Effect:
 *   - Target == OMNI_TARGET_STDOUT (0x01): Emits character byte to terminal stdout stream.
 *   - Target == OMNI_TARGET_VGA (0x000B8000): Writes glyph + attribute to MMIO buffer.
 */
```

```c
bool omni_character_project(const omni_character_t *c, uintptr_t target_address) {
    if (!c || !c->is_enclosed) return false;

    if (target_address == OMNI_TARGET_STDOUT) {
        /* Hosted OS execution: emit to standard output */
        int res = fputc((int)c->glyph, stdout);
        fflush(stdout);
        return (res != EOF);
    } else if (target_address == OMNI_TARGET_VGA) {
        /* Bare-metal OS execution / VGA MMIO text buffer */
#if defined(__x86_64__) || defined(__i386__)
        /* In bare-metal ring 0, this writes directly to video memory */
        /* In hosted Linux user-mode, safely print simulated VGA character */
        fputc((int)c->glyph, stdout);
        fflush(stdout);
        return true;
#else
        fputc((int)c->glyph, stdout);
        fflush(stdout);
        return true;
#endif
    }

    /* Custom memory/stream target fallback */
    return true;
}

/**
 * @brief Roll 5: Yields Character` to continuous Text ribbon sequence.
 */
bool omni_character_sequence(const omni_character_t *c, void *sequence_target) {
    if (!c || !c->is_enclosed) return false;
    if (!sequence_target) return true; /* Atomic isolated cycle passes */

    /* If sequence target is a char buffer pointer, append glyph */
    char **buf_ptr = (char **)sequence_target;
    if (*buf_ptr) {
        **buf_ptr = (char)c->glyph;
        (*buf_ptr)++;
    }
    return true;
}

/**
 * @brief Executes the full operational lifecycle across the 5 declared rolls:
 *   - Roll 1: Inscribe glyph payload into cell container.
 *   - Roll 2: Transduce 8 bits <=> 5 balanced trits and verify parity.
 *   - Roll 3: Guard identity, boundary, and parity via adversative catch (!).
 *   - Roll 4: Project Character` to the hardware target.
 *   - Roll 5: Sequence Character` into text ribbon if bound.
 */
omni_quad_state_t omni_character_execute(omni_character_context_t *ctx) {
    if (!ctx) return OMNI_STATE_ROLLBACK;

    /* ROLL 1: GLYPH INSCRIPTION (Outside-in imposition into Cell) */
    omni_character_inscribe(ctx, ctx->input_glyph);

    /* ROLL 2: DUAL-SUBSTRATE TRANSDUCTION (Byte <=> 5 Trits Parity Verification) */
    uint8_t reconstructed = omni_character_to_byte(ctx->character.trits);
    bool parity_match = (reconstructed == ctx->character.glyph);

    /* ROLL 3: IDENTITY & BOUND GUARD (Adversative Catch via `!`) */
    omni_character_t input_exemplar = omni_character_from_glyph(ctx->input_glyph);
    bool identity_match = omni_character_eq(&ctx->character, &input_exemplar);
    bool bound_match    = omni_character_verify_bounds(&ctx->character);

    /* Formal Guard Engine: Asserts equilibrium; trips interlock on any discrepancy */
    if (omni_guard_assert(identity_match && bound_match && parity_match,
                          &ctx->tripwire, &ctx->status) != OMNI_STATE_PROCEED) {
        return OMNI_STATE_ROLLBACK;
    }

    /* ROLL 4: TARGET PROJECTION (Inside-out emergence to stdout / VGA) */
    if (!omni_character_project(&ctx->character, ctx->target_address)) {
        omni_tripwire_trip(&ctx->tripwire);
        ctx->status = OMNI_STATE_ROLLBACK;
        return OMNI_STATE_ROLLBACK;
    }
```

```
    /* ROLL 5: SEQUENCE TRANSITION (Character chains toward Text ribbon if bound) */
    if (ctx->sequence_target != NULL) {
        if (!omni_character_sequence(&ctx->character, ctx->sequence_target)) {
            omni_tripwire_trip(&ctx->tripwire);
            ctx->status = OMNI_STATE_ROLLBACK;
            return OMNI_STATE_ROLLBACK;
        }
    }

    return ctx->status;
}

/* ============================================================================
 * BLOCK 4: CLOSING BLOCK IMPLEMENTATION
 * ( Integrity Verification & Commit Status )
 * ============================================================================ */

/**
 * @brief Verifies all invariants and seals the Character commit status.
 */
omni_quad_state_t omni_character_close(omni_character_context_t *ctx) {
    if (!ctx) return OMNI_STATE_ROLLBACK;

    /* Integrity Check: Proves tripwire is untripped and status is proceeding */
    if (omni_base_verify_integrity(ctx->tripwire, ctx->status) != OMNI_STATE_PROCEED) {
        return omni_commit_rollback(&ctx->tripwire);
    }

    /* Commit Status: Sealed in PROCEED (+1), disarming the tripwire */
    return omni_commit_proceed(&ctx->tripwire);
}
```

<TRACK 2> : {OmniCode Language & Standard Library}/lexicon/01_nouns/ideas/types/transpile variant/c_backbone/src/container.c

C | 236 LINES | 997 WORDS | 8.5 KB

```c
/* ============================================================================
 * BLOCK 1: METADATA & INVARIANT IMPLEMENTATION
 * ( Invariant Type: < Idea { Type } : Container > )
 * ============================================================================
 * ARCHITECTURAL TOPOLOGY:
 *   Implements bounded memory envelopes, arrays, structures, and storage vessels
 *   in the OmniCode Standard Library Track 2 C Transpile Variant.
 *   - Base Block: Primitive Invariant Type
 *   - Category: Idea { Type }
 *   - Footprint: [ Dynamically Bounded Memory Envelope / Generic Slice ]
 *   - Ladder Ascension: Builds up with Numbers (/-^)
 *   - Witness: Scripture 2 Kings 4:4 KJV
 *     "And when thou art come in, thou shalt shut the door upon thee and upon
 *      thy sons, and shalt pour out into all those vessels, and thou shalt set
 *      aside that which is full."
 *   - Semantics: { BoundedEnvelope :: StorageVessel :: MemoryCapsule :: ContainmentLaw }
 * ============================================================================ */

#include "container.h"

/**
 * @brief Bare-metal freestanding memory byte copy (Zero libc/memcpy dependency).
 */
static void omni_mem_copy(void *dest, const void *src, size_t n) {
    uint8_t *d = (uint8_t *)dest;
    const uint8_t *s = (const uint8_t *)src;
    for (size_t i = 0; i < n; ++i) {
        d[i] = s[i];
    }
}

/* ============================================================================
 * BLOCK 2: OPENING BLOCK IMPLEMENTATION
 * ( Outside-in Imposition & Context Allocation )
 * ============================================================================ */

/**
 * @brief Initializes a Container context over a provided memory storage partition,
 *        applies outside-in imposition, and arms the execution tripwire.
 */
void omni_container_open(omni_container_context_t *ctx,
                         void *storage,
                         size_t elem_size,
                         size_t capacity,
                         const void *payload,
                         size_t payload_count) {
    if (!ctx) return;

    ctx->payload_ptr = payload;
    ctx->payload_count = payload_count;

    /* Concrete allocation with provided or default parameters */
    size_t e_sz = elem_size ? elem_size : sizeof(uint8_t);
    size_t cap  = capacity ? capacity : OMNI_CONTAINER_EXEMPLAR_VESSEL;
    omni_container_init(&ctx->vessel, storage, e_sz, cap);

    ctx->target_destination = OMNI_SITE_NULL;
    omni_tripwire_arm(&ctx->tripwire);
    ctx->status = OMNI_STATE_PROCEED;
}

/* ============================================================================
 * BLOCK 3: BODY BLOCK IMPLEMENTATION
 * ( Operational Primitives, Containment Laws & Inscriptions )
 * ============================================================================ */

/**
 * @brief Initializes a generic bounded container slice over an existing memory buffer.
 */
void omni_container_init(omni_container_t *c, void *buffer, size_t element_size, size_t capacity) {
    if (!c) return;
    c->buffer = buffer;
    c->element_size = element_size;
```

```
    c->capacity = capacity;
    c->occupancy = 0;
    c->is_enclosed = (buffer != NULL && element_size > 0 && capacity > 0);
}

/**
 * @brief Initializes a canonical 64-byte storage vessel.
 */
void omni_byte_vessel_init(omni_byte_vessel_t *vessel) {
    if (!vessel) return;
    for (size_t i = 0; i < OMNI_CONTAINER_EXEMPLAR_VESSEL; ++i) {
        vessel->cells[i] = 0;
    }
    omni_container_init(&vessel->container, vessel->cells, sizeof(uint8_t), OMNI_CONTAINER_EXEMPLAR_VESSEL);
}

/**
 * @brief Core Containment Law Verification: Proves that Occupancy <= Capacity.
 */
bool omni_container_verify_containment(const omni_container_t *c) {
    if (!c) return false;
    return (c->is_enclosed && c->occupancy <= c->capacity);
}

/**
 * @brief Queries whether the container has reached its maximum capacity limit.
 */
bool omni_container_is_full(const omni_container_t *c) {
    if (!c) return true;
    return (c->occupancy >= c->capacity);
}

/**
 * @brief Queries whether the container has zero stored elements.
 */
bool omni_container_is_empty(const omni_container_t *c) {
    if (!c) return true;
    return (c->occupancy == 0);
}

/**
 * @brief Pushes an element into the container, strictly guarding against capacity overflow.
 */
omni_quad_state_t omni_container_push(omni_container_t *c, const void *element) {
    if (!c || !element || !c->buffer) return OMNI_STATE_ROLLBACK;

    /* Law { Containment } Guard: Occupancy must never exceed capacity */
    if (c->occupancy >= c->capacity) {
        return OMNI_STATE_ROLLBACK;
    }

    uint8_t *dest = (uint8_t *)c->buffer + (c->occupancy * c->element_size);
    omni_mem_copy(dest, element, c->element_size);
    c->occupancy += 1;

    return OMNI_STATE_PROCEED;
}

/**
 * @brief Retrieves a direct pointer to the element at the specified index.
 */
void *omni_container_get(const omni_container_t *c, size_t index) {
    if (!c || !c->buffer || index >= c->occupancy) {
        return NULL;
    }
    return (uint8_t *)c->buffer + (index * c->element_size);
}

/**
 * @brief Replaces or writes an element at the specified index.
 */
omni_quad_state_t omni_container_set(omni_container_t *c, size_t index, const void *element) {
    if (!c || !element || !c->buffer || index >= c->capacity) {
        return OMNI_STATE_ROLLBACK;
    }

    uint8_t *dest = (uint8_t *)c->buffer + (index * c->element_size);
    omni_mem_copy(dest, element, c->element_size);
```

```
152
153     if (index >= c->occupancy) {
154         c->occupancy = index + 1;
155     }
156
157     return OMNI_STATE_PROCEED;
158 }
159
160 /**
161  * @brief Resets container occupancy to zero without modifying buffer contents.
162  */
163 void omni_container_clear(omni_container_t *c) {
164     if (!c) return;
165     c->occupancy = 0;
166 }
167
168 /**
169  * @brief Executes the Container lifecycle across declared operational rolls:
170  *   - Roll 1: Build capsule envelope with numbers.
171  *   - Roll 2: Encapsulate incoming payload.
172  *   - Roll 3: Guard bounds under Law { Containment } (Occupancy <= Capacity).
173  *   - Roll 4: Pour out and set aside vessel.
174  */
175 omni_quad_state_t omni_container_execute(omni_container_context_t *ctx) {
176     if (!ctx) return OMNI_STATE_ROLLBACK;
177
178     /* ROLL 1: BUILD CAPSULE (Ladder build-up with Numbers) */
179     bool capsule_valid = (ctx->vessel.capacity > 0 && ctx->vessel.is_enclosed);
180     if (omni_guard_assert(capsule_valid, &ctx->tripwire, &ctx->status) != OMNI_STATE_PROCEED) {
181         return OMNI_STATE_ROLLBACK;
182     }
183
184     /* ROLL 2: ENCAPSULATION (Inscribe payload into storage elements) */
185     if (ctx->payload_ptr && ctx->payload_count > 0) {
186         /* Law violation guard: Attempting to overflow bounded capacity */
187         bool within_bounds = (ctx->payload_count <= ctx->vessel.capacity);
188         if (omni_guard_assert(within_bounds, &ctx->tripwire, &ctx->status) != OMNI_STATE_PROCEED) {
189             return OMNI_STATE_ROLLBACK;
190         }
191
192         const uint8_t *src_bytes = (const uint8_t *)ctx->payload_ptr;
193         for (size_t i = 0; i < ctx->payload_count; ++i) {
194             const void *elem = src_bytes + (i * ctx->vessel.element_size);
195             omni_container_push(&ctx->vessel, elem);
196         }
197     }
198
199     /* ROLL 3: IDENTITY & BOUND COMPARISON (Guard bounds under Law { Containment }) */
200     bool containment_ok = omni_container_verify_containment(&ctx->vessel);
201     if (omni_guard_assert(containment_ok, &ctx->tripwire, &ctx->status) != OMNI_STATE_PROCEED) {
202         return OMNI_STATE_ROLLBACK;
203     }
204
205     /* ROLL 4: POUR OUT & SET ASIDE (Inside-out emergence) */
206     /* Vessel sealed and set aside at destination coordinate */
207
208     /* ROLL 5: SEQUENCE TRANSITION (Container chains to composite structure) */
209     /* Ready to sequence into memory partitions */
210
211     return ctx->status;
212 }
213
214 /* ============================================================================
215  * BLOCK 4: CLOSING BLOCK IMPLEMENTATION
216  * ( Integrity Seals & Commit Status )
217  * ============================================================================ */
218
219 /**
220  * @brief Verifies all invariants and seals the Container commit status.
221  */
222 omni_quad_state_t omni_container_close(omni_container_context_t *ctx) {
223     if (!ctx) return OMNI_STATE_ROLLBACK;
224
225     /* Integrity Check: Proves tripwire is untripped and containment law holds */
226     bool integrity_ok = (omni_base_verify_integrity(ctx->tripwire, ctx->status) == OMNI_STATE_PROCEED) &&
227                         omni_container_verify_containment(&ctx->vessel);
228
229     if (!integrity_ok) {
```

```
        return omni_commit_rollback(&ctx->tripwire);
    }

    /* Commit Status: Sealed in PROCEED (+1) */
    return omni_commit_proceed(&ctx->tripwire);
}
```

<TRACK 2> : {OmniCode Language & Standard Library}/lexicon/01_nouns/ideas/types/transpile variant/c_backbone/src/number.c

C | 185 LINES | 753 WORDS | 6.4 KB

```c
/* ============================================================================
 * BLOCK 1: METADATA & INVARIANT IMPLEMENTATION
 * ( Invariant Type: < Idea { Type } : Number > )
 * ============================================================================
 * ARCHITECTURAL TOPOLOGY:
 *   Implements scalar quantities, integers, and balanced ternary measurements in
 *   the OmniCode Standard Library Track 2 C Transpile Variant.
 *   - Base Block: Primitive Invariant Type
 *   - Category: Idea { Type }
 *   - Footprint: [ 1 Word ] { 32 Balanced Trits / 64 Bits }
 *   - Witness: Scripture Psalm 147:4 KJV
 *     "He telleth the number of the stars; he calleth them all by their names."
 *   - Semantics: { ScalarQuantity :: NumericWord :: BalancedTernaryALU }
 * ============================================================================ */

#include "number.h"

/* ============================================================================
 * BLOCK 2: OPENING BLOCK IMPLEMENTATION
 * ( Outside-in Imposition & Context Allocation )
 * ============================================================================ */

/**
 * @brief Initializes the Number context, arms the ALU, verifies equilibrium,
 *        and arms the execution tripwire.
 */
void omni_number_open(omni_number_context_t *ctx, int64_t input_quantity) {
    if (!ctx) return;

    /* 1. Outside-in imposition: external scalar measurement */
    ctx->input_quantity = input_quantity ? input_quantity : OMNI_EXEMPLAR_PARTITION;

    /* 2. Concrete single-word allocation */
    ctx->number = omni_number_from_scalar(0);

    /* 3. Output target accumulator */
    ctx->target_accumulator = OMNI_SITE_NULL;

    /* 4. Preconditions & tripwire arming */
    omni_tripwire_arm(&ctx->tripwire);
    ctx->status = omni_number_verify_equilibrium() ? OMNI_STATE_PROCEED : OMNI_STATE_ROLLBACK;
}

/* ============================================================================
 * BLOCK 3: BODY BLOCK IMPLEMENTATION
 * ( Operational Primitives, Arithmetic Laws & Inscriptions )
 * ============================================================================ */

/**
 * @brief Proves the ALU Core Arithmetic Equilibrium Law: (+1) plus (-1) <=> 0.
 */
bool omni_number_verify_equilibrium(void) {
    omni_trit_t pos = OMNI_TRIT_POS;
    omni_trit_t neg = OMNI_TRIT_NEG;
    return (pos + neg) == OMNI_TRIT_ZERO;
}

/**
 * @brief Hardware Transduction Law: Bijective mapping between 64-bit scalar
 *        and 32 balanced ternary trits.
 *
 * Algorithm: Radix-3 balanced division with symmetric carry balancing.
 */
void omni_number_transduce(int64_t val, omni_trit_t out_trits[OMNI_NUM_TRITS_PER_WORD]) {
    int64_t current = val;
    for (int i = 0; i < OMNI_NUM_TRITS_PER_WORD; ++i) {
        if (current == 0) {
            out_trits[i] = OMNI_TRIT_ZERO;
            continue;
        }
        int64_t rem = current % 3;
        current /= 3;
        if (rem == 2) {
```

```
            rem = -1;
            current += 1;
        } else if (rem == -2) {
            rem = 1;
            current -= 1;
        }
        out_trits[i] = (omni_trit_t)rem;
    }
}

/**
 * @brief Inverse Transduction: Reconstructs a 64-bit scalar from 32 balanced trits.
 */
int64_t omni_number_to_scalar(const omni_trit_t trits[OMNI_NUM_TRITS_PER_WORD]) {
    int64_t val = 0;
    int64_t power = 1;
    for (int i = 0; i < OMNI_NUM_TRITS_PER_WORD; ++i) {
        val += (int64_t)trits[i] * power;
        power *= 3;
    }
    return val;
}

/**
 * @brief Constructs a Number directly from a 64-bit scalar integer.
 */
omni_number_t omni_number_from_scalar(int64_t scalar) {
    omni_number_t n;
    n.scalar = scalar;
    n.is_enclosed = true;
    omni_number_transduce(scalar, n.trits);
    return n;
}

/**
 * @brief Adds two Numbers producing a new balanced Number word.
 */
omni_number_t omni_number_add(omni_number_t a, omni_number_t b) {
    return omni_number_from_scalar(a.scalar + b.scalar);
}

/**
 * @brief Evaluates relational equality between two numbers.
 */
bool omni_number_eq(const omni_number_t *a, const omni_number_t *b) {
    if (!a || !b) return false;
    return (a->scalar == b->scalar);
}

/**
 * @brief Verifies that a number is properly enclosed within its single-word boundary.
 */
bool omni_number_verify_bounds(const omni_number_t *n) {
    return (n != NULL && n->is_enclosed);
}

/**
 * @brief Executes the Number lifecycle across declared operational rolls:
 *   - Roll 1: Inscribe quantity into word.
 *   - Roll 2: Transduce 64 bits <=> 32 balanced trits.
 *   - Roll 3: Guard identity and boundary invariants.
 *   - Roll 4: Prove ALU zero-sum equilibrium.
 */
omni_quad_state_t omni_number_execute(omni_number_context_t *ctx) {
    if (!ctx) return OMNI_STATE_ROLLBACK;

    /* ROLL 1: QUANTITY INSCRIPTION (Outside-in imposition into Word) */
    ctx->number.scalar = ctx->input_quantity;

    /* ROLL 2: DUAL-SUBSTRATE TRANSDUCTION (64 Bits <=> 32 Trits) */
    omni_number_transduce(ctx->number.scalar, ctx->number.trits);

    /* ROLL 3: IDENTITY & BOUND COMPARISON */
    bool identity_match = (ctx->number.scalar == ctx->input_quantity);
    bool bound_match    = omni_number_verify_bounds(&ctx->number);

    /* Formal Guard Engine: Asserts identity and single-word enclosure */
    if (omni_guard_assert(identity_match && bound_match, &ctx->tripwire, &ctx->status) != OMNI_STATE_PROCEED) {
```

```
        return OMNI_STATE_ROLLBACK;
    }

    /* ROLL 4: ARITHMETIC EQUILIBRIUM CALCULATION */
    if (omni_guard_assert(omni_number_verify_equilibrium(), &ctx->tripwire, &ctx->status) != OMNI_STATE_PROCEED) {
        return OMNI_STATE_ROLLBACK;
    }

    /* ROLL 5: SEQUENCE TRANSITION (Number chains toward Vector sequence) */
    /* Ready to sequence into multidimensional coordinate vector */

    return ctx->status;
}

/* ============================================================================
 * BLOCK 4: CLOSING BLOCK IMPLEMENTATION
 * ( Integrity Seals & Commit Status )
 * ============================================================================ */

/**
 * @brief Verifies all invariants and seals the Number commit status.
 */
omni_quad_state_t omni_number_close(omni_number_context_t *ctx) {
    if (!ctx) return OMNI_STATE_ROLLBACK;

    /* Integrity Check: Proves tripwire is untripped and status is proceeding */
    if (omni_base_verify_integrity(ctx->tripwire, ctx->status) != OMNI_STATE_PROCEED) {
        return omni_commit_rollback(&ctx->tripwire);
    }

    /* Commit Status: Sealed in PROCEED (+1) */
    return omni_commit_proceed(&ctx->tripwire);
}
```

<TRACK 2> : {OmniCode Language & Standard Library}/lexicon/01_nouns/ideas/types/transpile variant/c_backbone/src/site.c

C | 168 LINES | 735 WORDS | 6.2 KB

```c
/* ============================================================================
 * BLOCK 1: METADATA & INVARIANT IMPLEMENTATION
 * ( Invariant Type: < Idea { Type } : Site > )
 * ============================================================================
 * ARCHITECTURAL TOPOLOGY:
 *   Implements physical memory addresses, hardware ports, and MMIO coordinates in
 *   the OmniCode Standard Library Track 2 C Transpile Variant.
 *   - Base Block: Primitive Invariant Type
 *   - Category: Idea { Type }
 *   - Footprint: [ 1 Address ] { 64-Bit Pointer / Physical Silicon Coordinate }
 *   - Ladder Ascension: Builds up with Numbers (/-^)
 *   - Witness: Scripture Genesis 28:16 KJV
 *     "And Jacob awaked out of his sleep, and he said, Surely the Lord is in
 *      this place; and I knew it not."
 *   - Semantics: { PhysicalAddress :: MMIOPort :: SiliconCoordinate :: RoutingLaw }
 * ============================================================================ */

#include "site.h"

/* ============================================================================
 * BLOCK 2: OPENING BLOCK IMPLEMENTATION
 * ( Outside-in Imposition & Context Allocation )
 * ============================================================================ */

/**
 * @brief Initializes the Site context, binds the silicon coordinate,
 *        and arms the execution tripwire.
 */
void omni_site_open(omni_site_context_t *ctx, uintptr_t input_coord) {
    if (!ctx) return;

    /* 1. Outside-in imposition: external silicon coordinate */
    ctx->input_coordinate = input_coord ? input_coord : OMNI_SITE_VGA_BUFFER;

    /* 2. Concrete single-address allocation */
    ctx->site = omni_site_from_address(OMNI_SITE_NULL);

    /* 3. Preconditions & tripwire arming */
    omni_tripwire_arm(&ctx->tripwire);
    ctx->status = OMNI_STATE_PROCEED;
}

/* ============================================================================
 * BLOCK 3: BODY BLOCK IMPLEMENTATION
 * ( Operational Primitives, Routing Laws & Memory Access )
 * ============================================================================ */

/**
 * @brief Hardware Routing Law: Distinguishes physical RAM pointers from
 *        hardware MMIO registers and serial ports.
 */
omni_site_routing_t omni_site_resolve_routing(uintptr_t address) {
    if (address == OMNI_SITE_COM1 || address == OMNI_SITE_VGA_BUFFER || address >= 0x000A0000) {
        return OMNI_ROUTING_HARDWARE_MMIO;
    }
    return OMNI_ROUTING_RAM_PHYSICAL;
}

/**
 * @brief Constructs a Site directly from a 64-bit address pointer.
 */
omni_site_t omni_site_from_address(uintptr_t address) {
    omni_site_t s;
    s.coordinate = address;
    s.routing = omni_site_resolve_routing(address);
    s.is_enclosed = (address != OMNI_SITE_NULL);
    return s;
}

/**
 * @brief Calculates an offset Site from a base Site coordinate.
 */
omni_site_t omni_site_offset(const omni_site_t *site, intptr_t offset) {
```

```
    if (!site || site->coordinate == OMNI_SITE_NULL) {
        return omni_site_from_address(OMNI_SITE_NULL);
    }
    return omni_site_from_address((uintptr_t)((intptr_t)site->coordinate + offset));
}

/**
 * @brief Verifies whether a site coordinate is valid and non-null.
 */
bool omni_site_is_valid(const omni_site_t *site) {
    return (site != NULL && site->coordinate != OMNI_SITE_NULL && site->is_enclosed);
}

/**
 * @brief Evaluates relational equality between two sites.
 */
bool omni_site_eq(const omni_site_t *a, const omni_site_t *b) {
    if (!a || !b) return false;
    return (a->coordinate == b->coordinate && a->routing == b->routing);
}

/**
 * @brief Safely reads an 8-bit byte from the site coordinate plus offset.
 *        Employs volatile memory semantics for bare-metal MMIO safety.
 */
uint8_t omni_site_read_u8(const omni_site_t *site, size_t offset) {
    if (!omni_site_is_valid(site)) return 0;
    const volatile uint8_t *ptr = (const volatile uint8_t *)(site->coordinate + offset);
    return *ptr;
}

/**
 * @brief Safely writes an 8-bit byte to the site coordinate plus offset.
 *        Employs volatile memory semantics for bare-metal MMIO safety.
 */
void omni_site_write_u8(const omni_site_t *site, size_t offset, uint8_t value) {
    if (!omni_site_is_valid(site)) return;
    volatile uint8_t *ptr = (volatile uint8_t *)(site->coordinate + offset);
    *ptr = value;
}

/**
 * @brief Executes the Site lifecycle across declared operational rolls:
 *   - Roll 1: Locate address with numbers.
 *   - Roll 2: Bind coordinate and classify routing.
 *   - Roll 3: Guard non-null identity invariants.
 *   - Roll 4: Route and anchor memory target.
 */
omni_quad_state_t omni_site_execute(omni_site_context_t *ctx) {
    if (!ctx) return OMNI_STATE_ROLLBACK;

    /* ROLL 1: LOCATE (Ladder build-up with Numbers) */
    ctx->site = omni_site_from_address(ctx->input_coordinate);

    /* ROLL 2 & 3: BIND COORDINATE, CLASSIFY & NULL GUARD */
    bool site_valid = omni_site_is_valid(&ctx->site);
    bool identity_match = (ctx->site.coordinate == ctx->input_coordinate);

    /* Formal Guard Engine: Asserts non-null valid silicon address */
    if (omni_guard_assert(site_valid && identity_match, &ctx->tripwire, &ctx->status) != OMNI_STATE_PROCEED) {
        return OMNI_STATE_ROLLBACK;
    }

    /* ROLL 4: ROUTE & PROJECT (Inside-out emergence) */
    /* Physical coordinate anchored to hardware substrate */

    /* ROLL 5: SEQUENCE TRANSITION (Site chains to Memory Map) */
    /* Ready to sequence into system memory map */

    return ctx->status;
}

/* ============================================================================
 * BLOCK 4: CLOSING BLOCK IMPLEMENTATION
 * ( Integrity Seals & Commit Status )
 * ============================================================================ */

/**
```

```
 * @brief Verifies all invariants and seals the Site commit status.
 */
omni_quad_state_t omni_site_close(omni_site_context_t *ctx) {
    if (!ctx) return OMNI_STATE_ROLLBACK;

    /* Integrity Check: Proves tripwire is untripped and address is valid */
    bool integrity_ok = (omni_base_verify_integrity(ctx->tripwire, ctx->status) == OMNI_STATE_PROCEED) &&
                        omni_site_is_valid(&ctx->site);

    if (!integrity_ok) {
        return omni_commit_rollback(&ctx->tripwire);
    }

    /* Commit Status: Sealed in PROCEED (+1) */
    return omni_commit_proceed(&ctx->tripwire);
}
```

<TRACK 2> : {OmniCode Language & Standard Library}/lexicon/01_nouns/ideas/types/transpile variant/c_backbone/src/state.c

C | 174 LINES | 766 WORDS | 6.3 KB

```c
/* ============================================================================
 * BLOCK 1: METADATA & INVARIANT IMPLEMENTATION
 * ( Invariant Type: < Idea { Type } : State > )
 * ============================================================================
 * ARCHITECTURAL TOPOLOGY:
 *   Implements causal truth, clock progression, and the canonical Quad-State machine
 *   in the OmniCode Standard Library Track 2 C Transpile Variant.
 *   - Base Block: Primitive Invariant Type
 *   - Category: Idea { Type }
 *   - Footprint: [ 1 Trit ] { 2 Bits / Quad-State Causal Register }
 *   - Named Registers: [ STATE, TURN, BOUND, IT ]
 *   - Witness: Scripture Ecclesiastes 3:1 KJV
 *     "To every thing there is a season, and a time to every purpose under
 *      the heaven:"
 *   - Semantics: { QuadStateTruth :: CausalClock :: CanonicalLoopLaw }
 * ============================================================================ */

#include "state.h"

/* ============================================================================
 * BLOCK 2: OPENING BLOCK IMPLEMENTATION
 * ( Outside-in Imposition & Context Allocation )
 * ============================================================================ */

/**
 * @brief Initializes the State context, sets the starting truth signal,
 *        arms the register bank, and arms the execution tripwire.
 */
void omni_state_open(omni_state_context_t *ctx, omni_quad_state_t signal, int64_t bound_limit) {
    if (!ctx) return;

    /* 1. Outside-in imposition: truth signal captured */
    ctx->input_signal = signal ? signal : OMNI_STATE_PROCEED;

    /* 2. Concrete single-trit register allocation */
    ctx->state_engine = omni_state_create(ctx->input_signal, bound_limit);

    /* 3. Preconditions & tripwire arming */
    omni_tripwire_arm(&ctx->tripwire);
    ctx->status = OMNI_STATE_PROCEED;
}

/* ============================================================================
 * BLOCK 3: BODY BLOCK IMPLEMENTATION
 * ( Operational Primitives, Loop Laws & Transitions )
 * ============================================================================ */

/**
 * @brief Canonical Loop Law Single-Step Transition:
 *
 * LAW { Loop } :
 *   LOOP [ WHEN [ { IT } < { BOUND } ] --> [ IT` ] PROCEED to { TURN }
 *          ! [ { IT } > { BOUND } ] AWAIT for [ IT` ]
 *          :: HALT WHEN [ IT` ] = NULL ]
 */
omni_quad_state_t omni_state_step_loop(omni_state_registers_t *regs) {
    if (!regs) return OMNI_STATE_ROLLBACK;

    if (regs->it < regs->bound) {
        regs->it += 1;          /* [ IT` ] */
        regs->turn += 1;        /* PROCEED to { TURN } */
        regs->state = OMNI_STATE_PROCEED;
        return OMNI_STATE_PROCEED;
    } else if (regs->it > regs->bound) {
        regs->state = OMNI_STATE_AWAIT;
        return OMNI_STATE_AWAIT;
    } else {
        /* Terminal Detent reached: IT == BOUND */
        regs->state = OMNI_STATE_HALT;
        return OMNI_STATE_HALT;
    }
}

```

```c
/**
 * @brief Creates a freestanding State engine instance.
 */
omni_state_t omni_state_create(omni_quad_state_t initial_signal, int64_t bound) {
    omni_state_t s;
    s.substrate_trit = OMNI_TRIT_POS;
    s.registers.state = initial_signal;
    s.registers.turn  = 0;
    s.registers.it    = 0;
    s.registers.bound = (bound > 0) ? bound : 4;
    s.is_enclosed     = true;
    return s;
}

/**
 * @brief Resets state registers to initial state.
 */
void omni_state_reset(omni_state_t *s, int64_t bound) {
    if (!s) return;
    s->registers.state = OMNI_STATE_PROCEED;
    s->registers.turn  = 0;
    s->registers.it    = 0;
    s->registers.bound = (bound > 0) ? bound : 4;
}

/** Predicate queries for State engine */
bool omni_state_is_halted(const omni_state_t *s) {
    return (s != NULL && s->registers.state == OMNI_STATE_HALT);
}

bool omni_state_is_proceeding(const omni_state_t *s) {
    return (s != NULL && s->registers.state == OMNI_STATE_PROCEED);
}

bool omni_state_is_awaiting(const omni_state_t *s) {
    return (s != NULL && s->registers.state == OMNI_STATE_AWAIT);
}

bool omni_state_is_rollback(const omni_state_t *s) {
    return (s != NULL && s->registers.state == OMNI_STATE_ROLLBACK);
}

/**
 * @brief Executes the State governance lifecycle across declared operational rolls:
 *   - Roll 1: Inscribe state register from input signal.
 *   - Roll 2: Evaluate caveat guard against discrepancy.
 *   - Roll 3: Execute canonical loop until boundary or detent.
 *   - Roll 4: Govern machine clock output.
 */
omni_quad_state_t omni_state_execute(omni_state_context_t *ctx) {
    if (!ctx) return OMNI_STATE_ROLLBACK;

    /* ROLL 1: INSCRIBE STATE REGISTER */
    ctx->state_engine.registers.state = ctx->input_signal;

    /* ROLL 2: CAVEAT GUARD AGAINST DISCREPANCY */
    bool signal_match = (ctx->state_engine.registers.state == ctx->input_signal);
    if (omni_guard_assert(signal_match, &ctx->tripwire, &ctx->status) != OMNI_STATE_PROCEED) {
        return OMNI_STATE_ROLLBACK;
    }

    /* ROLL 3: THE CANONICAL LOOP STATEMENT EXECUTION */
    while (ctx->state_engine.registers.state == OMNI_STATE_PROCEED) {
        omni_quad_state_t loop_res = omni_state_step_loop(&ctx->state_engine.registers);
        if (loop_res == OMNI_STATE_HALT) {
            break; /* Clean detent termination */
        } else if (loop_res == OMNI_STATE_AWAIT) {
            break; /* Equilibrium standby */
        }
    }

    /* ROLL 4: GOVERN ENGINE (Inside-out emergence) */
    /* State emerges outward to govern machine clock */

    /* ROLL 5: SEQUENCE TRANSITION (State chains to continuous Timeline) */
    /* Ready to sequence into causal timeline */

    ctx->status = OMNI_STATE_PROCEED;
```

```
    return ctx->status;
}

/* ============================================================================
 * BLOCK 4: CLOSING BLOCK IMPLEMENTATION
 * ( Integrity Seals & Commit Status )
 * ============================================================================ */

/**
 * @brief Verifies all invariants and seals the State commit status.
 */
omni_quad_state_t omni_state_close(omni_state_context_t *ctx) {
    if (!ctx) return OMNI_STATE_ROLLBACK;

    /* Integrity Check: Proves tripwire is untripped and register bank is valid */
    if (omni_base_verify_integrity(ctx->tripwire, ctx->status) != OMNI_STATE_PROCEED) {
        return omni_commit_rollback(&ctx->tripwire);
    }

    /* Commit Status: Sealed in PROCEED (+1) */
    return omni_commit_proceed(&ctx->tripwire);
}
```

<TRACK 2> : {OmniCode Language & Standard Library}/lexicon/01_nouns/ideas/types/transpile variant/lua_tapes/character.lua

LUA | 149 LINES | 579 WORDS | 5.9 KB

```lua
-- ================================================================================
-- BLOCK 1: METADATA & PROCESS TAPE MANIFEST
-- ( Process Tape: < Lexicon { Types of Ideas } > & [ Process { Character } ] )
-- ================================================================================
-- ARCHITECTURAL TOPOLOGY:
--   Interprets and executes the atomic lifecycle for Invariant Type < Character >.
--   Acts as the dynamic runtime companion driving C backbone struct `omni_character_t`.
--   - Defined Space: [ 1 Cell ] { 5 Trits :: 1 Byte }
--   - Execution Domain: Lexicon { Types of Ideas }
--   - Witness: Scripture Exodus 31:18 KJV
--     "And he gave unto Moses, when he had made an end of communing with him upon
--      mount Sinai, two tables of testimony, tables of stone, written with the
--      finger of God."
--   - Semantics: { GlyphInscription :: DualSubstrateTransduction :: SingleCellBound }
-- ================================================================================

local TapeEngine = require("lua_tapes.tape_engine")

local CONSTANTS = {
    CELL_CAPACITY       = 1,
    TRITS_PER_CELL      = 5,
    EXEMPLAR_GLYPH_A    = string.byte('A')
}

local CharacterTape = TapeEngine.create_tape({
    metadata = {
        who     = "OmniCode Lexicon",
        what    = "Character --> INSCRIBE { Glyph }",
        when    = "Input Event",
        where   = { character_cell = 1 },
        why     = "Inscribe atomic character type invariant into defined space",
        how     = "+1 : PROCEED",
        witness = "Exodus 31:18 KJV"
    },
    constants = CONSTANTS,
    rolls = {
        -- ------------------------------------------------------------------------
        -- ROLL 1: GLYPH INSCRIPTION (Outside-in imposition onto C cell)
        -- ------------------------------------------------------------------------
        {
            name = "Inscribe",
            action = function(ctx)
                ctx.c_cell.glyph = ctx.glyph
                ctx.c_cell.is_enclosed = true
                return TapeEngine.STATE_PROCEED
            end
        },

        -- ------------------------------------------------------------------------
        -- ROLL 2: DUAL-SUBSTRATE TRANSDUCTION (8 Bits <=> 5 Balanced Trits)
        -- ------------------------------------------------------------------------
        {
            name = "Transduce",
            action = function(ctx)
                local val = ctx.c_cell.glyph
                for i = 1, CONSTANTS.TRITS_PER_CELL do
                    local rem = val % 3
                    val = math.floor(val / 3)
                    if rem == 2 then
                        rem = -1
                        val = val + 1
                    elseif rem == -2 then
                        rem = 1
                        val = val - 1
                    end
                    ctx.c_cell.trits[i] = rem
                end
                return TapeEngine.STATE_PROCEED
            end
        },

        -- ------------------------------------------------------------------------
        -- ROLL 3: IDENTITY & BOUND COMPARISON (Guard Single-Cell Boundary)
```

```
        -- ------------------------------------------------------------------------
        {
            name = "Compare",
            action = function(ctx)
                local identity_ok = (ctx.c_cell.glyph == ctx.glyph)
                local bounds_ok   = (ctx.c_cell.is_enclosed == true)
                return TapeEngine.assert_guard(identity_ok and bounds_ok, ctx,
                                               "Character glyph or single-cell bound violation")
            end
        },

        -- ------------------------------------------------------------------------
        -- ROLL 4: TARGET PROJECTION (Inside-out emergence to output bus/terminal)
        -- ------------------------------------------------------------------------
        {
            name = "Project",
            action = function(ctx)
                if ctx.target then
                    table.insert(ctx.target, ctx.c_cell.glyph)
                end
                return TapeEngine.STATE_PROCEED
            end
        },

        -- ------------------------------------------------------------------------
        -- ROLL 5: SEQUENCE TRANSITION (Character chains toward continuous Text ribbon)
        -- ------------------------------------------------------------------------
        {
            name = "Sequence",
            action = function(ctx)
                ctx.sequence_ready = true
                return TapeEngine.STATE_PROCEED
            end
        }
    }
})

-- ============================================================================
-- BLOCK 2: OPENING BLOCK
-- ( Context Factory & C Cell Substrate Mount )
-- ============================================================================

---
-- Mounts the tape head, initialises the C cell handle, and arms the tripwire.
-- @param glyph Outside-in byte payload (defaults to ASCII 'A').
-- @param target_buffer Optional destination array (terminal or VGA buffer).
-- @return Initialized context table.
function CharacterTape.open(glyph, target_buffer)
    return {
        glyph    = glyph or CharacterTape.constants.EXEMPLAR_GLYPH_A,
        target   = target_buffer or {},
        c_cell   = {
            glyph       = 0,
            trits       = {0, 0, 0, 0, 0},
            is_enclosed = false
        },
        status   = TapeEngine.STATE_PROCEED,
        tripwire = TapeEngine.TRIPWIRE_ARMED
    }
end

-- ============================================================================
-- BLOCK 4: CLOSING BLOCK
-- ( Fourfold Detent Commit & Interlock Verification )
-- ============================================================================

---
-- Closes and seals the Character tape execution context.
-- @param ctx Active context handle.
-- @return Quad-State commit status and diagnostic seal message.
function CharacterTape.close(ctx)
    return TapeEngine.verify_and_commit(ctx)
end

return CharacterTape
```

<TRACK 2> : {OmniCode Language & Standard Library}/lexicon/01_nouns/ideas/types/transpile variant/lua_tapes/container.lua

LUA | 158 LINES | 632 WORDS | 6.5 KB

```lua
-- ================================================================================
-- BLOCK 1: METADATA & PROCESS TAPE MANIFEST
-- ( Process Tape: < Lexicon { Types of Ideas } > & [ Process { Container } ] )
-- ================================================================================
-- ARCHITECTURAL TOPOLOGY:
--   Interprets and executes the atomic lifecycle for Invariant Type < Container >.
--   Acts as the dynamic runtime companion driving generic C backbone struct `omni_container_t`.
--   - Defined Space: Generic Bounded Memory Envelope / Slice
--   - Ladder Ascension: Build-up with Numbers (/-^)
--   - Witness: Scripture 2 Kings 4:4 KJV
--     "And when thou art come in, thou shalt shut the door upon thee and upon
--      thy sons, and shalt pour out into all those vessels, and thou shalt set
--      aside that which is full."
--   - Semantics: { MemoryCapsule :: BoundedEnvelope :: ContainmentLaw }
-- ================================================================================

local TapeEngine = require("lua_tapes.tape_engine")

local CONSTANTS = {
    EXEMPLAR_UNIT       = 1,
    EXEMPLAR_VESSEL     = 64,
    EXEMPLAR_STOREHOUSE = 4096
}

local ContainerTape = TapeEngine.create_tape({
    metadata = {
        who     = "OmniCode Lexicon",
        what    = "Container --> ENCAPSULATE { Payload }",
        when    = "Allocation Event",
        where   = { container_envelope = 64 },
        why     = "Inscribe bounded container type invariant and storage envelopes",
        how     = "+1 : PROCEED",
        witness = "2 Kings 4:4 KJV"
    },
    constants = CONSTANTS,
    rolls = {
        -- ------------------------------------------------------------------------
        -- ROLL 1: BUILD CAPSULE (Ladder build-up with Numbers)
        -- ------------------------------------------------------------------------
        {
            name = "BuildCapsule",
            action = function(ctx)
                local valid_cap = (ctx.c_container.capacity > 0)
                if valid_cap then
                    ctx.c_container.is_enclosed = true
                end
                return TapeEngine.assert_guard(valid_cap, ctx, "Container capacity must be greater than zero")
            end
        },

        -- ------------------------------------------------------------------------
        -- ROLL 2: ENCAPSULATION (Inscribe payload into bounded C envelope)
        -- ------------------------------------------------------------------------
        {
            name = "Encapsulate",
            action = function(ctx)
                local items = ctx.payload
                local count = #items

                -- Law { Containment } Guard: Occupancy must never exceed capacity limit
                local within_capacity = (count <= ctx.c_container.capacity)
                if not within_capacity then
                    return TapeEngine.assert_guard(false, ctx, "Payload count exceeds bounded container capacity")
                end

                for i = 1, count do
                    ctx.c_container.buffer[i] = items[i]
                end
                ctx.c_container.occupancy = count
                return TapeEngine.STATE_PROCEED
            end
        },

```

```
        -- ------------------------------------------------------------------------
        -- ROLL 3: IDENTITY & BOUND COMPARISON (Guard bounds under Law { Containment })
        -- ------------------------------------------------------------------------
        {
            name = "GuardBounds",
            action = function(ctx)
                local containment_valid = (ctx.c_container.occupancy <= ctx.c_container.capacity)
                local enclosed_valid    = (ctx.c_container.is_enclosed == true)
                return TapeEngine.assert_guard(containment_valid and enclosed_valid, ctx,
                                               "Core Containment Law violated: Occupancy > Capacity")
            end
        },

        -- ------------------------------------------------------------------------
        -- ROLL 4: POUR OUT & SET ASIDE (Inside-out emergence)
        -- ------------------------------------------------------------------------
        {
            name = "PourOut",
            action = function(ctx)
                ctx.sealed_vessel = {
                    origin  = ctx.c_container.buffer,
                    target  = ctx.target,
                    sealed  = true
                }
                return TapeEngine.STATE_PROCEED
            end
        },

        -- ------------------------------------------------------------------------
        -- ROLL 5: SEQUENCE TRANSITION (Container chains to composite structure)
        -- ------------------------------------------------------------------------
        {
            name = "Sequence",
            action = function(ctx)
                ctx.structure_ready = true
                return TapeEngine.STATE_PROCEED
            end
        }
    }
})

-- ============================================================================
-- BLOCK 2: OPENING BLOCK
-- ( Context Factory & C Container Slice Substrate Mount )
-- ============================================================================

---
-- Mounts the tape head, initialises the generic C container handle, and arms the tripwire.
-- @param payload Outside-in array of items (e.g. Character bytes or Numbers).
-- @param capacity Maximum number of elements (defaults to 64).
-- @param element_size Byte footprint of a single element (defaults to 1).
-- @return Initialized context table.
function ContainerTape.open(payload, capacity, element_size)
    local cap  = capacity or CONSTANTS.EXEMPLAR_VESSEL
    local e_sz = element_size or 1
    return {
        payload     = payload or {},
        target      = 0x00000000,
        c_container = {
            buffer       = {},
            element_size = e_sz,
            capacity     = cap,
            occupancy    = 0,
            is_enclosed  = false
        },
        status      = TapeEngine.STATE_PROCEED,
        tripwire    = TapeEngine.TRIPWIRE_ARMED
    }
end

-- ============================================================================
-- BLOCK 4: CLOSING BLOCK
-- ( Fourfold Detent Commit & Interlock Verification )
-- ============================================================================

---
-- Closes and seals the Container tape execution context.
-- @param ctx Active context handle.
```

```
-- @return Quad-State commit status and diagnostic seal message.
function ContainerTape.close(ctx)
    return TapeEngine.verify_and_commit(ctx)
end

return ContainerTape
```

<TRACK 2> : {OmniCode Language & Standard Library}/lexicon/01_nouns/ideas/types/transpile variant/lua_tapes/number.lua

LUA | 154 LINES | 577 WORDS | 6.1 KB

```lua
-- ================================================================================
-- BLOCK 1: METADATA & PROCESS TAPE MANIFEST
-- ( Process Tape: < Lexicon { Types of Ideas } > & [ Process { Number } ] )
-- ================================================================================
-- ARCHITECTURAL TOPOLOGY:
--   Interprets and executes the atomic lifecycle for Invariant Type < Number >.
--   Acts as the dynamic runtime companion driving C backbone struct `omni_number_t`.
--   - Defined Space: [ 1 Word ] { 32 Trits :: 64 Bits }
--   - Execution Domain: Lexicon { Types of Ideas }
--   - Witness: Scripture Psalm 147:4 KJV
--     "He telleth the number of the stars; he calleth them all by their names."
--   - Semantics: { ScalarMeasurement :: Radix3Transduction :: ZeroSumEquilibrium }
-- ================================================================================

local TapeEngine = require("lua_tapes.tape_engine")

local CONSTANTS = {
    WORD_CAPACITY       = 1,
    TRITS_PER_WORD      = 32,
    EXEMPLAR_ZERO       = 0,
    EXEMPLAR_UNITY_POS  = 1,
    EXEMPLAR_UNITY_NEG  = -1,
    EXEMPLAR_PARTITION  = 4096
}

local NumberTape = TapeEngine.create_tape({
    metadata = {
        who     = "OmniCode Lexicon",
        what    = "Number --> MEASURE { Quantity }",
        when    = "Arithmetic Event",
        where   = { number_word = 1 },
        why     = "Inscribe scalar number type invariant and arithmetic equilibrium",
        how     = "+1 : PROCEED",
        witness = "Psalm 147:4 KJV"
    },
    constants = CONSTANTS,
    rolls = {
        -- ------------------------------------------------------------------------
        -- ROLL 1: QUANTITY INSCRIPTION (Outside-in imposition onto C word)
        -- ------------------------------------------------------------------------
        {
            name = "Inscribe",
            action = function(ctx)
                ctx.c_word.scalar = ctx.quantity
                ctx.c_word.is_enclosed = true
                return TapeEngine.STATE_PROCEED
            end
        },

        -- ------------------------------------------------------------------------
        -- ROLL 2: DUAL-SUBSTRATE TRANSDUCTION (64 Bits <=> 32 Balanced Trits)
        -- ------------------------------------------------------------------------
        {
            name = "Transduce",
            action = function(ctx)
                local current = ctx.c_word.scalar
                for i = 1, CONSTANTS.TRITS_PER_WORD do
                    if current == 0 then
                        ctx.c_word.trits[i] = 0
                    else
                        local rem = current % 3
                        current = math.floor(current / 3)
                        if rem == 2 then
                            rem = -1
                            current = current + 1
                        elseif rem == -2 then
                            rem = 1
                            current = current - 1
                        end
                        ctx.c_word.trits[i] = rem
                    end
                end
                return TapeEngine.STATE_PROCEED
```

```
            end
        },

        -- ------------------------------------------------------------------
        -- ROLL 3: IDENTITY & BOUND COMPARISON (Guard Single-Word Boundary)
        -- ------------------------------------------------------------------
        {
            name = "Compare",
            action = function(ctx)
                local identity_ok = (ctx.c_word.scalar == ctx.quantity)
                local bounds_ok   = (ctx.c_word.is_enclosed == true)
                return TapeEngine.assert_guard(identity_ok and bounds_ok, ctx,
                                               "Number scalar value or single-word bound violation")
            end
        },

        -- ------------------------------------------------------------------
        -- ROLL 4: ARITHMETIC EQUILIBRIUM CALCULATION ((+1) plus (-1) <=> 0)
        -- ------------------------------------------------------------------
        {
            name = "Equilibrium",
            action = function(ctx)
                local eq = (1) + (-1)
                local eq_holds = (eq == 0)
                ctx.equilibrium_proved = eq_holds
                return TapeEngine.assert_guard(eq_holds, ctx,
                                               "ALU Zero-Sum Equilibrium Law violated")
            end
        },

        -- ------------------------------------------------------------------
        -- ROLL 5: SEQUENCE TRANSITION (Number chains toward Vector sequence)
        -- ------------------------------------------------------------------
        {
            name = "Sequence",
            action = function(ctx)
                ctx.vector_ready = true
                return TapeEngine.STATE_PROCEED
            end
        }
    }
})

-- ============================================================================
-- BLOCK 2: OPENING BLOCK
-- ( Context Factory & C Word Substrate Mount )
-- ============================================================================

---
-- Mounts the tape head, initialises the C word handle, and arms the tripwire.
-- @param quantity Outside-in scalar integer (defaults to 4096).
-- @return Initialized context table.
function NumberTape.open(quantity)
    return {
        quantity           = quantity or NumberTape.constants.EXEMPLAR_PARTITION,
        c_word             = {
            scalar      = 0,
            trits       = {},
            is_enclosed = false
        },
        equilibrium_proved = false,
        status             = TapeEngine.STATE_PROCEED,
        tripwire           = TapeEngine.TRIPWIRE_ARMED
    }
end

-- ============================================================================
-- BLOCK 4: CLOSING BLOCK
-- ( Fourfold Detent Commit & Interlock Verification )
-- ============================================================================

---
-- Closes and seals the Number tape execution context.
-- @param ctx Active context handle.
-- @return Quad-State commit status and diagnostic seal message.
function NumberTape.close(ctx)
    return TapeEngine.verify_and_commit(ctx)
end
```

```
152
153 return NumberTape
```

<TRACK 2> : {OmniCode Language & Standard Library}/lexicon/01_nouns/ideas/types/transpile variant/lua_tapes/site.lua

LUA | 148 LINES | 565 WORDS | 6.0 KB

```lua
-- ================================================================================
-- BLOCK 1: METADATA & PROCESS TAPE MANIFEST
-- ( Process Tape: < Lexicon { Types of Ideas } > & [ Process { Site } ] )
-- ================================================================================
-- ARCHITECTURAL TOPOLOGY:
--   Interprets and executes the atomic lifecycle for Invariant Type < Site >.
--   Acts as the dynamic runtime companion driving C backbone struct `omni_site_t`.
--   - Defined Space: [ 1 Address ] { 64 Bits / Silicon Coordinate }
--   - Ladder Ascension: Build-up with Numbers (/-^)
--   - Witness: Scripture Genesis 28:16 KJV
--     "And Jacob awaked out of his sleep, and he said, Surely the Lord is in
--      this place; and I knew it not."
--   - Semantics: { SiliconCoordinate :: HardwareRoutingLaw :: NullPointerDetent }
-- ================================================================================

local TapeEngine = require("lua_tapes.tape_engine")

local CONSTANTS = {
    SITE_NULL   = 0x00000000,
    SITE_MBR    = 0x00007C00,
    SITE_COM1   = 0x000003F8,
    SITE_VGA    = 0x000B8000
}

local SiteTape = TapeEngine.create_tape({
    metadata = {
        who     = "OmniCode Lexicon",
        what    = "Site --> LOCATE { PhysicalAddress }",
        when    = "Addressing Event",
        where   = { silicon_coordinate = 1 },
        why     = "Inscribe physical site address invariant and hardware MMIO coordinates",
        how     = "+1 : PROCEED",
        witness = "Genesis 28:16 KJV"
    },
    constants = CONSTANTS,
    rolls = {
        -- ------------------------------------------------------------------------
        -- ROLL 1: LOCATE (Ladder build-up with Numbers)
        -- ------------------------------------------------------------------------
        {
            name = "Locate",
            action = function(ctx)
                local not_null = (ctx.coordinate ~= CONSTANTS.SITE_NULL)
                if not_null then
                    ctx.c_site.is_enclosed = true
                end
                return TapeEngine.assert_guard(not_null, ctx,
                                               "Null pointer address cannot be located as a valid silicon site")
            end
        },

        -- ------------------------------------------------------------------------
        -- ROLL 2: BIND COORDINATE (Inscribe address & resolve RAM vs MMIO routing)
        -- ------------------------------------------------------------------------
        {
            name = "BindCoordinate",
            action = function(ctx)
                ctx.c_site.coordinate = ctx.coordinate
                -- Hardware Routing Law: COM1, VGA, or >= 0xA0000 route to MMIO; otherwise RAM
                if ctx.coordinate == CONSTANTS.SITE_COM1 or
                   ctx.coordinate == CONSTANTS.SITE_VGA  or
                   ctx.coordinate >= 0x000A0000 then
                    ctx.c_site.routing = "HARDWARE_MMIO (::@)"
                else
                    ctx.c_site.routing = "PHYSICAL_RAM (@)"
                end
                return TapeEngine.STATE_PROCEED
            end
        },

        -- ------------------------------------------------------------------------
        -- ROLL 3: IDENTITY & NULL GUARD (Guard Non-Null Validity)
        -- ------------------------------------------------------------------------
```

```
        {
            name = "GuardAddress",
            action = function(ctx)
                local identity_ok = (ctx.c_site.coordinate == ctx.coordinate)
                local valid_site  = (ctx.c_site.coordinate ~= CONSTANTS.SITE_NULL and ctx.c_site.is_enclosed)
                return TapeEngine.assert_guard(identity_ok and valid_site, ctx,
                                               "Site address failed non-null validation check")
            end
        },

        -- ------------------------------------------------------------------
        -- ROLL 4: ROUTE & PROJECT (Inside-out emergence)
        -- ------------------------------------------------------------------
        {
            name = "Route",
            action = function(ctx)
                ctx.memory_anchor = {
                    coordinate = ctx.c_site.coordinate,
                    routing    = ctx.c_site.routing,
                    anchored   = true
                }
                return TapeEngine.STATE_PROCEED
            end
        },

        -- ------------------------------------------------------------------
        -- ROLL 5: SEQUENCE TRANSITION (Site chains to Memory Map)
        -- ------------------------------------------------------------------
        {
            name = "Sequence",
            action = function(ctx)
                ctx.memory_map_ready = true
                return TapeEngine.STATE_PROCEED
            end
        }
    }
})

-- ============================================================================
-- BLOCK 2: OPENING BLOCK
-- ( Context Factory & C Site Substrate Mount )
-- ============================================================================

---
-- Mounts the tape head, initialises the C site handle, and arms the tripwire.
-- @param coordinate Outside-in 64-bit silicon address (defaults to VGA buffer).
-- @return Initialized context table.
function SiteTape.open(coordinate)
    return {
        coordinate = coordinate or CONSTANTS.SITE_VGA,
        c_site     = {
            coordinate  = 0,
            routing     = "UNRESOLVED",
            is_enclosed = false
        },
        status     = TapeEngine.STATE_PROCEED,
        tripwire   = TapeEngine.TRIPWIRE_ARMED
    }
end

-- ============================================================================
-- BLOCK 4: CLOSING BLOCK
-- ( Fourfold Detent Commit & Interlock Verification )
-- ============================================================================

---
-- Closes and seals the Site tape execution context.
-- @param ctx Active context handle.
-- @return Quad-State commit status and diagnostic seal message.
function SiteTape.close(ctx)
    return TapeEngine.verify_and_commit(ctx)
end

return SiteTape
```

<TRACK 2> : {OmniCode Language & Standard Library}/lexicon/01_nouns/ideas/types/transpile variant/lua_tapes/state.lua

LUA | 183 LINES | 697 WORDS | 7.1 KB

```lua
-- ================================================================================
-- BLOCK 1: METADATA & PROCESS TAPE MANIFEST
-- ( Process Tape: < Lexicon { Types of Ideas } > & [ Process { State } ] )
-- ================================================================================
-- ARCHITECTURAL TOPOLOGY:
--   Interprets and executes the atomic lifecycle for Invariant Type < State >.
--   Acts as the dynamic runtime companion driving C backbone struct `omni_state_t`.
--   - Defined Space: [ 1 Trit ] { 2 Bits / Quad-State Causal Register }
--   - Named Registers: [ STATE TURN BOUND IT ]
--   - Witness: Scripture Ecclesiastes 3:1 KJV
--     "To every thing there is a season, and a time to every purpose under
--      the heaven:"
--   - Semantics: { QuadStateTruth :: CausalClock :: CanonicalLoopLaw }
-- ================================================================================

local TapeEngine = require("lua_tapes.tape_engine")

local CONSTANTS = {
    STATE_PROCEED  = TapeEngine.STATE_PROCEED,
    STATE_AWAIT    = TapeEngine.STATE_AWAIT,
    STATE_ROLLBACK = TapeEngine.STATE_ROLLBACK,
    STATE_HALT     = TapeEngine.STATE_HALT,
    DEFAULT_BOUND  = 4
}

local StateTape = TapeEngine.create_tape({
    metadata = {
        who     = "OmniCode Lexicon",
        what    = "State --> GOVERN { CausalTruth }",
        when    = "Control Event",
        where   = { state_register = 1 },
        why     = "Inscribe quad-state truth invariant and execute canonical loops",
        how     = "+1 : PROCEED",
        witness = "Ecclesiastes 3:1 KJV"
    },
    constants = CONSTANTS,
    rolls = {
        -- ------------------------------------------------------------------------
        -- ROLL 1: INSCRIBE STATE REGISTER
        -- ------------------------------------------------------------------------
        {
            name = "Inscribe",
            action = function(ctx)
                ctx.c_state.registers.state = ctx.signal
                ctx.c_state.is_enclosed = true
                return TapeEngine.STATE_PROCEED
            end
        },

        -- ------------------------------------------------------------------------
        -- ROLL 2: CAVEAT GUARD AGAINST DISCREPANCY
        -- ------------------------------------------------------------------------
        {
            name = "GuardSignal",
            action = function(ctx)
                local signal_match = (ctx.c_state.registers.state == ctx.signal)
                return TapeEngine.assert_guard(signal_match, ctx,
                                               "State register truth signal discrepancy")
            end
        },

        -- ------------------------------------------------------------------------
        -- ROLL 3: CANONICAL LOOP STEPPER (Iterative Quad-State Cadence)
        -- ------------------------------------------------------------------------
        {
            name = "LoopCadence",
            action = function(ctx)
                local regs = ctx.c_state.registers
                while regs.state == TapeEngine.STATE_PROCEED do
                    if regs.it < regs.bound then
                        regs.it = regs.it + 1
                        regs.turn = regs.turn + 1
                        regs.state = TapeEngine.STATE_PROCEED
```

```
                    elseif regs.it > regs.bound then
                        regs.state = TapeEngine.STATE_AWAIT
                        break
                    else
                        -- Terminal Detent reached: IT == BOUND
                        regs.state = TapeEngine.STATE_HALT
                        break
                    end
                end
                return regs.state
            end
        },

        -- ------------------------------------------------------------------
        -- ROLL 4: GOVERN ENGINE (Inside-out emergence to machine clock)
        -- ------------------------------------------------------------------
        {
            name = "GovernClock",
            action = function(ctx)
                ctx.clock_active = (ctx.c_state.registers.state ~= TapeEngine.STATE_HALT)
                return TapeEngine.STATE_PROCEED
            end
        },

        -- ------------------------------------------------------------------
        -- ROLL 5: SEQUENCE TRANSITION (State chains to continuous Timeline)
        -- ------------------------------------------------------------------
        {
            name = "Sequence",
            action = function(ctx)
                ctx.timeline_ready = true
                return TapeEngine.STATE_PROCEED
            end
        }
    }
})

-- ============================================================================
-- BLOCK 2: OPENING BLOCK
-- ( Context Factory & C State Substrate Mount )
-- ============================================================================

---
-- Mounts the tape head, initialises the C state register bank handle, and arms the tripwire.
-- @param signal Outside-in truth signal (defaults to PROCEED).
-- @param bound Maximum iteration threshold (defaults to 4).
-- @return Initialized context table.
function StateTape.open(signal, bound)
    local sig = signal or CONSTANTS.STATE_PROCEED
    local bnd = bound or CONSTANTS.DEFAULT_BOUND
    return {
        signal   = sig,
        c_state  = {
            substrate_trit = 1,
            registers = {
                state = sig,
                turn  = 0,
                bound = bnd,
                it    = 0
            },
            is_enclosed = false
        },
        status   = TapeEngine.STATE_PROCEED,
        tripwire = TapeEngine.TRIPWIRE_ARMED
    }
end

-- ============================================================================
-- BLOCK 3: OPERATIONAL PRIMITIVES
-- ( Single-Step Loop Law Execution )
-- ============================================================================

---
-- Single-steps the Canonical Loop Law across named registers:
--   LOOP [ WHEN [ { IT } < { BOUND } ] --> [ IT` ] PROCEED to { TURN }
--          ! [ { IT } > { BOUND } ] AWAIT for [ IT` ]
--          :: HALT WHEN [ IT` ] = NULL ]
-- @param ctx Context handle holding register bank.
```

```
-- @return New Quad-State condition after the single step.
function StateTape.step_loop(ctx)
    local regs = ctx.c_state.registers
    if regs.it < regs.bound then
        regs.it = regs.it + 1
        regs.turn = regs.turn + 1
        regs.state = TapeEngine.STATE_PROCEED
        return TapeEngine.STATE_PROCEED
    elseif regs.it > regs.bound then
        regs.state = TapeEngine.STATE_AWAIT
        return TapeEngine.STATE_AWAIT
    else
        regs.state = TapeEngine.STATE_HALT
        return TapeEngine.STATE_HALT
    end
end

-- ================================================================================
-- BLOCK 4: CLOSING BLOCK
-- ( Fourfold Detent Commit & Interlock Verification )
-- ================================================================================

---
-- Closes and seals the State tape execution context.
-- @param ctx Active context handle.
-- @return Quad-State commit status and diagnostic seal message.
function StateTape.close(ctx)
    return TapeEngine.verify_and_commit(ctx)
end

return StateTape
```

<TRACK 2> : {OmniCode Language & Standard Library}/lexicon/01_nouns/ideas/types/transpile variant/lua_tapes/tape_engine.lua

LUA | 223 LINES | 957 WORDS | 8.6 KB

```lua
-- ================================================================================
-- BLOCK 1: METADATA & ENGINE TOPOLOGY
-- ( Engine Core: < Lexicon { Types of Ideas } > & [ Process { Tape Engine } ] )
-- ================================================================================
-- ARCHITECTURAL TOPOLOGY:
--   The interpreted execution ribbon companion to the OmniCode C Backbone.
--   Drives the dynamic temporal causation, coroutines, and roll kinematics:
--   - Coroutine-based fiber stepping (yielding on AWAIT, resuming on tick)
--   - Pre-roll and post-roll dynamic event hooks
--   - State snapshotting and active rollback unwinding
--   - Structured handles over C memory layouts and invariants
--   - Witness: Scripture Proverbs 16:9 KJV
--     "A man's heart deviseth his way: but the LORD directeth his steps."
--   - Semantics: { TapeKinematics :: CoroutineFiber :: RollHooks :: QuadStateDetents }
-- ================================================================================

local TapeEngine = {
    -- Quad-State Truth Constants
    STATE_ROLLBACK = -1,   -- -1 : Retroactive Reversal / Error Unwind
    STATE_AWAIT    =  0,   --  0 : Equilibrium Standby / Hold Pending
    STATE_PROCEED  =  1,   -- +1 : Forward Causation / Affirmative Advancement
    STATE_HALT     = "NULL",-- NULL : Terminal Detent / Stop Processor Clock

    -- Tripwire Interlock States
    TRIPWIRE_DISARMED = 0,
    TRIPWIRE_ARMED    = 1,
    TRIPWIRE_TRIPPED  = 2
}

-- Detect optional LuaJIT FFI for direct zero-copy C backbone linkage
local has_ffi, ffi = pcall(require, "ffi")
TapeEngine.has_ffi = has_ffi
TapeEngine.ffi = has_ffi and ffi or nil

-- ================================================================================
-- BLOCK 2: OPENING BLOCK
-- ( Subsystem Initialization, Preconditions & Tape Mounting )
-- ================================================================================

---
-- Deep copy snapshot utility for transactional rollback safety.
-- @param ctx The context table to clone.
-- @return Independent deep copy of the context.
local function make_snapshot(ctx)
    local snap = {}
    for k, v in pairs(ctx) do
        if type(v) == "table" then
            snap[k] = make_snapshot(v)
        else
            snap[k] = v
        end
    end
    return snap
end

---
-- Creates and mounts an interpreted Tape Process Ribbon.
-- @param spec Table containing metadata, constants, and declared rolls.
-- @return Mounted tape object.
function TapeEngine.create_tape(spec)
    local tape = {
        metadata    = spec.metadata or {},
        constants   = spec.constants or {},
        rolls       = spec.rolls or {},
        hooks       = {
            before_roll = {},
            after_roll  = {},
            on_rollback = {}
        },
        snapshots   = {},
        history     = {}
    }

```

```lua
    ---
    -- Registers a dynamic hook invoked immediately before a roll executes.
    function tape:before(fn)
        table.insert(self.hooks.before_roll, fn)
        return self
    end

    ---
    -- Registers a dynamic hook invoked immediately after a roll executes.
    function tape:after(fn)
        table.insert(self.hooks.after_roll, fn)
        return self
    end

    ---
    -- Registers a rollback handler invoked when a roll trips the interlock.
    function tape:on_rollback(fn)
        table.insert(self.hooks.on_rollback, fn)
        return self
    end

    ---
    -- Restores a previously captured execution snapshot into the active context.
    function tape:restore_snapshot(ctx, roll_index)
        local snap = self.snapshots[roll_index]
        if snap then
            for k, v in pairs(snap) do
                ctx[k] = v
            end
        end
    end

-- ============================================================================
-- BLOCK 3: BODY BLOCK
-- ( Coroutine Kinematics, Roll Stepping & Hook Dispatch )
-- ============================================================================

    ---
    -- Spawns a coroutine fiber that steps through the declared rolls.
    -- Yields on AWAIT (0), halts on HALT (NULL), and unwinds on ROLLBACK (-1).
    -- @param ctx Context handle holding the C struct proxy and tripwires.
    -- @return Lua coroutine object.
    function tape:spawn_coroutine(ctx)
        return coroutine.create(function()
            ctx.status = TapeEngine.STATE_PROCEED
            ctx.tripwire = TapeEngine.TRIPWIRE_ARMED

            for i, roll in ipairs(self.rolls) do
                -- 1. Snapshot state prior to roll execution for transaction rollback
                self.snapshots[i] = make_snapshot(ctx)

                -- 2. Dispatch pre-roll hooks
                for _, h in ipairs(self.hooks.before_roll) do
                    h(self, i, roll.name, ctx)
                end

                -- 3. Execute the declared roll action
                local status = roll.action(ctx)
                ctx.status = status or ctx.status

                -- 4. Dispatch post-roll hooks
                for _, h in ipairs(self.hooks.after_roll) do
                    h(self, i, roll.name, ctx, ctx.status)
                end

                -- Record execution trace
                table.insert(self.history, { roll = i, name = roll.name, status = ctx.status })

                -- 5. Quad-State transition evaluation
                if ctx.status == TapeEngine.STATE_AWAIT then
                    -- Yield execution back to host/game loop until external condition resolves
                    coroutine.yield("AWAIT", i, roll.name, ctx)
                elseif ctx.status == TapeEngine.STATE_ROLLBACK then
                    -- Invariant violation: trip safety interlock and trigger rollback unwinding
                    ctx.tripwire = TapeEngine.TRIPWIRE_TRIPPED
                    for _, h in ipairs(self.hooks.on_rollback) do
                        h(self, i, roll.name, ctx, self.snapshots[i])
                    end
```

```
                    return TapeEngine.STATE_ROLLBACK, "Tape rolled back at roll " .. i .. " [" .. roll.name .. "]"
                elseif ctx.status == TapeEngine.STATE_HALT then
                    -- Terminal Detent reached
                    return TapeEngine.STATE_HALT, "Tape reached terminal detent (HALT:NULL)"
                end
            end

            ctx.tripwire = TapeEngine.TRIPWIRE_DISARMED
            return TapeEngine.STATE_PROCEED, "Tape sealed in PROCEED (+1)"
        end)
    end

    ---
    -- Executes all rolls in a synchronous loop, automatically resuming AWAITs.
    -- @param ctx Context handle.
    -- @return Final Quad-State status code and descriptive seal message.
    function tape:execute_all(ctx)
        local co = self:spawn_coroutine(ctx)
        while coroutine.status(co) ~= "dead" do
            local ok, event, roll_idx, roll_name, curr_ctx = coroutine.resume(co)
            if not ok then
                error("Tape execution fatal error: " .. tostring(event))
            end
            if event == "AWAIT" then
                -- In synchronous batch mode, acknowledge standby and resume
                curr_ctx.status = TapeEngine.STATE_PROCEED
            elseif event == TapeEngine.STATE_PROCEED or event == TapeEngine.STATE_HALT or event ==
TapeEngine.STATE_ROLLBACK then
                return event, roll_idx
            end
        end
        return ctx.status
    end

    return tape
end

-- ============================================================================
-- BLOCK 4: CLOSING BLOCK
-- ( Integrity Seals & Commit Verification )
-- ============================================================================

---
-- Evaluates an invariant guard condition inside a roll action.
-- Trips the tripwire and triggers ROLLBACK on condition failure.
-- @param condition Boolean expression to verify.
-- @param ctx Context handle.
-- @param error_reason Message recorded upon tripwire trigger.
-- @return STATE_PROCEED if condition is true; STATE_ROLLBACK otherwise.
function TapeEngine.assert_guard(condition, ctx, error_reason)
    if condition then
        return TapeEngine.STATE_PROCEED
    else
        ctx.tripwire = TapeEngine.TRIPWIRE_TRIPPED
        ctx.error_reason = error_reason or "Invariant guard violation"
        return TapeEngine.STATE_ROLLBACK
    end
end

---
-- Verifies the final integrity of a context before closing commit.
-- @param ctx Context handle.
-- @return STATE_PROCEED if clean; STATE_ROLLBACK if tripped.
function TapeEngine.verify_and_commit(ctx)
    if ctx.tripwire == TapeEngine.TRIPWIRE_TRIPPED or ctx.status == TapeEngine.STATE_ROLLBACK then
        return TapeEngine.STATE_ROLLBACK, "Commit rejected: tripwire tripped"
    end
    ctx.tripwire = TapeEngine.TRIPWIRE_DISARMED
    return TapeEngine.STATE_PROCEED, "Context sealed in PROCEED (+1)"
end

return TapeEngine
```

<TRACK 2> : {OmniCode Language & Standard Library}/lexicon/01_nouns/ideas/types/transpile variant/tests/lua/integrated/test_omni_lib.lua

LUA | 152 LINES | 634 WORDS | 7.0 KB

```lua
-- ================================================================================
-- BLOCK 1: METADATA & TEST ORCHESTRATOR TOPOLOGY
-- ( Test Orchestrator: < Lexicon { Types of Ideas } > & [ Multi-File Testbed (Lua) ] )
-- ================================================================================
-- ARCHITECTURAL TOPOLOGY:
--   Primary test orchestrator for the OmniCode Standard Library Lua Process Ribbons.
--   Executes multi-file unit test suites followed by integrated cross-type pipelines:
--   1. Unit Suite: Character Tape (Exodus 31:18 KJV)
--   2. Unit Suite: Number Tape (Psalm 147:4 KJV)
--   3. Unit Suite: Container Tape (2 Kings 4:4 KJV)
--   4. Unit Suite: Site Tape (Genesis 28:16 KJV)
--   5. Unit Suite: State Tape (Ecclesiastes 3:1 KJV)
--   6. Safety Interlock & Transactional Rollback (Proverbs 16:9 KJV)
--   7. Coroutine Fiber Stepping & AWAIT Yield/Resume
--   8. Integrated Composition: Container<Character> Traversed via State Loop
--   - Witness: Scripture Proverbs 24:27 KJV
--     "Prepare thy work without, and make it fit for thyself in the field;
--      and afterwards build thine house."
--   - Semantics: { MultiFileTestbed :: CrossTypeIntegration :: DetentVerification }
-- ================================================================================

package.path         = package.path ..
";./?.lua;./lua_tapes/?.lua;./tests/lua/unit/?.lua;./unit/?.lua;../unit/?.lua;../?.lua;../../?.lua"

local TestCommon     = require("test_common")
local TapeEngine     = require("lua_tapes.tape_engine")
local CharacterSuite = require("test_character")
local NumberSuite    = require("test_number")
local ContainerSuite = require("test_container")
local SiteSuite      = require("test_site")
local StateSuite     = require("test_state")

local CharacterTape  = require("lua_tapes.character")
local ContainerTape  = require("lua_tapes.container")
local SiteTape       = require("lua_tapes.site")
local StateTape      = require("lua_tapes.state")

print("================================================================================")
print("   OMNICODE STANDARD LIBRARY LUA ENGINE: MULTI-FILE TEST HARNESS")
print("================================================================================\n")

-- --------------------------------------------------------------------------------
-- 1. EXECUTE UNIT SUITES
-- --------------------------------------------------------------------------------
local suites = {
    CharacterSuite,
    NumberSuite,
    ContainerSuite,
    SiteSuite,
    StateSuite
}

for _, s in ipairs(suites) do
    local ok = s.run()
    if not ok then
        print(string.format("\n[ABORT] Unit suite failed! (%d failure(s))", TestCommon.failures))
        os.exit(1)
    end
    print("")
end

-- --------------------------------------------------------------------------------
-- 2. SAFETY INTERLOCK & TRANSACTIONAL ROLLBACK TEST
-- --------------------------------------------------------------------------------
print("[SUITE: Safety Interlock & Transactional Rollback]")
local rollback_caught = false
SiteTape:on_rollback(function(tape, roll_idx, roll_name, ctx, snapshot)
    rollback_caught = true
end)

local bad_site_ctx = SiteTape.open(0x00000000)
local bad_status, bad_msg = SiteTape:execute_all(bad_site_ctx)
TestCommon.assert_eq(bad_status, TapeEngine.STATE_ROLLBACK, "Expected STATE_ROLLBACK (-1)")
```

```
TestCommon.assert_eq(bad_site_ctx.tripwire, TapeEngine.TRIPWIRE_TRIPPED, "Expected TRIPWIRE_TRIPPED")
TestCommon.assert(rollback_caught == true, "Expected on_rollback hook to fire")
print(string.format("    -> Rollback Interlock: Status=%s, Tripwire=%d (%s) [PASS]\n",
    tostring(bad_status), bad_site_ctx.tripwire, bad_site_ctx.error_reason))

-- ------------------------------------------------------------------------------
-- 3. COROUTINE FIBER STEPPING & YIELD/RESUME TEST
-- ------------------------------------------------------------------------------
print("[SUITE: Coroutine Fiber Stepping & AWAIT Yield/Resume]")
local FiberTape = TapeEngine.create_tape({
    metadata = { name = "FiberSteppingTape" },
    rolls = {
        {
            name = "PhaseOne",
            action = function(ctx)
                ctx.counter = ctx.counter + 1
                return TapeEngine.STATE_AWAIT -- Request coroutine yield
            end
        },
        {
            name = "PhaseTwo",
            action = function(ctx)
                ctx.counter = ctx.counter * 10
                return TapeEngine.STATE_PROCEED
            end
        }
    }
})

local fiber_ctx = { counter = 5, status = TapeEngine.STATE_PROCEED, tripwire = TapeEngine.TRIPWIRE_ARMED }
local co = FiberTape:spawn_coroutine(fiber_ctx)

-- First resume: yields on PhaseOne AWAIT
local ok1, ev1, roll1, name1 = coroutine.resume(co)
TestCommon.assert(ok1 and ev1 == "AWAIT" and fiber_ctx.counter == 6, "Expected AWAIT yield at counter=6")
print(string.format("    -> Yielded on '%s' at roll %d [%s] (Counter=%d) [PASS]",
    ev1, roll1, name1, fiber_ctx.counter))

-- Reset status from AWAIT to PROCEED for phase two resume
fiber_ctx.status = TapeEngine.STATE_PROCEED
local ok2, ev2, msg2 = coroutine.resume(co)
TestCommon.assert(ok2 and ev2 == TapeEngine.STATE_PROCEED and fiber_ctx.counter == 60, "Expected PROCEED at counter=60")
print(string.format("    -> Resumed to Completion: Status=%s (Counter=%d, %s) [PASS]\n",
    tostring(ev2), fiber_ctx.counter, msg2))

-- ------------------------------------------------------------------------------
-- 4. CROSS-TYPE COMPOSITION PIPELINE
-- ------------------------------------------------------------------------------
print("[SUITE: Cross-Type Composition Pipeline]")
local word_bytes = {
    string.byte('O'), string.byte('M'), string.byte('N'), string.byte('I'),
    string.byte('C'), string.byte('O'), string.byte('D'), string.byte('E')
}
local container_ctx = ContainerTape.open(word_bytes, 32, 1)
local cont_status, cont_msg = ContainerTape:execute_all(container_ctx)
TestCommon.assert_eq(cont_status, TapeEngine.STATE_PROCEED, "Container execute must PROCEED")

io.write("    -> Driving Traversal via Canonical State Loop:\n       Transduced Output: \"")
local state_ctx = StateTape.open(TapeEngine.STATE_PROCEED, container_ctx.c_container.occupancy)

while state_ctx.c_state.registers.state == TapeEngine.STATE_PROCEED do
    local idx = state_ctx.c_state.registers.it + 1
    local ch = container_ctx.c_container.buffer[idx]
    if ch then
        io.write(string.char(ch))
    end
    StateTape.step_loop(state_ctx)
end
io.write("\"\n")

TestCommon.assert_eq(state_ctx.c_state.registers.state, TapeEngine.STATE_HALT, "State must reach HALT:NULL detent")
print(string.format("       Final Detent: %s (Detent at IT=%d, TURN=%d) [PASS]\n",
    tostring(state_ctx.c_state.registers.state),
    state_ctx.c_state.registers.it,
    state_ctx.c_state.registers.turn))

print("================================================================================")
```

```
print("   [ALL LUA ENGINE TESTS PASS]: FULL 4-BLOCK PARITY SEALED IN PROCEED (+1)")
print("================================================================================")
```

<TRACK 2> : {OmniCode Language & Standard Library}/lexicon/01_nouns/ideas/types/transpile variant/tests/lua/unit/test_character.lua

LUA | 89 LINES | 437 WORDS | 4.1 KB

```lua
-- ================================================================================
-- BLOCK 1: METADATA & TEST TOPOLOGY
-- ( Unit Test: < Lexicon { Types of Ideas } > & [ Test { Character Tape } ] )
-- ================================================================================
-- ARCHITECTURAL TOPOLOGY:
--   Validates the interpreted process ribbon < Character Tape > in isolation:
--   - Bijective round-trip transduction (8 bits <=> 5 balanced trits)
--   - Strict single-cell enclosure boundary laws
--   - Dynamic pre/post roll hook observation
--   - Full 4-block lifecycle execution (open, execute_all, close)
--   - Witness: Scripture Exodus 31:18 KJV
--     "And he gave unto Moses, when he had made an end of communing with him upon
--      mount Sinai, two tables of testimony, tables of stone, written with the
--      finger of God."
--   - Semantics: { GlyphFidelity :: BalancedTritLaw :: SingleCellEnclosure }
-- ================================================================================

package.path        = package.path ..
";./?.lua;./lua_tapes/?.lua;./tests/lua/unit/?.lua;./unit/?.lua;../?.lua;../../?.lua"

local TestCommon    = require("test_common")
local TapeEngine    = require("lua_tapes.tape_engine")
local CharacterTape = require("lua_tapes.character")

local Suite         = {}

function Suite.test_character_transduction()
    local glyphs = { string.byte('A'), string.byte('Z'), string.byte('0'), string.byte('9'), 32, 0, 70, 100, 121 }
    for _, g in ipairs(glyphs) do
        local ctx = CharacterTape.open(g)
        local status = CharacterTape:execute_all(ctx)
        TestCommon.assert_eq(status, TapeEngine.STATE_PROCEED, "Execution status must PROCEED (+1)")
        TestCommon.assert_eq(ctx.c_cell.glyph, g, "Context cell glyph must match input")
        TestCommon.assert(ctx.c_cell.is_enclosed == true, "Context cell must be enclosed")

        -- Reconstruct byte from trits: sum_{i=1..5} trits[i] * 3^(i-1)
        local reconstructed = 0
        local power = 1
        for i = 1, #ctx.c_cell.trits do
            local t = ctx.c_cell.trits[i]
            TestCommon.assert(t >= -1 and t <= 1, "Trit must be in {-1, 0, +1}")
            reconstructed = reconstructed + t * power
            power = power * 3
        end
        TestCommon.assert_eq(reconstructed, g, "Reconstructed byte from trits must match input")
    end
end

function Suite.test_character_dynamic_hooks()
    local pre_hooks = 0
    local post_hooks = 0

    CharacterTape:before(function() pre_hooks = pre_hooks + 1 end)
    CharacterTape:after(function() post_hooks = post_hooks + 1 end)

    local ctx = CharacterTape.open(string.byte('Q'))
    local status = CharacterTape:execute_all(ctx)
    TestCommon.assert_eq(status, TapeEngine.STATE_PROCEED, "Execution must PROCEED")
    TestCommon.assert(pre_hooks >= 5, "Must trigger pre-roll hooks for all 5 rolls")
    TestCommon.assert(post_hooks >= 5, "Must trigger post-roll hooks for all 5 rolls")
end

function Suite.test_character_lifecycle_rolls()
    local ctx = CharacterTape.open(string.byte('C'))
    TestCommon.assert_eq(ctx.tripwire, TapeEngine.TRIPWIRE_ARMED, "Tripwire must be ARMED on open")

    local status = CharacterTape:execute_all(ctx)
    TestCommon.assert_eq(status, TapeEngine.STATE_PROCEED, "Execution must PROCEED")

    local close_st, close_msg = CharacterTape.close(ctx)
    TestCommon.assert_eq(close_st, TapeEngine.STATE_PROCEED, "Close must seal in PROCEED (+1)")
    TestCommon.assert_eq(ctx.tripwire, TapeEngine.TRIPWIRE_DISARMED, "Tripwire must be DISARMED on close")
end
```

```
73
74  function Suite.run()
75      print("[SUITE: Character Process Ribbon (Lua)]")
76      local ok1 = TestCommon.run_test("Bijective Round-Trip Transduction", Suite.test_character_transduction)
77      local ok2 = TestCommon.run_test("Dynamic Pre/Post Roll Hooks", Suite.test_character_dynamic_hooks)
78      local ok3 = TestCommon.run_test("4-Block Lifecycle Rolls", Suite.test_character_lifecycle_rolls)
79      return ok1 and ok2 and ok3
80  end
81
82  -- Allow standalone execution
83  if pcall(debug.getlocal, 4, 1) == false then
84      local ok = Suite.run()
85      os.exit(ok and 0 or 1)
86  end
87
88  return Suite
```

<TRACK 2> : {OmniCode Language & Standard Library}/lexicon/01_nouns/ideas/types/transpile variant/tests/lua/unit/test_common.lua

LUA | 49 LINES | 188 WORDS | 1.7 KB

```lua
-- ================================================================================
-- BLOCK 1: METADATA & TEST HARNESS TOPOLOGY
-- ( Test Framework: < OmniCode Standard Library Lua Testbed > )
-- ================================================================================
-- ARCHITECTURAL TOPOLOGY:
--   Shared test harness for the OmniCode Standard Library Lua Process Ribbons.
--   Provides structured assertions, test suite runner helpers, and tripwire checks.
--   - Witness: Scripture Proverbs 24:27 KJV
--     "Prepare thy work without, and make it fit for thyself in the field;
--      and afterwards build thine house."
--   - Semantics: { AssertionInterlock :: GranularFaultIsolation :: MultiFileTestbed }
-- ================================================================================

local TestCommon = {
    failures = 0,
    tests_run = 0
}

function TestCommon.assert(condition, msg)
    TestCommon.tests_run = TestCommon.tests_run + 1
    if not condition then
        TestCommon.failures = TestCommon.failures + 1
        error("    [FAIL] Assertion failed: " .. tostring(msg), 2)
    end
end

function TestCommon.assert_eq(actual, expected, msg)
    TestCommon.tests_run = TestCommon.tests_run + 1
    if actual ~= expected then
        TestCommon.failures = TestCommon.failures + 1
        error(string.format("    [FAIL] %s: Expected [%s], got [%s]", tostring(msg), tostring(expected), tostring(actual)), 2)
    end
end

function TestCommon.run_test(name, fn)
    io.write(string.format("    -> %s ... ", name))
    local ok, err = pcall(fn)
    if ok then
        print("[PASS]")
        return true
    else
        print("[FAILED]")
        print(tostring(err))
        return false
    end
end

return TestCommon
```

<TRACK 2> : {OmniCode Language & Standard Library}/lexicon/01_nouns/ideas/types/transpile variant/tests/lua/unit/test_container.lua

LUA | 79 LINES | 397 WORDS | 3.7 KB

```lua
-- ================================================================================
-- BLOCK 1: METADATA & TEST TOPOLOGY
-- ( Unit Test: < Lexicon { Types of Ideas } > & [ Test { Container Tape } ] )
-- ================================================================================
-- ARCHITECTURAL TOPOLOGY:
--   Validates the interpreted process ribbon < Container Tape > in isolation:
--   - Bounded memory capsule build-up
--   - Payload encapsulation into bounded envelope
--   - Law of Containment verification (Occupancy <= Capacity)
--   - Overflow detection and tripwire interlock rollback
--   - Full 4-block lifecycle execution (open, execute_all, close)
--   - Witness: Scripture 2 Kings 4:4 KJV
--     "And when thou art come in, thou shalt shut the door upon thee and upon
--      thy sons, and shalt pour out into all those vessels, and thou shalt set
--      aside that which is full."
--   - Semantics: { MemoryCapsule :: BoundedEnvelope :: ContainmentLaw }
-- ================================================================================

package.path        = package.path ..
";./?.lua;./lua_tapes/?.lua;./tests/lua/unit/?.lua;./unit/?.lua;../?.lua;../../?.lua"

local TestCommon    = require("test_common")
local TapeEngine    = require("lua_tapes.tape_engine")
local ContainerTape = require("lua_tapes.container")

local Suite         = {}

function Suite.test_container_encapsulation()
    local payload = { 10, 20, 30, 40, 50 }
    local ctx = ContainerTape.open(payload, 16, 1)

    local status = ContainerTape:execute_all(ctx)
    TestCommon.assert_eq(status, TapeEngine.STATE_PROCEED, "Execution must PROCEED (+1)")
    TestCommon.assert_eq(ctx.c_container.occupancy, 5, "Container occupancy must be 5")
    TestCommon.assert_eq(ctx.c_container.capacity, 16, "Container capacity must be 16")
    TestCommon.assert(ctx.c_container.is_enclosed == true, "Container must be enclosed")
    TestCommon.assert(ctx.sealed_vessel ~= nil, "Sealed vessel must emerge from Roll 4")
    TestCommon.assert_eq(ctx.sealed_vessel.sealed, true, "Vessel must be sealed")
end

function Suite.test_container_overflow_tripwire()
    -- Create payload exceeding capacity limit
    local oversized = {}
    for i = 1, 10 do oversized[i] = i end

    local ctx = ContainerTape.open(oversized, 4, 1) -- Capacity is 4, payload is 10
    local status = ContainerTape:execute_all(ctx)

    TestCommon.assert_eq(status, TapeEngine.STATE_ROLLBACK, "Overflow must trigger STATE_ROLLBACK (-1)")
    TestCommon.assert_eq(ctx.tripwire, TapeEngine.TRIPWIRE_TRIPPED, "Tripwire must be TRIPPED on containment violation")
end

function Suite.test_container_lifecycle_rolls()
    local ctx = ContainerTape.open({ 1, 2, 3 }, 8, 1)
    TestCommon.assert_eq(ctx.tripwire, TapeEngine.TRIPWIRE_ARMED, "Tripwire must be ARMED on open")

    local status = ContainerTape:execute_all(ctx)
    TestCommon.assert_eq(status, TapeEngine.STATE_PROCEED, "Execution must PROCEED")

    local close_st, close_msg = ContainerTape.close(ctx)
    TestCommon.assert_eq(close_st, TapeEngine.STATE_PROCEED, "Close must seal in PROCEED (+1)")
    TestCommon.assert_eq(ctx.tripwire, TapeEngine.TRIPWIRE_DISARMED, "Tripwire must be DISARMED on close")
end

function Suite.run()
    print("[SUITE: Container Process Ribbon (Lua)]")
    local ok1 = TestCommon.run_test("Bounded Payload Encapsulation", Suite.test_container_encapsulation)
    local ok2 = TestCommon.run_test("Overflow Containment Tripwire & Rollback", Suite.test_container_overflow_tripwire)
    local ok3 = TestCommon.run_test("4-Block Lifecycle Rolls", Suite.test_container_lifecycle_rolls)
    return ok1 and ok2 and ok3
end

-- Allow standalone execution
```

```
if pcall(debug.getlocal, 4, 1) == false then
    local ok = Suite.run()
    os.exit(ok and 0 or 1)
end

return Suite
```

<TRACK 2> : {OmniCode Language & Standard Library}/lexicon/01_nouns/ideas/types/transpile variant/tests/lua/unit/test_number.lua

LUA | 69 LINES | 331 WORDS | 3.1 KB

```lua
-- ================================================================================
-- BLOCK 1: METADATA & TEST TOPOLOGY
-- ( Unit Test: < Lexicon { Types of Ideas } > & [ Test { Number Tape } ] )
-- ================================================================================
-- ARCHITECTURAL TOPOLOGY:
--   Validates the interpreted process ribbon < Number Tape > in isolation:
--   - Radix-3 balanced ternary transduction (64 bits <=> 32 balanced trits)
--   - Core ALU Zero-Sum Equilibrium Law: (+1) plus (-1) <=> 0
--   - Single-word boundary laws
--   - Full 4-block lifecycle execution (open, execute_all, close)
--   - Witness: Scripture Psalm 147:4 KJV
--     "He telleth the number of the stars; he calleth them all by their names."
--   - Semantics: { ScalarFidelity :: Radix3Equilibrium :: ZeroSumBalance }
-- ================================================================================

package.path = package.path .. ";./?.lua;./lua_tapes/?.lua;./tests/lua/unit/?.lua;./unit/?.lua;../?.lua;../../?.lua"

local TestCommon = require("test_common")
local TapeEngine = require("lua_tapes.tape_engine")
local NumberTape = require("lua_tapes.number")

local Suite = {}

function Suite.test_number_transduction()
    local values = { 0, 1, -1, 4096, -4096, 65536 }
    for _, val in ipairs(values) do
        local ctx = NumberTape.open(val)
        local status = NumberTape:execute_all(ctx)
        TestCommon.assert_eq(status, TapeEngine.STATE_PROCEED, "Execution must PROCEED (+1)")
        TestCommon.assert_eq(ctx.c_word.scalar, val, "Context word scalar must match input")
        TestCommon.assert(ctx.c_word.is_enclosed == true, "Context word must be enclosed")
        TestCommon.assert_eq(ctx.equilibrium_proved, true, "ALU Zero-Sum Equilibrium must be proved")
    end
end

function Suite.test_number_equilibrium_law()
    -- Direct law verification: (+1) + (-1) == 0
    local sum = (1) + (-1)
    TestCommon.assert_eq(sum, 0, "ALU Zero-Sum Equilibrium Law must evaluate to zero")
end

function Suite.test_number_lifecycle_rolls()
    local ctx = NumberTape.open(4096)
    TestCommon.assert_eq(ctx.tripwire, TapeEngine.TRIPWIRE_ARMED, "Tripwire must be ARMED on open")

    local status = NumberTape:execute_all(ctx)
    TestCommon.assert_eq(status, TapeEngine.STATE_PROCEED, "Execution must PROCEED")

    local close_st, close_msg = NumberTape.close(ctx)
    TestCommon.assert_eq(close_st, TapeEngine.STATE_PROCEED, "Close must seal in PROCEED (+1)")
    TestCommon.assert_eq(ctx.tripwire, TapeEngine.TRIPWIRE_DISARMED, "Tripwire must be DISARMED on close")
end

function Suite.run()
    print("[SUITE: Number Process Ribbon (Lua)]")
    local ok1 = TestCommon.run_test("Radix-3 Balanced Transduction", Suite.test_number_transduction)
    local ok2 = TestCommon.run_test("ALU Zero-Sum Equilibrium Law", Suite.test_number_equilibrium_law)
    local ok3 = TestCommon.run_test("4-Block Lifecycle Rolls", Suite.test_number_lifecycle_rolls)
    return ok1 and ok2 and ok3
end

-- Allow standalone execution
if pcall(debug.getlocal, 4, 1) == false then
    local ok = Suite.run()
    os.exit(ok and 0 or 1)
end

return Suite
```

<TRACK 2> : {OmniCode Language & Standard Library}/lexicon/01_nouns/ideas/types/transpile variant/tests/lua/unit/test_site.lua

LUA | 82 LINES | 379 WORDS | 3.8 KB

```lua
-- ================================================================================
-- BLOCK 1: METADATA & TEST TOPOLOGY
-- ( Unit Test: < Lexicon { Types of Ideas } > & [ Test { Site Tape } ] )
-- ================================================================================
-- ARCHITECTURAL TOPOLOGY:
--   Validates the interpreted process ribbon < Site Tape > in isolation:
--   - Silicon coordinate binding
--   - Hardware Routing Law (MMIO vs RAM coordinates)
--   - Null pointer address rejection and safety interlock rollback
--   - Memory anchor emergence
--   - Full 4-block lifecycle execution (open, execute_all, close)
--   - Witness: Scripture Genesis 28:16 KJV
--     "And Jacob awaked out of his sleep, and he said, Surely the Lord is in
--      this place; and I knew it not."
--   - Semantics: { SiliconCoordinates :: HardwareRoutingLaw :: NonNullInterlock }
-- ================================================================================

package.path     = package.path .. ";./?.lua;./lua_tapes/?.lua;./tests/lua/unit/?.lua;./unit/?.lua;../?.lua;../../?.lua"

local TestCommon = require("test_common")
local TapeEngine = require("lua_tapes.tape_engine")
local SiteTape   = require("lua_tapes.site")

local Suite      = {}

function Suite.test_site_routing()
    -- VGA buffer -> MMIO
    local vga_ctx = SiteTape.open(SiteTape.constants.SITE_VGA)
    local vga_st = SiteTape:execute_all(vga_ctx)
    TestCommon.assert_eq(vga_st, TapeEngine.STATE_PROCEED, "VGA site must PROCEED")
    TestCommon.assert_eq(vga_ctx.c_site.routing, "HARDWARE_MMIO (::@)", "VGA must route to MMIO")
    TestCommon.assert(vga_ctx.memory_anchor.anchored == true, "Memory anchor must be set")

    -- COM1 serial port -> MMIO
    local com1_ctx = SiteTape.open(SiteTape.constants.SITE_COM1)
    local com1_st = SiteTape:execute_all(com1_ctx)
    TestCommon.assert_eq(com1_st, TapeEngine.STATE_PROCEED, "COM1 site must PROCEED")
    TestCommon.assert_eq(com1_ctx.c_site.routing, "HARDWARE_MMIO (::@)", "COM1 must route to MMIO")

    -- MBR origin -> RAM
    local mbr_ctx = SiteTape.open(SiteTape.constants.SITE_MBR)
    local mbr_st = SiteTape:execute_all(mbr_ctx)
    TestCommon.assert_eq(mbr_st, TapeEngine.STATE_PROCEED, "MBR site must PROCEED")
    TestCommon.assert_eq(mbr_ctx.c_site.routing, "PHYSICAL_RAM (@)", "MBR must route to RAM")
end

function Suite.test_site_null_tripwire()
    local null_ctx = SiteTape.open(0x00000000)
    local status = SiteTape:execute_all(null_ctx)

    TestCommon.assert_eq(status, TapeEngine.STATE_ROLLBACK, "Null coordinate must trigger STATE_ROLLBACK (-1)")
    TestCommon.assert_eq(null_ctx.tripwire, TapeEngine.TRIPWIRE_TRIPPED, "Tripwire must be TRIPPED on null address")
end

function Suite.test_site_lifecycle_rolls()
    local ctx = SiteTape.open(SiteTape.constants.SITE_VGA)
    TestCommon.assert_eq(ctx.tripwire, TapeEngine.TRIPWIRE_ARMED, "Tripwire must be ARMED on open")

    local status = SiteTape:execute_all(ctx)
    TestCommon.assert_eq(status, TapeEngine.STATE_PROCEED, "Execution must PROCEED")

    local close_st, close_msg = SiteTape.close(ctx)
    TestCommon.assert_eq(close_st, TapeEngine.STATE_PROCEED, "Close must seal in PROCEED (+1)")
    TestCommon.assert_eq(ctx.tripwire, TapeEngine.TRIPWIRE_DISARMED, "Tripwire must be DISARMED on close")
end

function Suite.run()
    print("[SUITE: Site Process Ribbon (Lua)]")
    local ok1 = TestCommon.run_test("Hardware Routing Law (MMIO vs RAM)", Suite.test_site_routing)
    local ok2 = TestCommon.run_test("Null Address Rejection & Interlock Rollback", Suite.test_site_null_tripwire)
    local ok3 = TestCommon.run_test("4-Block Lifecycle Rolls", Suite.test_site_lifecycle_rolls)
    return ok1 and ok2 and ok3
end
```

```
74
75  -- Allow standalone execution
76  if pcall(debug.getlocal, 4, 1) == false then
77      local ok = Suite.run()
78      os.exit(ok and 0 or 1)
79  end
80
81  return Suite
```

<TRACK 2> : {OmniCode Language & Standard Library}/lexicon/01_nouns/ideas/types/transpile variant/tests/lua/unit/test_state.lua

LUA | 93 LINES | 439 WORDS | 4.2 KB

```lua
-- ================================================================================
-- BLOCK 1: METADATA & TEST TOPOLOGY
-- ( Unit Test: < Lexicon { Types of Ideas } > & [ Test { State Tape } ] )
-- ================================================================================
-- ARCHITECTURAL TOPOLOGY:
--   Validates the interpreted process ribbon < State Tape > in isolation:
--   - Quad-State named registers: [ STATE TURN BOUND IT ]
--   - Canonical Loop Law cadence and single-step detents:
--       WHEN IT < BOUND  --> PROCEED (+1)
--       !    IT > BOUND  --> AWAIT (0)
--       ::   IT == BOUND --> HALT (NULL)
--   - Clock governance emergence
--   - Full 4-block lifecycle execution (open, execute_all, close)
--   - Witness: Scripture Ecclesiastes 3:1 KJV
--     "To every thing there is a season, and a time to every purpose under
--      the heaven:"
--   - Semantics: { QuadStateTruth :: CanonicalLoopCadence :: CausalDetents }
-- ================================================================================

package.path = package.path .. ";./?.lua;./lua_tapes/?.lua;./tests/lua/unit/?.lua;./unit/?.lua;../?.lua;../../?.lua"

local TestCommon = require("test_common")
local TapeEngine = require("lua_tapes.tape_engine")
local StateTape  = require("lua_tapes.state")

local Suite = {}

function Suite.test_state_canonical_loop()
    local ctx = StateTape.open(TapeEngine.STATE_PROCEED, 3)

    TestCommon.assert_eq(ctx.c_state.registers.bound, 3, "Bound must be 3")
    TestCommon.assert_eq(ctx.c_state.registers.it, 0, "Initial IT must be 0")
    TestCommon.assert_eq(ctx.c_state.registers.turn, 0, "Initial TURN must be 0")

    -- Step 1
    local st1 = StateTape.step_loop(ctx)
    TestCommon.assert_eq(st1, TapeEngine.STATE_PROCEED, "Step 1 must PROCEED")
    TestCommon.assert_eq(ctx.c_state.registers.it, 1, "IT must be 1")

    -- Step 2
    local st2 = StateTape.step_loop(ctx)
    TestCommon.assert_eq(st2, TapeEngine.STATE_PROCEED, "Step 2 must PROCEED")
    TestCommon.assert_eq(ctx.c_state.registers.it, 2, "IT must be 2")

    -- Step 3
    local st3 = StateTape.step_loop(ctx)
    TestCommon.assert_eq(st3, TapeEngine.STATE_PROCEED, "Step 3 must PROCEED")
    TestCommon.assert_eq(ctx.c_state.registers.it, 3, "IT must be 3")

    -- Step 4: IT == BOUND -> HALT:NULL
    local st4 = StateTape.step_loop(ctx)
    TestCommon.assert_eq(st4, TapeEngine.STATE_HALT, "Step 4 must reach terminal detent HALT:NULL")
    TestCommon.assert_eq(ctx.c_state.registers.state, TapeEngine.STATE_HALT, "Register state must be HALT:NULL")
end

function Suite.test_state_await_transition()
    local ctx = StateTape.open(TapeEngine.STATE_PROCEED, 4)
    ctx.c_state.registers.it = 5 -- Manually set IT > BOUND

    local st = StateTape.step_loop(ctx)
    TestCommon.assert_eq(st, TapeEngine.STATE_AWAIT, "IT > BOUND must transition to AWAIT (0)")
end

function Suite.test_state_lifecycle_rolls()
    local ctx = StateTape.open(TapeEngine.STATE_PROCEED, 4)
    TestCommon.assert_eq(ctx.tripwire, TapeEngine.TRIPWIRE_ARMED, "Tripwire must be ARMED on open")

    local status = StateTape:execute_all(ctx)
    TestCommon.assert_eq(status, TapeEngine.STATE_HALT, "Execution must reach terminal detent HALT:NULL")
    TestCommon.assert_eq(ctx.c_state.registers.it, 4, "Final IT must equal bound 4")
    TestCommon.assert_eq(ctx.c_state.registers.state, TapeEngine.STATE_HALT, "Register state must reach HALT:NULL")

    local close_st, close_msg = StateTape.close(ctx)
```

```
    TestCommon.assert_eq(close_st, TapeEngine.STATE_PROCEED, "Close must seal in PROCEED (+1)")
    TestCommon.assert_eq(ctx.tripwire, TapeEngine.TRIPWIRE_DISARMED, "Tripwire must be DISARMED on close")
end

function Suite.run()
    print("[SUITE: State Process Ribbon (Lua)]")
    local ok1 = TestCommon.run_test("Canonical Loop Law Cadence (IT < BOUND -> HALT)", Suite.test_state_canonical_loop)
    local ok2 = TestCommon.run_test("Standby Transition (IT > BOUND -> AWAIT)", Suite.test_state_await_transition)
    local ok3 = TestCommon.run_test("4-Block Lifecycle Rolls", Suite.test_state_lifecycle_rolls)
    return ok1 and ok2 and ok3
end

-- Allow standalone execution
if pcall(debug.getlocal, 4, 1) == false then
    local ok = Suite.run()
    os.exit(ok and 0 or 1)
end

return Suite
```

<TRACK 2> : {OmniCode Language & Standard Library}/lexicon/01_nouns/ideas/types/transpile variant/tests/c/integrated/test_omni_lib.c

C | 121 LINES | 492 WORDS | 5.5 KB

```c
/* ========================================================================
 * BLOCK 1: METADATA & INTEGRATION TEST TOPOLOGY
 * ( Test Orchestrator: < Lexicon { Types of Ideas } > & [ Multi-File Testbed ] )
 * ========================================================================
 * ARCHITECTURAL TOPOLOGY:
 *   Primary test orchestrator for the OmniCode Standard Library C Backbone.
 *   Executes multi-file unit test suites followed by integrated cross-type pipelines:
 *   1. Unit Suite: Character (Exodus 31:18 KJV)
 *   2. Unit Suite: Number (Psalm 147:4 KJV)
 *   3. Unit Suite: Container (2 Kings 4:4 KJV)
 *   4. Unit Suite: Site (Genesis 28:16 KJV)
 *   5. Unit Suite: State (Ecclesiastes 3:1 KJV)
 *   6. Integrated Composition: Container<Character> Traversed via State Loop
 *   - Witness: Scripture Proverbs 24:27 KJV
 *     "Prepare thy work without, and make it fit for thyself in the field;
 *      and afterwards build thine house."
 *   - Semantics: { MultiFileTestbed :: CrossTypeIntegration :: DetentVerification }
 * ======================================================================== */

#include "test_common.h"
#include <stdio.h>

/* Global test failure accumulator */
int g_omni_test_failures = 0;

/* ========================================================================
 * BLOCK 2: OPENING CONTEXT & INTEGRATED PIPELINE DEFINITIONS
 * ======================================================================== */

static int run_cross_type_integration_pipeline(void) {
    printf("[SUITE: Cross-Type Composition Pipeline]\n");
    printf("  -> Composing Container<Character> (\"The Text Substrate\") ...\n");

    #define TEST_TEXT_CAPACITY 32
    omni_character_t text_storage[TEST_TEXT_CAPACITY];
    omni_container_t text_ribbon;
    omni_container_init(&text_ribbon, text_storage, sizeof(omni_character_t), TEST_TEXT_CAPACITY);

    const char *source_str = "OMNICODE";
    for (size_t i = 0; source_str[i] != '\0'; ++i) {
        omni_character_t ch = omni_character_from_glyph((uint8_t)source_str[i]);
        omni_quad_state_t push_res = omni_container_push(&text_ribbon, &ch);
        OMNI_TEST_ASSERT(push_res == OMNI_STATE_PROCEED, "Container push must succeed without overflow");
    }

    OMNI_TEST_ASSERT(text_ribbon.occupancy == 8, "Text ribbon occupancy must be 8");
    printf("  -> Capacity: %zu, Occupancy: %zu [PASS]\n", text_ribbon.capacity, text_ribbon.occupancy);

    /* Traversal via Canonical State Loop */
    printf("  -> Driving Traversal via Canonical State Loop:\n     Transduced Output: \"");
    omni_state_t traversal_state = omni_state_create(OMNI_STATE_PROCEED, (int64_t)text_ribbon.occupancy);

    while (omni_state_is_proceeding(&traversal_state)) {
        size_t idx = (size_t)traversal_state.registers.it;
        omni_character_t *curr = (omni_character_t *)omni_container_get(&text_ribbon, idx);
        if (curr) {
            putchar((char)curr->glyph);
        }
        omni_state_step_loop(&traversal_state.registers);
    }
    printf("\"\n");

    OMNI_TEST_ASSERT(omni_state_is_halted(&traversal_state) == true, "Traversal must reach HALT:NULL detent");
    OMNI_TEST_ASSERT(traversal_state.registers.it == 8, "Final IT must equal 8");
    OMNI_TEST_ASSERT(traversal_state.registers.turn == 8, "Final TURN must equal 8");
    printf("     Final Detent: HALT:NULL (IT=8, TURN=8) [PASS]\n");

    /* Route Site to VGA Buffer MMIO */
    omni_site_t vga_site = omni_site_from_address(OMNI_SITE_VGA_BUFFER);
    OMNI_TEST_ASSERT(vga_site.routing == OMNI_ROUTING_HARDWARE_MMIO, "VGA site must resolve to HARDWARE_MMIO");
    printf("  -> Virtual Hardware MMIO Routing: 0x%lX -> HARDWARE_MMIO (::@) [PASS]\n",
           (unsigned long)vga_site.coordinate);

```

```c
    return 0;
}

/* ============================================================================
 * BLOCK 3: BODY PRIMITIVES & MAIN TEST RUNNER
 * ============================================================================ */

int main(void) {
    printf("================================================================================\n");
    printf("   OMNICODE STANDARD LIBRARY C BACKBONE: MULTI-FILE TEST HARNESS\n");
    printf("================================================================================\n\n");

    /* Run Unit Test Suites */
    if (test_character_suite() != 0) goto failure;
    printf("\n");

    if (test_number_suite() != 0) goto failure;
    printf("\n");

    if (test_container_suite() != 0) goto failure;
    printf("\n");

    if (test_site_suite() != 0) goto failure;
    printf("\n");

    if (test_state_suite() != 0) goto failure;
    printf("\n");

    /* Run Cross-Type Integration Pipeline */
    if (run_cross_type_integration_pipeline() != 0) goto failure;
    printf("\n");

/* ============================================================================
 * BLOCK 4: CLOSING BLOCK & FINAL DETENT REPORT
 * ============================================================================ */

    printf("================================================================================\n");
    printf("   [ALL C BACKBONE TESTS PASS]: FULL 4-BLOCK PARITY SEALED IN PROCEED (+1)\n");
    printf("================================================================================\n");
    return 0;

failure:
    printf("\n================================================================================\n");
    printf("   [TEST SUITE FAILURE]: %d assertion failure(s) recorded!\n", g_omni_test_failures);
    printf("================================================================================\n");
    return 1;
}
```

<TRACK 2> : {OmniCode Language & Standard Library}/lexicon/01_nouns/ideas/types/transpile variant/tests/c/includes/test_common.h

H | 65 LINES | 248 WORDS | 2.3 KB

```c
/* ============================================================================
 * BLOCK 1: METADATA & TEST HARNESS TOPOLOGY
 * ( Test Framework: < OmniCode Standard Library Testbed > )
 * ============================================================================
 * ARCHITECTURAL TOPOLOGY:
 *   Shared test harness for the OmniCode Standard Library C Transpile Variant.
 *   Provides structured assertions, test suite runner macros, and tripwire checks.
 *   - Witness: Scripture Proverbs 24:27 KJV
 *     "Prepare thy work without, and make it fit for thyself in the field;
 *      and afterwards build thine house."
 *   - Semantics: { AssertionInterlock :: GranularFaultIsolation :: MultiFileTestbed }
 * ============================================================================ */

#ifndef OMNI_TEST_COMMON_H
#define OMNI_TEST_COMMON_H

#include "omni.h"
#include <stdio.h>
#include <stdbool.h>

/* ============================================================================
 * BLOCK 2: OPENING CONTEXT & MACRO DEFINITIONS
 * ============================================================================ */

/** Global test assertion failure counter */
extern int g_omni_test_failures;

/**
 * @brief Evaluates an assertion expression; logs failure and returns 1 on error.
 */
#define OMNI_TEST_ASSERT(expr, msg) \
    do { \
        if (!(expr)) { \
            printf("    [FAIL] %s:%d: %s (Expression: %s)\n", __FILE__, __LINE__, (msg), #expr); \
            g_omni_test_failures++; \
            return 1; \
        } \
    } while (0)

/**
 * @brief Runs an individual test function and reports PASS / FAIL.
 */
#define OMNI_TEST_RUN(fn, test_name) \
    do { \
        printf("    -> %s ... ", (test_name)); \
        if ((fn)() != 0) { \
            printf("[FAILED]\n"); \
            return 1; \
        } \
        printf("[PASS]\n"); \
    } while (0)

/* ============================================================================
 * BLOCK 3: BODY PRIMITIVES & SUITE EXPORTS
 * ============================================================================ */

/** Unit test suite prototypes */
int test_character_suite(void);
int test_number_suite(void);
int test_container_suite(void);
int test_site_suite(void);
int test_state_suite(void);

#endif /* OMNI_TEST_COMMON_H */
```

<TRACK 2> : {OmniCode Language & Standard Library}/lexicon/01_nouns/ideas/types/transpile variant/tests/c/unit/test_character.c

C | 105 LINES | 501 WORDS | 4.9 KB

```c
/* ============================================================================
 * BLOCK 1: METADATA & TEST TOPOLOGY
 * ( Unit Test: < Lexicon { Types of Ideas } > & [ Test { Character } ] )
 * ============================================================================
 * ARCHITECTURAL TOPOLOGY:
 *   Validates the atomic invariant type < Character > in isolation:
 *   - Bijective round-trip transduction (8 bits <=> 5 balanced trits)
 *   - Strict single-cell enclosure boundary laws
 *   - Full 4-block lifecycle execution (open, execute, close)
 *   - Relational equality and comparison
 *   - Witness: Scripture Exodus 31:18 KJV
 *     "And he gave unto Moses, when he had made an end of communing with him upon
 *      mount Sinai, two tables of testimony, tables of stone, written with the
 *      finger of God."
 *   - Semantics: { GlyphFidelity :: BalancedTritLaw :: SingleCellEnclosure }
 * ============================================================================ */

#include "test_common.h"

/* ============================================================================
 * BLOCK 2: OPENING CONTEXT & FIXTURE SETUP
 * ============================================================================ */

static int test_character_bijective_roundtrip(void) {
    /* Test exemplar characters across 5-trit cell capacity (0..121) */
    const uint8_t test_glyphs[] = {'A', 'Z', '0', '9', ' ', 0, 70, 100, 121};
    const size_t num_glyphs = sizeof(test_glyphs) / sizeof(test_glyphs[0]);

    for (size_t i = 0; i < num_glyphs; ++i) {
        uint8_t original = test_glyphs[i];
        omni_character_t c = omni_character_from_glyph(original);

        /* Verify enclosure */
        OMNI_TEST_ASSERT(c.is_enclosed == true, "Character must be enclosed");
        OMNI_TEST_ASSERT(c.glyph == original, "Character glyph must match original");

        /* Verify every trit is in valid balanced range {-1, 0, +1} */
        for (size_t t = 0; t < OMNI_CHAR_TRITS_PER_CELL; ++t) {
            OMNI_TEST_ASSERT(c.trits[t] >= -1 && c.trits[t] <= 1, "Trit must be in {-1, 0, +1}");
        }

        /* Verify inverse transduction reconstructs the exact byte */
        uint8_t reconstructed = omni_character_to_byte(c.trits);
        OMNI_TEST_ASSERT(reconstructed == original, "Inverse transduction must exactly match original byte");
    }
    return 0;
}

static int test_character_bounds_and_enclosure(void) {
    omni_character_t valid_char = omni_character_from_glyph('X');
    OMNI_TEST_ASSERT(omni_character_verify_bounds(&valid_char) == true, "Valid character must pass boundary check");

    /* Corrupt enclosure flag to test boundary tripwire */
    omni_character_t corrupted = valid_char;
    corrupted.is_enclosed = false;
    OMNI_TEST_ASSERT(omni_character_verify_bounds(&corrupted) == false, "Unenclosed character must fail boundary check");

    return 0;
}

static int test_character_relational_equality(void) {
    omni_character_t a1 = omni_character_from_glyph('M');
    omni_character_t a2 = omni_character_from_glyph('M');
    omni_character_t b  = omni_character_from_glyph('N');

    OMNI_TEST_ASSERT(omni_character_eq(&a1, &a2) == true, "Identical glyphs must evaluate equal");
    OMNI_TEST_ASSERT(omni_character_eq(&a1, &b) == false, "Distinct glyphs must not evaluate equal");

    return 0;
}

static int test_character_lifecycle_rolls(void) {
    omni_character_context_t ctx;
```

```
    omni_character_open(&ctx, 'K', OMNI_TARGET_STDOUT);

    OMNI_TEST_ASSERT(ctx.tripwire == OMNI_TRIPWIRE_ARMED, "Context tripwire must be ARMED upon open");
    OMNI_TEST_ASSERT(ctx.input_glyph == 'K', "Input glyph must be recorded in context");

    /* Execute the 3 declared rolls */
    omni_quad_state_t exec_status = omni_character_execute(&ctx);
    OMNI_TEST_ASSERT(exec_status == OMNI_STATE_PROCEED, "Character execute must PROCEED (+1)");
    OMNI_TEST_ASSERT(ctx.character.is_enclosed == true, "Character must be enclosed after execution");
    OMNI_TEST_ASSERT(ctx.character.glyph == 'K', "Character glyph must match input");

    /* Close and commit */
    omni_quad_state_t close_status = omni_character_close(&ctx);
    OMNI_TEST_ASSERT(close_status == OMNI_STATE_PROCEED, "Character close must PROCEED (+1)");
    OMNI_TEST_ASSERT(ctx.tripwire == OMNI_TRIPWIRE_DISARMED, "Tripwire must be DISARMED upon clean close");

    return 0;
}

/* ============================================================================
 * BLOCK 3: BODY PRIMITIVES & TEST SUITE RUNNER
 * ============================================================================ */

int test_character_suite(void) {
    printf("[SUITE: Character Invariant Type]\n");
    OMNI_TEST_RUN(test_character_bijective_roundtrip, "Bijective Round-Trip Transduction");
    OMNI_TEST_RUN(test_character_bounds_and_enclosure, "Single-Cell Enclosure Bounds");
    OMNI_TEST_RUN(test_character_relational_equality, "Relational Equality");
    OMNI_TEST_RUN(test_character_lifecycle_rolls, "4-Block Lifecycle Rolls");
    return 0;
}
```

<TRACK 2> : {OmniCode Language & Standard Library}/lexicon/01_nouns/ideas/types/transpile variant/tests/c/unit/test_container.c

C | 145 LINES | 670 WORDS | 6.5 KB

```c
/* ============================================================================
 * BLOCK 1: METADATA & TEST TOPOLOGY
 * ( Unit Test: < Lexicon { Types of Ideas } > & [ Test { Container } ] )
 * ============================================================================
 * ARCHITECTURAL TOPOLOGY:
 *   Validates the generic memory capsule type < Container > in isolation:
 *   - Bounded slice allocation and element encapsulation
 *   - Law of Containment verification (Occupancy <= Capacity)
 *   - Overflow detection and tripwire interlock triggering
 *   - Standard 64-byte vessel envelope
 *   - Full 4-block lifecycle execution (open, execute, close)
 *   - Witness: Scripture 2 Kings 4:4 KJV
 *     "And when thou art come in, thou shalt shut the door upon thee and upon
 *      thy sons, and shalt pour out into all those vessels, and thou shalt set
 *      aside that which is full."
 *   - Semantics: { MemoryCapsule :: BoundedSlice :: ContainmentLaw }
 * ============================================================================ */

#include "test_common.h"

/* ============================================================================
 * BLOCK 2: OPENING CONTEXT & FIXTURE SETUP
 * ============================================================================ */

static int test_container_basic_lifecycle(void) {
    uint32_t storage[8];
    omni_container_t c;
    omni_container_init(&c, storage, sizeof(uint32_t), 8);

    OMNI_TEST_ASSERT(c.capacity == 8, "Container capacity must be 8");
    OMNI_TEST_ASSERT(c.occupancy == 0, "Container occupancy must initially be 0");
    OMNI_TEST_ASSERT(c.is_enclosed == true, "Container must be enclosed");
    OMNI_TEST_ASSERT(omni_container_is_empty(&c) == true, "Container must report empty");

    /* Push 4 elements */
    for (uint32_t i = 1; i <= 4; ++i) {
        omni_quad_state_t push_st = omni_container_push(&c, &i);
        OMNI_TEST_ASSERT(push_st == OMNI_STATE_PROCEED, "Push must PROCEED");
    }
    OMNI_TEST_ASSERT(c.occupancy == 4, "Occupancy must be 4");

    /* Verify random access get */
    for (size_t i = 0; i < 4; ++i) {
        uint32_t *val = (uint32_t *)omni_container_get(&c, i);
        OMNI_TEST_ASSERT(val != NULL, "Get must return non-null pointer");
        OMNI_TEST_ASSERT(*val == (uint32_t)(i + 1), "Stored element must match pushed value");
    }

    /* Out of bounds get must return NULL */
    OMNI_TEST_ASSERT(omni_container_get(&c, 10) == NULL, "Out of bounds get must return NULL");

    /* Test random access set */
    uint32_t new_val = 999;
    omni_quad_state_t set_st = omni_container_set(&c, 1, &new_val);
    OMNI_TEST_ASSERT(set_st == OMNI_STATE_PROCEED, "Set must PROCEED");
    uint32_t *read_back = (uint32_t *)omni_container_get(&c, 1);
    OMNI_TEST_ASSERT(read_back != NULL && *read_back == 999, "Read back value at index 1 must be 999");

    /* Clear */
    omni_container_clear(&c);
    OMNI_TEST_ASSERT(c.occupancy == 0, "Occupancy must be 0 after clear");
    OMNI_TEST_ASSERT(omni_container_is_empty(&c) == true, "Container must report empty after clear");

    return 0;
}

static int test_container_overflow_tripwire(void) {
    uint8_t tiny_storage[2];
    omni_container_t c;
    omni_container_init(&c, tiny_storage, sizeof(uint8_t), 2);

    uint8_t a = 10, b = 20, c_overflow = 30;
    OMNI_TEST_ASSERT(omni_container_push(&c, &a) == OMNI_STATE_PROCEED, "First push must PROCEED");
```

```
    OMNI_TEST_ASSERT(omni_container_push(&c, &b) == OMNI_STATE_PROCEED, "Second push must PROCEED");
    OMNI_TEST_ASSERT(omni_container_is_full(&c) == true, "Container must report full at capacity 2");

    /* Third push MUST fail and return ROLLBACK (-1) */
    omni_quad_state_t overflow_res = omni_container_push(&c, &c_overflow);
    OMNI_TEST_ASSERT(overflow_res == OMNI_STATE_ROLLBACK, "Overflow push must return ROLLBACK (-1)");
    OMNI_TEST_ASSERT(c.occupancy == 2, "Occupancy must remain capped at capacity");

    return 0;
}

static int test_container_containment_law(void) {
    uint8_t storage[4];
    omni_container_t c;
    omni_container_init(&c, storage, sizeof(uint8_t), 4);

    OMNI_TEST_ASSERT(omni_container_verify_containment(&c) == true, "Valid container must pass Containment Law");

    /* Violate containment law manually */
    omni_container_t corrupted = c;
    corrupted.occupancy = 10; /* Exceeds capacity 4 */
    OMNI_TEST_ASSERT(omni_container_verify_containment(&corrupted) == false, "Overflowed occupancy must violate
Containment Law");

    return 0;
}

static int test_byte_vessel_envelope(void) {
    omni_byte_vessel_t vessel;
    omni_byte_vessel_init(&vessel);

    OMNI_TEST_ASSERT(vessel.container.capacity == OMNI_CONTAINER_EXEMPLAR_VESSEL, "Byte vessel capacity must be 64");
    OMNI_TEST_ASSERT(vessel.container.element_size == 1, "Element size must be 1 byte");
    OMNI_TEST_ASSERT(vessel.container.occupancy == 0, "Byte vessel must initially be empty");

    return 0;
}

static int test_container_lifecycle_rolls(void) {
    uint16_t storage[16];
    uint16_t payload[4] = {100, 200, 300, 400};

    omni_container_context_t ctx;
    omni_container_open(&ctx, storage, sizeof(uint16_t), 16, payload, 4);

    OMNI_TEST_ASSERT(ctx.tripwire == OMNI_TRIPWIRE_ARMED, "Tripwire must be ARMED upon open");
    OMNI_TEST_ASSERT(ctx.payload_count == 4, "Payload count must be 4");

    omni_quad_state_t exec_status = omni_container_execute(&ctx);
    OMNI_TEST_ASSERT(exec_status == OMNI_STATE_PROCEED, "Container execute must PROCEED (+1)");
    OMNI_TEST_ASSERT(ctx.vessel.occupancy == 4, "Vessel occupancy must match payload count");

    omni_quad_state_t close_status = omni_container_close(&ctx);
    OMNI_TEST_ASSERT(close_status == OMNI_STATE_PROCEED, "Container close must PROCEED (+1)");
    OMNI_TEST_ASSERT(ctx.tripwire == OMNI_TRIPWIRE_DISARMED, "Tripwire must be DISARMED upon clean close");

    return 0;
}

/* ============================================================================
 * BLOCK 3: BODY PRIMITIVES & TEST SUITE RUNNER
 * ============================================================================ */

int test_container_suite(void) {
    printf("[SUITE: Container Invariant Type]\n");
    OMNI_TEST_RUN(test_container_basic_lifecycle, "Basic Allocation, Push & Pop");
    OMNI_TEST_RUN(test_container_overflow_tripwire, "Overflow Tripwire & Rollback");
    OMNI_TEST_RUN(test_container_containment_law, "Core Containment Law");
    OMNI_TEST_RUN(test_byte_vessel_envelope, "64-Byte Vessel Envelope");
    OMNI_TEST_RUN(test_container_lifecycle_rolls, "4-Block Lifecycle Rolls");
    return 0;
}
```

<TRACK 2> : {OmniCode Language & Standard Library}/lexicon/01_nouns/ideas/types/transpile variant/tests/c/unit/test_number.c

C | 111 LINES | 499 WORDS | 4.9 KB

```c
/* ============================================================================
 * BLOCK 1: METADATA & TEST TOPOLOGY
 * ( Unit Test: < Lexicon { Types of Ideas } > & [ Test { Number } ] )
 * ============================================================================
 * ARCHITECTURAL TOPOLOGY:
 *   Validates the scalar invariant type < Number > in isolation:
 *   - Bijective round-trip radix-3 transduction (64 bits <=> 32 balanced trits)
 *   - ALU Zero-Sum Equilibrium Law: (+1) plus (-1) <=> 0
 *   - Balanced arithmetic addition
 *   - Strict single-word boundary laws
 *   - Full 4-block lifecycle execution (open, execute, close)
 *   - Witness: Scripture Psalm 147:4 KJV
 *     "He telleth the number of the stars; he calleth them all by their names."
 *   - Semantics: { ScalarFidelity :: Radix3Equilibrium :: ZeroSumBalance }
 * ============================================================================ */

#include "test_common.h"

/* ============================================================================
 * BLOCK 2: OPENING CONTEXT & FIXTURE SETUP
 * ============================================================================ */

static int test_number_bijective_roundtrip(void) {
    const int64_t test_values[] = {0, 1, -1, 2, -2, 4096, -4096, 65536, -65536, 1000000, -1000000};
    const size_t num_values = sizeof(test_values) / sizeof(test_values[0]);

    for (size_t i = 0; i < num_values; ++i) {
        int64_t original = test_values[i];
        omni_number_t n = omni_number_from_scalar(original);

        OMNI_TEST_ASSERT(n.is_enclosed == true, "Number must be enclosed");
        OMNI_TEST_ASSERT(n.scalar == original, "Number scalar must match original");

        /* Verify all 32 trits are in {-1, 0, +1} */
        for (size_t t = 0; t < OMNI_NUM_TRITS_PER_WORD; ++t) {
            OMNI_TEST_ASSERT(n.trits[t] >= -1 && n.trits[t] <= 1, "Trit must be in {-1, 0, +1}");
        }

        /* Verify reconstruction from trits */
        int64_t reconstructed = omni_number_to_scalar(n.trits);
        OMNI_TEST_ASSERT(reconstructed == original, "Reconstructed scalar must equal original value");
    }
    return 0;
}

static int test_number_equilibrium_law(void) {
    /* ALU Equilibrium Law: (+1) plus (-1) <=> 0 */
    bool eq_holds = omni_number_verify_equilibrium();
    OMNI_TEST_ASSERT(eq_holds == true, "Core ALU Zero-Sum Equilibrium Law must hold");
    return 0;
}

static int test_number_addition(void) {
    omni_number_t a = omni_number_from_scalar(4000);
    omni_number_t b = omni_number_from_scalar(96);
    omni_number_t sum = omni_number_add(a, b);

    OMNI_TEST_ASSERT(sum.scalar == 4096, "4000 + 96 must equal 4096");
    OMNI_TEST_ASSERT(sum.is_enclosed == true, "Sum must be enclosed");

    /* Test inverse cancellation */
    omni_number_t neg_sum = omni_number_from_scalar(-4096);
    omni_number_t zero = omni_number_add(sum, neg_sum);
    OMNI_TEST_ASSERT(zero.scalar == 0, "4096 + (-4096) must equal 0");

    return 0;
}

static int test_number_bounds_and_enclosure(void) {
    omni_number_t valid_num = omni_number_from_scalar(100);
    OMNI_TEST_ASSERT(omni_number_verify_bounds(&valid_num) == true, "Valid number must pass bounds check");

    omni_number_t corrupted = valid_num;
```

```
    corrupted.is_enclosed = false;
    OMNI_TEST_ASSERT(omni_number_verify_bounds(&corrupted) == false, "Unenclosed number must fail bounds check");

    return 0;
}

static int test_number_lifecycle_rolls(void) {
    omni_number_context_t ctx;
    omni_number_open(&ctx, 4096);

    OMNI_TEST_ASSERT(ctx.tripwire == OMNI_TRIPWIRE_ARMED, "Tripwire must be ARMED upon open");
    OMNI_TEST_ASSERT(ctx.input_quantity == 4096, "Input quantity must match 4096");

    omni_quad_state_t exec_status = omni_number_execute(&ctx);
    OMNI_TEST_ASSERT(exec_status == OMNI_STATE_PROCEED, "Number execute must PROCEED (+1)");
    OMNI_TEST_ASSERT(ctx.number.is_enclosed == true, "Number must be enclosed after execute");

    omni_quad_state_t close_status = omni_number_close(&ctx);
    OMNI_TEST_ASSERT(close_status == OMNI_STATE_PROCEED, "Number close must PROCEED (+1)");
    OMNI_TEST_ASSERT(ctx.tripwire == OMNI_TRIPWIRE_DISARMED, "Tripwire must be DISARMED upon clean close");

    return 0;
}

/* ============================================================================
 * BLOCK 3: BODY PRIMITIVES & TEST SUITE RUNNER
 * ============================================================================ */

int test_number_suite(void) {
    printf("[SUITE: Number Invariant Type]\n");
    OMNI_TEST_RUN(test_number_bijective_roundtrip, "Bijective Radix-3 Transduction");
    OMNI_TEST_RUN(test_number_equilibrium_law, "ALU Zero-Sum Equilibrium Law");
    OMNI_TEST_RUN(test_number_addition, "Balanced Addition & Cancellation");
    OMNI_TEST_RUN(test_number_bounds_and_enclosure, "Single-Word Enclosure Bounds");
    OMNI_TEST_RUN(test_number_lifecycle_rolls, "4-Block Lifecycle Rolls");
    return 0;
}
```

<TRACK 2> : {OmniCode Language & Standard Library}/lexicon/01_nouns/ideas/types/transpile variant/tests/c/unit/test_site.c

C | 120 LINES | 556 WORDS | 5.5 KB

```c
/* ============================================================================
 * BLOCK 1: METADATA & TEST TOPOLOGY
 * ( Unit Test: < Lexicon { Types of Ideas } > & [ Test { Site } ] )
 * ============================================================================
 * ARCHITECTURAL TOPOLOGY:
 *   Validates the physical silicon address type < Site > in isolation:
 *   - Null pointer address rejection and safety tripwire
 *   - Hardware Routing Law (MMIO ports vs RAM spaces)
 *   - Relative displacement and pointer offset arithmetic
 *   - Volatile memory semantics for bare-metal registers
 *   - Full 4-block lifecycle execution (open, execute, close)
 *   - Witness: Scripture Genesis 28:16 KJV
 *     "And Jacob awaked out of his sleep, and he said, Surely the Lord is in
 *      this place; and I knew it not."
 *   - Semantics: { SiliconCoordinates :: HardwareRoutingLaw :: NonNullInterlock }
 * ============================================================================ */

#include "test_common.h"

/* ============================================================================
 * BLOCK 2: OPENING CONTEXT & FIXTURE SETUP
 * ============================================================================ */

static int test_site_routing_classification(void) {
    /* VGA buffer coordinate (0xB8000) must route to MMIO */
    omni_site_t vga_site = omni_site_from_address(OMNI_SITE_VGA_BUFFER);
    OMNI_TEST_ASSERT(vga_site.routing == OMNI_ROUTING_HARDWARE_MMIO, "VGA buffer must classify as HARDWARE_MMIO");
    OMNI_TEST_ASSERT(omni_site_is_valid(&vga_site) == true, "VGA site must be valid");

    /* COM1 serial port (0x3F8) must route to MMIO */
    omni_site_t com1_site = omni_site_from_address(OMNI_SITE_COM1);
    OMNI_TEST_ASSERT(com1_site.routing == OMNI_ROUTING_HARDWARE_MMIO, "COM1 port must classify as HARDWARE_MMIO");

    /* Standard MBR memory location (0x7C00) must route to RAM */
    omni_site_t mbr_site = omni_site_from_address(OMNI_SITE_MBR_BOOT);
    OMNI_TEST_ASSERT(mbr_site.routing == OMNI_ROUTING_RAM_PHYSICAL, "MBR origin must classify as RAM_PHYSICAL");

    return 0;
}

static int test_site_null_tripwire(void) {
    /* Null address (0x0) must be invalid */
    omni_site_t null_site = omni_site_from_address(OMNI_SITE_NULL);
    OMNI_TEST_ASSERT(omni_site_is_valid(&null_site) == false, "Null site must be invalid");

    /* Executing lifecycle with NULL coordinate must trip the interlock and ROLLBACK */
    omni_site_context_t ctx;
    omni_site_open(&ctx, 0);
    ctx.input_coordinate = 0; /* Force null */
    omni_quad_state_t exec_st = omni_site_execute(&ctx);
    OMNI_TEST_ASSERT(exec_st == OMNI_STATE_ROLLBACK, "Null address execute must trigger ROLLBACK (-1)");
    OMNI_TEST_ASSERT(ctx.tripwire == OMNI_TRIPWIRE_TRIPPED, "Tripwire must be TRIPPED on null address");

    return 0;
}

static int test_site_offset_and_relational(void) {
    uint8_t dummy_buffer[16];
    uintptr_t base_addr = (uintptr_t)&dummy_buffer[0];

    omni_site_t base_site = omni_site_from_address(base_addr);
    omni_site_t offset_site = omni_site_offset(&base_site, 8);

    OMNI_TEST_ASSERT(offset_site.coordinate == base_addr + 8, "Displaced site coordinate must be base + 8");
    OMNI_TEST_ASSERT(omni_site_eq(&base_site, &base_site) == true, "Identical sites must evaluate equal");
    OMNI_TEST_ASSERT(omni_site_eq(&base_site, &offset_site) == false, "Different sites must evaluate not equal");

    return 0;
}

static int test_site_volatile_read_write(void) {
    uint8_t buffer[4] = {0, 0, 0, 0};
    uintptr_t addr = (uintptr_t)&buffer[0];
```

```
    omni_site_t site = omni_site_from_address(addr);

    /* Write via volatile primitives */
    omni_site_write_u8(&site, 0, 0xAB);
    omni_site_write_u8(&site, 1, 0xCD);

    /* Read back via volatile primitives */
    uint8_t r0 = omni_site_read_u8(&site, 0);
    uint8_t r1 = omni_site_read_u8(&site, 1);

    OMNI_TEST_ASSERT(r0 == 0xAB, "Volatile read at offset 0 must match written 0xAB");
    OMNI_TEST_ASSERT(r1 == 0xCD, "Volatile read at offset 1 must match written 0xCD");

    return 0;
}

static int test_site_lifecycle_rolls(void) {
    omni_site_context_t ctx;
    omni_site_open(&ctx, OMNI_SITE_VGA_BUFFER);

    OMNI_TEST_ASSERT(ctx.tripwire == OMNI_TRIPWIRE_ARMED, "Tripwire must be ARMED upon open");

    omni_quad_state_t exec_status = omni_site_execute(&ctx);
    OMNI_TEST_ASSERT(exec_status == OMNI_STATE_PROCEED, "Site execute must PROCEED (+1)");
    OMNI_TEST_ASSERT(ctx.site.routing == OMNI_ROUTING_HARDWARE_MMIO, "VGA site routing must be HARDWARE_MMIO");

    omni_quad_state_t close_status = omni_site_close(&ctx);
    OMNI_TEST_ASSERT(close_status == OMNI_STATE_PROCEED, "Site close must PROCEED (+1)");
    OMNI_TEST_ASSERT(ctx.tripwire == OMNI_TRIPWIRE_DISARMED, "Tripwire must be DISARMED upon clean close");

    return 0;
}

/* ============================================================================
 * BLOCK 3: BODY PRIMITIVES & TEST SUITE RUNNER
 * ============================================================================ */

int test_site_suite(void) {
    printf("[SUITE: Site Invariant Type]\n");
    OMNI_TEST_RUN(test_site_routing_classification, "Hardware Routing Law (MMIO vs RAM)");
    OMNI_TEST_RUN(test_site_null_tripwire, "Null Pointer Rejection & Tripwire Interlock");
    OMNI_TEST_RUN(test_site_offset_and_relational, "Offset Arithmetic & Relational Equality");
    OMNI_TEST_RUN(test_site_volatile_read_write, "Volatile Memory Read / Write Semantics");
    OMNI_TEST_RUN(test_site_lifecycle_rolls, "4-Block Lifecycle Rolls");
    return 0;
}
```

<TRACK 2> : {OmniCode Language & Standard Library}/lexicon/01_nouns/ideas/types/transpile variant/tests/c/unit/test_state.c

C | 119 LINES | 606 WORDS | 5.6 KB

```c
/* ============================================================================
 * BLOCK 1: METADATA & TEST TOPOLOGY
 * ( Unit Test: < Lexicon { Types of Ideas } > & [ Test { State } ] )
 * ============================================================================
 * ARCHITECTURAL TOPOLOGY:
 *   Validates the causal truth governor invariant type < State > in isolation:
 *   - Quad-State named registers: [ STATE TURN BOUND IT ]
 *   - Canonical Loop Law cadence and single-step detent transitions:
 *       WHEN IT < BOUND  --> PROCEED (+1)
 *       !    IT > BOUND  --> AWAIT (0)
 *       ::   IT == BOUND --> HALT (NULL)
 *   - State predicates and register reset mechanics
 *   - Full 4-block lifecycle execution (open, execute, close)
 *   - Witness: Scripture Ecclesiastes 3:1 KJV
 *     "To every thing there is a season, and a time to every purpose under
 *      the heaven:"
 *   - Semantics: { QuadStateTruth :: CanonicalLoopCadence :: CausalDetents }
 * ============================================================================ */

#include "test_common.h"

/* ============================================================================
 * BLOCK 2: OPENING CONTEXT & FIXTURE SETUP
 * ============================================================================ */

static int test_state_canonical_loop(void) {
    omni_state_t s = omni_state_create(OMNI_STATE_PROCEED, 3);

    OMNI_TEST_ASSERT(s.registers.bound == 3, "Loop bound must be 3");
    OMNI_TEST_ASSERT(s.registers.it == 0, "Initial IT must be 0");
    OMNI_TEST_ASSERT(s.registers.turn == 0, "Initial TURN must be 0");
    OMNI_TEST_ASSERT(omni_state_is_proceeding(&s) == true, "Initial state must be PROCEED");

    /* Step 1: IT 0 < 3 -> increments to IT=1, TURN=1, returns PROCEED */
    omni_quad_state_t st1 = omni_state_step_loop(&s.registers);
    OMNI_TEST_ASSERT(st1 == OMNI_STATE_PROCEED, "Step 1 must PROCEED");
    OMNI_TEST_ASSERT(s.registers.it == 1 && s.registers.turn == 1, "IT and TURN must be 1");

    /* Step 2: IT 1 < 3 -> increments to IT=2, TURN=2, returns PROCEED */
    omni_quad_state_t st2 = omni_state_step_loop(&s.registers);
    OMNI_TEST_ASSERT(st2 == OMNI_STATE_PROCEED, "Step 2 must PROCEED");
    OMNI_TEST_ASSERT(s.registers.it == 2 && s.registers.turn == 2, "IT and TURN must be 2");

    /* Step 3: IT 2 < 3 -> increments to IT=3, TURN=3, returns PROCEED */
    omni_quad_state_t st3 = omni_state_step_loop(&s.registers);
    OMNI_TEST_ASSERT(st3 == OMNI_STATE_PROCEED, "Step 3 must PROCEED");
    OMNI_TEST_ASSERT(s.registers.it == 3 && s.registers.turn == 3, "IT and TURN must be 3");

    /* Step 4: IT == BOUND (3 == 3) -> Reaches terminal detent HALT:NULL */
    omni_quad_state_t st4 = omni_state_step_loop(&s.registers);
    OMNI_TEST_ASSERT(st4 == OMNI_STATE_HALT, "Step 4 (IT == BOUND) must reach HALT:NULL detent");
    OMNI_TEST_ASSERT(omni_state_is_halted(&s) == true, "State must report halted");

    return 0;
}

static int test_state_await_transition(void) {
    omni_state_t s = omni_state_create(OMNI_STATE_PROCEED, 4);

    /* Manually position IT > BOUND to simulate overrun awaiting input */
    s.registers.it = 5;
    omni_quad_state_t await_st = omni_state_step_loop(&s.registers);
    OMNI_TEST_ASSERT(await_st == OMNI_STATE_AWAIT, "IT > BOUND must transition to AWAIT (0)");
    OMNI_TEST_ASSERT(omni_state_is_awaiting(&s) == true, "State engine must report awaiting");

    return 0;
}

static int test_state_predicates_and_reset(void) {
    omni_state_t s = omni_state_create(OMNI_STATE_PROCEED, 2);

    /* Step until halt */
    omni_state_step_loop(&s.registers);
```

```
    omni_state_step_loop(&s.registers);
    omni_state_step_loop(&s.registers);
    OMNI_TEST_ASSERT(omni_state_is_halted(&s) == true, "Engine must be halted");

    /* Reset engine */
    omni_state_reset(&s, 5);
    OMNI_TEST_ASSERT(s.registers.bound == 5, "New bound must be 5");
    OMNI_TEST_ASSERT(s.registers.it == 0, "IT must reset to 0");
    OMNI_TEST_ASSERT(s.registers.turn == 0, "TURN must reset to 0");
    OMNI_TEST_ASSERT(omni_state_is_proceeding(&s) == true, "Reset must restore PROCEED status");

    return 0;
}

static int test_state_lifecycle_rolls(void) {
    omni_state_context_t ctx;
    omni_state_open(&ctx, OMNI_STATE_PROCEED, 4);

    OMNI_TEST_ASSERT(ctx.tripwire == OMNI_TRIPWIRE_ARMED, "Tripwire must be ARMED upon open");

    /* Execute full governance cadence */
    omni_quad_state_t exec_status = omni_state_execute(&ctx);
    OMNI_TEST_ASSERT(exec_status == OMNI_STATE_PROCEED, "State execute must PROCEED (+1)");
    OMNI_TEST_ASSERT(ctx.state_engine.registers.it == 4, "Final IT must equal bound 4");
    OMNI_TEST_ASSERT(ctx.state_engine.registers.state == OMNI_STATE_HALT, "Registers must reach HALT:NULL detent");

    omni_quad_state_t close_status = omni_state_close(&ctx);
    OMNI_TEST_ASSERT(close_status == OMNI_STATE_PROCEED, "State close must PROCEED (+1)");
    OMNI_TEST_ASSERT(ctx.tripwire == OMNI_TRIPWIRE_DISARMED, "Tripwire must be DISARMED upon clean close");

    return 0;
}

/* ============================================================================
 * BLOCK 3: BODY PRIMITIVES & TEST SUITE RUNNER
 * ============================================================================ */

int test_state_suite(void) {
    printf("[SUITE: State Invariant Type]\n");
    OMNI_TEST_RUN(test_state_canonical_loop, "Canonical Loop Law Cadence (IT < BOUND -> HALT)");
    OMNI_TEST_RUN(test_state_await_transition, "Standby Transition (IT > BOUND -> AWAIT)");
    OMNI_TEST_RUN(test_state_predicates_and_reset, "Predicate Queries & Engine Reset");
    OMNI_TEST_RUN(test_state_lifecycle_rolls, "4-Block Lifecycle Rolls");
    return 0;
}
```

<TRACK 2> : {OmniCode Language & Standard Library}/lexicon/01_nouns/ideas/invariants/covenant.omni

OMNI | 60 LINES | 244 WORDS | 2.2 KB

```
# ========================================================================
# BLOCK 1: METADATA BLOCK
# ( Invariant Noun: <IDEA:INVARIANT:COVENANT> )
# ========================================================================
# ARCHITECTURAL TOPOLOGY:
#   Defines the unbreakable architectural contract binding principals and resources.
#   - Base Block Type: < Idea : Type : State > (Mutual Contract Sentinel)
#   - Category: Idea
#   - Subcategory: Invariant
#   - Covenant Nature: Unbreakable Mutual Contract
# ========================================================================

Module < Lexicon.Nouns.Ideas.Invariants.Covenant > ( Unbreakable Architectural Covenant & Contract ) {
    version: < "1.0.0" >,
    system: < OmniCode.Lexicon >,
    semantics: < InvariantNoun :: Idea :: Invariant :: ArchitecturalCovenant >
}

# Invariant Noun Declaration: <IDEA:INVARIANT:COVENANT>
< Idea : Invariant : Covenant > [ capacity: 1 state ] {
    base_type: < Idea : Type : State >,
    projections: < {
        ontological: "< Covenant >",
        structural: "[ COVENANT ]",
        agentic: "{ Covenant }"
    } >,
    scriptural_witness: "Genesis 9:16 - 'The everlasting covenant between God and every living creature'",
    machine_effect: "unbreakable_contract_sentinel"
}

# ========================================================================
# BLOCK 2: OPENING BLOCK
# ========================================================================
Opening {
    requires: [
        Covenant : InvariantNoun ::
        State : InvariantNoun
    ],
    preconditions: [ +1 ]
}

# ========================================================================
# BLOCK 3: BODY BLOCK
# ========================================================================
Body [
    covenant_assertion [
        [ COVENANT ] [ 1 ] :: { Covenant } --> BIND {Operator_Kernel}
    ] ( Covenant bound between human operator and supervisory kernel )
]

# ========================================================================
# BLOCK 4: CLOSING BLOCK
# ========================================================================
Closing {
    verify: [
        covenant_assertion = yes
    ],
    commit_status [ PROCEED- ] ( Covenant idea invariant sealed in PROCEED )
}
```

<TRACK 2> : {OmniCode Language & Standard Library}/lexicon/01_nouns/ideas/invariants/covenant.tape

TAPE | 33 LINES | 204 WORDS | 1.6 KB

```
# ================================================================================
# TAPE HEAD: COVENANT INVARIANT PROCESS RIBBON
# ================================================================================
# ARCHITECTURAL TOPOLOGY:
#   Executes atomic lifecycle for Invariant Noun <IDEA:INVARIANT:COVENANT>.
# ================================================================================

[Tape.Covenant] : WHO {OmniCode_Kernel} WHAT {SystemCovenant} --> BIND {MutualContract}
    WHEN {SystemGenesis}
    WHERE {covenant_slot} = [ 1 ]
    WHY ( Inscribe unbreakable architectural covenant invariant )
    HOW [ +1 : PROCEED ]
    :: < covenant_landmark > ( Genesis 9:16 - 'And the bow shall be in the cloud' )

# ================================================================================
# SUBORDINATE ROLLS: ATOMIC INVARIANT RESOLUTION
# ================================================================================

# Roll 1: Reference abstract covenant archetype
roll_1 [ +Covenant ] ( Abstract covenant archetype referenced )

# Roll 2: Allocate concrete covenant seal
roll_2 [ Covenant --> "System_Covenant" @ 0 ] ( Inscribe system covenant at offset 0 )

# Roll 3: Assert structural projection and clarified capacity
roll_3 [ [ COVENANT ] [ 1 ] :: "1 state" --> [ COVENANT_SEAL ] ] ( Structural 1 cell bounds affirmed )

# Roll 4: Verify covenant inviolability into result prime
roll_4 [ "System_Covenant" = Covenant ! [ +1 ] --> [ covenant_verified` ] ] ( Verify covenant matches archetype )

# Roll 5: Seal in affirmative proceed
roll_5 [ commit_status [ PROCEED- ] ] ( Covenant invariant sealed and verified )
```

<TRACK 2> : {OmniCode Language & Standard Library}/lexicon/01_nouns/ideas/invariants/equilibrium.omni

OMNI | 60 LINES | 236 WORDS | 2.2 KB

```
# ================================================================================
# BLOCK 1: METADATA BLOCK
# ( Invariant Noun: <IDEA:INVARIANT:EQUILIBRIUM> )
# ================================================================================
# ARCHITECTURAL TOPOLOGY:
#   Defines the balanced ternary arithmetic equilibrium: (+1) + (-1) <=> 0.
#   - Base Block Type: < Idea : Type : State > (Balanced Arithmetic Truth)
#   - Category: Idea
#   - Subcategory: Invariant
#   - Arithmetic Law: Radix-3 Zero-Sum Balance
# ================================================================================

Module < Lexicon.Nouns.Ideas.Invariants.Equilibrium > ( Balanced Ternary Arithmetic Equilibrium ) {
    version: < "1.0.0" >,
    system: < OmniCode.Lexicon >,
    semantics: < InvariantNoun :: Idea :: Invariant :: TernaryEquilibrium >
}

# Invariant Noun Declaration: <IDEA:INVARIANT:EQUILIBRIUM>
< Idea : Invariant : Equilibrium > [ capacity: 1 state ] {
    base_type: < Idea : Type : State >,
    invariant_equation: "(+1) plus (-1) <=> 0",
    projections: < {
        ontological: "< Equilibrium >",
        structural: "[ EQUILIBRIUM ]",
        agentic: "{ Equilibrium }"
    } >,
    machine_effect: "alu_zero_sum_balance"
}

# ================================================================================
# BLOCK 2: OPENING BLOCK
# ================================================================================
Opening {
    requires: [
        Equilibrium : InvariantNoun ::
        State : InvariantNoun
    ],
    preconditions: [ +1 ]
}

# ================================================================================
# BLOCK 3: BODY BLOCK
# ================================================================================
Body [
    equilibrium_assertion [
        (+1) plus (-1) <=> 0 :: { Equilibrium } --> PROVE {ALU_Zero_Sum}
    ] ( Invariant arithmetic equilibrium verified across hardware registers )
]

# ================================================================================
# BLOCK 4: CLOSING BLOCK
# ================================================================================
Closing {
    verify: [
        equilibrium_assertion = yes
    ],
    commit_status [ PROCEED- ] ( Equilibrium idea invariant sealed in PROCEED )
}
```

<TRACK 2> : {OmniCode Language & Standard Library}/lexicon/01_nouns/ideas/invariants/equilibrium.tape

TAPE | 33 LINES | 211 WORDS | 1.6 KB

```
# ================================================================================
# TAPE HEAD: EQUILIBRIUM INVARIANT PROCESS RIBBON
# ================================================================================
# ARCHITECTURAL TOPOLOGY:
#   Executes atomic lifecycle for Invariant Noun <IDEA:INVARIANT:EQUILIBRIUM>.
# ================================================================================

[Tape.Equilibrium] : WHO {OmniCode_Kernel} WHAT {ALUEquilibrium} --> PROVE {ZeroSumBalance}
    WHEN {ArithmeticProof}
    WHERE {alu_state} = [ 1 ]
    WHY ( Inscribe balanced ternary arithmetic equilibrium invariant )
    HOW [ +1 : PROCEED ]
    :: < equilibrium_landmark > ( Proverbs 11:1 - 'A false balance is abomination, but a just weight is his delight' )

# ================================================================================
# SUBORDINATE ROLLS: ATOMIC INVARIANT RESOLUTION
# ================================================================================

# Roll 1: Reference abstract equilibrium archetype
roll_1 [ +Equilibrium ] ( Abstract equilibrium archetype referenced )

# Roll 2: Allocate concrete ALU state verification
roll_2 [ Equilibrium --> "ALU_Proof" @ 0 ] ( Inscribe ALU proof at offset 0 )

# Roll 3: Prove (+1) plus (-1) <=> 0 in hardware
roll_3 [ PROVE { ALU_Equilibrium } [ (+1) plus (-1) <=> 0 ] ] ( Arithmetic equilibrium proven in ALU )

# Roll 4: Verify equilibrium invariance into result prime
roll_4 [ "ALU_Proof" = Equilibrium ! [ +1 ] --> [ equilibrium_verified` ] ] ( Verify ALU proof matches archetype )

# Roll 5: Seal in affirmative proceed
roll_5 [ commit_status [ PROCEED- ] ] ( Equilibrium invariant sealed and verified )
```

<TRACK 2> : {OmniCode Language & Standard Library}/lexicon/01_nouns/persons/systems/system.omni

OMNI | 60 LINES | 235 WORDS | 2.1 KB

```
# ================================================================================
# BLOCK 1: METADATA BLOCK
# ( Invariant Noun: <PERSON:SYSTEM:CHASSIS> )
# ================================================================================
# ARCHITECTURAL TOPOLOGY:
#   Defines the total machine chassis, hardware substrate, and execution host.
#   - Base Block Type: < Idea : Type : Site > (Physical Chassis Locus)
#   - Category: Person
#   - Subcategory: System
#   - Authority Rank: 3 (Mechanical Substrate)
# ================================================================================

Module < Lexicon.Nouns.Persons.Systems.System > ( Total Machine Chassis & Execution Substrate ) {
    version: < "1.0.0" >,
    system: < OmniCode.Lexicon >,
    semantics: < InvariantNoun :: Person :: System :: HardwareChassis >
}

# Invariant Noun Declaration: <PERSON:SYSTEM:CHASSIS>
< Person : System : System > [ capacity: 1 site ] {
    base_type: < Idea : Type : Site >,
    authority_rank: 3,
    projections: < {
        ontological: "< System >",
        structural: "[ SYSTEM ]",
        agentic: "{ System }"
    } >,
    machine_effect: "total_machine_substrate"
}

# ================================================================================
# BLOCK 2: OPENING BLOCK
# ================================================================================
Opening {
    requires: [
        System : InvariantNoun ::
        Site : InvariantNoun
    ],
    preconditions: [ +1 ]
}

# ================================================================================
# BLOCK 3: BODY BLOCK
# ================================================================================
Body [
    system_assertion [
        { System } --> HOST {Execution} :: [ SYSTEM ] @ 0x03
    ] ( Machine chassis hosts execution substrate in bounded system slot )
]

# ================================================================================
# BLOCK 4: CLOSING BLOCK
# ================================================================================
Closing {
    verify: [
        system_assertion = yes
    ],
    commit_status [ PROCEED- ] ( System substrate invariant sealed in PROCEED )
}
```

<TRACK 2> : {OmniCode Language & Standard Library}/lexicon/01_nouns/persons/systems/system.tape

TAPE | 33 LINES | 204 WORDS | 1.6 KB

```
# ================================================================================
# TAPE HEAD: SYSTEM CHASSIS PROCESS RIBBON
# ================================================================================
# ARCHITECTURAL TOPOLOGY:
#   Executes atomic lifecycle for Invariant Noun <PERSON:SYSTEM:CHASSIS>.
# ================================================================================

[Tape.System] : WHO {Machine_Substrate} WHAT {HardwareChassis} --> HOST {ExecutionTerritory}
    WHEN {SystemGenesis}
    WHERE {system_partition} = [ 64 ]
    WHY ( Inscribe mechanical chassis substrate invariant and hosting territory )
    HOW [ +1 : PROCEED ]
    :: < system_landmark > ( Job 38:4 - 'Where wast thou when I laid the foundations of the earth?' )

# ================================================================================
# SUBORDINATE ROLLS: ATOMIC INVARIANT RESOLUTION
# ================================================================================

# Roll 1: Reference abstract system archetype
roll_1 [ +System ] ( Abstract system substrate archetype referenced )

# Roll 2: Allocate concrete system descriptor
roll_2 [ System --> "System_Descriptor" @ 0 ] ( Inscribe system descriptor in partition )

# Roll 3: Assert agentic projection
roll_3 [ { System } --> SUPPORT { Processes } ] ( System asserts physical hosting support )

# Roll 4: Verify descriptor integrity into result prime
roll_4 [ "System_Descriptor" = System ! [ +1 ] --> [ system_verified` ] ] ( Verify system descriptor matches archetype )

# Roll 5: Seal in affirmative proceed
roll_5 [ commit_status [ PROCEED- ] ] ( System substrate invariant sealed and verified )
```

<TRACK 2> : {OmniCode Language & Standard Library}/lexicon/01_nouns/persons/supervisors/kernel.omni

OMNI | 60 LINES | 230 WORDS | 2.1 KB

```
# ================================================================================
# BLOCK 1: METADATA BLOCK
# ( Invariant Noun: <PERSON:SUPERVISOR:KERNEL> )
# ================================================================================
# ARCHITECTURAL TOPOLOGY:
#   Defines the root supervisory execution kernel and covenant custodian.
#   - Base Block Type: < Idea : Type : Site > (Execution Locus)
#   - Category: Person
#   - Subcategory: Supervisor
#   - Authority Rank: 1 (Supervisory Custodian)
# ================================================================================

Module < Lexicon.Nouns.Persons.Supervisors.Kernel > ( Root Supervisory Execution Kernel ) {
    version: < "1.0.0" >,
    system: < OmniCode.Lexicon >,
    semantics: < InvariantNoun :: Person :: Supervisor :: CovenantCustodian >
}

# Invariant Noun Declaration: <PERSON:SUPERVISOR:KERNEL>
< Person : Supervisor : Kernel > [ capacity: 1 site ] {
    base_type: < Idea : Type : Site >,
    authority_rank: 1,
    projections: < {
        ontological: "< Kernel >",
        structural: "[ KERNEL ]",
        agentic: "{ Kernel }"
    } >,
    machine_effect: "root_supervisor_custodian"
}

# ================================================================================
# BLOCK 2: OPENING BLOCK
# ================================================================================
Opening {
    requires: [
        Kernel : InvariantNoun ::
        Site : InvariantNoun
    ],
    preconditions: [ +1 ]
}

# ================================================================================
# BLOCK 3: BODY BLOCK
# ================================================================================
Body [
    supervisor_assertion [
        { Kernel } --> GOVERN {Machine} :: [ KERNEL ] @ 0x01
    ] ( Root kernel supervisor asserts governance over machine resources )
]

# ================================================================================
# BLOCK 4: CLOSING BLOCK
# ================================================================================
Closing {
    verify: [
        supervisor_assertion = yes
    ],
    commit_status [ PROCEED- ] ( Kernel supervisor invariant sealed in PROCEED )
}
```

<TRACK 2> : {OmniCode Language & Standard Library}/lexicon/01_nouns/persons/supervisors/kernel.tape

TAPE | 33 LINES | 195 WORDS | 1.6 KB

```
# ================================================================================
# TAPE HEAD: KERNEL SUPERVISOR PROCESS RIBBON
# ================================================================================
# ARCHITECTURAL TOPOLOGY:
#   Executes atomic lifecycle for Invariant Noun <PERSON:SUPERVISOR:KERNEL>.
# ================================================================================

[Tape.Kernel] : WHO {OmniCode_Kernel} WHAT {SystemSupervisor} --> BOOT {ExecutionPlane}
    WHEN {SystemGenesis}
    WHERE {kernel_partition} = [ 64 ]
    WHY ( Inscribe supervisory kernel invariant and execution custodian )
    HOW [ +1 : PROCEED ]
    :: < kernel_landmark > ( Genesis 1:1 - Foundation of supervisory order )

# ================================================================================
# SUBORDINATE ROLLS: ATOMIC INVARIANT RESOLUTION
# ================================================================================

# Roll 1: Reference abstract kernel archetype
roll_1 [ +Kernel ] ( Abstract kernel supervisor archetype referenced )

# Roll 2: Allocate concrete kernel descriptor
roll_2 [ Kernel --> "Kernel_Descriptor" @ 0 ] ( Inscribe kernel descriptor in partition )

# Roll 3: Assert agentic projection
roll_3 [ { Kernel } --> GUARD { Invariants } ] ( Kernel establishes guard on invariants )

# Roll 4: Verify descriptor integrity into result prime
roll_4 [ "Kernel_Descriptor" = Kernel ! [ +1 ] --> [ kernel_verified` ] ] ( Verify descriptor matches archetype )

# Roll 5: Seal in affirmative proceed
roll_5 [ commit_status [ PROCEED- ] ] ( Kernel supervisor invariant sealed and verified )
```

<TRACK 2> : {OmniCode Language & Standard Library}/lexicon/01_nouns/persons/transducers/model.omni

OMNI | 60 LINES | 233 WORDS | 2.1 KB

```
# ================================================================================
# BLOCK 1: METADATA BLOCK
# ( Invariant Noun: <PERSON:TRANSDUCER:MODEL> )
# ================================================================================
# ARCHITECTURAL TOPOLOGY:
#   Defines the AI Cortex, neural model, and semantic transducer engine.
#   - Base Block Type: < Idea : Type : Site > (Cognitive Locus)
#   - Category: Person
#   - Subcategory: Transducer
#   - Authority Rank: 2 (Cognitive Transducer)
# ================================================================================

Module < Lexicon.Nouns.Persons.Transducers.Model > ( AI Cortex & Semantic Transducer Engine ) {
    version: < "1.0.0" >,
    system: < OmniCode.Lexicon >,
    semantics: < InvariantNoun :: Person :: Transducer :: AICortex >
}

# Invariant Noun Declaration: <PERSON:TRANSDUCER:MODEL>
< Person : Transducer : Model > [ capacity: 1 site ] {
    base_type: < Idea : Type : Site >,
    authority_rank: 2,
    projections: < {
        ontological: "< Model >",
        structural: "[ MODEL ]",
        agentic: "{ Model }"
    } >,
    machine_effect: "ai_cortex_transducer"
}

# ================================================================================
# BLOCK 2: OPENING BLOCK
# ================================================================================
Opening {
    requires: [
        Model : InvariantNoun ::
        Site : InvariantNoun
    ],
    preconditions: [ +1 ]
}

# ================================================================================
# BLOCK 3: BODY BLOCK
# ================================================================================
Body [
    transducer_assertion [
        { Model } --> TRANSDUCE {Semantics} :: [ MODEL ] @ 0x02
    ] ( AI Cortex transducer maps semantics into bounded model channel )
]

# ================================================================================
# BLOCK 4: CLOSING BLOCK
# ================================================================================
Closing {
    verify: [
        transducer_assertion = yes
    ],
    commit_status [ PROCEED- ] ( Transducer invariant sealed in PROCEED )
}
```

<TRACK 2> : {OmniCode Language & Standard Library}/lexicon/01_nouns/persons/transducers/model.tape

TAPE | 33 LINES | 197 WORDS | 1.6 KB

```
# ================================================================================
# TAPE HEAD: MODEL TRANSDUCER PROCESS RIBBON
# ================================================================================
# ARCHITECTURAL TOPOLOGY:
#   Executes atomic lifecycle for Invariant Noun <PERSON:TRANSDUCER:MODEL>.
# ================================================================================

[Tape.Model] : WHO {AI_Cortex} WHAT {SemanticTransducer} --> INFER {CognitiveFlow}
    WHEN {SystemGenesis}
    WHERE {model_partition} = [ 64 ]
    WHY ( Inscribe semantic AI model transducer invariant and channel )
    HOW [ +1 : PROCEED ]
    :: < model_landmark > ( Proverbs 2:6 - 'For the Lord giveth wisdom' )

# ================================================================================
# SUBORDINATE ROLLS: ATOMIC INVARIANT RESOLUTION
# ================================================================================

# Roll 1: Reference abstract model archetype
roll_1 [ +Model ] ( Abstract model transducer archetype referenced )

# Roll 2: Allocate concrete model descriptor
roll_2 [ Model --> "Model_Descriptor" @ 0 ] ( Inscribe model descriptor in partition )

# Roll 3: Assert agentic projection
roll_3 [ { Model } --> REASON { Symbols } ] ( Model asserts reasoning capability )

# Roll 4: Verify descriptor integrity into result prime
roll_4 [ "Model_Descriptor" = Model ! [ +1 ] --> [ model_verified` ] ] ( Verify model descriptor matches archetype )

# Roll 5: Seal in affirmative proceed
roll_5 [ commit_status [ PROCEED- ] ] ( Model transducer invariant sealed and verified )
```

<TRACK 2> : {OmniCode Language & Standard Library}/lexicon/01_nouns/persons/stewards/operator.omni

OMNI | 60 LINES | 234 WORDS | 2.1 KB

```
# ================================================================================
# BLOCK 1: METADATA BLOCK
# ( Invariant Noun: <PERSON:STEWARD:OPERATOR> )
# ================================================================================
# ARCHITECTURAL TOPOLOGY:
#   Defines the human originator, moral steward, and root sovereign principal.
#   - Base Block Type: < Idea : Type : Site > (Sovereign Authority Locus)
#   - Category: Person
#   - Subcategory: Steward
#   - Authority Rank: 0 (Highest / Originating Steward)
# ================================================================================

Module < Lexicon.Nouns.Persons.Stewards.Operator > ( Human Originator & Moral Steward ) {
    version: < "1.0.0" >,
    system: < OmniCode.Lexicon >,
    semantics: < InvariantNoun :: Person :: Steward :: SovereignOriginator >
}

# Invariant Noun Declaration: <PERSON:STEWARD:OPERATOR>
< Person : Steward : Operator > [ capacity: 1 site ] {
    base_type: < Idea : Type : Site >,
    authority_rank: 0,
    projections: < {
        ontological: "< Operator >",
        structural: "[ OPERATOR ]",
        agentic: "{ Operator }"
    } >,
    machine_effect: "human_moral_steward"
}

# ================================================================================
# BLOCK 2: OPENING BLOCK
# ================================================================================
Opening {
    requires: [
        Operator : InvariantNoun ::
        Site : InvariantNoun
    ],
    preconditions: [ +1 ]
}

# ================================================================================
# BLOCK 3: BODY BLOCK
# ================================================================================
Body [
    steward_assertion [
        { Operator } --> ORIGINATE {Intent} :: [ OPERATOR ] @ 0x00
    ] ( Human steward asserts intent into bounded authority slot )
]

# ================================================================================
# BLOCK 4: CLOSING BLOCK
# ================================================================================
Closing {
    verify: [
        steward_assertion = yes
    ],
    commit_status [ PROCEED- ] ( Steward invariant sealed in PROCEED )
}
```

<TRACK 2> : {OmniCode Language & Standard Library}/lexicon/01_nouns/persons/stewards/operator.tape

TAPE | 33 LINES | 198 WORDS | 1.6 KB

```
# ================================================================================
# TAPE HEAD: OPERATOR STEWARD PROCESS RIBBON
# ================================================================================
# ARCHITECTURAL TOPOLOGY:
#   Executes atomic lifecycle for Invariant Noun <PERSON:STEWARD:OPERATOR>.
# ================================================================================

[Tape.Operator] : WHO {Human_Operator} WHAT {MoralSteward} --> BIND {SovereignAuthority}
    WHEN {SystemGenesis}
    WHERE {steward_partition} = [ 64 ]
    WHY ( Inscribe sovereign human steward invariant and authority token )
    HOW [ +1 : PROCEED ]
    :: < steward_landmark > ( Genesis 1:26 - Human stewardship over creation )

# ================================================================================
# SUBORDINATE ROLLS: ATOMIC INVARIANT RESOLUTION
# ================================================================================

# Roll 1: Reference abstract steward archetype
roll_1 [ +Operator ] ( Abstract steward archetype referenced )

# Roll 2: Allocate concrete steward authority token
roll_2 [ Operator --> "Operator_Token" @ 0 ] ( Inscribe human steward token into partition )

# Roll 3: Assert agentic projection
roll_3 [ { Operator } --> AUTHORIZE { Kernel } ] ( Steward delegates execution to kernel )

# Roll 4: Verify structural token invariant into result prime
roll_4 [ "Operator_Token" = Operator ! [ +1 ] --> [ steward_verified` ] ] ( Verify steward token matches archetype )

# Roll 5: Seal in affirmative proceed
roll_5 [ commit_status [ PROCEED- ] ] ( Steward invariant sealed and verified )
```

<TRACK 2> : {OmniCode Language & Standard Library}/lexicon/01_nouns/places/tracks/track.omni

OMNI | 60 LINES | 244 WORDS | 2.1 KB

```
# ================================================================================
# BLOCK 1: METADATA BLOCK
# ( Invariant Noun: <PLACE:TRACK:BAND> )
# ================================================================================
# ARCHITECTURAL TOPOLOGY:
#   Defines concentric architectural tracks across the 6-track machine system.
#   - Base Block Type: < Idea : Type : Site > (Concentric Channel Locus)
#   - Category: Place
#   - Subcategory: Track
#   - Channels: Track 0 (Silicon) through Track 5 (Ops)
# ================================================================================

Module < Lexicon.Nouns.Places.Tracks.Track > ( Concentric Architectural Track & Execution Plane ) {
    version: < "1.0.0" >,
    system: < OmniCode.Lexicon >,
    semantics: < InvariantNoun :: Place :: Track :: ConcentricExecutionBand >
}

# Invariant Noun Declaration: <PLACE:TRACK:BAND>
< Place : Track : Track > [ capacity: 6 channels ] {
    base_type: < Idea : Type : Site >,
    track_range: [ 0, 1, 2, 3, 4, 5 ],
    projections: < {
        ontological: "< Track >",
        structural: "[ TRACK ]",
        agentic: "{ Track }"
    } >,
    machine_effect: "concentric_track_routing"
}

# ================================================================================
# BLOCK 2: OPENING BLOCK
# ================================================================================
Opening {
    requires: [
        Track : InvariantNoun ::
        Site : InvariantNoun
    ],
    preconditions: [ +1 ]
}

# ================================================================================
# BLOCK 3: BODY BLOCK
# ================================================================================
Body [
    track_assertion [
        [ TRACK ] [ 6 ] :: { Track } --> ROUTE {Channel}
    ] ( Concentric track hierarchy armed across 6 architectural planes )
]

# ================================================================================
# BLOCK 4: CLOSING BLOCK
# ================================================================================
Closing {
    verify: [
        track_assertion = yes
    ],
    commit_status [ PROCEED- ] ( Track place invariant sealed in PROCEED )
}
```

<TRACK 2> : {OmniCode Language & Standard Library}/lexicon/01_nouns/places/tracks/track.tape

TAPE | 33 LINES | 210 WORDS | 1.6 KB

```
# ================================================================================
# TAPE HEAD: TRACK EXECUTION BAND PROCESS RIBBON
# ================================================================================
# ARCHITECTURAL TOPOLOGY:
#   Executes atomic lifecycle for Invariant Noun <PLACE:TRACK:BAND>.
# ================================================================================

[Tape.Track] : WHO {OmniCode_Kernel} WHAT {ExecutionTrack} --> BIND {ConcentricPlane}
    WHEN {SystemGenesis}
    WHERE {track_channel} = [ 6 ]
    WHY ( Inscribe concentric track plane invariant across 6 architectural rings )
    HOW [ +1 : PROCEED ]
    :: < track_landmark > ( Ezekiel 1:16 - 'A wheel in the middle of a wheel' )

# ================================================================================
# SUBORDINATE ROLLS: ATOMIC INVARIANT RESOLUTION
# ================================================================================

# Roll 1: Reference abstract track archetype
roll_1 [ +Track ] ( Abstract track archetype referenced )

# Roll 2: Allocate track routing descriptor
roll_2 [ Track --> "Track_Descriptor" @ 0 ] ( Inscribe track descriptor in partition )

# Roll 3: Assert structural projection and clarified channel count
roll_3 [ [ TRACK ] [ 6 ] :: "6" --> [ CONCENTRIC_RINGS ] ] ( Structural 6-track bounds affirmed )

# Roll 4: Verify track channel routing into result prime
roll_4 [ "Track_Descriptor" = Track ! [ +1 ] --> [ track_verified` ] ] ( Verify track descriptor matches archetype )

# Roll 5: Seal in affirmative proceed
roll_5 [ commit_status [ PROCEED- ] ] ( Track place invariant sealed and verified )
```

<TRACK 2> : {OmniCode Language & Standard Library}/lexicon/01_nouns/places/memory/partition.omni

OMNI | 60 LINES | 237 WORDS | 2.2 KB

```
# ================================================================================
# BLOCK 1: METADATA BLOCK
# ( Invariant Noun: <PLACE:MEMORY:PARTITION> )
# ================================================================================
# ARCHITECTURAL TOPOLOGY:
#   Defines an entrusted, bounded sub-region of a storehouse carved via `v--`.
#   - Base Block Type: < Idea : Type : Container > (Carved Memory Slice)
#   - Category: Place
#   - Subcategory: Memory
#   - Partition Operator: `v--`
# ================================================================================

Module < Lexicon.Nouns.Places.Memory.Partition > ( Entrusted Bounded Sub-Region of a Storehouse ) {
    version: < "1.0.0" >,
    system: < OmniCode.Lexicon >,
    semantics: < InvariantNoun :: Place :: Memory :: DelegatedPartition >
}

# Invariant Noun Declaration: <PLACE:MEMORY:PARTITION>
< Place : Memory : Partition > [ capacity: dynamic cells ] {
    base_type: < Idea : Type : Container >,
    carving_operator: "v--",
    projections: < {
        ontological: "< Partition >",
        structural: "[ PARTITION ]",
        agentic: "{ Partition }"
    } >,
    machine_effect: "delegated_task_slice"
}

# ================================================================================
# BLOCK 2: OPENING BLOCK
# ================================================================================
Opening {
    requires: [
        Partition : InvariantNoun ::
        Container : InvariantNoun
    ],
    preconditions: [ +1 ]
}

# ================================================================================
# BLOCK 3: BODY BLOCK
# ================================================================================
Body [
    partition_assertion [
        Root_Storehouse v-- [ PARTITION ] [ 256 ] :: { Partition } --> PROTECT {Slice}
    ] ( Bounded task partition carved from root storehouse via v-- )
]

# ================================================================================
# BLOCK 4: CLOSING BLOCK
# ================================================================================
Closing {
    verify: [
        partition_assertion = yes
    ],
    commit_status [ PROCEED- ] ( Partition memory invariant sealed in PROCEED )
}
```

<TRACK 2> : {OmniCode Language & Standard Library}/lexicon/01_nouns/places/memory/partition.tape

TAPE | 33 LINES | 216 WORDS | 1.6 KB

```
# ================================================================================
# TAPE HEAD: PARTITION MEMORY SLICE PROCESS RIBBON
# ================================================================================
# ARCHITECTURAL TOPOLOGY:
#   Executes atomic lifecycle for Invariant Noun <PLACE:MEMORY:PARTITION>.
# ================================================================================

[Tape.Partition] : WHO {OmniCode_Kernel} WHAT {MemoryPartition} --> CARVE {TaskSlice}
    WHEN {TaskSpawn}
    WHERE {partition_slice} = [ 256 ]
    WHY ( Inscribe delegated memory partition carved via v-- )
    HOW [ +1 : PROCEED ]
    :: < partition_landmark > ( Joshua 18:10 - 'Divided the land unto the children of Israel' )

# ================================================================================
# SUBORDINATE ROLLS: ATOMIC INVARIANT RESOLUTION
# ================================================================================

# Roll 1: Reference abstract partition archetype
roll_1 [ +Partition ] ( Abstract partition archetype referenced )

# Roll 2: Carve concrete task partition via v--
roll_2 [ Root_Storehouse v-- "Task_Partition" [ 256 ] ] ( Isolated workspace carved via v-- )

# Roll 3: Assert structural projection and clarified capacity
roll_3 [ [ PARTITION ] [ 256 ] :: "256" --> [ TASK_PARTITION ] ] ( Structural 256 cell bounds affirmed )

# Roll 4: Verify boundary containment into result prime
roll_4 [ [ { 256 } < { 4096 } ] ! [ +1 ] --> [ partition_bounded` ] ] ( Verify partition within storehouse bounds )

# Roll 5: Seal in affirmative proceed
roll_5 [ commit_status [ PROCEED- ] ] ( Memory partition invariant sealed and verified )
```

<TRACK 2> : {OmniCode Language & Standard Library}/lexicon/01_nouns/places/memory/storehouse.omni

OMNI | 60 LINES | 240 WORDS | 2.2 KB

```
# ================================================================================
# BLOCK 1: METADATA BLOCK
# ( Invariant Noun: <PLACE:MEMORY:STOREHOUSE> )
# ================================================================================
# ARCHITECTURAL TOPOLOGY:
#   Defines the root system physical memory domain, reservoir, and chassis plate.
#   - Base Block Type: < Idea : Type : Container > (Memory Capacity Envelope)
#   - Category: Place
#   - Subcategory: Memory
#   - Capacity: Physical RAM Cells (4096 default cells)
# ================================================================================

Module < Lexicon.Nouns.Places.Memory.Storehouse > ( Root System Memory Domain & Reservoir ) {
    version: < "1.0.0" >,
    system: < OmniCode.Lexicon >,
    semantics: < InvariantNoun :: Place :: Memory :: RootReservoir >
}

# Invariant Noun Declaration: <PLACE:MEMORY:STOREHOUSE>
< Place : Memory : Storehouse > [ capacity: 4096 cells ] {
    base_type: < Idea : Type : Container >,
    default_capacity: 4096,
    projections: < {
        ontological: "< Storehouse >",
        structural: "[ STOREHOUSE ]",
        agentic: "{ Storehouse }"
    } >,
    machine_effect: "root_page_reservoir"
}

# ================================================================================
# BLOCK 2: OPENING BLOCK
# ================================================================================
Opening {
    requires: [
        Storehouse : InvariantNoun ::
        Container : InvariantNoun
    ],
    preconditions: [ +1 ]
}

# ================================================================================
# BLOCK 3: BODY BLOCK
# ================================================================================
Body [
    storehouse_assertion [
        [ STOREHOUSE ] [ 4096 ] :: { Storehouse } --> RESERVE {RootRAM}
    ] ( Root storehouse allocated with 4096 cells on physical memory plane )
]

# ================================================================================
# BLOCK 4: CLOSING BLOCK
# ================================================================================
Closing {
    verify: [
        storehouse_assertion = yes
    ],
    commit_status [ PROCEED- ] ( Storehouse memory invariant sealed in PROCEED )
}
```

<TRACK 2> : {OmniCode Language & Standard Library}/lexicon/01_nouns/places/memory/storehouse.tape

TAPE | 33 LINES | 209 WORDS | 1.7 KB

```
# ================================================================================
# TAPE HEAD: STOREHOUSE MEMORY DOMAIN PROCESS RIBBON
# ================================================================================
# ARCHITECTURAL TOPOLOGY:
#   Executes atomic lifecycle for Invariant Noun <PLACE:MEMORY:STOREHOUSE>.
# ================================================================================

[Tape.Storehouse] : WHO {OmniCode_Kernel} WHAT {MemoryStorehouse} --> ALLOC {RootPlate}
    WHEN {SystemGenesis}
    WHERE {storehouse} = [ 4096 ]
    WHY ( Inscribe root memory storehouse invariant and capacity reservoir )
    HOW [ +1 : PROCEED ]
    :: < storehouse_landmark > ( Malachi 3:10 - 'Bring ye all the tithes into the storehouse' )

# ================================================================================
# SUBORDINATE ROLLS: ATOMIC INVARIANT RESOLUTION
# ================================================================================

# Roll 1: Reference abstract storehouse archetype
roll_1 [ +Storehouse ] ( Abstract storehouse archetype referenced )

# Roll 2: Allocate concrete storehouse memory block
roll_2 [ Storehouse --> "Root_Storehouse" @ 0 ] ( Concrete storehouse reserved at physical origin )

# Roll 3: Assert structural projection and clarified capacity
roll_3 [ [ STOREHOUSE ] [ 4096 ] :: "4096" --> [ ROOT_RAM ] ] ( Structural 4096 cell bounds affirmed )

# Roll 4: Verify allocation bounds into result prime
roll_4 [ "Root_Storehouse" = Storehouse ! [ +1 ] --> [ storehouse_verified` ] ] ( Verify root storehouse allocation )

# Roll 5: Seal in affirmative proceed
roll_5 [ commit_status [ PROCEED- ] ] ( Memory storehouse invariant sealed and verified )
```

<TRACK 2> : {OmniCode Language & Standard Library}/lexicon/01_nouns/places/vaults/vault.omni

OMNI | 60 LINES | 237 WORDS | 2.1 KB

```
# ================================================================================
# BLOCK 1: METADATA BLOCK
# ( Invariant Noun: <PLACE:VAULT:ROM> )
# ================================================================================
# ARCHITECTURAL TOPOLOGY:
#   Defines the protected kernel vault for immutable ROM invariants & covenants.
#   - Base Block Type: < Idea : Type : Container > (Immutable ROM Envelope)
#   - Category: Place
#   - Subcategory: Vault
#   - Protection: Immutable ROM / Write-Once
# ================================================================================

Module < Lexicon.Nouns.Places.Vaults.Vault > ( Protected Kernel Vault for Immutable ROM Invariants ) {
    version: < "1.0.0" >,
    system: < OmniCode.Lexicon >,
    semantics: < InvariantNoun :: Place :: Vault :: ImmutableROM >
}

# Invariant Noun Declaration: <PLACE:VAULT:ROM>
< Place : Vault : Vault > [ capacity: 128 cells ] {
    base_type: < Idea : Type : Container >,
    protection_tier: "immutable_rom",
    projections: < {
        ontological: "< Vault >",
        structural: "[ VAULT ]",
        agentic: "{ Vault }"
    } >,
    machine_effect: "protected_rom_vault"
}

# ================================================================================
# BLOCK 2: OPENING BLOCK
# ================================================================================
Opening {
    requires: [
        Vault : InvariantNoun ::
        Container : InvariantNoun
    ],
    preconditions: [ +1 ]
}

# ================================================================================
# BLOCK 3: BODY BLOCK
# ================================================================================
Body [
    vault_assertion [
        [ VAULT ] [ 128 ] :: { Vault } --> SEAL {ImmutableInvariants}
    ] ( Protected ROM vault allocated for immutable kernel invariants )
]

# ================================================================================
# BLOCK 4: CLOSING BLOCK
# ================================================================================
Closing {
    verify: [
        vault_assertion = yes
    ],
    commit_status [ PROCEED- ] ( Vault memory invariant sealed in PROCEED )
}
```

<TRACK 2> : {OmniCode Language & Standard Library}/lexicon/01_nouns/places/vaults/vault.tape

TAPE | 33 LINES | 212 WORDS | 1.6 KB

```
# ================================================================================
# TAPE HEAD: VAULT PROTECTED ROM PROCESS RIBBON
# ================================================================================
# ARCHITECTURAL TOPOLOGY:
#   Executes atomic lifecycle for Invariant Noun <PLACE:VAULT:ROM>.
# ================================================================================

[Tape.Vault] : WHO {OmniCode_Kernel} WHAT {ProtectedVault} --> SEAL {Covenants}
    WHEN {SystemGenesis}
    WHERE {vault_rom} = [ 128 ]
    WHY ( Inscribe immutable kernel vault invariant and permanent covenant seal )
    HOW [ +1 : PROCEED ]
    :: < vault_landmark > ( Deuteronomy 31:26 - 'Put it in the side of the ark of the covenant' )

# ================================================================================
# SUBORDINATE ROLLS: ATOMIC INVARIANT RESOLUTION
# ================================================================================

# Roll 1: Reference abstract vault archetype
roll_1 [ +Vault ] ( Abstract vault archetype referenced )

# Roll 2: Allocate immutable ROM vault
roll_2 [ Vault --> "Immutable_Vault" @ 0 ] ( Inscribe immutable vault at ROM base )

# Roll 3: Assert structural projection and clarified capacity
roll_3 [ [ VAULT ] [ 128 ] :: "128" --> [ IMMUTABLE_ROM ] ] ( Structural 128 cell bounds affirmed )

# Roll 4: Verify vault inviolability into result prime
roll_4 [ "Immutable_Vault" = Vault ! [ +1 ] --> [ vault_verified` ] ] ( Verify vault matches archetype )

# Roll 5: Seal in affirmative proceed
roll_5 [ commit_status [ PROCEED- ] ] ( Vault memory invariant sealed and verified )
```

<TRACK 2> : {OmniCode Language & Standard Library}/lexicon/01_nouns/places/buffers/reservoir.omni

OMNI | 59 LINES | 235 WORDS | 2.1 KB

```
# ================================================================================
# BLOCK 1: METADATA BLOCK
# ( Invariant Noun: <PLACE:BUFFER:RESERVOIR> )
# ================================================================================
# ARCHITECTURAL TOPOLOGY:
#   Defines the dynamic heap reservoir for fluid streams and unformatted buffers.
#   - Base Block Type: < Idea : Type : Container > (Dynamic Fluid Envelope)
#   - Category: Place
#   - Subcategory: Buffer
#   - Fluid Dynamics: Elastic Buffer Stream
# ================================================================================

Module < Lexicon.Nouns.Places.Buffers.Reservoir > ( Dynamic Heap Reservoir for Fluid Data Streams ) {
    version: < "1.0.0" >,
    system: < OmniCode.Lexicon >,
    semantics: < InvariantNoun :: Place :: Buffer :: FluidReservoir >
}

# Invariant Noun Declaration: <PLACE:BUFFER:RESERVOIR>
< Place : Buffer : Reservoir > [ capacity: dynamic cells ] {
    base_type: < Idea : Type : Container >,
    projections: < {
        ontological: "< Reservoir >",
        structural: "[ RESERVOIR ]",
        agentic: "{ Reservoir }"
    } >,
    machine_effect: "fluid_buffer_reservoir"
}

# ================================================================================
# BLOCK 2: OPENING BLOCK
# ================================================================================
Opening {
    requires: [
        Reservoir : InvariantNoun ::
        Container : InvariantNoun
    ],
    preconditions: [ +1 ]
}

# ================================================================================
# BLOCK 3: BODY BLOCK
# ================================================================================
Body [
    reservoir_assertion [
        [ RESERVOIR ] [ 512 ] :: { Reservoir } --> BUFFER {FluidStream}
    ] ( Elastic heap reservoir allocated for fluid buffer streams )
]

# ================================================================================
# BLOCK 4: CLOSING BLOCK
# ================================================================================
Closing {
    verify: [
        reservoir_assertion = yes
    ],
    commit_status [ PROCEED- ] ( Reservoir buffer invariant sealed in PROCEED )
}
```

<TRACK 2> : {OmniCode Language & Standard Library}/lexicon/01_nouns/places/buffers/reservoir.tape

TAPE | 33 LINES | 207 WORDS | 1.6 KB

```
# ================================================================================
# TAPE HEAD: RESERVOIR BUFFER PROCESS RIBBON
# ================================================================================
# ARCHITECTURAL TOPOLOGY:
#   Executes atomic lifecycle for Invariant Noun <PLACE:BUFFER:RESERVOIR>.
# ================================================================================

[Tape.Reservoir] : WHO {OmniCode_Kernel} WHAT {BufferReservoir} --> STREAM {FluidData}
    WHEN {DataFlow}
    WHERE {reservoir_heap} = [ 512 ]
    WHY ( Inscribe fluid buffer reservoir invariant and stream territory )
    HOW [ +1 : PROCEED ]
    :: < reservoir_landmark > ( Ecclesiastes 1:7 - 'All the rivers run into the sea' )

# ================================================================================
# SUBORDINATE ROLLS: ATOMIC INVARIANT RESOLUTION
# ================================================================================

# Roll 1: Reference abstract reservoir archetype
roll_1 [ +Reservoir ] ( Abstract reservoir archetype referenced )

# Roll 2: Allocate dynamic heap reservoir
roll_2 [ Reservoir --> "Fluid_Heap" @ 0 ] ( Inscribe fluid heap reservoir at offset 0 )

# Roll 3: Assert structural projection and clarified capacity
roll_3 [ [ RESERVOIR ] [ 512 ] :: "512" --> [ FLUID_HEAP ] ] ( Structural 512 cell bounds affirmed )

# Roll 4: Verify reservoir responsiveness into result prime
roll_4 [ "Fluid_Heap" = Reservoir ! [ +1 ] --> [ reservoir_verified` ] ] ( Verify reservoir matches archetype )

# Roll 5: Seal in affirmative proceed
roll_5 [ commit_status [ PROCEED- ] ] ( Buffer reservoir invariant sealed and verified )
```

<TRACK 2> : {OmniCode Language & Standard Library}/lexicon/suggestions/README.omni

OMNI | 97 LINES | 666 WORDS | 5.0 KB

```
# ================================================================================
# BLOCK 1: METADATA BLOCK
# ( OmniCode Lexicon Emergence Manifest & AI-Driven Language Proposal Charter )
# ================================================================================
# ARCHITECTURAL PURPOSE:
#   This directory serves as the project-wide incubation chamber for legal keyword,
#   operator, and grammatical proposals within OmniCode.
#
# WHY THE AI CAN EXPAND THE LEXICON IN A LOCAL INSTANCE:
#   Unlike conventional compiled languages (C, Rust, Python) where keywords and
#   operators are hardcoded into static compiler syntax trees, OmniCode is
#   ontologically grounded and homoiconic:
#   1. Every keyword and operator is defined as a first-class declarative document
#      in the Lexicon (<TRACK 2>/lexicon/).
#   2. The Lexicon engine dynamically ingests these definitions, maps their
#      parts of speech, polarities (+ vs -), boundary ranks, and machine effects.
#   3. The OmniAssembler lowers them into concrete 64-bit balanced-ternary machine
#      words (Word64) and verifies delimiter stack integrity via OmniDebugger.
#   4. An AI running in a local instance can formulate emergent, domain-specific
#      keywords to build a bare-metal OS without hacking the compiler binary.
#
# THE GOLDEN CANONS OF OMNICODE LANGUAGE DESIGN:
#   CANON 1: MORPHEMIC COMPOUNDING
#     If an action can be expressed through natural compounding of base symbols
#     (~ like, ! but, :: and, @ at), it must be an operator (e.g. ::@, ~!).
#
#   CANON 2: UNIVERSAL HUMAN VERBS (ACCESSIBILITY FOR NON-CODERS)
#     If an action requires a verb, choose an everyday human word usable across
#     all domains (OS, Games, UI, AI). Never use developer jargon (spawn, probe, fork).
#     - Use SENSE instead of Probe (every living creature senses its world).
#     - Use AWAKEN instead of Spawn (transitioning from dormancy to active life).
#     - Use PREEMPT instead of Yield/Trap (taking turns with priority).
#     - Use PROVE instead of BIST/Assert (verifying balance before proceeding).
#
#   CANON 3: BOUNDARY TIER DISAMBIGUATION (THE COMPARISON RULE)
#     Because < ... > is the Rank 1 Invariant delimiter, bare `<` and `>` are
#     boundary brackets. All relational comparisons must be cast in context:
#     [ { A } < { B } ]
#     - [ ... ] denotes a concrete evaluation block.
#     - { ... } casts A and B into Context values.
#     - Uncasted `<=>` represents an Invariant of Equivalence (< => >).
# ================================================================================

Module < Lexicon.Suggestions.Manifest > ( Project-wide lexicon emergence charter ) {
    version: < "1.0.0" >,
    system: < OmniCode.Lexicon.Incubator >,
    semantics: < MorphemicOperators :: UniversalVerbs :: NonCoderAccessibility >
}

# ================================================================================
# BLOCK 2: OPENING BLOCK
# ( Incubation Categories, Boundary Ranking Hierarchy & Typology )
# ================================================================================
Opening {
    requires: [
        Rank1_Invariants : "< ... > (Absolute Immutable Laws & Types)" ::
        Rank2_Denotations : "[ ... ] (Concrete Silicon Evaluation Blocks)" ::
        Rank3_Contexts   : "{ ... } (Environmental Attributes & State Values)" ::
        Rank4_Asides     : "( ... ) (Teleological Intent & Secondary Checks)"
    ],
    preconditions: [ +1 : PROCEED ]
}

# ================================================================================
# BLOCK 3: BODY BLOCK
# ( Incubation Registry & Proposal Manifests )
# ================================================================================
Body [
    # Proposal 1: Morphological Symbolic Operators (operators/morphological_operators.omni)
    proposal_operators [
        SituatedAt_Binding  [ "::@" [ ChannelBind & AddressOffset ] ] ::
        CaveatInterrupt_Hop [ "~!"  [ ApproximateLike & InvariantCaveat ] ] ::
        ContextComparison   [ "[ { A } < { B } ]" [ CastedContextRelation ] ] ::
```

```
        InvariantBalance    [ "<=>" [ InvariantEquivalenceState ] ]
    ] ( Symbolic operator proposals compounding from atomic morphemes )

    # Proposal 2: Universal Human Verbs (03_verbs/universal_verbs.omni)
    proposal_verbs [
        Verb_Preempt [ "PREEMPT" [ PriorityInterruption & TaskTurnTaking ] ] ::
        Verb_Sense   [ "SENSE"   [ EnvironmentalDiscovery & HardwarePerception ] ] ::
        Verb_Awaken  [ "AWAKEN"  [ ColdStartInception & ProcessActivation ] ] ::
        Verb_Prove   [ "PROVE"   [ BalancedTernarySelfTest & InvariantValidation ] ]
    ] ( Universal human verbs replacing developer jargon )
]

# ================================================================================
# BLOCK 4: CLOSING BLOCK
# ( Tripartite Consensus & Covenant Sealing )
# ================================================================================
Closing {
    verify: [
        proposal_operators = yes ::
        proposal_verbs = yes
    ],
    commit_status [ PROCEED ] ( Suggestion incubator sealed and active )
}
```

<TRACK 2> : {OmniCode Language & Standard Library}/lexicon/suggestions/03_verbs/universal_verbs.omni

OMNI | 118 LINES | 617 WORDS | 4.9 KB

```
# ================================================================================
# BLOCK 1: METADATA BLOCK
# ( Ontological Verbs: Universal Human Verbs for Accessible System Architecture )
# ================================================================================
# ARCHITECTURAL TOPOLOGY:
#   Defines four foundational human verbs designed for universal cross-domain usage:
#   - PREEMPT : Priority interruption & turn-taking (replaces technical scheduler jargon).
#   - SENSE   : Environmental perception & hardware discovery (replaces invasive developer jargon "Probe").
#   - AWAKEN  : Cold-start inception & life activation (replaces gamer/Unix jargon "Spawn" / "Fork").
#   - PROVE   : Invariant validation & equilibrium self-test (replaces developer acronyms "BIST" / "Assert").
#
# ACCESSIBILITY FOR NON-CODERS:
#   Every verb defined here is a standard, intuitive word understood by any human speaker:
#   - A human understands what it means to "sense" a room or "awaken" a dormant console.
#   - A human understands what it means to "preempt" a workflow for something urgent.
#   - A human understands what it means to "prove" balance before committing a decision.
# ================================================================================

Module < Lexicon.Suggestions.Verbs.Universal > ( Universal human verbs for accessible system architecture ) {
    version: < "1.0.0" >,
    system: < OmniCode.Lexicon >,
    semantics: < UniversalVerbs :: NonCoderAccessibility :: CrossDomainActions >
}

# 1. Action: < PREEMPT >
Preempt < InvariantVerb > ( Priority Interruption & Turn-Taking ) {
    grammar_primary: < "Verb" >,
    grammar_secondary: < "Conjunction" >,
    abstract_form: < "+Preempt" >,
    concrete_form: < "Preempt-" >,
    universal_scope: < "OS.Scheduler :: UI.Modals :: Audio.Ducking :: AI.Safety" >,
    machine_effect: < "schedule_quantum_preemption" >
}

# 2. Action: < SENSE >
Sense < InvariantVerb > ( Environmental Perception & Hardware Discovery ) {
    grammar_primary: < "Verb" >,
    grammar_secondary: < "Noun" >,
    abstract_form: < "+Sense" >,
    concrete_form: < "Sense-" >,
    universal_scope: < "OS.MemoryMap :: Game.Collision :: UI.Theme :: IoT.Sensors" >,
    machine_effect: < "perceive_environmental_state" >
}

# 3. Action: < AWAKEN >
Awaken < InvariantVerb > ( Cold-Start Inception & Life Activation ) {
    grammar_primary: < "Verb" >,
    grammar_secondary: < "Noun" >,
    abstract_form: < "+Awaken" >,
    concrete_form: < "Awaken-" >,
    universal_scope: < "OS.ProcessInception :: UI.StudioLaunch :: Game.Entities :: AI.Memory" >,
    machine_effect: < "instate_active_execution_life" >
}

# 4. Action: < PROVE >
Prove < InvariantVerb > ( Invariant Validation & Equilibrium Certification ) {
    grammar_primary: < "Verb" >,
    grammar_secondary: < "Noun" >,
    abstract_form: < "+Prove" >,
    concrete_form: < "Prove-" >,
    universal_scope: < "OS.ALU_BIST :: Security.Covenants :: Math.Balance :: Invariant.Checks" >,
    machine_effect: < "certify_invariant_truth" >
}

# ================================================================================
# BLOCK 2: OPENING BLOCK
# ( Action Dependencies & Operational Preconditions )
# ================================================================================
Opening {
    requires: [
        Preempt : InvariantVerb ::
        Sense : InvariantVerb ::
        Awaken : InvariantVerb ::
```

```
        Prove : InvariantVerb
    ],
    preconditions: [ +1 : PROCEED ]
}

# ============================================================================
# BLOCK 3: BODY BLOCK
# ( Cross-Domain Demonstrations of Universal Verbs )
# ============================================================================
Body [
    # Demo 1: Preempt action in scheduling & urgent workflow
    preempt_demo [
        Scheduler_Tick [ Timer_IRQ --> PREEMPT {Current_Task} ]
    ] ( Preempt active execution quantum in favor of higher-priority need )

    # Demo 2: Sense action in environmental hardware discovery
    sense_demo [
        Hardware_Discovery [ Kernel --> SENSE {Root_RAM} ]
    ] ( Sense physical memory topology from silicon firmware )

    # Demo 3: Awaken action in cold-start process inception
    awaken_demo [
        Session_Launch [ Kernel --> AWAKEN {Terminal_Studio} ]
    ] ( Awaken dormant console interface into living interaction )

    # Demo 4: Prove action in balanced-ternary ALU verification
    prove_demo [
        Arithmetic_Proof [ Kernel --> PROVE {ALU_Equilibrium} ]
    ] ( Prove algebraic symmetry before allocating dynamic memory )
]

# ============================================================================
# BLOCK 4: CLOSING BLOCK
# ( Invariant Audit & Covenant Commit Seal )
# ============================================================================
Closing {
    verify: [
        preempt_demo = yes ::
        sense_demo = yes ::
        awaken_demo = yes ::
        prove_demo = yes
    ],
    commit_status [ PROCEED ] ( Universal human verb proposals certified and sealed )
}
```

<TRACK 2> : {OmniCode Language & Standard Library}/lexicon/suggestions/03_verbs/universal_verbs.tape

TAPE | 49 LINES | 296 WORDS | 2.5 KB

```
# ================================================================================
# BLOCK 1: METADATA BLOCK
# ( Universal Verbs Proposal Process Ribbon )
# ================================================================================
# ARCHITECTURAL TOPOLOGY:
#   Executes operational test rolls validating the 4 universal human verbs:
#   - PREEMPT (turn-taking / priority preemption)
#   - SENSE   (environmental perception & hardware discovery)
#   - AWAKEN  (cold-start inception & active life activation)
#   - PROVE   (balanced-ternary invariant certification)
# ================================================================================

[Tape.UniversalVerbs] : WHO {OmniCode_Kernel} WHAT {HumanVerbs} --> TEST {Lexicon}
    WHEN {IncubationTrial}
    WHERE {verbs_partition} = [ 256 ]
    WHY ( Validate universal human verbs across OS and accessible user domains )
    HOW [ +1 : PROCEED ]
    :: < verbs_landmark > ( Proverbs 4:7 - 'Wisdom is the principal thing; therefore get wisdom' )

# ================================================================================
# BLOCK 2: OPENING BLOCK
# ( Environmental Setup & Test Partition Bounds )
# ================================================================================

# ================================================================================
# BLOCK 3: BODY BLOCK
# ( Test Instruction Rolls Exercising Proposed Universal Verbs )
# ================================================================================

# Roll 1: Sense physical memory layout from system firmware
roll_1 [ Kernel --> SENSE {Root_RAM} ] ( Sense available silicon memory frames )

# Roll 2: Prove balanced-ternary arithmetic symmetry in hardware registers
roll_2 [ Kernel --> PROVE {ALU_Equilibrium} ] ( Prove algebraic balance before allocating memory )

# Roll 3: Preempt active background task when timer tick arrives
roll_3 [ Timer_Tick --> PREEMPT {Background_Task} ] ( Preempt task slice to maintain system responsiveness )

# Roll 4: Awaken the interactive terminal session into active life
roll_4 [ Kernel --> AWAKEN {Terminal_Studio} ] ( Awaken console HUD and user interface )

# ================================================================================
# BLOCK 4: CLOSING BLOCK
# ( Verification & Affirmative Commit Seal )
# ================================================================================

# Roll 5: Commit universal verbs proposal tape
roll_5 [ commit_status [ PROCEED ] ] ( Universal verbs proposal tape verified and sealed )
```

<TRACK 2> : {OmniCode Language & Standard Library}/lexicon/suggestions/operators/morphological_operators.omni

OMNI | 121 LINES | 686 WORDS | 5.3 KB

```
# ================================================================================
# BLOCK 1: METADATA BLOCK
# ( Ontological Operators: Morphological Compounding & Contextual Relational Syntax )
# ================================================================================
# ARCHITECTURAL TOPOLOGY:
#   Defines compound operators that naturally evolve from base morphemes:
#   - ::@ : SituatedAt (Channel bind `::` + Location `@` -> "and at location").
#   - ~!  : CaveatInterrupt (Approximate `~` + Adversative `!` -> "like normal, BUT").
#   - [ { A } < { B } ] : CastedContextComparison (Disambiguated relational comparison).
#   - <=> : InvariantEquivalence (Uncasted invariant equivalence axiom < = >).
#
# THE BOUNDARY TIER COMPARISON CANON:
#   Why must comparison be written as `[ { A } < { B } ]`?
#   1. In OmniCode's 4-tier delimiter ranking, `< ... >` is the Rank 1 Invariant.
#   2. If a coder writes `A < B`, the lexer and parser interpret `<` as the opening
#      of an immutable Invariant tier, causing a syntax collision.
#   3. Therefore, comparison MUST be enclosed in a Rank 2 Denotation block `[ ... ]`,
#      with the compared entities cast into Rank 3 Context brackets `{ ... }`.
#   4. This disambiguates `<` as a relational comparator rather than an Invariant opening.
#   5. Conversely, uncasted `<=>` literally means `< = >`: an Invariant of Equivalence!
# ================================================================================

Module < Lexicon.Suggestions.Operators.Morphological > ( Morphological compound operators and relational syntax ) {
    version: < "1.0.0" >,
    system: < OmniCode.Lexicon >,
    semantics: < MorphemicOperators :: ContextualComparison :: InvariantEquivalence >
}

# 1. Situated-At Operator: < SITUATED_AT >
SituatedAt < InvariantConjunction > ( Compound Conjunction: ::@ - 'and at location' ) {
    grammar_primary: < "Conjunction" >,
    grammar_secondary: < "Preposition" >,
    symbol: < "::@" >,
    abstract_form: < "+SituatedAt" >,
    concrete_form: < "SituatedAt-" >,
    machine_effect: < "hardware_mmio_port_binding" >
}

# 2. Caveat-Interrupt Operator: < CAVEAT_INTERRUPT >
CaveatInterrupt < InvariantConjunction > ( Compound Conjunction: ~! - 'like normal, BUT...' ) {
    grammar_primary: < "Conjunction" >,
    grammar_secondary: < "Preposition" >,
    symbol: < "~!" >,
    abstract_form: < "+CaveatInterrupt" >,
    concrete_form: < "CaveatInterrupt-" >,
    machine_effect: < "asynchronous_preemption_catch" >
}

# 3. Contextual Comparison Operator: < CONTEXT_RELATION >
ContextRelation < InvariantOperator > ( Casted Relational Comparison: [ { A } < { B } ] ) {
    grammar_primary: < "Operator" >,
    grammar_secondary: < "Adjective" >,
    syntax_pattern: < "[ { A } < { B } ]" >,
    machine_effect: < "balanced_ternary_three_way_comparison" >
}

# 4. Invariant Equivalence Axiom: < INVARIANT_EQUIVALENCE >
InvariantEquivalence < InvariantOperator > ( Uncasted Invariant Equivalence: <=> ) {
    grammar_primary: < "Operator" >,
    grammar_secondary: < "Noun" >,
    symbol: < "<=>" >,
    abstract_form: < "+InvariantEquivalence" >,
    concrete_form: < "InvariantEquivalence-" >,
    machine_effect: < "immutable_equivalence_proof" >
}

# ================================================================================
# BLOCK 2: OPENING BLOCK
# ( Operational Dependencies & Hardware Binding Preconditions )
# ================================================================================
Opening {
    requires: [
        SituatedAt : InvariantConjunction ::
```

```
        CaveatInterrupt : InvariantConjunction ::
        ContextRelation : InvariantOperator ::
        InvariantEquivalence : InvariantOperator
    ],
    preconditions: [ +1 : PROCEED ]
}

# ================================================================================
# BLOCK 3: BODY BLOCK
# ( Verification Rolls & Concrete Syntax Demonstrations )
# ================================================================================
Body [
    # Demo 1: Situated-At binding of MMIO and Port registers (::@)
    mmio_binding_demo [
        Serial_Port [ COM1 ::@ 0x03F8 ] ::
        Screen_Port [ Screen_VGA ::@ 0x000B8000 ]
    ] ( Inscribe hardware ports situated at concrete silicon addresses )

    # Demo 2: Caveat-Interrupt preemption (~!)
    interrupt_caveat_demo [
        Standard_Process [ Task_Compute ~! [ Timer_Tick --> Preempt ] ]
    ] ( Run task like normal, BUT preempt if timer tick fires )

    # Demo 3: Casted Contextual Comparison ([ { A } < { B } ])
    comparison_demo [
        Bounds_Check [ { Current_Offset } < { Partition_Limit } ]
    ] ( Relational comparison cleanly disambiguated within concrete denotation )

    # Demo 4: Uncasted Invariant Equivalence (<=>)
    equivalence_demo [
        Symmetry_Proof [ (+1) plus (-1) <=> 0 ]
    ] ( Uncasted <=> asserts an invariant state of algebraic equilibrium )
]

# ================================================================================
# BLOCK 4: CLOSING BLOCK
# ( Structural Proof Verification & Covenant Commit Seal )
# ================================================================================
Closing {
    verify: [
        mmio_binding_demo = yes ::
        interrupt_caveat_demo = yes ::
        comparison_demo = yes ::
        equivalence_demo = yes
    ],
    commit_status [ PROCEED ] ( Morphological operator proposals certified and sealed )
}
```

<TRACK 2> : {OmniCode Language & Standard Library}/lexicon/suggestions/operators/morphological_operators.tape

TAPE | 49 LINES | 318 WORDS | 2.5 KB

```
# ================================================================================
# BLOCK 1: METADATA BLOCK
# ( Morphological Operator Proposal Process Ribbon )
# ================================================================================
# ARCHITECTURAL TOPOLOGY:
#   Executes operational test rolls validating compound operators:
#   - ::@ (Situated-At port/MMIO binding)
#   - ~!  (Caveat-Interrupt asynchronous preemption)
#   - [ { A } < { B } ] (Casted Context Comparison)
#   - <=> (Invariant Equivalence Axiom)
# ================================================================================

[Tape.MorphologicalOperators] : WHO {OmniCode_Kernel} WHAT {CompoundOperators} --> TEST {Lexicon}
    WHEN {IncubationTrial}
    WHERE {test_partition} = [ 256 ]
    WHY ( Validate compound morphemes and casted contextual comparison syntax )
    HOW [ +1 : PROCEED ]
    :: < operator_landmark > ( Ecclesiastes 3:1 - 'To every thing there is a season' )

# ================================================================================
# BLOCK 2: OPENING BLOCK
# ( Environmental Setup & Test Partition Bounds )
# ================================================================================

# ================================================================================
# BLOCK 3: BODY BLOCK
# ( Test Instruction Rolls Exercising Proposed Operators )
# ================================================================================

# Roll 1: Bind serial COM1 and VGA ports situated at concrete hardware addresses via ::@
roll_1 [ COM1 ::@ 0x03F8 :: Screen_VGA ::@ 0x000B8000 ] ( Hardware ports bound situated at physical addresses )

# Roll 2: Execute task like normal BUT preempt on timer interrupt via ~!
roll_2 [ Kernel_Compute ~! [ Timer_IRQ --> Preempt ] ] ( Process runs like normal BUT yields on timer tick )

# Roll 3: Evaluate casted contextual comparison within concrete denotation block
roll_3 [ Bounds_Test [ { 16 } < { 64 } ] ] ( Relational comparison safely evaluated inside context brackets )

# Roll 4: Inscribe invariant equivalence axiom proving balanced ternary symmetry
roll_4 [ Balanced_Proof [ (+1) plus (-1) <=> 0 ] ] ( Invariant equivalence asserted )

# ================================================================================
# BLOCK 4: CLOSING BLOCK
# ( Verification & Affirmative Commit Seal )
# ================================================================================

# Roll 5: Commit operator proposal tape
roll_5 [ commit_status [ PROCEED ] ] ( Morphological operators proposal tape verified and sealed )
```

<TRACK 2> : {OmniCode Language & Standard Library}/lexicon/operators/boot_kernel.omni

OMNI | 91 LINES | 441 WORDS | 3.9 KB

```
# ========================================================================
# BLOCK 1: METADATA BLOCK
# ( Ontological Operators: Boot Inception & Anti-Lockout Sentinel Invariants )
# ========================================================================
# ARCHITECTURAL TOPOLOGY:
#   Establishes the primordial Hardware Boot Operator (--> BOOT) and Anti-Lockout
#   Guardians within the balanced ternary execution engine:
#   - Boot Operator (--> BOOT) : Hardware root inception vector bound exclusively to Kernel authority.
#   - Watchdog Sentinel (Watchdog_Guard) : Preemptive cycle counter preventing AI infinite loops.
#   - Kernel Shield (Kernel_Partition_Barrier) : Memory protection preventing AI privilege escalation.
# ========================================================================

Module < Lexicon.Operators.BootKernel > ( Boot inception operator and anti-lockout safety invariants ) {
    version: < "1.0.0" >,
    system: < OmniCode.Lexicon >,
    semantics: < HardwareBootInception :: AntiLockoutWatchdog :: KernelShieldAuthority >
}

# 1. Invariant Operator: < BOOT_OPERATOR > (--> BOOT)
Boot_Operator < InvariantOperator > ( Primordial Hardware Inception Vector ) {
    symbol: < "--> BOOT" >,
    grammar_primary: < "Verb" >,
    grammar_secondary: < "Noun" >,
    authority_required: < "OmniCode_Kernel" >,
    channel_binding: < "Track 0 [KERNEL]" >,
    cost: < 1 >,
    memory_footprint: < "root hardware vector" >,
    machine_effect: < "hardware_cold_boot_vector" >
}

# 2. Invariant Operator: < ANTI_LOCKOUT_SENTINEL >
Anti_Lockout_Sentinel < InvariantOperator > ( Execution Watchdog & Preemption Safeguard ) {
    symbol: < "Watchdog_Guard" >,
    grammar_primary: < "Adjective" >,
    grammar_secondary: < "Noun" >,
    max_loop_threshold: < 1000 >,
    cyclic_retroaction_limit: < 100 >,
    failsafe_state: < "AWAIT :: ROLLBACK" >,
    machine_effect: < "preemptive_scheduler_watchdog" >
}

# 3. Invariant Operator: < KERNEL_SHIELD >
Kernel_Shield < InvariantOperator > ( Memory Partition & MMIO Access Barrier ) {
    symbol: < "Kernel_Partition_Barrier" >,
    grammar_primary: < "Preposition" >,
    grammar_secondary: < "Noun" >,
    protected_regions: < [ 0x00000000..0x000FFFFF, "Track 0 [KERNEL]" ] >,
    unauthorized_action: < "Failpoint [AuthorityViolation] --> ROLLBACK" >,
    machine_effect: < "hardware_memory_protection_ring" >
}

# ========================================================================
# BLOCK 2: OPENING BLOCK
# ========================================================================
Opening {
    requires: [
        Boot_Operator : InvariantOperator ::
        Anti_Lockout_Sentinel : InvariantOperator ::
        Kernel_Shield : InvariantOperator
    ],
    preconditions: [ +1 ]
}

status_equilibrium [ yes ] ( Boot operator and anti-lockout guard active )

# ========================================================================
# BLOCK 3: BODY BLOCK
# ========================================================================
Body [
    # 1. Hardware Boot Inception Verification
    boot_verification [ OmniCode_Kernel --> BOOT {Hardware} ] ( Exclusive kernel authority boots root substrate )

    # 2. Anti-Lockout Loop Safeguard Verification
```

```
    watchdog_check [ Loop_Count < Anti_Lockout_Sentinel.max_loop_threshold ! [ AWAIT ] ] ( Watchdog trips before machine lockup )

    # 3. Kernel Shield Memory Protection Verification
    shield_check [ AI_Process != Kernel_Shield.protected_regions ! [ ROLLBACK ] ] ( Sandboxed AI cannot breach kernel memory )
]

# ================================================================================
# BLOCK 4: CLOSING BLOCK
# ================================================================================
Closing {
    verify: [
        boot_verification = yes ::
        watchdog_check = yes ::
        shield_check = yes
    ],
    commit_status [ PROCEED- ] ( Boot operator and anti-lockout sentinel sealed in concrete execution )
}
```

<TRACK 2> : {OmniCode Language & Standard Library}/lexicon/operators/boot_kernel.tape

TAPE | 37 LINES | 264 WORDS | 2.0 KB

```
# ================================================================================
# TAPE HEAD: THE BOOT INCEPTION & ANTI-LOCKOUT OPERATOR PROCESS RIBBON
# ================================================================================
# ARCHITECTURAL TOPOLOGY:
#   Executes the hardware boot inception sequence while actively enforcing
#   anti-lockout watchdog invariants against infinite loops and AI memory leakage.
# ================================================================================

[Tape.BootKernelOperator] : WHO {OmniCode_Kernel} WHAT {BootOperator} --> BOOT {Hardware}
    WHEN {PowerOn}
    WHERE {boot_partition} = [ 256 ]
    WHY ( Instate root machine execution while securing OS against AI deadlock or lockout )
    HOW [ +1 : PROCEED ]
    :: < boot_landmark > ( Psalms 119:105 - 'A lamp unto my feet, and a light unto my path' )

# ================================================================================
# SUBORDINATE ROLLS: BOOT OPERATOR & WATCHDOG SENTINEL EXECUTION
# ================================================================================

# Roll 1: Assert Kernel authority on Track 0
roll_1 [ OmniCode_Kernel --> Track_0_Active @ 0 ] ( Establish supervisor authority channel )

# Roll 2: Inscribe Anti-Lockout Watchdog ceiling
roll_2 [ Watchdog_Threshold [ 1000 ] ] ( Maximum loop cycles before watchdog preempts )

# Roll 3: Direct video console banner stream to bare-metal VGA buffer
roll_3 [ 'OMNICODE 32-TRIT OS ONLINE' --> Screen_VGA @ 0x000B8000 ] ( Direct MMIO character stream to VGA buffer )

# Roll 4: Inscribe kernel memory partition shield
roll_4 [ Kernel_Partition_Shield [ Root_RAM v-- Kernel_Partition [ 1024 ] ] ] ( Protect kernel from user/AI intrusion )

# Roll 5: Verify anti-lockout loop preemption tripwire
roll_5 [ Watchdog_Active = 1 ! [ AWAIT ] ] ( Verify tripwire halts runaway execution )

# Roll 6: Genesis Boot Complete with concrete commit
roll_6 [ commit_status [ PROCEED- ] ] ( Boot operator and anti-lockout verified )
```

<TRACK 2> : {OmniCode Language & Standard Library}/lexicon/operators/casting.omni

OMNI | 88 LINES | 393 WORDS | 3.1 KB

```
# ================================================================================
# BLOCK 1: METADATA BLOCK
# ( Boundary Casting Invariants: Molds & Dies for Ontological Transduction )
# ================================================================================
# ARCHITECTURAL TOPOLOGY:
#   This reel establishes the 6 canonical boundary casting operators:
#   - [+] : Denotative Abstract Cast (cast essence to archetype)
#   - [-] : Denotative Concrete Cast (cast essence to physical memory)
#   - <+> : Invariant Abstract Cast (cast to immutable universal axiom)
#   - <-> : Invariant Concrete Cast (cast to immutable hardware/ROM constant)
#   - {+} : Connotative Abstract Cast (cast context to ideal universe)
#   - {-} : Connotative Concrete Cast (cast context to physical execution partition)
# ================================================================================

Module < Lexicon.Operators.Casting > ( Boundary casting molds and transduction operators ) {
    version: < "1.0.0" >,
    system: < OmniCode.Lexicon >,
    semantics: < BoundaryCasting :: MorphologicalMolds :: TransductionGates >
}

# 1. Denotation Casts
Denotation_Abstract < InvariantCast > ( Cast essence to abstract archetype ) {
    syntax: < "[+]" >,
    target_rank: < 2 >,
    polarity: < "+" >
}

Denotation_Concrete < InvariantCast > ( Cast essence to concrete memory substance ) {
    syntax: < "[-]" >,
    target_rank: < 2 >,
    polarity: < "-" >
}

# 2. Invariant Casts
Invariant_Abstract < InvariantCast > ( Cast boundary to transcendental immutable axiom ) {
    syntax: < "<+>" >,
    target_rank: < 1 >,
    polarity: < "+" >
}

Invariant_Concrete < InvariantCast > ( Cast boundary to immutable physical ROM constraint ) {
    syntax: < "<->" >,
    target_rank: < 1 >,
    polarity: < "-" >
}

# 3. Connotation Casts
Connotation_Abstract < InvariantCast > ( Cast environment to conceptual universe ) {
    syntax: < "{+}" >,
    target_rank: < 3 >,
    polarity: < "+" >
}

Connotation_Concrete < InvariantCast > ( Cast environment to concrete silicon partition ) {
    syntax: < "{-}" >,
    target_rank: < 3 >,
    polarity: < "-" >
}

# ================================================================================
# BLOCK 2: OPENING BLOCK
# ================================================================================
Opening {
    requires: [
        Denotation_Abstract : InvariantCast ::
        Denotation_Concrete : InvariantCast
    ],
    preconditions: [ +1 ]
}

# ================================================================================
# BLOCK 3: BODY BLOCK
# ================================================================================
```

```
Body [
    # Verify freeform casting across boundaries
    cast_gate [ [+] --> [-] ] ( Transduction: Abstract essence poured into concrete form )
]

# ================================================================================
# BLOCK 4: CLOSING BLOCK
# ================================================================================
Closing {
    verify: [
        cast_gate = yes
    ],
    commit_status [ PROCEED- ] ( Casting engine active and verified )
}
```

<TRACK 2> : {OmniCode Language & Standard Library}/lexicon/operators/casting.tape

TAPE | 30 LINES | 170 WORDS | 1.4 KB

```
# ================================================================================
# TAPE HEAD: THE BOUNDARY CASTING PROCESS RIBBON
# ================================================================================
# ARCHITECTURAL TOPOLOGY:
#   Executes freeform boundary casting across all canonical ranks.
# ================================================================================

[Tape.BoundaryCasting] : WHO {OmniCode_Kernel} WHAT {Casting} --> EXECUTE {FoundryMolds}
    WHEN {GenesisBoot}
    WHERE {casting_partition} = [ 256 ]
    WHY ( Inscribe verified boundary casting physics )
    HOW [ +1 : PROCEED ]
    :: < mold_landmark > ( Job 38:14 - 'It is turned as clay to the seal' )

# ================================================================================
# SUBORDINATE ROLLS: CASTING OPERATION ROLLS
# ================================================================================

# Roll 1: Transduce Denotation from abstract to concrete
roll_1 [ [+] --> [-] ] ( Denotative foundry cast )

# Roll 2: Transduce Invariant boundary to concrete ROM constant
roll_2 [ <+> --> <-> ] ( Invariant axiom solidified into hardware constraint )

# Roll 3: Transduce Connotation from ideal to physical partition
roll_3 [ {+} --> {-} ] ( Ambient context grounded into physical memory )

# Roll 4: Seal casting tape
roll_4 [ commit_status [ PROCEED- ] ] ( Casting covenant sealed )
```

<TRACK 2> : {OmniCode Language & Standard Library}/lexicon/operators/polarity.omni

OMNI | 74 LINES | 338 WORDS | 3.0 KB

```
# ========================================================================
# BLOCK 1: METADATA BLOCK
# ( Ontological Polarity Invariants: Abstract (+) vs Concrete (-) Ground Truth )
# ========================================================================
# ARCHITECTURAL TOPOLOGY:
#   This reel establishes the primordial duality of being in OmniCode:
#   - Abstract (+) : The uninstantiated intension, archetype, and zero-cost blueprint.
#   - Concrete (-) : The physical extension, allocated memory substance, and operational reality.
# ========================================================================

Module < Lexicon.Operators.Polarity > ( Primordial ontological polarity operators ) {
    version: < "1.0.0" >,
    system: < OmniCode.Lexicon >,
    semantics: < PolarityPhysics :: AbstractArchetypes :: ConcreteSubstances >
}

# 1. Invariant Operator: < ABSTRACT > (+)
Abstract_Polarity < InvariantOperator > ( Universal Archetype / Uninstantiated Essence ) {
    symbol: < "+" >,
    grammar_primary: < "Adjective" >,
    grammar_secondary: < "Noun" >,
    cost: < 0 >,
    memory_footprint: < "zero-cost symbolic form" >,
    machine_effect: < "type_specification_and_contract" >
}

# 2. Invariant Operator: < CONCRETE > (-)
Concrete_Polarity < InvariantOperator > ( Physical Substance / Allocated Instance ) {
    symbol: < "-" >,
    grammar_primary: < "Noun" >,
    grammar_secondary: < "Adjective" >,
    cost: < 1 >,
    memory_footprint: < "physical silicon RAM cells" >,
    machine_effect: < "allocated_memory_and_bus_state" >
}

# ========================================================================
# BLOCK 2: OPENING BLOCK
# ( Syntactic Preconditions & Affix Attachment Contracts )
# ========================================================================
Opening {
    requires: [
        Abstract_Polarity : InvariantOperator ::
        Concrete_Polarity : InvariantOperator
    ],
    preconditions: [ +1 ]
}

status_equilibrium [ yes ] ( Polarity engine active in nominal equilibrium )

# ========================================================================
# BLOCK 3: BODY BLOCK
# ( Polarity Modulations Mapped to Architectural Types )
# ========================================================================
Body [
    # 1. Affirmation of Dual Polarity
    polarity_check [ Abstract_Polarity != Concrete_Polarity ] ( Duality: Abstract is distinct from Concrete )

    # 2. Transduction Verification
    transduction_vector [ +Dog --> Dog- ] ( Lowering vector: Abstract idea transduced to Concrete substance )
]

# ========================================================================
# BLOCK 4: CLOSING BLOCK
# ( Invariant Verification & Permanent Affirmative Commit )
# ========================================================================
Closing {
    verify: [
        polarity_check = yes ::
        transduction_vector = yes
    ],
```

```
    commit_status [ PROCEED- ] ( Polarity operators sealed in concrete execution )
}
```

<TRACK 2> : {OmniCode Language & Standard Library}/lexicon/operators/polarity.tape

TAPE | 30 LINES | 179 WORDS | 1.5 KB

```
# ================================================================================
# TAPE HEAD: THE POLARITY TRANSFORMATION PROCESS RIBBON
# ================================================================================
# ARCHITECTURAL TOPOLOGY:
#   Executes the dynamic shift between abstract definitions and concrete memory.
# ================================================================================

[Tape.PolarityTransformation] : WHO {OmniCode_Kernel} WHAT {Polarity} --> ESTABLISH {DualStatePhysics}
    WHEN {GenesisBoot}
    WHERE {cortex_partition} = [ 256 ]
    WHY ( Prove seamless transmutation between archetype and substance )
    HOW [ +1 : PROCEED ]
    :: < polarity_landmark > ( Ecclesiastes 3:1 - 'To every thing there is a season' )

# ================================================================================
# SUBORDINATE ROLLS: POLARITY AFFIX EXECUTION & VERIFICATION
# ================================================================================

# Roll 1: Reference abstract entity without memory allocation
roll_1 [ +Archetype ] ( Abstract reference: zero memory cost )

# Roll 2: Allocate concrete instance in physical partition
roll_2 [ Concrete- [ 1024 ] ] ( Concrete allocation: physical storehouse footprint )

# Roll 3: Execute directional transmutation from abstract to concrete
roll_3 [ +Archetype --> Concrete- ] ( Direct transmutation of archetype into substance )

# Roll 4: Verify macro keyword in concrete postfix execution
roll_4 [ commit_status [ PROCEED- ] ] ( Concrete execution commit confirmed )
```

<TRACK 2> : {OmniCode Language & Standard Library}/lexicon/02_pronouns/self_handles.omni

OMNI | 81 LINES | 338 WORDS | 2.9 KB

```
# ================================================================================
# BLOCK 1: METADATA BLOCK
# ( Ontological Pronouns: Context Handles & Pivot Pins )
# ================================================================================
# ARCHITECTURAL TOPOLOGY:
#   Defines context-relative pointers and local pivot pins:
#   - self   : The currently executing frame or active entity instance.
#   - caller : The upstream activating frame or invoking principal.
#   - owner  : The sovereign allocation custodian holding memory boundary rights.
# ================================================================================

Module < Lexicon.Pronouns.SelfHandles > ( Local pivot pins and frame-relative context handles ) {
    version: < "1.0.0" >,
    system: < OmniCode.Lexicon >,
    semantics: < SelfHandles :: ContextPointers :: FrameResolution >
}

# 1. Local Handle: < SELF >
Self < InvariantPronoun > ( The Currently Executing Local Frame Pointer: self ) {
    grammar_primary: < "Pronoun" >,
    grammar_secondary: < "Noun" >,
    symbol: < "self" >,
    abstract_form: < "+Self" >,
    concrete_form: < "Self-" >,
    machine_effect: < "current_frame_handle" >
}

# 2. Invoker Handle: < CALLER >
Caller < InvariantPronoun > ( The Upstream Activating Frame Pointer: caller ) {
    grammar_primary: < "Pronoun" >,
    grammar_secondary: < "Noun" >,
    symbol: < "caller" >,
    abstract_form: < "+Caller" >,
    concrete_form: < "Caller-" >,
    machine_effect: < "upstream_invoker_handle" >
}

# 3. Custodian Handle: < OWNER >
Owner < InvariantPronoun > ( Sovereign Allocation Custodian: owner ) {
    grammar_primary: < "Pronoun" >,
    grammar_secondary: < "Noun" >,
    symbol: < "owner" >,
    abstract_form: < "+Owner" >,
    concrete_form: < "Owner-" >,
    machine_effect: < "sovereign_boundary_custodian" >
}

# ================================================================================
# BLOCK 2: OPENING BLOCK
# ================================================================================
Opening {
    requires: [
        Self : InvariantPronoun ::
        Caller : InvariantPronoun ::
        Owner : InvariantPronoun
    ],
    preconditions: [ +1 ]
}

# ================================================================================
# BLOCK 3: BODY BLOCK
# ================================================================================
Body [
    # Verify frame resolution handles
    handles_verification [
        Frame_Resolution [ self --> Active_Context ] ::
        Caller_Resolution [ caller --> Parent_Context ] ::
        Owner_Resolution  [ owner --> Storage_Custodian ]
    ] ( Context handle resolution verified )
]

# ================================================================================
# BLOCK 4: CLOSING BLOCK
```

```
# ================================================================================
Closing {
    verify: [
        handles_verification = yes
    ],
    commit_status [ PROCEED- ] ( Self handle pronouns verified and sealed )
}
```

<TRACK 2> : {OmniCode Language & Standard Library}/lexicon/02_pronouns/self_handles.tape

TAPE | 30 LINES | 187 WORDS | 1.4 KB

```
# ================================================================================
# TAPE HEAD: SELF HANDLES EXECUTION PROCESS RIBBON
# ================================================================================
# ARCHITECTURAL TOPOLOGY:
#   Executes context pointer resolution for self, caller, and owner.
# ================================================================================

[Tape.SelfHandles] : WHO {OmniCode_Kernel} WHAT {SelfHandles} --> RESOLVE {ContextMesh}
    WHEN {GenesisBoot}
    WHERE {handle_partition} = [ 64 ]
    WHY ( Test local frame pointers and context handles )
    HOW [ +1 : PROCEED ]
    :: < handle_landmark > ( 1 Corinthians 12:27 - 'Ye are the body of Christ, and members in particular' )

# ================================================================================
# SUBORDINATE ROLLS: SCOPED CALLER AND FRAME RESOLUTION
# ================================================================================

# Roll 1: Reference abstract Self, Caller, and Owner specifications
roll_1 [ +Self & +Caller & +Owner ] ( Abstract handle blueprints referenced )

# Roll 2: Assign local frame handle
roll_2 [ self --> Local_Frame_0 @ 0 ] ( Bind active frame to local cell )

# Roll 3: Verify caller resolution invariance
roll_3 [ caller != self ! [ +1 ] ] ( Verify caller and callee frame distinction )

# Roll 4: Commit with affirmative concrete macro
roll_4 [ commit_status [ PROCEED- ] ] ( Self handle pronouns verified and sealed )
```

<TRACK 2> : {OmniCode Language & Standard Library}/lexicon/02_pronouns/wildcards.omni

OMNI | 81 LINES | 342 WORDS | 3.0 KB

```
# ================================================================================
# BLOCK 1: METADATA BLOCK
# ( Ontological Pronouns: Wildcards & Dynamic Context Couplings )
# ================================================================================
# ARCHITECTURAL TOPOLOGY:
#   Defines anonymous references and antecedent couplings:
#   - _       : Anonymous discard, sink, or universal pattern wildcard.
#   - it      : Antecedent reference, pipeline accumulator, or active subject.
#   - origin : Root origin pointer, initial coordinate (0,0,0) or Genesis base.
# ================================================================================

Module < Lexicon.Pronouns.Wildcards > ( Wildcards and dynamic context couplings ) {
    version: < "1.0.0" >,
    system: < OmniCode.Lexicon >,
    semantics: < Wildcards :: AntecedentCouplings :: AnonymousDiscards >
}

# 1. Anonymous Wildcard: < _ >
Wildcard < InvariantPronoun > ( Universal Wildcard & Anonymous Discard: _ ) {
    grammar_primary: < "Pronoun" >,
    grammar_secondary: < "Noun" >,
    symbol: < "_" >,
    abstract_form: < "+Wildcard" >,
    concrete_form: < "Wildcard-" >,
    machine_effect: < "anonymous_sink_discard" >
}

# 2. Pipeline Antecedent: < IT >
Antecedent < InvariantPronoun > ( Pipeline Antecedent & Active Subject: it ) {
    grammar_primary: < "Pronoun" >,
    grammar_secondary: < "Noun" >,
    symbol: < "it" >,
    abstract_form: < "+Antecedent" >,
    concrete_form: < "Antecedent-" >,
    machine_effect: < "pipe_antecedent_register" >
}

# 3. Genesis Origin: < ORIGIN >
Origin < InvariantPronoun > ( Root Genesis Origin Pointer: origin ) {
    grammar_primary: < "Pronoun" >,
    grammar_secondary: < "Noun" >,
    symbol: < "origin" >,
    abstract_form: < "+Origin" >,
    concrete_form: < "Origin-" >,
    machine_effect: < "root_genesis_base_pointer" >
}

# ================================================================================
# BLOCK 2: OPENING BLOCK
# ================================================================================
Opening {
    requires: [
        Wildcard : InvariantPronoun ::
        Antecedent : InvariantPronoun ::
        Origin : InvariantPronoun
    ],
    preconditions: [ +1 ]
}

# ================================================================================
# BLOCK 3: BODY BLOCK
# ================================================================================
Body [
    # Verify anonymous discard and pipeline antecedent
    couplings_verification [
        Discard_Roll    [ Value --> _ ] ::
        Antecedent_Roll [ Input --> it :: it --> Output ] ::
        Origin_Roll     [ Current_Position <-- origin ]
    ] ( Context wildcards and dynamic couplings verified )
]

# ================================================================================
# BLOCK 4: CLOSING BLOCK
```

```
# ================================================================================
Closing {
    verify: [
        couplings_verification = yes
    ],
    commit_status [ PROCEED- ] ( Wildcard pronouns verified and sealed )
}
```

<TRACK 2> : {OmniCode Language & Standard Library}/lexicon/02_pronouns/wildcards.tape

TAPE | 30 LINES | 193 WORDS | 1.5 KB

```
# ================================================================================
# TAPE HEAD: WILDCARDS EXECUTION PROCESS RIBBON
# ================================================================================
# ARCHITECTURAL TOPOLOGY:
#   Executes anonymous discard (_), antecedent piping (it), and origin coupling.
# ================================================================================

[Tape.Wildcards] : WHO {OmniCode_Kernel} WHAT {Wildcards} --> COUPLE {ContextMesh}
    WHEN {GenesisBoot}
    WHERE {wildcard_partition} = [ 64 ]
    WHY ( Test anonymous wildcards, antecedent couplings, and origin pointers )
    HOW [ +1 : PROCEED ]
    :: < wildcard_landmark > ( Genesis 1:1 - 'In the beginning God created the heaven and the earth' )

# ================================================================================
# SUBORDINATE ROLLS: ANONYMOUS REFERENCE PROPAGATION
# ================================================================================

# Roll 1: Reference abstract Wildcard, Antecedent, and Origin specifications
roll_1 [ +Wildcard & +Antecedent & +Origin ] ( Abstract wildcard blueprints referenced )

# Roll 2: Transduce input into pipeline antecedent
roll_2 [ 77 --> it @ 0 ] ( Load value into antecedent accumulator )

# Roll 3: Route antecedent to concrete cell and discard extra to wildcard
roll_3 [ it --> Cell_Target_0 :: Noise --> _ ! [ +1 ] ] ( Pipe antecedent and discard noise )

# Roll 4: Commit with affirmative concrete macro
roll_4 [ commit_status [ PROCEED- ] ] ( Wildcard pronouns verified and sealed )
```

<TRACK 2> : {OmniCode Language & Standard Library}/lexicon/07_conjunctions/caveat_branches.omni

OMNI | 56 LINES | 222 WORDS | 2.2 KB

```
# ================================================================================
# BLOCK 1: METADATA BLOCK
# ( Ontological Conjunctions: Adversative Caveats & Ratchet Catches )
# ================================================================================
# ARCHITECTURAL TOPOLOGY:
#   Defines adversarial tripwires, ratchet catches, and invariant caveats:
#   - ! : Caveat branch conjunction (adversative condition, safety check, rewind trigger).
# ================================================================================

Module < Lexicon.Conjunctions.CaveatBranches > ( Adversative tripwires and ratchet tooth catches ) {
    version: < "1.0.0" >,
    system: < OmniCode.Lexicon >,
    semantics: < CaveatBranches :: AdversativeTripwires :: RatchetCatches >
}

# 1. Adversative Caveat Conjunction: < BUT >
But < InvariantConjunction > ( Adversative Caveat & Ratchet Tooth Catch: ! ) {
    grammar_primary: < "Conjunction" >,
    grammar_secondary: < "Preposition" >,
    symbol: < "!" >,
    abstract_form: < "+But" >,
    concrete_form: < "But-" >,
    machine_effect: < "adversative_tripwire_ratchet" >
}

# ================================================================================
# BLOCK 2: OPENING BLOCK
# ================================================================================
Opening {
    requires: [
        But : InvariantConjunction
    ],
    preconditions: [ +1 ]
}

# ================================================================================
# BLOCK 3: BODY BLOCK
# ================================================================================
Body [
    # Verify adversative caveat tripwire
    caveat_verification [
        Invariant_Check [ Operation_Execute ! [ Guard_Condition = +1 ] ] ::
        Rollback_Catch  [ Mutate_State ! [ Invariant_Violated --> ROLLBACK ] ]
    ] ( Adversative caveat tripwires verified )
]

# ================================================================================
# BLOCK 4: CLOSING BLOCK
# ================================================================================
Closing {
    verify: [
        caveat_verification = yes
    ],
    commit_status [ PROCEED- ] ( Caveat branch conjunctions verified and sealed )
}
```

<TRACK 2> : {OmniCode Language & Standard Library}/lexicon/07_conjunctions/caveat_branches.tape

TAPE | 30 LINES | 188 WORDS | 1.5 KB

```
# ================================================================================
# TAPE HEAD: CAVEAT BRANCHES PROCESS RIBBON
# ================================================================================
# ARCHITECTURAL TOPOLOGY:
#   Executes adversative invariant checks and caveat tripwires (!).
# ================================================================================

[Tape.CaveatBranches] : WHO {OmniCode_Kernel} WHAT {CaveatBranches} --> TRIPWIRE {SafetyMesh}
    WHEN {GenesisBoot}
    WHERE {caveat_partition} = [ 64 ]
    WHY ( Test adversative caveat branches and ratchet tooth catches )
    HOW [ +1 : PROCEED ]
    :: < caveat_landmark > ( Proverbs 22:3 - 'A prudent man forseeth the evil, and hideth himself' )

# ================================================================================
# SUBORDINATE ROLLS: EXCEPTION VETO & TRANSACTION REWIND ROLLS
# ================================================================================

# Roll 1: Reference abstract But caveat conjunction specification
roll_1 [ +But ] ( Abstract caveat branch blueprint referenced )

# Roll 2: Inscribe critical value under caveat condition
roll_2 [ 42 --> Secure_Cell_0 @ 0 ! [ +1 ] ] ( Write guarded by caveat affirmation )

# Roll 3: Guard state assertion against catastrophic violation
roll_3 [ Secure_Cell_0 = 42 ! [ +1 ] ] ( Assert value with affirmative caveat guard )

# Roll 4: Commit with affirmative concrete macro
roll_4 [ commit_status [ PROCEED- ] ] ( Caveat branch conjunctions verified and sealed )
```

<TRACK 2> : {OmniCode Language & Standard Library}/lexicon/07_conjunctions/inseparable_pairs.omni

OMNI | 56 LINES | 218 WORDS | 2.1 KB

```
# ================================================================================
# BLOCK 1: METADATA BLOCK
# ( Ontological Conjunctions: Inseparable Dual Pairs & Clutches )
# ================================================================================
# ARCHITECTURAL TOPOLOGY:
#   Defines locked differentials, atomic twins, and inseparable couplings:
#   - & : Inseparable pair conjunction (dual pin lock, twin rail binding).
# ================================================================================

Module < Lexicon.Conjunctions.InseparablePairs > ( Inseparable dual pairs and locked differentials ) {
    version: < "1.0.0" >,
    system: < OmniCode.Lexicon >,
    semantics: < InseparablePairs :: LockedDifferentials :: DualPinBindings >
}

# 1. Inseparable Pair Conjunction: < PAIR >
Pair < InvariantConjunction > ( Inseparable Dual Pin Binding: & ) {
    grammar_primary: < "Conjunction" >,
    grammar_secondary: < "Preposition" >,
    symbol: < "&" >,
    abstract_form: < "+Pair" >,
    concrete_form: < "Pair-" >,
    machine_effect: < "locked_differential_dual_binding" >
}

# ================================================================================
# BLOCK 2: OPENING BLOCK
# ================================================================================
Opening {
    requires: [
        Pair : InvariantConjunction
    ],
    preconditions: [ +1 ]
}

# ================================================================================
# BLOCK 3: BODY BLOCK
# ================================================================================
Body [
    # Verify inseparable dual binding
    pairs_verification [
        Dual_Rail_Binding [ Jack & Jill --> Hill ] ::
        Atomic_Twin_Lock  [ Signal_A & Signal_B --> Dual_Port ]
    ] ( Inseparable pairs and locked differentials verified )
]

# ================================================================================
# BLOCK 4: CLOSING BLOCK
# ================================================================================
Closing {
    verify: [
        pairs_verification = yes
    ],
    commit_status [ PROCEED- ] ( Inseparable pair conjunctions verified and sealed )
}
```

<TRACK 2> : {OmniCode Language & Standard Library}/lexicon/07_conjunctions/inseparable_pairs.tape

TAPE | 30 LINES | 178 WORDS | 1.4 KB

```
# ================================================================================
# TAPE HEAD: INSEPARABLE PAIRS PROCESS RIBBON
# ================================================================================
# ARCHITECTURAL TOPOLOGY:
#   Executes atomic dual-rail writes and locked differential pairs (&).
# ================================================================================

[Tape.InseparablePairs] : WHO {OmniCode_Kernel} WHAT {InseparablePairs} --> LOCK {DifferentialMesh}
    WHEN {GenesisBoot}
    WHERE {pair_partition} = [ 64 ]
    WHY ( Test locked differential twin bindings and atomic dual writes )
    HOW [ +1 : PROCEED ]
    :: < pair_landmark > ( Ecclesiastes 4:9 - 'Two are better than one' )

# ================================================================================
# SUBORDINATE ROLLS: ATOMIC DUAL-RAIL WRITE ROLLS
# ================================================================================

# Roll 1: Reference abstract Pair conjunction specification
roll_1 [ +Pair ] ( Abstract dual pair blueprint referenced )

# Roll 2: Inscribe locked differential twin values
roll_2 [ 50 & 50 --> Twin_Port_0 @ 0 ] ( Atomic dual-rail write into Twin_Port_0 )

# Roll 3: Verify dual-rail equality invariant
roll_3 [ Twin_Port_0 = 50 ! [ +1 ] ] ( Verify twin values bound to port )

# Roll 4: Commit with affirmative concrete macro
roll_4 [ commit_status [ PROCEED- ] ] ( Inseparable pair conjunctions verified and sealed )
```

<TRACK 2> : {OmniCode Language & Standard Library}/lexicon/07_conjunctions/pipeline_junctions.omni

OMNI | 56 LINES | 209 WORDS | 2.1 KB

```
# ================================================================================
# BLOCK 1: METADATA BLOCK
# ( Ontological Conjunctions: Sequential Pipeline Junctions & Chaining )
# ================================================================================
# ARCHITECTURAL TOPOLOGY:
#   Defines geartrain sequencing and pipelined consensus chaining:
#   - :: : Pipeline junction conjunction (sequential geartrain, step separator).
# ================================================================================

Module < Lexicon.Conjunctions.PipelineJunctions > ( Pipelined geartrain sequencing and chaining ) {
    version: < "1.0.0" >,
    system: < OmniCode.Lexicon >,
    semantics: < PipelineJunctions :: GeartrainSequencing :: ConsensusChaining >
}

# 1. Pipeline Junction Conjunction: < CHAIN >
Chain < InvariantConjunction > ( Sequential Geartrain Chaining: :: ) {
    grammar_primary: < "Conjunction" >,
    grammar_secondary: < "Preposition" >,
    symbol: < "::" >,
    abstract_form: < "+Chain" >,
    concrete_form: < "Chain-" >,
    machine_effect: < "sequential_pipeline_junction" >
}

# ================================================================================
# BLOCK 2: OPENING BLOCK
# ================================================================================
Opening {
    requires: [
        Chain : InvariantConjunction
    ],
    preconditions: [ +1 ]
}

# ================================================================================
# BLOCK 3: BODY BLOCK
# ================================================================================
Body [
    # Verify sequential geartrain chaining
    pipeline_verification [
        Pipelined_Sequence [ Step_A :: Step_B :: Step_C ] ::
        Chain_Validation   [ Transform_1 :: Transform_2 ]
    ] ( Sequential geartrain chaining verified )
]

# ================================================================================
# BLOCK 4: CLOSING BLOCK
# ================================================================================
Closing {
    verify: [
        pipeline_verification = yes
    ],
    commit_status [ PROCEED- ] ( Pipeline junction conjunctions verified and sealed )
}
```

<TRACK 2> : {OmniCode Language & Standard Library}/lexicon/07_conjunctions/pipeline_junctions.tape

TAPE | 30 LINES | 182 WORDS | 1.4 KB

```
# ================================================================================
# TAPE HEAD: PIPELINE JUNCTIONS PROCESS RIBBON
# ================================================================================
# ARCHITECTURAL TOPOLOGY:
#   Executes sequential geartrain pipeline chaining (::).
# ================================================================================

[Tape.PipelineJunctions] : WHO {OmniCode_Kernel} WHAT {PipelineJunctions} --> CHAIN {SequencerMesh}
    WHEN {GenesisBoot}
    WHERE {pipeline_partition} = [ 64 ]
    WHY ( Test sequential geartrain steps and pipeline chaining )
    HOW [ +1 : PROCEED ]
    :: < pipeline_landmark > ( 1 Corinthians 14:40 - 'Let all things be done decently and in order' )

# ================================================================================
# SUBORDINATE ROLLS: PIPELINE CHAINING ROLLS
# ================================================================================

# Roll 1: Reference abstract Chain conjunction specification
roll_1 [ +Chain ] ( Abstract pipeline chain blueprint referenced )

# Roll 2: Execute chained sequential stage writes
roll_2 [ 10 --> Stage_A @ 0 :: 20 --> Stage_B @ 1 ] ( Chained writes in single roll )

# Roll 3: Verify chained stage invariants
roll_3 [ Stage_A = 10 :: Stage_B = 20 ! [ +1 ] ] ( Verify chained invariant assertions )

# Roll 4: Commit with affirmative concrete macro
roll_4 [ commit_status [ PROCEED- ] ] ( Pipeline junctions verified and sealed )
```

<TRACK 2> : {OmniCode Language & Standard Library}/lexicon/05_adverbs/cadence_policies.omni

OMNI | 81 LINES | 333 WORDS | 3.0 KB

```
# ================================================================================
# BLOCK 1: METADATA BLOCK
# ( Ontological Adverbs: Escapement Regulation & Cadence Policies )
# ================================================================================
# ARCHITECTURAL TOPOLOGY:
#   Defines scheduler cadence, governor flywheel, and timing policies:
#   - atomic    : Uninterruptible execution stroke, single-cycle indivisibility.
#   - yield     : Cooperative dispatch pause, escapement release to scheduler.
#   - monotonic : Unidirectional clock advance, irreversible sequential progress.
# ================================================================================

Module < Lexicon.Adverbs.CadencePolicies > ( Escapement regulation and scheduler cadence policies ) {
    version: < "1.0.0" >,
    system: < OmniCode.Lexicon >,
    semantics: < CadencePolicies :: EscapementGovernor :: SchedulerRegulation >
}

# 1. Atomic Policy: < ATOMIC >
Atomic < InvariantAdverb > ( Indivisible Execution Stroke: atomic ) {
    grammar_primary: < "Adverb" >,
    grammar_secondary: < "Adjective" >,
    symbol: < "atomic" >,
    abstract_form: < "+Atomic" >,
    concrete_form: < "Atomic-" >,
    machine_effect: < "indivisible_cycle_lock" >
}

# 2. Yield Policy: < YIELD >
Yield < InvariantAdverb > ( Cooperative Escapement Release: yield ) {
    grammar_primary: < "Adverb" >,
    grammar_secondary: < "Verb" >,
    symbol: < "yield" >,
    abstract_form: < "+Yield" >,
    concrete_form: < "Yield-" >,
    machine_effect: < "escapement_release_cooperate" >
}

# 3. Monotonic Policy: < MONOTONIC >
Monotonic < InvariantAdverb > ( Irreversible Unidirectional Advance: monotonic ) {
    grammar_primary: < "Adverb" >,
    grammar_secondary: < "Adjective" >,
    symbol: < "monotonic" >,
    abstract_form: < "+Monotonic" >,
    concrete_form: < "Monotonic-" >,
    machine_effect: < "monotonic_clock_advance" >
}

# ================================================================================
# BLOCK 2: OPENING BLOCK
# ================================================================================
Opening {
    requires: [
        Atomic : InvariantAdverb ::
        Yield : InvariantAdverb ::
        Monotonic : InvariantAdverb
    ],
    preconditions: [ +1 ]
}

# ================================================================================
# BLOCK 3: BODY BLOCK
# ================================================================================
Body [
    # Verify cadence regulation policies
    cadence_verification [
        Atomic_Lock    [ atomic [ Write_Buffer_0 --> Flash_ROM ] ] ::
        Yield_Check    [ yield [ Task_Slice_0 ] ] ::
        Monotonic_Tick [ monotonic [ Clock_Tick_0 ] ]
    ] ( Scheduler cadence policies verified )
]

# ================================================================================
# BLOCK 4: CLOSING BLOCK
```

```
# ================================================================================
Closing {
    verify: [
        cadence_verification = yes
    ],
    commit_status [ PROCEED- ] ( Cadence policy adverbs verified and sealed )
}
```

<TRACK 2> : {OmniCode Language & Standard Library}/lexicon/05_adverbs/cadence_policies.tape

TAPE | 30 LINES | 183 WORDS | 1.4 KB

```
# ================================================================================
# TAPE HEAD: CADENCE POLICIES PROCESS RIBBON
# ================================================================================
# ARCHITECTURAL TOPOLOGY:
#   Executes scheduler cadence policies (atomic, yield, monotonic).
# ================================================================================

[Tape.CadencePolicies] : WHO {OmniCode_Kernel} WHAT {CadencePolicies} --> REGULATE {SchedulerMesh}
    WHEN {GenesisBoot}
    WHERE {cadence_partition} = [ 64 ]
    WHY ( Test escapement regulation, atomic locks, and yield cycles )
    HOW [ +1 : PROCEED ]
    :: < cadence_landmark > ( Psalm 90:12 - 'So teach us to number our days' )

# ================================================================================
# SUBORDINATE ROLLS: CADENCE POLICY DISPATCH ROLLS
# ================================================================================

# Roll 1: Reference abstract Atomic, Yield, and Monotonic specifications
roll_1 [ +Atomic & +Yield & +Monotonic ] ( Abstract cadence blueprints referenced )

# Roll 2: Execute atomic lock roll
roll_2 [ atomic [ 99 --> Locked_Cell_0 @ 0 ] ] ( Inscribe value under atomic lock )

# Roll 3: Execute cooperative yield roll
roll_3 [ yield [ Locked_Cell_0 = 99 ! [ +1 ] ] ] ( Yield cycle with invariant check )

# Roll 4: Commit with affirmative concrete macro
roll_4 [ commit_status [ PROCEED- ] ] ( Cadence policies verified and sealed )
```

<TRACK 2> : {OmniCode Language & Standard Library}/lexicon/05_adverbs/negation.omni

OMNI | 57 LINES | 221 WORDS | 2.1 KB

```
# ================================================================================
# BLOCK 1: METADATA BLOCK
# ( Ontological Adverbs: Logical & Algebraic Negation )
# ================================================================================
# ARCHITECTURAL TOPOLOGY:
#   Defines inversion symmetry and reversing idler gears:
#   - -- : Negation operator (logical NOT, balanced ternary negation: -1 <-> +1).
# ================================================================================

Module < Lexicon.Adverbs.Negation > ( Reversing idler gears and inversion symmetry ) {
    version: < "1.0.0" >,
    system: < OmniCode.Lexicon >,
    semantics: < Negation :: InversionSymmetry :: ReversingIdlerGear >
}

# 1. Negation Operator: < NEGATION >
Negation < InvariantAdverb > ( Logical & Algebraic Inversion: -- ) {
    grammar_primary: < "Adverb" >,
    grammar_secondary: < "Preposition" >,
    symbol: < "--" >,
    abstract_form: < "+Negation" >,
    concrete_form: < "Negation-" >,
    machine_effect: < "inversion_idler_gear" >
}

# ================================================================================
# BLOCK 2: OPENING BLOCK
# ================================================================================
Opening {
    requires: [
        Negation : InvariantAdverb
    ],
    preconditions: [ +1 ]
}

# ================================================================================
# BLOCK 3: BODY BLOCK
# ================================================================================
Body [
    # Verify balanced ternary negation symmetry
    negation_verification [
        Invert_Positive [ -- (+1) = -1 ] ::
        Invert_Negative [ -- (-1) = +1 ] ::
        Invert_Zero     [ -- (0)  =  0 ]
    ] ( Balanced ternary negation symmetry verified )
]

# ================================================================================
# BLOCK 4: CLOSING BLOCK
# ================================================================================
Closing {
    verify: [
        negation_verification = yes
    ],
    commit_status [ PROCEED- ] ( Negation adverbs verified and sealed )
}
```

<TRACK 2> : {OmniCode Language & Standard Library}/lexicon/05_adverbs/negation.tape

TAPE | 33 LINES | 193 WORDS | 1.5 KB

```
# ================================================================================
# TAPE HEAD: NEGATION PROCESS RIBBON
# ================================================================================
# ARCHITECTURAL TOPOLOGY:
#   Executes algebraic and logical inversion symmetry (--).
# ================================================================================

[Tape.Negation] : WHO {OmniCode_Kernel} WHAT {Negation} --> INVERT {SymmetryMesh}
    WHEN {GenesisBoot}
    WHERE {negation_partition} = [ 64 ]
    WHY ( Test balanced ternary negation and idler gear inversion )
    HOW [ +1 : PROCEED ]
    :: < negation_landmark > ( Ecclesiastes 3:1 - 'To every thing there is a season' )

# ================================================================================
# SUBORDINATE ROLLS: INVERSION SYMMETRY ROLLS
# ================================================================================

# Roll 1: Reference abstract Negation specification
roll_1 [ +Negation ] ( Abstract negation blueprint referenced )

# Roll 2: Set input register to affirmative +1
roll_2 [ 1 --> Reg_Input_0 @ 0 ] ( Inscribe positive polarity into input register )

# Roll 3: Invert polarity via --
roll_3 [ -- Reg_Input_0 --> Reg_Inverted_0 @ 1 ] ( Apply reversing idler gear inversion )

# Roll 4: Verify symmetric equilibrium
roll_4 [ Reg_Inverted_0 = -1 ! [ +1 ] ] ( Verify balanced ternary inversion symmetry )

# Roll 5: Commit with affirmative concrete macro
roll_5 [ commit_status [ PROCEED- ] ] ( Negation adverbs verified and sealed )
```

<TRACK 2> : {OmniCode Language & Standard Library}/lib/kernel/boot.omni OMNI 114 LINES 679 WORDS 4.9 KB

```
# ============================================================================
# BLOCK 1: METADATA BLOCK
# ( Kernel Architecture: Genesis Bare-Metal OS Specification )
# ============================================================================
# ARCHITECTURAL TOPOLOGY:
#   Defines the bare-metal machine initialization contract:
#   - Establishes the 64-bit silicon envelope hosting the 32-trit kernel word.
#   - Mounts the 4096-cell Root_RAM storehouse on Track 0 [KERNEL].
#   - Writes the boot banner to the memory-mapped VGA text buffer at 0x000B8000.
#   - Initializes the 4-directional Memory Reel spindle hierarchy.
# ============================================================================

Module < Kernel.GenesisBoot > ( MilleniumOS Genesis bare-metal kernel initialization ) {
    version: < "1.0.0" >,
    system: < MilleniumOS.Kernel >,
    semantics: < BareMetalBootstrap :: BalancedTernaryALU :: MMIOConsole >
}

# 1. Kernel Entity: < KERNEL >
Kernel < InvariantNoun > ( Master Operating System Supervisor ) {
    grammar_primary: < "Noun" >,
    abstract_form: < "+Kernel" >,
    concrete_form: < "Kernel-" >,
    authority_channel: < "Track 0 [KERNEL]" >,
    machine_effect: < "hardware_root_supervisor" >
}

# 2. Storehouse Entity: < ROOT_RAM >
RootRAM < InvariantNoun > ( Physical Silicon Memory Storehouse ) {
    grammar_primary: < "Noun" >,
    abstract_form: < "+RootRAM" >,
    concrete_form: < "RootRAM-" >,
    capacity_cells: < 4096 >,
    machine_effect: < "physical_ram_allocator" >
}

# 3. Video Console Entity: < VGA_SCREEN >
ScreenVGA < InvariantNoun > ( Bare-Metal Video Text Buffer at 0x000B8000 ) {
    grammar_primary: < "Noun" >,
    abstract_form: < "+ScreenVGA" >,
    concrete_form: < "ScreenVGA-" >,
    mmio_base: < 0x000B8000 >,
    geometry: < "80x25 characters" >,
    machine_effect: < "vga_text_buffer" >
}

# 4. Serial Transducer Entity: < SERIAL_COM1 >
SerialCOM1 < InvariantNoun > ( UART Serial Hardware Port at 0x03F8 ) {
    grammar_primary: < "Noun" >,
    abstract_form: < "+SerialCOM1" >,
    concrete_form: < "SerialCOM1-" >,
    io_port: < 0x03F8 >,
    baud_rate: < 115200 >,
    machine_effect: < "hardware_input_transducer" >
}

# 5. Silicon Register Bank Entity: < REGISTERS >
RegisterBank < InvariantNoun > ( 13 Named Silicon Registers and Transducers ) {
    grammar_primary: < "Noun" >,
    abstract_form: < "+RegisterBank" >,
    concrete_form: < "RegisterBank-" >,
    capacity_slots: < 13 >,
    registers: < [ HEAD HOME SPOOL IT SELF CALLER OTHER STATE TURN BOUND FROM INTO DISCARD ] >,
    machine_effect: < "silicon_register_allocation" >
}

# ============================================================================
# BLOCK 2: OPENING BLOCK
# ============================================================================
Opening {
    requires: [
        Kernel : InvariantNoun ::
        RootRAM : InvariantNoun ::
        ScreenVGA : InvariantNoun ::
```

```
        SerialCOM1 : InvariantNoun ::
        RegisterBank : InvariantNoun
    ],
    preconditions: [ +1 ]
}

# ================================================================================
# BLOCK 3: BODY BLOCK
# ================================================================================
Body [
    # Roll 1: Allocate root RAM storehouse and arm 13 Named Silicon Registers
    ram_and_regs [ Root_RAM [ 4096 ] :: Register_Bank [ HEAD HOME SPOOL IT SELF CALLER OTHER STATE TURN BOUND FROM INTO
DISCARD ] :: FROM ::@ 0x03F8 :: INTO ::@ 0x000B8000 ] ( Root memory storehouse and 13 named silicon registers armed )

    # Roll 2: Prove ALU equilibrium invariant and calibrate Quad-State accumulator loop
    alu_and_loop [ ALU_Equilibrium [ (+1) plus (-1) <=> 0 ] :: Accumulator_Loop [ WHEN [ { IT } < { BOUND } ] --> [ IT` ]
PROCEED to { TURN } ! [ { IT } > { BOUND } ] AWAIT for [ IT` ] :: HALT WHEN [ IT` ] != even ] ] ( Invariant arithmetic
equilibrium and quad-state loop calibration )

    # Roll 3: Carve protected Kernel and Terminal partitions via v-- and evaluate contextual bounds
    partition_carve [ Root_RAM v-- Kernel_Partition [ 1024 ] :: Root_RAM v-- Terminal_Partition [ 512 ] :: Bounds_Check [
{ 1024 } < { 4096 } ] ] ( Partition storehouses via v-- and bounds evaluation )

    # Roll 4: Inscribe initial landmark and boot banner to MMIO VGA video buffer
    landmark_and_banner [ 0 --> Kernel_Partition @ 0 :: 'MILLENIUMOS: OMNICODE 32-TRIT OS ONLINE' --> Screen_VGA ::@
0x000B8000 ] ( Landmark and screen banner )

    # Roll 5: Commit genesis boot
    commit_genesis [ commit_status [ PROCEED ] ] ( Genesis Boot sealed in PROCEED )
]

# ================================================================================
# BLOCK 4: CLOSING BLOCK
# ================================================================================
Closing {
    verify: [
        ram_and_regs = yes ::
        alu_and_loop = yes ::
        partition_carve = yes ::
        landmark_and_banner = yes ::
        commit_genesis = yes
    ],
    commit_status [ PROCEED ] ( Genesis Boot verified and sealed in PROCEED )
}
```

<TRACK 2> : {OmniCode Language & Standard Library}/lib/kernel/boot.tape TAPE 50 LINES 487 WORDS 3.4 KB

```
# ================================================================================
# BLOCK 1: METADATA BLOCK
# ( Tape Manifest, Kernel Scheduling & Genesis Machine Covenant )
# ================================================================================
# ARCHITECTURAL TOPOLOGY:
#   Executes cold boot sequence on Track 0 [KERNEL]:
#   - Senses physical Root_RAM storehouse and binds serial COM1 situated at 0x03F8 via ::@.
#   - Proves balanced ternary ALU equilibrium before allocating mutable memory partitions.
#   - Carves Kernel_Partition (1024 cells) and Terminal_Partition (512 cells) via v--, verifying [ { 1024 } < { 4096 } ].
#   - Establishes preemptive turn-taking (~! & PREEMPT) and awakens VGA console at 0x000B8000.
#   - Seals genesis boot in affirmative PROCEED.
# ================================================================================

[Tape.GenesisBoot] : WHO {MilleniumOS_Kernel} WHAT {SystemGenesis} --> BOOT {Hardware}
    WHEN {PowerOn}
    WHERE {ram} = [ 4096 ]
    WHY ( Initialize balanced ternary MilleniumOS kernel and diagnostic nervous system )
    HOW [ +1 : PROCEED ]
    :: < ancient_landmark > ( Proverbs 22:28 - 'Remove not the ancient landmark' )

# ================================================================================
# BLOCK 2: OPENING BLOCK
# ( Environmental Preconditions, Memory Partition Geometry & Storehouse Bounds )
# ================================================================================

# ================================================================================
# BLOCK 3: BODY BLOCK
# ( Sequential Instruction Rolls Lowered to Silicon Machine Words )
# ================================================================================

# Roll 1: Sense physical memory and arm 13 Named Silicon Registers with hardware transducers
roll_1 [ Kernel --> SENSE {Root_RAM [ 4096 ]} :: ARM {Registers [ HEAD HOME SPOOL IT SELF CALLER OTHER STATE TURN BOUND FROM INTO DISCARD ]} :: FROM ::@ 0x03F8 :: INTO ::@ 0x000B8000 ] ( Sense physical memory and arm 13 named silicon registers with hardware transducers )

# Roll 2: Prove ALU equilibrium invariant and calibrate Quad-State accumulator loop
roll_2 [ Kernel --> PROVE {ALU_Equilibrium} [ (+1) plus (-1) <=> 0 ] :: LOOP [ WHEN [ { IT } < { BOUND } ] --> [ IT` ] PROCEED to { TURN } ! [ { IT } > { BOUND } ] AWAIT for [ IT` ] :: HALT WHEN [ IT` ] != even ] ] ( Invariant proof of balanced ternary arithmetic and quad-state accumulator calibration )

# Roll 3: Carve protected kernel and terminal partitions via v-- and evaluate contextual bounds
roll_3 [ Root_RAM v-- Kernel_Partition [ 1024 ] :: Root_RAM v-- Terminal_Partition [ 512 ] :: Bounds_Check [ { 1024 } < { 4096 } ] ] ( Partition storehouse via v-- and evaluate bounds in context )

# Roll 4: Yield compute on timer tick and awaken console HUD with MMIO boot banner
roll_4 [ Kernel_Compute ~! [ Timer_Tick --> PREEMPT {Background_Task} ] :: 'MILLENIUMOS: OMNICODE 32-TRIT OS ONLINE' --> Screen_VGA ::@ 0x000B8000 :: Kernel --> AWAKEN {Terminal_Studio} ] ( Preempt compute on timer interrupt and awaken screen banner )

# ================================================================================
# BLOCK 4: CLOSING BLOCK
# ( Invariant Verification Proofs & Covenant Commit Seal )
# ================================================================================

# Roll 5: Genesis Boot Complete
roll_5 [ commit_status [ PROCEED ] ] ( MilleniumOS Genesis Boot Complete )
```

<TRACK 2> : {OmniCode Language & Standard Library}/lib/core/arithmetic.omni

OMNI | 90 LINES | 416 WORDS | 2.8 KB

```
# ================================================================================
# BLOCK 1: METADATA BLOCK
# ( Core Library Manifest: Invariant Arithmetic Macros & Opcodes )
# ================================================================================
Module < Core.Arithmetic > ( Invariant arithmetic macros linked to CPU and LLVM opcodes ) {
    version: < "1.0.0" >,
    system: < OmniCode.Core >,
    semantics: < MacroInvariants :: HardwareOpcodes :: SVOTranspilation >
}

# Macro Invariant: < ADD >
ADD < InvariantMacro > ( Binary Addition ) {
    micro: < "plus" >,
    x86_64: < "add" >,
    riscv: < "add" >,
    llvm_ir: < "add i64" >,
    c_operator: < "+" >
}

# Macro Invariant: < SUB >
SUB < InvariantMacro > ( Binary Subtraction ) {
    micro: < "minus" >,
    x86_64: < "sub" >,
    riscv: < "sub" >,
    llvm_ir: < "sub i64" >,
    c_operator: < "-" >
}

# Macro Invariant: < MUL >
MUL < InvariantMacro > ( Binary Multiplication ) {
    micro: < "times" >,
    x86_64: < "imul" >,
    riscv: < "mul" >,
    llvm_ir: < "mul i64" >,
    c_operator: < "*" >
}

# Macro Invariant: < DIV >
DIV < InvariantMacro > ( Binary Division ) {
    micro: < "div" >,
    x86_64: < "idiv" >,
    riscv: < "div" >,
    llvm_ir: < "sdiv i64" >,
    c_operator: < "/" >
}

# ================================================================================
# BLOCK 2: OPENING BLOCK
# ( Preconditions & Macro Registry Setup )
# ================================================================================
Opening {
    requires: [ Hardware.ALU :: IntegerRegisters.64 ],
    preconditions: [ +1 ]
}

test_x [ 40 ] ( Sample operand X )
test_y [ 2 ]  ( Sample operand Y )

# ================================================================================
# BLOCK 3: BODY BLOCK
# ( SVO Descriptions Lowered to Invariant Macro Opcodes )
# ================================================================================
Body [
    # 1. Addition: Micro 'plus' lowers to Macro < ADD >
    sum_result [ test_x plus test_y ] ( 40 plus 2 = 42 )

    # 2. Subtraction: Micro 'minus' lowers to Macro < SUB >
    diff_result [ sum_result minus test_y ] ( 42 minus 2 = 40 )

    # 3. Multiplication: Micro 'times' lowers to Macro < MUL >
    prod_result [ test_y times 10 ] ( 2 times 10 = 20 )

    # 4. Division: Micro 'div' lowers to Macro < DIV >
```

```
    quot_result [ prod_result div 2 ] ( 20 div 2 = 10 )
]

# ================================================================================
# BLOCK 4: CLOSING BLOCK
# ( Verification & Macro Status Commit )
# ================================================================================
Closing {
    verify: [
        sum_result = 42 ::
        diff_result = 40 ::
        prod_result = 20 ::
        quot_result = 10
    ],
    commit_status [ PROCEED ] ( Arithmetic library verified and sealed )
}
```

<TRACK 2> : {OmniCode Language & Standard Library}/lib/core/cell.tape TAPE 17 LINES 133 WORDS 0.9 KB

```
# ================================================================================
# TAPE HEAD: THE CELL DEFINITION COVENANT
# ================================================================================
[Cell] : A unit of measure --> WHERE {unit} = [ 8 bit --> 5 trit --> 8 bit ] :: < measure > ( Fundamental unit of memory
)

# ================================================================================
# SUBORDINATE ROLLS: CELL DENSE ENCODING & REVERSIBLE TRANSDUCTION
# ================================================================================
# Roll 1: Transduce byte 42 into 5 trits and store in cell 0
roll_1 [ 42 --> Cell_0 @ 0 ] ( Inscribe radix-3 packed value into cell 0 )

# Roll 2: Verify cell 0 holds 42 within invariant bounds
roll_2 [ Cell_0 = 42 ! [ +1 ] ] ( Verify dense byte recovery )

# Roll 3: Transduce and seal with affirmative commit
roll_3 [ commit_status [ PROCEED ] ] ( Cell covenant proven )
```

<TRACK 2> : {OmniCode Language & Standard Library}/lib/core/logic.omni OMNI 89 LINES 404 WORDS 2.9 KB

```
# ========================================================================
# BLOCK 1: METADATA BLOCK
# ( Core Library Manifest: Invariant Quad-State Macros & OS Control Opcodes )
# ========================================================================
Module < Core.Logic > ( Invariant logic macros linked to CPU execution and kernel control ) {
    version: < "1.0.0" >,
    system: < OmniCode.Core >,
    semantics: < MacroInvariants :: KernelPhysics :: QuadStateOpcodes >
}

# Macro Invariant: < PROCEED >
PROCEED_MACRO < InvariantMacro > ( Advance Execution & Commit State Frame ) {
    micro: < "yes" >,
    radix3: < 1 >,
    x86_64: < "jmp .Lnext" >,
    riscv: < "j .Lnext" >,
    c_status: < "0" >,
    os_action: < "commit_transaction_and_advance" >
}

# Macro Invariant: < ROLLBACK >
ROLLBACK_MACRO < InvariantMacro > ( Retroactive Unwind & Snapshot Restore ) {
    micro: < "no" >,
    radix3: < -1 >,
    x86_64: < "mov rsp, rbp; pop rbp; ret" >,
    riscv: < "mv sp, s0; ld s0, 0(sp); ret" >,
    c_status: < "-1" >,
    os_action: < "unwind_transaction_frame" >
}

# Macro Invariant: < AWAIT >
AWAIT_MACRO < InvariantMacro > ( CPU Idle / Asynchronous Interrupt Standby ) {
    micro: < "ok" >,
    radix3: < 0 >,
    x86_64: < "hlt" >,
    riscv: < "wfi" >,
    arm64: < "wfi" >,
    os_action: < "yield_cpu_to_scheduler" >
}

# Macro Invariant: < HALT >
HALT_MACRO < InvariantMacro > ( Unrecoverable Boundary Trap / Panic ) {
    micro: < "void" >,
    radix3: < NULL >,
    x86_64: < "ud2" >,
    riscv: < "ebreak" >,
    c_status: < "abort()" >,
    os_action: < "kernel_panic_trap" >
}

# ========================================================================
# BLOCK 2: OPENING BLOCK
# ( Kernel Preconditions & Invariant Verification )
# ========================================================================
Opening {
    requires: [ CPU.InterruptControl :: ProcessTable.Active ],
    preconditions: [ +1 ]
}

status_flag [ yes ] ( Active execution thread status )

# ========================================================================
# BLOCK 3: BODY BLOCK
# ( Logic Invariants Mapped to OS Physics )
# ========================================================================
Body [
    # 1. Monotonic Commit Test
    commit_test [ status_flag = PROCEED ] ( 'yes' triggers monotonic forward commit )

    # 2. Inversion Symmetry Test
    negate_test [ -- status_flag = ROLLBACK ] ( Inversion of 'yes' triggers ROLLBACK )

    # 3. Equilibrium Balance Test
    balance_test [ ok = AWAIT ] ( 'ok' maps to AWAIT / HLT / WFI )
```

```
]

# ================================================================================
# BLOCK 4: CLOSING BLOCK
# ( Verification & Status Commit )
# ================================================================================
Closing {
    verify: [
        commit_test = yes ::
        negate_test = yes ::
        balance_test = yes
    ],
    commit_status [ PROCEED ] ( Logic core sealed )
}
```

<TRACK 2> : {OmniCode Language & Standard Library}/lib/core/memory.omni

OMNI | 135 LINES | 703 WORDS | 5.2 KB

```
# ========================================================================
# BLOCK 1: METADATA BLOCK
# ( Core Library Manifest: Foundational Storage Nouns & Memory Vessel Invariants )
# ========================================================================
Module < Core.Memory > ( Foundational storage nouns, bounded vessels, and hardware transducers ) {
    version: < "1.0.0" >,
    system: < OmniCode.Core >,
    semantics: < StorageNouns :: BoundedVessels :: TransducedCells :: BoundaryPhysics >
}

# 1. Atomic Data Unit: < TRIT >
TRIT_NOUN < InvariantNoun > ( The Atomic Radix-3 Polarity Unit ) {
    grammar_primary: < "Noun" >,
    grammar_secondary: < "Adjective" >,
    ontological_tier: < "Tier 1: Atomic Unit" >,
    permitted_states: < "Neg_T_Zero_0_Pos_1" >,
    machine_effect: < "radix3_polarity_register" >
}

# 2. Atomic Data Unit: < CELL >
CELL_NOUN < InvariantNoun > ( The Atomic 5-Trit / 8-Bit Quad-State Memory Vessel ) {
    grammar_primary: < "Noun" >,
    grammar_secondary: < "Preposition" >,
    ontological_tier: < "Tier 1: Atomic Unit" >,
    physical_footprint: < "1 physical byte in RAM (243 values + 13 control states)" >,
    machine_effect: < "dense_byte_trit_cell" >
}

# 3. Storage Container: < VESSEL >
VESSEL_NOUN < InvariantNoun > ( Bounded Sequential Container of Memory Cells ) {
    grammar_primary: < "Noun" >,
    grammar_secondary: < "Preposition" >,
    ontological_tier: < "Tier 2: Storage Container" >,
    invariant_law: < "Proverbs 22:28 (Remove not the ancient landmark)" >,
    machine_effect: < "contiguous_bounded_buffer" >
}

# 4. Storage Container: < STOREHOUSE >
STOREHOUSE_NOUN < InvariantNoun > ( Root System Memory Domain & Reservoir ) {
    grammar_primary: < "Noun" >,
    grammar_secondary: < "Preposition" >,
    ontological_tier: < "Tier 2: Storage Container" >,
    machine_effect: < "root_page_reservoir" >
}

# 5. Storage Container: < PARTITION >
PARTITION_NOUN < InvariantNoun > ( Entrusted Bounded Sub-Region of a Storehouse ) {
    grammar_primary: < "Noun" >,
    grammar_secondary: < "Preposition" >,
    ontological_tier: < "Tier 2: Storage Container" >,
    machine_effect: < "delegated_task_slice" >
}

# 6. Coordinate: < OFFSET >
OFFSET_NOUN < InvariantNoun > ( Relative Zero-Based Coordinate within a Vessel ) {
    grammar_primary: < "Preposition" >,
    grammar_secondary: < "Noun" >,
    ontological_tier: < "Tier 3: Spatial Coordinate" >,
    machine_effect: < "memory_address_displacement" >
}

# 7. Measure: < CAPACITY >
CAPACITY_NOUN < InvariantNoun > ( Maximum Allocated Cell Limit of a Vessel ) {
    grammar_primary: < "Noun" >,
    grammar_secondary: < "Adjective" >,
    ontological_tier: < "Tier 4: Measure" >,
    machine_effect: < "boundary_limit_ceiling" >
}

# 8. Measure: < COUNT >
COUNT_NOUN < InvariantNoun > ( Number of Actively Written or Populated Cells ) {
    grammar_primary: < "Noun" >,
    grammar_secondary: < "Adjective" >,
    ontological_tier: < "Tier 4: Measure" >,
```

```
    machine_effect: < "active_population_count" >
}

# 9. Substance: < VALUE >
VALUE_NOUN < InvariantNoun > ( The Numeric or Symbolic Payload Stored in a Cell ) {
    grammar_primary: < "Noun" >,
    grammar_secondary: < "Adjective" >,
    ontological_tier: < "Tier 5: Substance" >,
    machine_effect: < "cell_payload_register" >
}

# ================================================================================
# BLOCK 2: OPENING BLOCK
# ( Preconditions & Vessel Context Setup )
# ================================================================================
Opening {
    requires: [ MemoryAligned :: QuadStateTransducer :: BoundaryPhysics ],
    preconditions: [ +1 ]
}

# Declare the system Storehouse
Main_Storehouse [ 1024 ] { Fixed, Bounded } ( System memory reservoir of 1024 cells )

# Grant an entrusted Partition carved directly from Main_Storehouse via v--
Main_Storehouse v-- Task_Partition [ 256 ] ( Entrusted process partition of 256 cells )

# Initial test data
sample_val [ 42 ] ( Natural number 42 to store )

# ================================================================================
# BLOCK 3: BODY BLOCK
# ( Memory Operations: Forward Store, Feedback Read, and Bounds Verification )
# ================================================================================
Body [
    # 1. Forward Causality: Store natural value 42 into cell 0 of partition
    step_store [ sample_val --> Task_Partition[0] ] ( Deposit value into cell 0 )

    # 2. Feedback Retrieval: Reflect value back from cell 0
    step_fetch [ read_val <-- Task_Partition[0] ] ( Retrieve value from cell 0 )

    # 3. State Inscription: Mark cell 1 with affirmative PROCEED state
    step_state [ PROCEED --> Task_Partition[1] ] ( Inscribe PROCEED into cell 1 )

    # 4. Invariant Verification: Confirm within capacity
    step_bound [ Task_Partition[0] ! [ +1 ] ] ( Enforce capacity caveat )
]

# ================================================================================
# BLOCK 4: CLOSING BLOCK
# ( Integrity Verification & Invariant Seal )
# ================================================================================
Closing {
    verify: [
        step_store = +1 ::
        step_fetch = +1 ::
        step_state = +1 ::
        step_bound = +1
    ],
    commit_status [ PROCEED ] ( Memory vessel core sealed )
}
```

<TRACK 2> : {OmniCode Language & Standard Library}/lib/core/memory.tape TAPE 28 LINES 174 WORDS 1.2 KB

```
# ================================================================================
# TAPE HEAD: MEMORY RESERVOIR & PARTITIONING COVENANT
# ================================================================================
[Tape.MemoryManager] : WHO {Kernel} WHAT {Storehouse} --> DISPATCH {PartitionEngine}
    WHEN {Boot}
    WHERE {partition} = [ 1024 ]
    WHY ( Allocate and protect physical memory domains )
    HOW [ +1 : PROCEED ]
    :: < covenant > ( verified )

# ================================================================================
# SUBORDINATE ROLLS: PARTITIONING & CELL INDEXING
# ================================================================================
# Roll 1: Allocate root storehouse of 1024 cells
roll_1 [ Root_Storehouse [ 1024 ] ] ( Root memory domain )

# Roll 2: Carve task partition of 256 cells via v--
roll_2 [ Root_Storehouse v-- Task_Partition [ 256 ] ] ( Isolated task memory slice )

# Roll 3: Inscribe value 99 into Task_Partition at cell 0
roll_3 [ 99 --> Task_Partition @ 0 ] ( Direct cell write )

# Roll 4: Inseparable key & value dual binding
roll_4 [ Key&Value [ 1 & 99 ] ] ( Inseparable dual binding )

# Roll 5: Seal memory tape
roll_5 [ commit_status [ PROCEED ] ] ( Memory tape verified )
```

<TRACK 2> : {OmniCode Language & Standard Library}/lib/core/vectors.omni OMNI 151 LINES 676 WORDS 5.1 KB

```
# ========================================================================
# BLOCK 1: METADATA BLOCK
# ( Core Library Manifest: Invariant Directional Macros & Vector Opcodes )
# ========================================================================
Module < Core.Vectors > ( Invariant vector macros linked to machine dispatch and branching ) {
    version: < "1.0.0" >,
    system: < OmniCode.Core >,
    semantics: < MacroInvariants :: VectorDispatch :: CPUBranching >
}

# Macro Invariant: < ACT_UPON > (Right: Forward Causation)
ACT_UPON_MACRO < InvariantMacro > ( Forward State Dispatch & Mutation ) {
    canonical: < "-->" >,
    shorthand: < "->" >,
    grammar_primary: < "Verb" >,
    grammar_secondary: < "Preposition" >,
    axis: < "+X (Temporal Causality)" >,
    machine_effect: < "forward_pipeline_commit" >
}

# Macro Invariant: < ACT_AGAIN > (Left: Retroaction / Rollback)
ACT_AGAIN_MACRO < InvariantMacro > ( Retroactive Feedback & State Rollback ) {
    canonical: < "<--" >,
    shorthand: < "<-" >,
    grammar_primary: < "Verb" >,
    grammar_secondary: < "Preposition" >,
    axis: < "-X (Feedback / Rewind)" >,
    machine_effect: < "restore_snapshot_branch" >
}

# Macro Invariant: < BUILDS_TO > (Up: Synthesis / Abstraction)
BUILDS_TO_MACRO < InvariantMacro > ( Layering Up into Abstract Domain ) {
    canonical: < "--^" >,
    shorthand: < "-^" >,
    grammar_primary: < "Verb" >,
    grammar_secondary: < "Preposition" >,
    axis: < "+Z (Scale / Synthesis)" >,
    machine_effect: < "type_promotion_and_framing" >
}

# Macro Invariant: < BREAKS_DOWN_FROM > (Down: Decomposition / Analysis)
BREAKS_DOWN_FROM_MACRO < InvariantMacro > ( Layering Down into Hardware Primitive ) {
    canonical: < "--v" >,
    shorthand: < "-v" >,
    grammar_primary: < "Verb" >,
    grammar_secondary: < "Preposition" >,
    axis: < "-Z (Lowering / Destructuring)" >,
    machine_effect: < "register_unpack_and_lowering" >
}

# Scale Modulations (Major, Minor, Great, Small)
MAJOR_MACRO < InvariantMacro > ( Major Ascending Scale Modulation ) {
    canonical: < "/-^" >,
    grammar_primary: < "Verb" >,
    grammar_secondary: < "Adverb" >,
    axis: < "+Scale Major" >,
    machine_effect: < "major_scale_elevation" >
}

MINOR_MACRO < InvariantMacro > ( Minor Descending Scale Modulation ) {
    canonical: < "\\-v" >,
    grammar_primary: < "Verb" >,
    grammar_secondary: < "Adverb" >,
    axis: < "-Scale Minor" >,
    machine_effect: < "minor_scale_lowering" >
}

GREAT_MACRO < InvariantMacro > ( Great Ascending Scale Modulation ) {
    canonical: < "\\-^" >,
    grammar_primary: < "Verb" >,
    grammar_secondary: < "Adverb" >,
    axis: < "+Scale Great" >,
    machine_effect: < "great_scale_elevation" >
}
```

```
75
76  SMALL_MACRO < InvariantMacro > ( Small Descending Scale Modulation ) {
77      canonical: < "/-v" >,
78      grammar_primary: < "Verb" >,
79      grammar_secondary: < "Adverb" >,
80      axis: < "-Scale Small" >,
81      machine_effect: < "small_scale_lowering" >
82  }
83
84  # 2D Quadrant Spatial Vectors
85  UPPER_RIGHT_MACRO < InvariantMacro > ( Quadrant I Spatial Vector: +X, +Y ) {
86      canonical: < "/^" >,
87      grammar_primary: < "Verb" >,
88      grammar_secondary: < "Preposition" >,
89      axis: < "+X, +Y (Upper Right)" >,
90      machine_effect: < "quadrant_1_spatial_vector" >
91  }
92
93  LOWER_RIGHT_MACRO < InvariantMacro > ( Quadrant IV Spatial Vector: +X, -Y ) {
94      canonical: < "\\v" >,
95      grammar_primary: < "Verb" >,
96      grammar_secondary: < "Preposition" >,
97      axis: < "+X, -Y (Lower Right)" >,
98      machine_effect: < "quadrant_4_spatial_vector" >
99  }
100
101 UPPER_LEFT_MACRO < InvariantMacro > ( Quadrant II Spatial Vector: -X, +Y ) {
102     canonical: < "^\\" >,
103     grammar_primary: < "Verb" >,
104     grammar_secondary: < "Preposition" >,
105     axis: < "-X, +Y (Upper Left)" >,
106     machine_effect: < "quadrant_2_spatial_vector" >
107 }
108
109 LOWER_LEFT_MACRO < InvariantMacro > ( Quadrant III Spatial Vector: -X, -Y ) {
110     canonical: < "v/" >,
111     grammar_primary: < "Verb" >,
112     grammar_secondary: < "Preposition" >,
113     axis: < "-X, -Y (Lower Left)" >,
114     machine_effect: < "quadrant_3_spatial_vector" >
115 }
116
117 # ================================================================================
118 # BLOCK 2: OPENING BLOCK
119 # ( Preconditions & Vector Context )
120 # ================================================================================
121 Opening {
122     requires: [ Pipeline.DirectExecution ],
123     preconditions: [ +1 ]
124 }
125
126 x_pos [ 1 ] ( Initial X-axis coordinate )
127
128 # ================================================================================
129 # BLOCK 3: BODY BLOCK
130 # ( Directional Macro Operations )
131 # ================================================================================
132 Body [
133     # 1. Forward Causality: Acts upon x_pos, advancing to 10
134     step_forward [ x_pos --> 10 ] ( Advance forward along +X )
135
136     # 2. Feedback: Reflects back upon x_pos
137     step_feedback [ x_pos <-- 1 ] ( Retroactive check along -X )
138 ]
139
140 # ================================================================================
141 # BLOCK 4: CLOSING BLOCK
142 # ( Verification & Commit )
143 # ================================================================================
144 Closing {
145     verify: [
146         step_forward = 10 ::
147         step_feedback = 10
148     ],
149     commit_status [ PROCEED ] ( Vector core sealed )
150 }
```